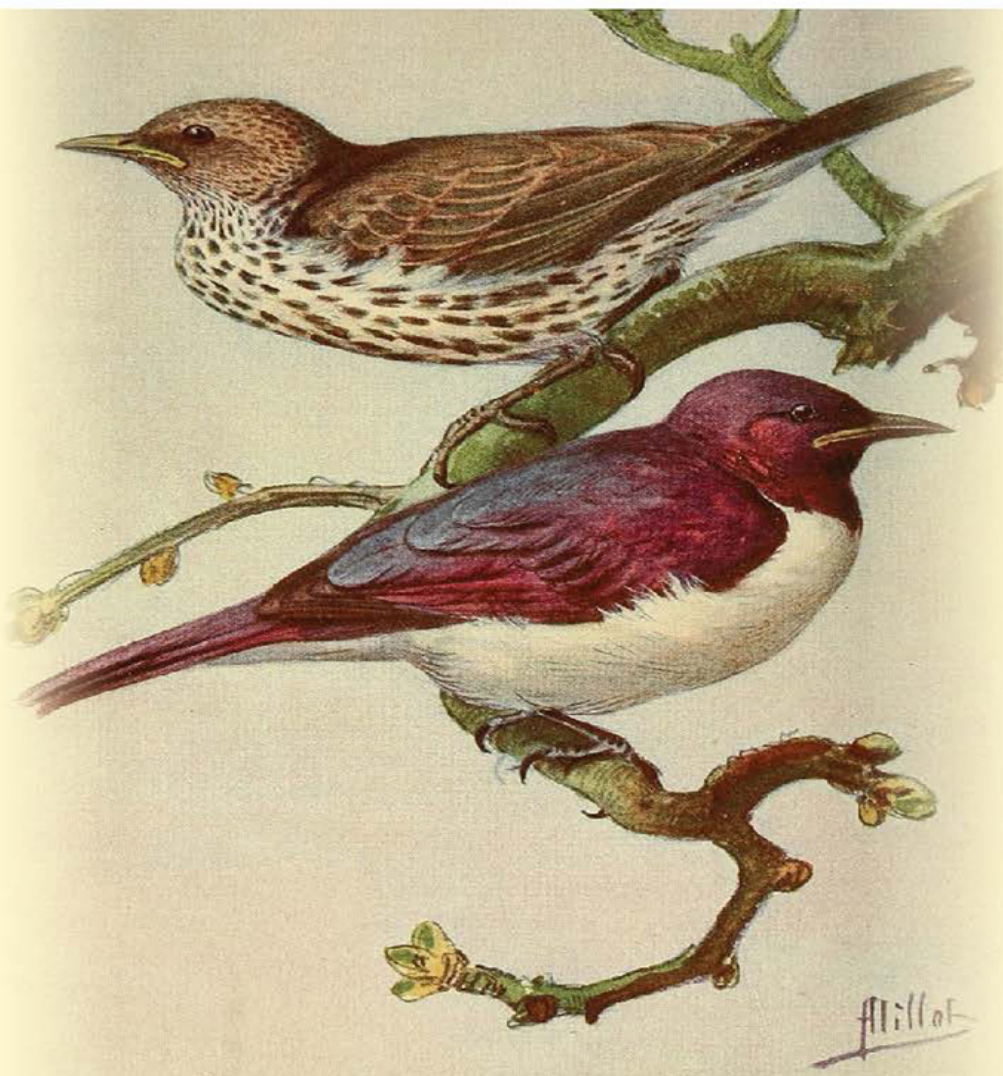


精通计算机系列

精通Rust：初学者指南



译者：chszs

主页：<https://blog.csdn.net/chszs>



CRC Press
Taylor & Francis Group

CO: 前言

- 2022.9.21

掌握 Rust 可以帮助读者掌握强大的 Rust 编程语言，以创建稳定且通用的应用程序和项目。

Rust 是一种可靠且健壮的编程语言，它是根据当今的需求创建的，这是其他几种脚本语言所缺乏的。

Rust 的开发旨在提供可与 C 和 C++ 相媲美的高级功能，并且专注于代码完整性，这可以说这是 C 等语言所缺乏的。Rust 是一种强调性能和可靠性的动态类型语言，尤其是在并行性和存储的组织方面。**Rust 允许您将数据存储在塔式(tower)或卸载式(shedload)上**，并且它认识到性能优化的重要性。与同类中的大多数其他编程语言相比，它可以实现更高效的内存利用以及更快的内存管理。

别搞错了——Rust 是一种学习曲线很陡峭的编程语言，即使是最有经验的开发人员也认为它很复杂。学习 Rust 的回报很多，但学习过程本身需要具备很大的决心和努力。

尽管如此，Rust 旨在提供一种安全、并发且实用的系统语言，这也是为什么 Rust 通常是构建复杂且高度稳定的应用程序的首选语言的主要原因。在表现力、速度、**声音设计(sound design)**和内存存储方面，Rust 拥有优于许多其他编程语言的优势。虽然语言是新的并且随着时间不断变化，但在这个领域有一个很好的未来就业机会。也就是说，要学习可靠的 Rust 语言，你需要有一个同样可靠的同伴指南，这就是掌握 Rust 的用武之地。

掌握 Rust，学习 Rust 编程语言成为一种魅力，无疑将帮助读者推进他们的职业生涯。

《Mastering Computer Science》系列由在计算领域拥有十多年经验的作家和教育家 Sufyan bin Uzayr 编辑。

《Mastering Computer Science》系列前言

《Mastering Computer Science》系列涵盖了广泛的主题，涵盖编程语言以及现代技术和框架。该系列特别关注初学者级别的内容，并以易于理解的方式呈现，包括：

- 清晰的文本，涵盖按相关性排序的各种主题
- 特别注重实践练习，提供大量代码示例和程序
- 引导式编程方法，为绝对初学者提供分步教程
- 强调技能在现实世界中的实用性，从而减少冗余和很少使用的概念，转而关注行业流行的编码范式
- 广泛的参考资料和资源可帮助初学者和中级开发人员充分利用书籍

Mastering Computer Science 系列书籍从核心概念开始，然后快速转向行业标准的编码实践，以帮助学习者在尽可能短的时间内获得高效和关键的技能。这些书假设没有编码知识，因此即使是绝对新手编码员也可以从本系列中受益。

译者说

我的初衷并非是要翻译这本书，而是有感于近两年在学习和使用 Rust 时，总觉得自己还做不到丝滑般的编写 Rust 代码。仔细思考，问题出在我对 Rust 语言的基础掌握的还不够牢固。有鉴于此，我买了这本书。在学习本书的过程中，我不时地记笔记，随后转念一想，我何不完整的把此书翻译完呢？既可以查漏补缺，又可以分享给别人，可谓是一举两得。

读完/译完这本书，我的体会是这本书说不上经典，也不觉得特别好。作为一本初学者指南，还行。

不好的地方如下：

1. 书中的小错误有很多，不知道是印刷问题还是原稿问题，又或者是审稿问题。
2. 书中的描述不够精确，比如对于某个函数，其原文中有时称它为函数，有时又称之为方法。我的看法是，既然是同一本书，对于名词或术语的称呼一定要保持一致。
3. 描述不够精炼，口水话过多。尽管是面向初学者的书籍，但仍然是一本专业书籍，那么简洁、清晰地表达内容就应该贯穿始终。

限于译者的水平有限，一些地方为了准确表达原文的原意，读起来有些绕口。不过还好好的是，这样的情况并不多见。

译者：chszs

博客主页：<https://blog.csdn.net/chszs>

发布日期：2023年1月9日

C1: 开始使用 Rust

- 2022.9.21

在这一章当中，涵盖如下内容：

- 入门
- 安装
- 编程基础

为初学者和专家演示了 Rust 编程语言。Rust 编程语言旨在提高内存安全性；但是，它仍在发展中。

究竟什么是 Rust？

Rust 是一种系统编程语言，由 Mozilla 工程师 Graydon Hoare 于 2006 年创建。他称其为“安全、并发且实用的编程语言”，支持函数式和命令式范式。

Rust 的语法与 C++ 编程语言的语法相当。Rust 是免费且开源的软件，这意味着任何人都可以免费使用它，并且源代码是公开提供的，任何人都可以增强其产品的设计。

在 2016 年、2017 年和 2018 年，Stack Overflow 开发者调查将 Rust 评为“最喜欢的编程语言”之一。

Rust 中没有诸如 `calloc` 或 `malloc` 之类的直接内存管理的東西。Rust 在内部管理内存。

Rust 的开发旨在提供与 C 和 C++ 相当的优异性能，同时优先考虑代码的安全性，这是 C 和 C++ 这两种语言的致命弱点。Rust 编程语言目前被 Firefox、Dropbox 和 Cloudflare 等知名软件巨头采用。从初创公司到大型企业，许多公司都在生产中采用这种技术。

Rust 适合谁

由于各种原因，Rust 编程语言对许多人来说非常出色。

我们来看一下：

- **开发团队：**Rust 已被证明对“开发团队”非常有效。低层编程代码存在测试人员必须彻底测试的缺陷。但是，在使用 Rust 的情况下，如果代码包含缺陷，编译器会拒绝编译。通过与编译器并行工作，开发人员可以专注于程序的逻辑而不是缺陷。
- **学生：**许多人可以学习使用 Rust 构建操作系统。Rust 团队正在努力使公众更容易理解系统原理，尤其是那些刚接触编程的人。

- 公司：大型和小型企业都使用 Rust 来完成各种需求。命令行工具、Web 服务、DevOps 工具、嵌入式设备、音频和视频分析和转码、加密货币、生物信息学、搜索引擎、物联网应用程序、机器学习，甚至大部分 Firefox 网络浏览器都在其中。
- 开源软件的开发者：Rust 是一种开源语言；源代码对公众开放。因此，他们可能会利用源代码来增强 Rust 的设计。

Rust 越来越受欢迎

根据 2020 年 Stack Overflow 开发人员调查，Rust 是最受欢迎的编程语言，该调查对大约 65,000 名开发人员进行了调查。它也连续第六年获得冠军。

此外，Linux 内核工程师在 2020 年建议使用 Rust 创建新的 Linux 内核代码。要明确的是，他们不希望重做最初用 C 构建的整个内核，而是希望使用 Rust 添加新的内核代码，当前的内核基础设施保持不动。开源操作系统 Linux 的创始人 Linus Thorvalds 支持这一想法，并渴望看到该项目的成果。

Facebook 加入了 Rust 基金会，进一步扩大了与 Rust 的联系，该基金会成立于 2021 年，旨在促进 Rust 开发并使其成为“系统编程及其他领域的主流语言选择”。Rust 被 Facebook、亚马逊、谷歌、华为、微软和 Mozilla 使用。

是什么让 Rust 编程语言与众不同？

Rust 是一种强类型编程语言，它优先考虑速度和安全性以及非常安全的并发和内存管理。它的语法与 C++ 相当。这是一个始于 Mozilla Research 的开源项目。Rust 基金会已接过火炬，并在 2021 年引领该语言的发展。

Rust 解决了 C/C++ 开发人员长期存在的两个问题：内存错误和并发编程。这被认为是它的主要优势。

当然，有人可能会争辩说，当代 C++ 更多地关注内存安全（例如，通过实现智能指针），但许多问题仍未解决。

其中之一是“释放后使用错误”，当程序在被释放后继续使用指针时会发生这种情况，例如，在释放其引用捕获的对象后运行 lambda 函数。

另一方面，在 Rust 中，存在**借用检查器(borrow checker)**，即编译器组件会保证引用的存活时间不会超过它们所引用的数据。此功能有助于消除内存违规错误。此类问题在编译过程中得到确认；因此，不需要收集垃圾。

此外，在 Rust 中，每个引用都有一个生命周期，它指定了该引用的有效范围。这一创新克服了无效引用的问题，同时也将 Rust 与 C 和 C++ 区分开来。

当我们考虑到过去 12 年内存安全问题已占 Microsoft 产品中所有安全漏洞的 70% 以上时，正确的内存管理的必要性立即变得清晰。Google Chrome 也有类似的数据。

用 Rust 编写代码有两种方式：Safe Rust 和 Unsafe Rust。Safe Rust 对程序员施加了额外的限制（例如，**对象所有权管理(object ownership management)**），以保证代码的正常运行。Unsafe Rust 允许程序员有更多的自由（例如，它可以在原始类 C 指针上工作），但代码可能会失败。

有风险的 Rust 模式（即 `unsafe rust`）为我们提供了更多可能性，但我们必须格外小心以确保我们的代码是安全的。为此，我们可以将其封装在更高级别的抽象中，以确保所有抽象的应用程序都是安全的。与其他编程语言一样，应谨慎处理使用不安全代码，以阻止未定义的行为并减少因内存不安全而导致的**内存段错误（segfaults）**和漏洞的危险。Rust 最显著的优势之一是它的**双模式（dual-mode）**概念。另一方面，在 C++ 中，我们从未意识到我们已经编写了危险的代码，直到我们的产品出现崩溃或发生安全漏洞。

LQ 注1: Segmentation fault, 经常被缩写为 segfault, 内存区段错误, 又译为内存段错误, 也称为总线错误 (bus error)、访问权限冲突 (access violation), 是一种程序错误。内存段错误出现在当程序企图访问 CPU 无法寻址的内存区段时。当错误发生时, 硬件会通知操作系统, 产生了内存访问权限冲突的状况。操作系统通常会产生核心转储 (core dump), 以方便程序员进行除错。通常该错误是由调用一个地址, 而该地址为空(NULL)造成的。比如链表中调用一个未分配地址的空链表单元的元素。数组访问越界也可能产生这个错误。

并发编程已被简化

Rust 通过在构建时防止数据冲突来简化并发编程。当来自不同线程的至少两条独立指令尝试同时访问同一内存位置时，其中至少一条尝试写入内容，并且没有同步以在各种访问之间创建任何精确的顺序，就会发生数据竞争（**data race**）。没有同步的内存访问是未定义的(**undefined**)。

在 Rust 中检测到数据竞争。假设给定的对象不允许多个线程访问（即，它没有被指定为适合的 **trait**）。在这种情况下，它必须由一个互斥锁（**mutex**）同步，它为其其他线程锁定对该特定对象的访问。

只有一个线程可以访问一个对象，以保证其操作不会损坏它。

从其他线程的角度来看，对该对象的操作是原子的，这意味着观察到的对象状态始终是有效的。我们无法见证其他线程对该对象的操作所产生的任何中间状态。Rust 编程语言可以在构建时检测和报告对此类对象的不当操作。

其他语言有同步技术，但它们与它们所引用的对象无关。开发者的责任是确保对象在使用前被锁定。

使用 Rust 编程时需要克服的一些挑战

当然，并非一切都是完美的。因为 Rust 是一项新技术，满足某些特定需求的库可能还没有。尽管如此，Rust 软件包库 `crates.io` 自 2016 年以来一直在迅速增长，强大的 Rust 开发者社区充满了活力。

此外，对于不习惯使用在编译时识别代码中问题的开发人员来说，看到大量错误消息可能会加剧这种印象。因此，与 Python 等更流行的编程语言相比，创建代码所需的时间更长。另一方面，Rust 的开发人员正在努力使这些错误消息尽可能有用且可操作。

即使在编码时看到这么多错误警告可能很烦人，也要关注更广泛的情况。在构建时实施的内存安全可以避免在我们的软件发布后发生缺陷和安全漏洞。在这个阶段，纠正问题无疑会耗费我们的精力和金钱。

最后但同样重要的是，由于入门门槛相对较高，创建 Rust 代码需要额外的工作。我们必须花一些时间来学习语言。还需要对 C++ 或其他面向对象的编程语言有扎实的理解。学习曲线远非平坦。

但是，如果我们能够克服所有这些挑战，那么利用 Rust 的优势将是我们工作的最终回报。

Rust 的目标是什么？

Rust 是一项已在业界使用的成熟技术。作为一种系统编程语言，它可以帮助我们控制低级细节。我们可以将数据存储在栈上（用于静态内存分配）或堆上（用于动态内存分配）。RAII（Resource Acquisition Is Initialization）是一个基本的编程习惯，主要与 C++ 相关，但也存在于 Rust 中：一旦对象退出作用域，它的析构函数就会被调用，并且它拥有的资源会被释放。我们不必手动执行它，而且我们可以安全地避免资源泄漏问题。

LQ 注2：RAII 即 Resource Acquisition Is Initialization，是在一些面向对象语言中的一种惯用法。RAII 源于 C++，在 Java，C#，D，Ada，Vala 和 Rust 中也有应用。1984~1989 年期间，比雅尼·斯特劳斯特鲁普和安德鲁·柯尼希在设计 C++ 异常时，为解决资源管理时的异常安全性而使用了该用法，后来比雅尼·斯特劳斯特鲁普将其称为 RAII。RAII 要求，资源的有效期与持有资源的对象的生命期严格绑定，即由对象的构造函数完成资源的分配(获取)，同时由析构函数完成资源的释放。在这种要求下，只要对象能正确地析构，就不会出现资源泄露问题。

结果，可以更有效地使用存储器。Tilde 在他们的 Skylight 产品中使用了 Rust 来重建了一些 Java HTTP 端点（*LQ：REST API 服务*）。因此，他们能够将内存使用量从 5GiB 减少到 50MiB。

LQ 注3：GiB、MiB 和 GB、MB 的区别在于，GB、MB 等单位是以 10 为底数的指数；而 GiB、MiB 等单位是以 2 为底数的指数。也即 $1KB=10^3=1000$ ， $1MB=10^6=1000000=1000KB$ ， $1GB=10^9=1000000000=1000MB$ ；而 $1KiB=2^{10}=1024$ ， $1MiB=2^{20}=1048576=1024KiB$

因为 Rust 没有活动的垃圾收集器，其他编程语言可能会通过外部函数接口 (foreign-function interfaces) 将其项目用作库。对于现有项目而言，这是一个很好的案例，在这些项目中，高性能同时保持内存安全至关重要。在这种情况下，Rust 代码可能会替换某些对速度至关重要的软件模块或组件，而无需重新构建整个产品。

LQ 注4: FFI, Foreign Function Interface , 外部函数接口, 是用来与其它编程语言交互的接口。在一些语言中称为语言绑定(Language bindings), 在 Java 语言中通常称为 JNI(Java Native Interface) 或 JNA(Java Native Access)。由于现实中很多程序是由不同编程语言写的, 必然会涉及到跨语言调用, 比如 A 语言写的函数如果想在 B 语言里面调用, 这时一般有两种解决方案: 一种是将函数做成一个服务, 通过进程间通信(IPC)或网络协议通信(RPC, RESTful 等); 另一种就是直接通过 FFI 调用。前者需要至少两个独立的进程才能实现, 而后者直接将其它语言的接口内嵌到本语言中, 所以调用效率比前者高。

Rust 是一种低层 (low-level) 编程语言, 可提供对硬件和内存的直接访问, 使其成为嵌入式和裸机开发的绝佳选择。Rust 可用于创建操作系统和微控制器应用程序。实际上, 存在几个用 Rust 编写的操作系统, 包括 Redox、intermezzOS、QuiltOS、Rux 和 Tock。正如我之前所说, 有计划在 Rust 中创建额外的 Linux 内核增强功能。Mozilla 在创建浏览器引擎时使用了 Rust。

LQ 注5: Low-Level 如果按常规译为‘低级’, 会引入其它歧义, 低级的通俗含义中带有贬义, 会对读者产生不好的诱导, 从而不客观。译为‘底层’也不够妥当, 底层默认的含义中带有最下面的一层的意味, 而硬件层之上的底层语言非汇编语言莫属, 故译为低层更合适。

Rust 的高性能和安全性等特性吸引了科学家并导致它被用于大数据分析领域。Rust 的速度非常快, 使其成为必须快速处理大量数据的计算生物学 (computational biology) 和机器学习的绝佳选择。

在 CodiLime, 我们还在测试 Rust 是否可以在需要高数据速度的网络环境中替代 C。在我们的概念验证中, 我们用 Rust 创建了一个应用程序, 它使用 DPDK 库 (用 C 编写) 来确保速度, 同时确保内存安全。

LQ 注 6: [DPDK](<https://www.dpdk.org/>), Data Plane Development Kit, 数据平面开发工具包, 由 6WIND、Intel 等多家公司联合开发, 用于加速在各种 CPU 体系架构上运行的数据包处理工作负载的库组成, 主要在 Linux 用户空间中运行, 可以极大提高数据处理的性能和吞吐量, 提高数据平面应用程序的工作效率。

为什么我们应该使用 Rust?

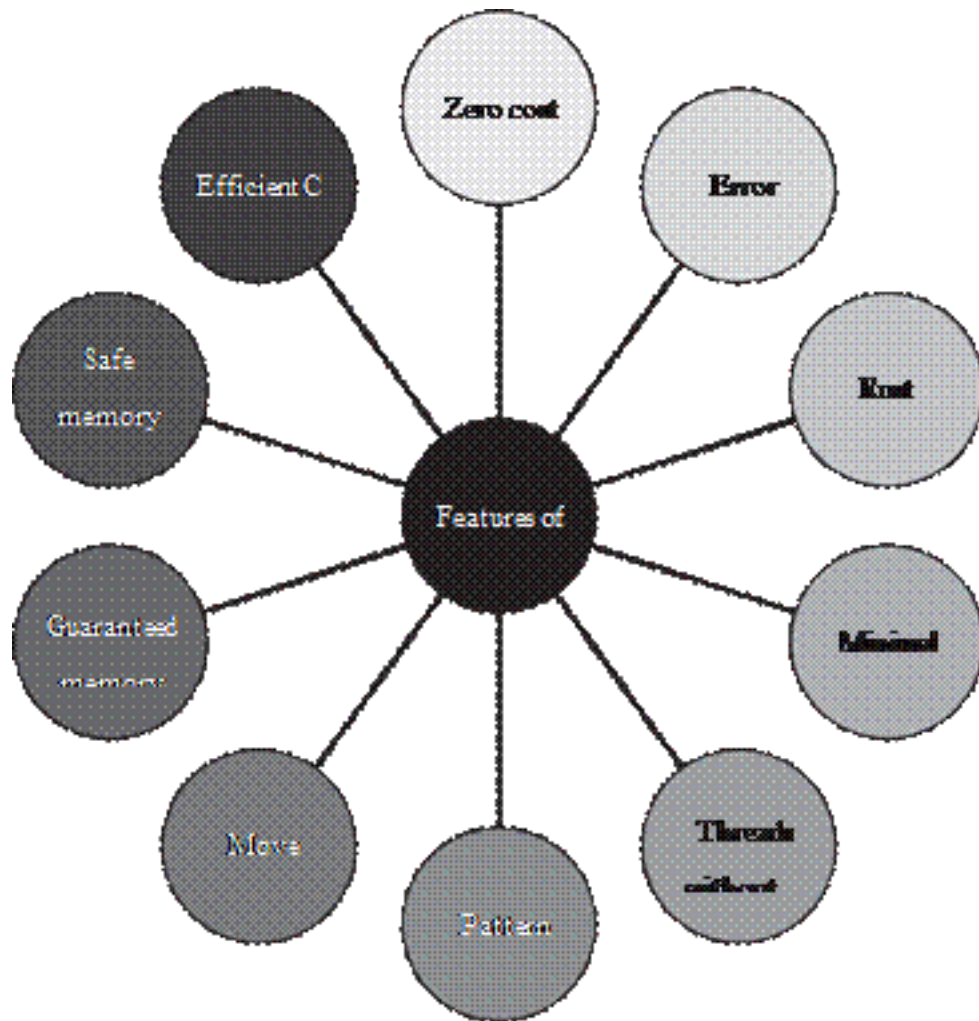
总而言之, 在我们的下一个软件项目中包含 Rust 的显著优势如下:

- 在保持高性能的同时确保内存安全。
- 支持并发编程。
- crates.io 存储库中有越来越多的 Rust 软件包。
- 一个充满活力的社区正在推动语言的发展。
- 向后兼容性和稳定性都得到保证。

Rust 的特点

Rust 是一种系统编程语言, 它具有以下功能:

- 抽象零开销
- 错误信息
- 类型推断
- 移动语义
- 没有数据竞争的线程
- 模式匹配
- 保证内存安全
- 高效的 C 绑定
- 安全的内存空间分配



Rust 的特点

1. 抽象零开销（zero cost abstraction）：我们可以在不影响代码的运行时性能的情况下引入抽象。这在不牺牲运行时效率的情况下提升了代码质量和可读性。

2. 错误消息 (error messages)：与 GCC 相比，C++ 编程显著改进了错误消息。在错误消息的清晰度方面，Rust 更进一步。在 Rust 应用程序中，错误消息的显示包含了格式和颜色，而且支持拼写错误检查。
3. 类型推断 (type inference)：Rust 具有类型推断功能，可以自动检测表达式的类型。
4. 移动语义 (move semantic)：当源对象是临时对象时，可以用 move 操作代替 copy 动作。
5. 没有数据竞争的线程 (threads without the data races)：当两个或多个线程同时访问同一内存地址时，这称为数据竞争。由于所有权机制，Rust 支持没有数据竞争的线程。只有单独对象的所有者通过所有权机制被发送到各个线程，并且两个线程永远不能同时拥有对同一个变量的写访问权限。
6. 模式匹配 (pattern matching)：Rust 具有模式匹配能力。Rust 的模式匹配将模式与“匹配”表达式结合使用，以提供对程序控制流的额外控制。以下是一些模式组合：- 字面量 - 数组、枚举、结构或元组 - 变量 - 通配符 - 占位符
7. 保证内存安全 (guaranteed memory safety)：Rust 采用所有权的思想保证了内存安全。所有权折中解决了 C 的内存控制和 Java 的垃圾收集。Rust 程序中的内存空间由变量所拥有，并且其他变量可以临时借用。这使 Rust 能够在编译时确保内存安全，而不是依赖垃圾收集器。
8. 高效的 C 绑定 (efficient bindings)：Rust 中“高效的 C 绑定”的特性意味着 Rust 语言可以与 C 语言进行通信时与它进行交互。Rust 提供了一个“外部函数接口 (FFI)”来连接 C API，同时还利用其所有权结构来确保内存安全。
9. 安全的内存空间分配 (safe memory space allocation)：Rust 使用手动内存管理，这意味着程序员可以完全控制创建和释放内存的位置和时间。在 C 中，我们使用 malloc 函数来分配内存并随后对其进行初始化，而 Rust 使用单个“~”运算符拒绝这两个操作。此操作将智能指针 (smart pointer) 转换为 int。智能指针是一种调节项何时释放的值。智能指针之所以称为“智能”，是因为它们不仅可以监视对象的位置，而且还知道如何清除它。

安装 Rust

第一步是安装 Rust。首先，使用 rustup 下载 Rust，这是一个用于管理所有 Rust 版本和相关工具的命令行应用程序。

在 Windows 安装 Rust

- 在 Windows 上，导航到 <https://www.rust-lang.org/install.html> 并按照安装说明进行操作。
- Rust 的 PATH 变量会在安装后自动添加到您系统的 PATH 环境变量中。
- 打开命令提示符，并执行以下命令：

```
$ rustc --version
```

- 运行此命令后，我们应该会看到版本号、提交哈希和提交日期。
- 如果这样做了，则表示 Rust 已成功安装。

在 Linux 或 macOS 上安装 Rust:

- 如果我们使用的是 Linux 或 macOS，请打开终端并输入以下命令：

```
$ curl https://sh.rustup.rs -sSf | sh
```

- 上述程序下载脚本并开始安装 rustup 实用程序。这将安装最新版本的 Rust。如果安装成功，将显示以下消息：

“Rust is now installed.”

- 重启终端 Shell 环境，Rust 相关的环境变量会生效，上面的安装会自动将 Rust 添加到系统路径中。如果您希望在不重新启动终端的情况下让 Rust 环境生效，那么可以在您的 shell 中运行以下命令手动使之生效：

```
$ source $HOME/.cargo/env
```

- 安装后还需要 linker 链接器。当尝试运行我们的 Rust 程序时，我们将收到一条错误消息，指出无法执行链接器。它表明我们的系统中还没有链接器。C 编译器就是我们所需的东西。设置 C 编译器。此外，一些 Rust 软件包依赖于 C 代码并需要 C 编译器。

更新和卸载 Rust

更新：使用“rustup”安装 Rust 后，更新到最新版本。要升级到最新版本，请使用以下命令：

```
$ rustup update
```

删除：要卸载 Rust，请在 shell 中使用以下命令：

```
$ rustup self uninstall
```

第一个 Rust 程序

让我们用 Rust 编程语言编写一个基本程序。打开记事本文件并键入以下代码：

```
fn main() {  
    println!("Hello, Rust!");  
}
```

- `main()`：它始终是所有 Rust 项目的入口点。大括号 `{}` 包围 `main()` 函数。`main()` 函数不接受任何参数，也不返回任何值。
- `println!`：它是 Rust 语言中的一个宏。如果它调用该方法，它不包含“!”。
- “Hello, Rust!”：它是作为参数传递给 `println!` 的字符串，并打印到控制台。

创建、编译和运行程序的过程：

1. 打开记事本文件，在里面写入代码。
2. 使用扩展名为 `.rs` 保存文件，文件名自行定义，这里假设为 `filename.rs`。
3. 启动命令提示符。
4. 进入上面源代码文件的目录。
5. 运行 “`rustc filename.rs`” 命令编译上述程序。
6. 最后，使用命令 `filename.exe` 执行应用程序。

RUST 适合未来的原因

在 Stack Overflow 2020 调查中，Rust 被评为最受欢迎的编程语言，86% 的工程师表示他们将继续使用它。对于语言设计师来说，这并不是什么新鲜事：自 2016 年以来，Rust 每年都在这项研究中获胜。在 Tiobe 指数上，Rust 也越来越受欢迎，在 9 月份最受欢迎的语言中排名第 18 位。它在我们基于情感的函数式编程语言排名中也得分很高，在最有利的情感类别中排名第一。

Rust 由前 Mozilla 工程师 Graydon Hoare 于 2006 年创建，作为 C++ 的替代品，它正在开发者社区稳步创建一个名字。它目前用于构建在线应用程序、嵌入式计算机、分布式服务和命令行界面。这种流行的一个很好的例子是微软逐渐从 C++（由于越来越多的安全问题）转向 Rust。

当有这么多不同的语言可供选择时，为什么 Rust 在程序员中如此受欢迎？一个关键原因是 Rust 解决了其他语言中的几个棘手的难题，以至于一些开发人员声称 Rust 的架构师必须知道 C++ 中可见的所有可能问题。

Rust 与其他编程语言的比较

与目前在开发人员中流行的其他几种语言相比，Rust 提供了一些优势。让我们来看看其中的几个。

1、Rust 相比于 C++ 的优势

与 C++ 相比，Rust 更安全。Rust 保护它的抽象和开发人员创建的抽象，而 C++ 没有。C++ 中的特定错误可能会导致任意的行为，但 Rust 可以帮助您专注于真正重要的事情。而且，虽然 C/C++ 仍然是最流行的编程语言之一，但它经常引起问题。Rust 更容易学习；学习曲线不如 C++ 那样陡峭，Rust 没有技术债务，它支持更多的并发性，并且它的速度与 C++ 相当。Rust 允许我们编写不安全的代码，但它始终默认为安全代码。

2、Rust 相比于 Java 的优势

当谈到 Java 时，事实证明它比 Rust 慢得多。最重要的是，我们应该考虑更快的启动时间和更少的内存占用。Java 使用垃圾收集器进行内存管理，这会降低速度（但值得注意的是它简化了代码）。

3、Rust 相比于 Python 的优势

Rust 是经过深思熟虑的。Rust 允许我们将语句包装到 `lambda` 中，并且一切都是表达式，这使得组合语言的特定部分变得容易。Python 不这样做。Rust 没有类；因此，面向对象不如 Python 那样发达。Python 还遇到需要编写额外的测试和生产中断或运行时问题。Rust 降低了识别和修复潜在问题的成本。

4、Rust 相比于 Go 的优势

Go 的表现力不足。Rust 是一种通用且富有表现力的架构，可以创建能够容纳各种组件、泛型、`traits` 和代数数据类型的新容器类型。使用 Go 时，我们对资源和内存的控制较少。

是什么让 Rust 代表编程的未来？

1、Rust 提高了内存的安全性

内存管理困难是开发人员面临的最严重的问题之一。在其他语言中，很容易忽略编码问题，从而导致可怕的错误代码以及发现和解决它所需的努力。当代码错误导致当今环境中的安全漏洞时，就会出现一个更严重的问题，这太冒险了。主要和流行的网站或应用程序中的数据安全漏洞发生的频率远远超出我们的预期。什么原因？应用程序缺陷或错误配置通常是罪魁祸首。

Rust 被认为是内存安全的。拖拽指针、缓冲区溢出和其他与内存相关的错误不允许使用 Rust 编程。它允许您描述应该如何处理内存以及应该如何将值放入其中，同时还可以在不牺牲性能的情况下处理控制线和安全线，使其成为 Rust 的宝贵资产。提高内存安全性是 Rust 创造者的主要目标之一，也是该语言的主要卖点之一。他们的代码编译器非常严格，并且会自动检查使用的每个变量或内存位置。

假设检测到任何语法问题、空值、悬空修饰符或内存安全问题。在这种情况下，Rust 不会编译代码并对不安全代码发出警告，从而可以快速识别和纠正潜在问题。它是如何做到这一点的？

Rust 中的每个值都有一个“所有权”。当一个值被传递或返回时，所有权一次转移到一个新的范围。如果值的“所有者”移出范围，则该值将被丢弃。Rust 跟踪内存并以这种方式自动释放它，防止所有缺陷进入主代码。这意味着在运行时不会有任何意外。

该系统将在构建时检查内存管理，从而更容易发现和修复问题并消除垃圾收集的需要。

然而，内存远不止这些。构建复杂的系统通常与大量计算绑定的活动相关联，并会产生大量的临时内存。这种卸载会显着影响任何 Java 虚拟机的性能，而包括 Scala 在内的许多编程语言都无法处理它。

但是，Rust 可以。它允许创建内存效率高的代码，而无需内存管理或使用闭包等现代便利，它运行时几乎没有运行时开销，使其适用于实时或嵌入式项目，并易于与其他语言或项目集成。

2、Rust 社区在逐渐扩大

当然，Rust 社区和库的数量远不及 C++ 之类的大。然而，随着 Rust 编程语言越来越受欢迎，加入的开发人员和爱好者的数量也在增加。社区创建的框架、库和开发工具（称为“crate”）的数量现在接近 57,000 个，并且每天都在增加。

除了越来越多的工具和框架集合之外，Rust 还拥有一个人活跃且引人入胜的社区。如果我们需要帮助解决问题或了解有关使用 Rust 的更多信息，有很多地方可以去。除了社区聊天和用户论坛之外，Rust 还提供了一个活跃的子 reddit 部分。

3、Rust 快速且适应性强

Rust 被认为是一种非常快的语言。在使用通用代码时，它可以比 Scala 运行得快得多，尤其是在性能关键的工作中。在某些地方，它很有可能比 Scala 或 Java 快三倍。获取和使用其他软件包很简单，Rust 的 cargo 构建工具简化了代码。

由于以下几个因素，Rust 非常快：

- 因为它是静态类型和编译的，编译器可以优化代码的速度。
- 它提供了静态 trait 分派，类似于 C++ 模板，但更简洁。
- 它明确地监控变量的所有权。Rust 并不假定每个具有地址的变量都可以随时更改，从而允许优化而不会对代码产生负面影响。
- 它假定变量在默认情况下是不可变的（因此优化更容易）。
- 它提供代数数据类型，这对优化也很有用。
- 它没有任何开销，没有运行时，也没有由于缺乏垃圾收集而导致的意外延迟。
- 它提供了无开销的抽象。

Rust 也可以被认为是一种可与 C++ 相媲美的编程语言，但它更容易优化，并且很少有令人不快的高级编程功能。

4、Rust 的应用范围很广

如果一个项目需要一个安全可靠的执行环境，而且需要性能和低层优化（高级 Rust 函数式编程方法），Rust 应该能满足需求。Rust 被公认为最适合于最佳系统、嵌入式编程和其他性能关键型应用程序的低层语言。它也用于 3D 视频游戏。

5、几家大公司都在使用 Rust

尽管是一种相对较新的语言，但开发人员已经认识到 Rust 是朝着提高编程安全性和使用便利性的正确方向迈出的一步。然而，不仅编码人员对 Rust 感兴趣；很多公司，尤其是大公司，已经迁移或打算转向 Rust。Ryan Levick（微软云开发布道者）在 2020 年 5 月的 AllThingsOpen 虚拟会议上表示，Rust 作为一种语言是“业界正面解决多个问题的最大希望”。虽然微软不打算很快放弃 C++，但它的许多基础设施逐渐过渡到 Rust。

微软并不是唯一一家看到转向 Rust 编程语言潜在好处的公司。Dropbox、Sentry、Amazon 和 Mozilla 都是使用 Rust 编程语言的组织。

在本章中，我们介绍了 Rust 的概念，以及它的历史、特征、目的和应用。此外，我们了解了 Rust 安装、第一个 Rust 程序以及 Rust 代表编程未来的原因。

C2: 常见的编程概念

- 2022.9.22

在这一章当中，涵盖如下内容：

- 变量和可变性
- 数据类型
- 函数
- 元组
- 数组
- 注释
- 控制流

上一章介绍了 Rust 的定义，以及它的历史、好处和用途。我们还了解了 Rust 的安装和基本程序。

本章讨论几乎所有编程语言中的概念以及 Rust 是如何运作的。从本质上讲，许多编程语言有很多共同点。本章中描述的概念都不是 Rust 独有的，但我们将在 Rust 的上下文中探索它们并解释如何使用它们。详细介绍变量、基本类型、函数、注释和控制流。这些基础知识将在任何 Rust 程序中都能遇到，尽早了解它们有助于打下一个坚实的基础。

Rust 的变量

变量是程序可以访问的一种命名存储。简单地说，变量是一种允许程序存储值的数据结构。在 Rust 中，变量与特定的数据类型相关联。数据类型决定了变量的内存大小和布局、存储在该内存中的值的范围以及对变量的操作集。

变量的命名规则

本节将介绍命名变量的各种规则：

- 变量名可以是字母、数字和下划线字符。
- 它以字母或下划线开头。
- 由于 Rust 区分大小写，所以大写和小写字母是分开的。

语法

在 Rust 中声明变量时，数据类型是可选的。分配给变量的值决定了数据类型。

定义变量的语法如下：

```
let variable_name = value; // 没有指定数据类型
let variable_name: dataType = value; // 指定了数据类型
```

例子：

```
fn main() {  
    let fees = 35_000;  
    let salary:f64 = 45_000.00;  
    println!("fees {} and salary is {}", fees, salary);  
}
```

不可变

默认情况下，Rust 语言中的变量是不变的。换句话说，一旦值绑定到某个变量名，该变量的值就无法更改。

让我们以一个例子理解这一点：

```
fn main() {  
    let fees = 25_000;  
    println!("fees is {}", fees);  
    fees = 35_000;  
    println!("fees changes is {}", fees);  
}
```

编译源代码就可以看到这样的错误消息：

```
error[E0384]: cannot assign twice to immutable variable `fees`  
--> r3.rs:4:5  
2 |     let fees = 25_000;  
   |     ----  
   |     |  
   |     first assignment to `fees`  
   |     help: consider making this binding mutable: `mut fees`  
3 |     println!("fees is {}", fees);  
4 |     fees = 35_000;  
   |     ^^^^^^^^^^^^^^^ cannot assign twice to immutable variable
```

此错误消息中指出了错误的来源；我们不能两次为不可变的变量 `fees` 设置值。这仅是 Rust 允许程序员编写代码同时受益于并发的安全性和易用性的众多方式之一。

可变的

默认情况下，变量是不可变的。要使变量可变，请在其前面加上术语 `mutable`，然后可变变量的值就可以改变。

定义可变变量的语法如下所示：

```
let mut variable_name = value;  
let mut variable_name:dataType = value;
```

例如：

```
fn main() {  
    let mut fees = 25_000;  
    println!("fees is {}", fees);  
    fees = 35_000;  
    println!("fees changes is {}", fees);  
}
```

变量和常量：有什么区别？

无法修改变量的值让我们想起了在大多数其他语言中看到的另一个编程概念：常量。与不可变变量一样，常量是附加在名称上的值，不能更改，但常量和变量之间有一些区别。

首先，我们不允许将 `mut` 与常量一起使用。默认情况下，常量不仅是不可变的；它们也始终是不可变的。

常量是用 `const` 关键字而不是 `let` 关键字声明的，并且值类型必须注解（`annotate`）。暂时不要担心技术问题；我将在“[Rust 数据类型](#)”章节介绍类型和类型注释。只要记住总是注解类型。

常量可以在任何范围内声明，包括全局范围，这使得它们对于需要被多个代码区域知道的变量很有帮助。

最后一个区别是常量只能使用常量表达式赋值，而不能通过只能在运行时计算出结果的赋值。

这是一个名为 `THREE_HOURS_IN_SECONDS` 的常量声明示例，其值设置为 60（一分钟的秒数）乘以 60（一小时的分钟数）乘以 3（我们要计算的小时数）的结果在这个程序中：

```
const THREE_HOURS_IN_SECONDS: u32 = 60 * 60 * 3;
```

Rust 的常量命名约定都是大写的，单词之间有以下划线。因为编译器只能在构建时评估有限数量的操作，所以我们可以选择以更易于理解和可验证的方式写出这个值，而不是将值 10,800 分配给这个常量。

常量在程序执行期间声明它们的范围内有效。常量对于您的应用程序域中的数字很重要，程序的许多部分可能需要知道这些数字，例如每个游戏参与者被允许获得的最大点数或光速。

将整个软件中使用的硬编码值命名为常量有助于未来的代码维护者理解这些值代表什么。如果将来需要修改硬编码的值，在我们的代码中只有一个需要更改的位置也很有用。

Rust 中的数据类型

类型系统（Type System）代表编程语言的许多不同类型的值。类型系统在软件存储或操作它们之前检查提供的值。这可确保代码的正常运行。使用类型系统还可以提供更好的代码提示和自动文档。

Rust 是一种静态类型的编程语言。在 Rust 中，每个值都有自己的数据类型。基于提供给变量的值，编译器可以自动确定其数据类型。

变量声明

要声明变量，请使用 `let` 关键字。

```
fn main() {  
    let company_string = "RustPoint";    // string 类型  
    let rating_float = 3.5;    // float 类型  
    let is_growing_boolean = true;    // boolean 类型  
    let icon_char = '♥';    // unicode 字符类型  
    println!("company name:{}", company_string);  
    println!("company rating on 5:{}", rating_float);  
    println!("company is growing :{}", is_growing_boolean);  
    println!("company icon:{}", icon_char);  
}
```

上例中变量的数据类型将从分配给它们的值推导出来。例如，Rust 会将 `string` 数据类型分配给变量 `company_string`，将 `float` 数据类型分配给 `rating_float`。

`println!` 宏接受两个参数：

1. 独特的语法 `{}`，充当占位符
2. 变量或常量的名称

变量的值用于替换占位符。

标量类型

标量类型（Scalar Type）是只有一个值的值。例如，`10`, `3.14`, `'c'`。Rust 有四种不同的标量类型。

1. 整数
2. 浮点数
3. 布尔值
4. 字符

整数

一个没有小数部分的数字被称为整数。简单地说，整数数据类型是用来表示整个整数的。

整数被进一步细分为有符号和无符号。负值和正值可以存储在有符号的整数中。在无符号整数中只能存储正值。下面是对整数类型的完整列表。

序号	大小	有符号	无符号
1	8 位	i8	u8
2	16 位	i16	u16
3	32 位	i32	u32
4	64 位	i64	u64
5	128 位	i128	u128
6	Arch	isize	usize

一个整数的大小可以是 **Arch** 的，这表明机器的架构将决定数据类型的大小。一个大小为 arch 的整数在 x86 机器上是 32 位，在 x64 机器上是 64 位。arch 类型的整数通常用做索引某种类型的集合。

LQ 注1: 这里的 Arch 是 Architecture（架构）的简写，我认为在这里不译更好，理解其含义即可。

```
fn main() {  
    let result = 20;    // 默认为 i32 类型  
    let age:u32 = 30;  
    let sum:i32 = 5-25;  
    let mark:isize = 20;  
    let count:usize = 40;  
    println!("result value is {}", result);  
    println!("sum {} and age {}", sum, age);  
    println!("mark {} and count {}", mark, count);  
}
```

整数范围

有符号的整数变量的范围为 $-2^{(n-1)} \sim 2^{(n-1)}-1$ ，其中 n 是使用的位数。例如，i8 的范围为 $-2^7 \sim 2^7-1$ ；在这种情况下，我们用 8 代替 n 。

无符号整数变量的范围为 $0 \sim 2^n-1$ 。例如，u8 的范围为 $0 \sim 2^8-1$ ，即 0~255 的整数。

整数溢出

当分配给一个整数变量的值超过其数据类型的指定范围时，就会发生整数溢出。让我用一个例子来说明这个问题。

```
fn main() {  
    let age:u8 = 255;  
    // 变量 age 的范围是 0~255  
    let weight:u8 = 256;    // the overflow value is 0  
    let height:u8 = 257;    // the overflow value is 1  
}
```

}

255 之后，溢出值将以 0、1、2 等开头。

浮点数

型:

}

自动类型转换

在 Rust 中，不允许自动类型转换。看下面的代码。浮点数变量想要转换为整数值。

}

编译时会出现如下的错误提示:

error[E0308]: mismatched types

```
--> r3.rs:2:24
```

```
2 | let interest:f32 = 9;
    |               ^
    |               expected `f32`, found integer
    |               help: use a float literal: `9.0`
```

数字分隔符

为了使大数字更容易阅读，我们可以添加下划线分隔符来分隔数字。比如 60,000 可以写成 60_000。这在以下示例中进行了演示：

```
fn main() {  
    let float_with_separator = 11_000.545_001;  
    println!("float value {}", float_with_separator);  
    let int_with_separator = 60_000;  
    println!("int value {}", int_with_separator);  
}
```

布尔型

true 或 false 是布尔类型仅有的两个可能值。要声明布尔变量，请使用 bool 关键字。

例子：

```
fn main() {  
    let isfun:bool = true;  
    println!("Rust Programming Fun ? {}", isfun);  
}
```

字符类型

Rust 的字符数据类型接受整数、字母、Unicode 和特殊字符。要声明字符类型的变量，请使用 char 关键字。Rust 中的 char 类型代表一个 Unicode 标量值，这意味着它代表的不仅仅是 ASCII。Unicode 标量值从 U+0000 到 U+D7FF 以及从 U+E000 到 U+10FFFF。

让我们看一个示例来了解字符类型的更多信息：

```
fn main() {  
    let special_character = '@';  
    let alphabet:char = 'D';  
    let emoji:char = '\u{e415}'; // 注意这里 Unicode 表情符号的写法  
    println!("special character {}", special_character);  
    println!("alphabet {}", alphabet);  
    println!("emoji {}", emoji);  
}
```

复合类型

复合类型可以将多个值组合成一个类型。元组和数组是 Rust 中的两种原始的复合类型。

元组类型

元组（Tuple）是一种将多种不同数据类型的项组合成一个复合类型的通用方法。元组有一个固定的长度：一旦声明，它们的大小就不能增加或减少。

通过将逗号分隔的值列表放在圆括号内来形成元组。元组中的每个位置都有一个数据类型，元组中元素的数据类型不必相同。在这个例子中，我们包含了可选类型的注解：

```
fn main() {  
    let tup:(i32,f64,u8) = (600, 7.4, 2);  
    println!("{:?}", tup);  
}
```

因为元组被视为单个复合元素，所以变量 `tup` 绑定到整个元组。为了从元组中提取单个值，我们可以使用模式匹配来解构元组值，如下所示：

```
fn main() {  
    let tup = (600, 7.4, 2);  
    let (_a, b, _c) = tup;  
    println!("The value of b is: {}", b);  
}
```

LQ 注：上面的例子中，由于解构元组值定义的变量 `a` 和 `c` 在后面的语句中未使用，故加上下划线做前缀，可以避免编译时报错“if this is intentional, prefix it with an underscore”

该程序首先创建一个元组并将其分配给变量 `tup`。然后，它采用一种模式将 `tup` 划分为三个不同的变量：`a`、`b` 和 `c`。这被称为解构，因为它将一个元组分成三部分。最后，程序输出 `b` 的值，即 7.4。

除了模式匹配之外，我们还可以使用句点 (.) 后跟要检索的值的索引来直接访问元组元素。考虑以下场景：

```
fn main() {  
    let x: (i32, f64, u8) = (600, 7.4, 1);  
    let a = x.0;  
    let b = x.1;  
    let c = x.2;  
    println!("The values are: {}, {}, {}", a, b, c);  
}
```

该程序生成一个元组 `x`，然后根据每个元素的索引为每个元素创建新变量。元组的初始索引是 0，就像大多数计算机语言编程中的使用习惯那样。

没有值的元组 `()` 是一种只有一个值的特殊类型，它也可以表示为 `Unit`。这种类型称为单位类型（unit type），而值称为单位值（unit value）。如果表达式不返回任何其他值，它会隐式返回单位值。

数组类型

数组（Array）是另一种创建多个值集合的方法。与元组不同，数组中的所有元素必须具有相同的数据类型。Rust 中的数组与其他语言中的数组的不同之处在于它们具有固定长度，类似于元组。

在 Rust 中，数组值通过方括号内的逗号分隔列表进行表示：

```
fn main() {  
    let a = [11, 22, 33, 44, 55];  
    println!("{:?}", a);  
}
```

当我们希望将数据分配在栈（stack）而不是堆（heap）上时，或者我们拥有固定数量的元素时，使用数组是很方便的。

LQ 注：Rust 语言中堆和栈是存储信息的两个不同的内存位置。栈内存空间很有限，栈上的分配和释放非常快，栈是 Rust 默认分配内存的地方，分配仅限于函数调用，存储的数据的大小在编译时是已知的，存储的典型数据是按顺序排列的函数帧：函数参数、返回地址、局部变量（基元和指针），LIFO（后进先出）。堆内存分配的大小无限制，堆上的分配和释放内存较慢，内存由 Rust 程序显式分配给堆，堆上的内存是全局可访问的，存储的典型数据是全局变量和复杂的数据结构，没有特别的顺序。

但是，数组的适应性不如向量（vector）。向量是 Rust 标准库提供的类似集合的数据类型，可以扩展或收缩大小。如果我们不确定是使用数组还是向量，请使用向量。

我们可能想在需要知道一年中月份名的程序中使用数组而不是向量。因为这样的程序不太可能需要增加或减少月份，故可以使用数组，因为我们知道月份数组总是只有 12 个元素：

```
let months = ["Jan", "Feb", "Mar", "April", "May", "June", "July", "Aug",  
             "Sep", "Oct", "Nov", "Dec"];
```

左边的方括号中，先定义数组元素的数据类型，然后定义数组元素的数量；右边的方括号中，是具体的数组元素项。如下所示：

```
let a: [i32; 5] = [11, 22, 33, 44, 55];
```

在这种情况下，每个元素都是 i32 类型。分号后面的数字 5 表示数组有五个元素。

这种编写数组类型的语法似乎类似于初始化数组的语法：如果我们想为每个成员构造一个具有相同值的数组，我们指定初始值，后跟分号，然后是数组的长度在方括号中，如此处所示：

```
let a = [33; 5];
```

名为 a 的数组将包含 5 个元素，并将数组中所有元素的初始值设置为值 33。这与编写 let a = [33, 33, 33, 33, 33] 相同，但更简洁。

访问数组元素

数组是可以在栈内存上分配的单个固定大小的内存块。索引可用于访问数组中的项，如下所示：

```
fn main() {  
    let x = [11, 22, 33, 44, 55];  
    let first = x[0];  
    let second = x[1];  
    println!("first: {}, second: {}", first, second);  
}
```

因为数组中位置 [0] 的值为 11，所以第一个变量将接收 11。变量 `second` 将从数组的索引 [1] 接收值 22。

无效的数组元素访问

如果尝试访问超出数组末尾的数组元素会发生什么？考虑以下示例：

```
use std::io;  
fn main() {  
    let x = [11, 22, 33, 44, 55];  
    println!("Enter array index.");  
    let mut index = String::new();  
    io::stdin()  
        .read_line(&mut index)  
        .expect("Failed to read the line");  
    let index: usize = index.trim().parse().expect("Index entered was not number");  
    let element = x[index];  
    println!("Value of the element at index {} is: {}", index, element);  
}
```

此代码已成功编译。如果我们使用 `cargo run` 运行此代码并输入 0、1、2、3 或 4，程序将输出数组中该索引处的值。如果我们在数组末尾输入一个数字，例如 15，我们会得到一个错误。

当程序在索引过程中使用不正确的值时，程序会生成运行时错误。程序以错误消息终止，最后一个 `println!` 指令未执行。当使用索引访问元素时，Rust 检查我们指定的索引是否小于数组长度。如果索引大于或等于长度，Rust 会恐慌。这种检查必须在运行时进行，尤其是在这种情况下，因为编译器不知道用户在执行代码时会提供什么值。

// 执行程序后输入 12，可以看到如下的错误提示

```
thread 'main' panicked at 'index out of bounds: the len is 5 but the index is 12', r3.rs:10:19  
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```


这是 Rust 的内存安全理念之一。这种类型检查在许多低层语言中没有，如果提供了错误的索引，那么可以访问无效的内存。Rust 通过立即离开而不是继续允许内存访问来保护我们免受此类错误的影响。

常量

常量是不能改变的值。当我们声明一个常量时，它的值不能改变。术语 `const` 表示常数。常量必须显式键入。声明常量的语法如下：

```
const VARIABLE_NAME: dataType = value;
```

Rust 中的常量命名约定

Rust 中，常量的命名约定是所有字符通常都大写。声明常量时，使用 `const` 关键字。

在以下示例中，我们使用了 Rust 中的常量：

```
fn main() {  
    const USER_LIMIT: i32 = 110; // 声明整数常量  
    const PI : f32 = 3.14; // 声明浮点数常量  
    println!("user limit {}", USER_LIMIT);  
    println!("pi value {}", PI);  
}
```

常量 vs. 变量

本节将介绍常量和变量之间的区别。

- `const` 关键字用于声明常量，而 `let` 关键字用于声明变量。
- 变量声明可能包含也可能不包含数据类型，但常量声明必须包含数据类型。因此，“`const USER_LIMIT = 110`”这样的常量定义语句在编译时会报错。
- 默认情况下，使用 `let` 关键字声明的变量是不可变的，但可以使用 `mut` 关键字来使其可变。常数是不可变的。
- 常量只能设置为常量表达式，而不是函数调用的结果或在运行时计算的任何其他值。
- 常量可以在任何范围内声明，包括全局范围，使它们方便地用于需要被代码的多个部分知道的变量。

变量和常量的遮蔽

在 Rust 中，程序员可以指定具有相同名称的变量。在这种情况下，新变量优先于先前的变量。

```
#[warn(unused_variables)]  
fn main() {  
    let salary = 110.00;  
    let salary = 2.50;
```

```
// salary 定义了两次，后者优先
println!("Value of salary: {}", salary);
}
```

上面的代码声明了两个名为 `salary` 的变量。第一个声明值为 110.00，而第二个声明值为 2.50。在显示输出时，第二个变量隐藏第一个变量。

在遮蔽时，Rust 支持各种数据类型的变量。考虑以下场景：

下面的代码使用术语 `uname` 来声明两个变量。第一个声明被赋予一个字符串值，而第二个声明被赋予一个整数值。`len` 函数确定字符串值中有多少个字符。

```
fn main() {
    let uname = "Mohtash";
    let uname = uname.len();
    println!("name changed to the integer: {}", uname);
}
```

与变量不同，常量不能被遮蔽。如果将上述程序中的变量替换为常量，编译器将产生错误。

```
fn main() {
    const NAME:&str = "Mohtash";
    const NAME:usize = NAME.len();
    println!("name changed to the integer: {}", NAME);
    // 编译时报错: `NAME` must be defined only once in the value namespace of this block
}
```

字符串类型

在 Rust 中，字符串数据类型分为以下几类：

- 字符串字面量（`&str`）
- 字符串对象（`String`）

字符串字面量

如果在构建时知道字符串的值，那么应使用字符串字面量。字符串字面量是硬编码到变量中的字符集合。字符串字面量可以在 `std::str` 包中找到。字符串字面量（`String Literal`）也称为字符串切片（`String Slice`）。

字符串字面量的声明示例如下：

```
fn main() {
    let company:&str = "RustPoint";
    let location:&str = "Ludhiana";
    println!("company: {}, location: {}", company, location);
}
```

默认情况下，字符串字面量是静态的。这确保了字符串字面量在整个程序中都是有效的。我们也可以直接将变量指定为静态，如下所示：

```
fn main() {
    let company:&'static str = "RustPoint";
    let location:&'static str = "Ludhiana";
    println!("company: {}, location: {}", company, location);
}
```

字符串对象

String 对象类型在 Rust 标准库中可用。与字符串字面量不同，String 对象类型不是核心语言的一部分。标准库 pub struct String 将其定义为公共结构。字符串是一个可以扩展的集合。它是一种使用 UTF-8 编码的可变类型。String 对象类型可用于表示在运行时发送到程序的字符串值。堆内存用于分配 String 对象。

语法

我们可以使用以下任何语法来构建 String 对象：

String::new()

上述语法生成一个空字符串。

String::from()

这会生成一个字符串，其中包含了作为参数传递给 from() 函数的默认值。

下面的示例展示了如何使用 String 对象。

```
fn main() {
    let empty_string = String::new();
    println!("length {}", empty_string.len());
    let content_string = String::from("RustPoint");
    println!("length {}", content_string.len());
}
```

上面的示例使用 new 方法生成一个空的 String 对象，并使用 from 方法从字符串字面量构建一个 String 对象。

String 对象的常用方法

序号	方法	签名	描述
1	new()	pub const fn new() → String	生成一个新的空字符串对象
2	to_string()	fn to_string(&self) → String	将字符串字面量转换为字符串对象
3	replace()	pub fn replace<'a, P>(&'a self, from: P, to: &str) → String	在字符串中遍历查找，将所有满足模式匹配的内容替换为给定的字符串
4	as_str()	pub fn as_str(&self) → &str	将字符串对象转换为字符串字面量
5	push()	pub fn push(&mut self, ch: char)	向字符串尾部添加指定的字符

序号	方法	签名	描述
6	push_str()	pub fn push_str(&mut self, string:&str)	将指定的字符串切片附加到此字符串的末尾
7	len()	pub fn len(&self) → usize	返回以字节为单位的字符串的长度
8	trim()	pub fn trim(&self) → &str	删除字符串切片前后的空格
9	split_whitespace()	pub fn split_whitespace(&self) → SplitWhitespace	用空格分割字符串后返回一个迭代器
10	split()	pub fn split<'a,C>(&'a self,pat:C) → Split<'a,C>, 其中 C 是模式, 可以是 &str、char 或确定拆分的闭包	用给定的字符或模式分割字符串后返回一个迭代器
11	chars()	pub fn chars(&self) → Chars	将给定的字符串分割为单个字符后返回一个迭代器

new()

new() 函数构造一个空字符串对象。看下面的例子：

```
fn main() {
    let mut a = String::new();
    a.push_str("hello");
    println!("{}", a);
}
```

to_string()

要访问 String 对象的所有方法，可以使用 to_string() 函数先将字符串字面量转换为字符串对象类型。例如：

```
fn main() {
    let names = "Hello RustPoint, Hello!".to_string();
    println!("{}", names);
}
```

replace()

replace() 函数接受两个参数：第一个参数是要搜索的字符串模式，第二个参数是用于替换的新值。在前面的示例中，Hello 在 names 字符串中出现了两次。下面的示例使用 replace() 替换函数将字符串对象中的所有 Hello 字符串替换为字符串 Rowdy。

```
fn main() {
    let names = "Hello RustPoint, Hello!".to_string();
    println!("{}", names);
    let name2 = names.replace("Hello", "Rowdy");
    println!("{}", name2);
}
```

as_str()

`as_str()` 函数返回一个包含整个字符串的字符串字面量。

```
fn main() {
    let example_string = String::from("example_string");
    print_literal(example_string.as_str());
}
fn print_literal(data:&str) {
    println!("displaying the string literal {}", data);
}
```

push()

`push()` 函数将指定的字符添加到字符串的尾部。

```
fn main() {
    let mut company1 = "Rust".to_string();
    company1.push('s');
    println!("{}", company1);
}
```

push_str()

`push_str()` 函数将指定的字符串切片添加到字符串的尾部。

```
fn main() {
    let mut company1 = "Rust".to_string();
    company1.push_str(" Point");
    println!("{}", company1);
}
```

len()

`len()` 函数返回字符串中包含的字符总数（包括空格）。

```
fn main() {
    let fullnames = " Rust Point";
    println!("length {}", fullnames.len());
}
```

trim()

`trim()` 函数用于消除给定字符串首尾的空格以及制表符（比如、）。要注意，它不会删除字符串中非首尾的空格。

```
fn main() {
    let fullnames = " Rust Point \r\n";
    println!("Before-trim ");
    println!("length {}", fullnames.len());
    println!("After-trim ");
}
```

```
println!("length {}", fullnames.trim().len());
}
```

split_whitespace()

split_whitespace() 函数可以将输入的字符串分割成数个字符串。

```
fn main() {
    let msg = "Rust Point has good tutorials".to_string();
    let mut x = 1;
    for token in msg.split_whitespace() {
        println!("token {} {}", x, token);
        x += 1;
    }
}
```

split()

split() 函数返回一个迭代器，该迭代器循环访问由与模式匹配的字符划分的字符串切片的子字符串。split() 函数具有不能存储结果以供以后使用的约束。可以使用 collect() 函数将 split() 的结果另存为向量。

```
fn main() {
    let fullname = "Canan, Rudhasharan, Rustpoint";
    for token in fullname.split(",") {
        println!("token is {}", token);
    }
    // 存储到向量中
    let tokens:Vec<&str> = fullname.split(",").collect();
    println!("firstName {}", tokens[0]);
    println!("lastName {}", tokens[1]);
    println!("company {}", tokens[2]);
}
```

chars()

chars() 函数可以将给定的字符串分割为单个字符后返回一个迭代器。要理解它，看下面的例子：

```
fn main() {
    let nm1 = "Rust".to_string();
    for nm in nm1.chars() {
        println!("{}", nm);
    }
}
```

使用 + 运算符连接字符串

可以将一个字符串的值附加到另一个字符串。这称为连接（concatenation）或插值（interpolation）。字符串连接产生一个新的字符串对象。在内部，+ 运算符采

用 `add()` 函数。`add()` 函数的语法接受两个参数。第一个参数是 `self`，它是字符串对象自身，第二个参数是对第二个字符串对象的引用。看下面的片段描述：

```
// add() 函数
add(self, &str) -> String {
    // returns String object
}
```

字符串连接的例子：

```
fn main() {
    let nm1 = "Rust".to_string();
    let nm2 = "Points".to_string();
    let nm3 = nm1 + &nm2; // n2 reference is passed
    println!("{}", nm3);
}
```

类型转换的例子：

```
fn main() {
    let number = 2022;
    let number_as_string = number.to_string();
    // 把数字转换为字符串
    println!("{}", number_as_string);
    println!("{}", number_as_string=="2022");
}
```

Format! 宏

名为 `format` 的宏函数也可用于将 `String` 对象组合在一起。看下面的例子：

```
fn main() {
    let nm1 = "Rust".to_string();
    let nm2 = "Points".to_string();
    let nm3 = format!("{}", nm1, nm2);
    println!("{}", nm3);
}
```

运算符

运算符（operator）指定了将应用于数据的函数。运算符操作的数据称为操作数（operands）。考虑以下表达式：

$6 + 4 = 10$

在这种情况下，操作数是 6、4 和 10，而运算符是 + 和 =。

Rust 中使用的主要运算符如下：

- 算术

- 按位
- 比较
- 逻辑
- 条件

算术运算符

假设变量 x 和 y 的值分别为 11 和 4。

序号	运算符	描述	示例
1	+	返回操作数的和	$x + y$ 的和为 15
2	-	返回操作数的差	$x - y$ 的差为 7
3	*	返回操作数的乘积	$x * y$ 的乘积是 44
4	/	除法运算	x / y 相除为 2
5	%	求余运算	$x \% y$ 求余为 3

关系运算符

关系运算符用于测试或指定两个项之间存在的关系类型。在比较两个或多个值时，使用关系运算符。关系运算符产生作为布尔值的 **true** 或 **false**。

假设 X 等于 15， Y 等于 25。

序号	运算符	描述	示例
1	>	大于	$(X > Y)$ 为假
2	<	小于	$(X < Y)$ 为真
3	>=	大于或等于	$(X >= Y)$ 为假
4	<=	小于或等于	$(X <= Y)$ 为真
5	==	相等	$(X == Y)$ 为假
6	!=	不等于	$(X != Y)$ 为真

逻辑运算符

逻辑运算符用于将两个或多个条件连接在一起。与关系运算符一样，逻辑运算符产生一个布尔值。假设变量 X 的值为 15，变量 Y 的值为 25。

序号	运算符	描述	示例
1	&&	逻辑与	$(X > 15 \ \&\& \ Y > 15)$ 为假
2		逻辑或	$(X > 15)$
3	!	逻辑非	$!(X > 15)$ 为真

位运算符

假设变量 X 的值为 3（u8 数据类型，00000011），变量 Y 的值为 4（u8 数据类型，00000100）。

序号	运算符	描述	示例
1	&（按位与）	其整数参数的每一位都执行布尔与运算	(X & Y) 为 0
2	（按位或）	其整数参数的每一位都执行布尔或运算	(X Y) 为 7
3	^（按位异或）	其整数参数的每一位都执行布尔异或运算	(X ^ Y) 为 7
4	!（按位非）	它是反转所有操作数位的一元运算符	(!Y) 为 251
5	<<（左移）	将第一个操作数的所有位左移 N 位，N 位由第二个操作数指定，零用于填充新位。将值左移一位相当于将其乘以 2，将其移至两位相当于将其乘以 4，依此类推	(X << 1) 为 6
6	>>（右移）	将第一个操作数的所有位右移 N 位，N 位由第二个操作数指定，零用于填充新位。	(X >> 1) 为 1

LQ 注: Rust 语言不支持 >>> 运算符，原书存在错误。

执行决策

程序员定义一个或多个由程序评估或测试的条件，如果条件判断为真则执行一些语句，并且可选地，如果条件判断为假则运行另一些语句。

1、if 语句 2、if...else 语句 3、else...if 和嵌套 if 语句 4、match 语句

if 语句

在执行一段代码之前，if...else 结构会评估一个条件。

语法：

```
if boolean_expression {  
    // 如果 boolean_expression 布尔表达式为真，执行下面的语句  
}
```

如果布尔表达式返回 true，则执行 if 语句中的代码。如果布尔表达式返回 false，则执行 if 语句结束后（右花括号后）的第一行代码。

```
fn main() {  
    let num:i32 = 9;  
    if num>0 {  
        println!("positive number");  
    }  
}
```

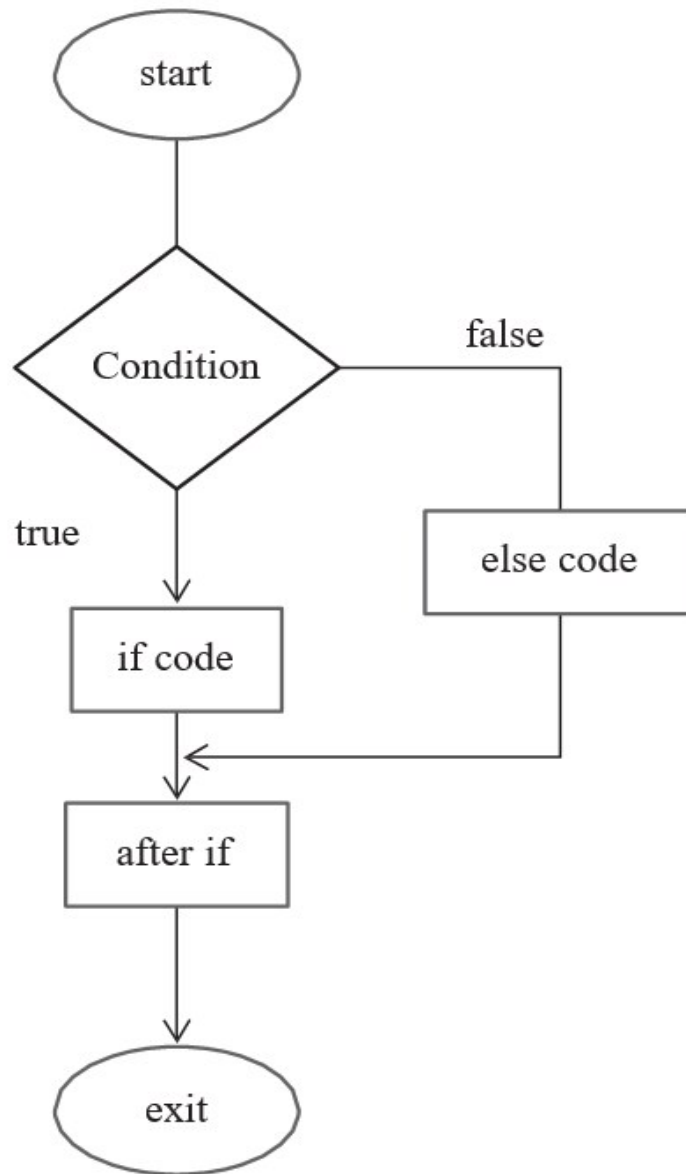
只要 if 块声明的条件为真，上面的例子就会输出“positive number”。

if else 语句

if...else 语句，检查 if 后的布尔表达式，如果返回 false，则将执行 else 的代码块。

语法：

```
if boolean_expression {  
    // 如果 boolean_expression 布尔表达式为真，执行下面的语句  
} else {  
    // 如果 boolean_expression 布尔表达式为假，执行下面的语句  
}
```



if...else 语句的流程图

if 语句紧跟布尔表达式。如果布尔表达式的计算结果为真，则执行与 if 语句相关的块。如果布尔表达式的计算结果为假，则执行与 else 块相关联的代码块。

```
fn main() {  
    let num = 15;  
    if num%2==0 {  
        println!("Evennum");  
    } else {  
        println!("Oddnum");  
    }  
}
```

if 嵌套

else ...if 阶梯适用于存在测试许多情况的场景。语法如下所示：

```
if boolean_expression1 {  
    // 如果 boolean_expression1 布尔表达式为真，执行下面的语句  
}  
else if boolean_expression2 {  
    // 如果 boolean_expression2 布尔表达式为真，执行下面的语句  
}  
else {  
    // 如果 boolean_expression1 和 boolean_expression2 布尔表达式都为假，  
    执行下面的语句  
}  
  
fn main() {  
    let numb = 4;  
    if numb>0 {  
        println!("{}", positive, numb);  
    } else if numb<0 {  
        println!("{}", negative, numb);  
    } else {  
        println!("{}", neither positive nor negative, numb);  
    }  
}
```

match 语句

match 语句检查当前值是否与值列表中的一个匹配；它类似于 C 语言中的 switch 语句。首先，match 关键字后面的表达式不需要括在括号中。语法：

```
let expressionResult=match variable_expression {  
    constant_expr1 => {  
        // 语句;  
    },  
    constant_expr2 => {  
        // 语句;  
    },  
    _=> {  
        // 语句;  
    }  
}
```

```
    }  
};
```

在以下示例中，将 `state_code` 与 MH、KL、KA 和 GA 一组值进行比较；如果找到匹配项，则将字符串值返回给变量 `state`；如果未找到匹配项，则将默认值 `Unknown` 返回给变量 `state`。例子：

```
fn main() {  
    let state_code = "IN";  
    let state = match state_code {  
        "DL" => {println!("Found match!"); "Delhi"},  
        "IN" => "India",  
        "KL" => "Kolkatta",  
        "AB" => "Ahmdabad",  
        _ => "Unknown"  
    };  
    println!("State name: {}", state);  
}
```

在“let”语句中使用“if”语句

“if”表达式用在 `let` 语句的右侧，“if”表达式的值被赋值给“let”语句。语法：

```
let variable_name = if condition {  
    // 代码块 1  
} else {  
    // 代码块 2  
}
```

如果条件为真，则将“if”表达式的值赋给变量；如果条件为假，则将“else”表达式的值分配给变量。例子：

```
fn main() {  
    let x = if true {  
        2  
    } else {  
        4  
    };  
    println!("value of x is: {}", x);  
}
```

例 2:

```
fn main() {  
    let x = if false {  
        10  
    } else {  
        "Rustpoint"  
    };  
};
```

```
println!("value of x is : {}", x);  
}
```

在此示例中，“if”块的计算结果为整数值，而“else”部分的计算结果为字符串值。这样在编译时会产生错误，错误信息为“if and else have incompatible types”，因为两个块包含了不同类型的值。

循环

如果我们想多次运行语句块，我们可以使用循环的思想。循环将循环体内的代码执行到最后，然后从头重新开始。Rust 由三种类型的循环组成：

1. loop 循环
2. for 循环
3. while 循环

loop 循环

它不是条件循环，关键字 loop 告诉 Rust 需要一遍又一遍地执行代码块，直到你明确地停止循环。语法：

```
loop {  
    // 语句块  
}
```

上述语法中的块语句无限运行。

```
fn main() {  
    loop {  
        println!("Hello everyone");  
    }  
}
```

在这个例子中，“Hello everyone”会重复显示，直到我们手动停止循环。在大多数情况下，按下“ctrl+c”命令用于退出循环。

从循环中退出

要退出循环，请使用“break”关键字。如果没有提供“break”关键字，则循环将无限期地运行。例如：

```
fn main() {  
    let mut x = 1;  
    loop {  
        println!("Hello everyone");  
        if x == 7 {  
            break;  
        }  
        x += 1;  
    }  
}
```



```
}  
}
```

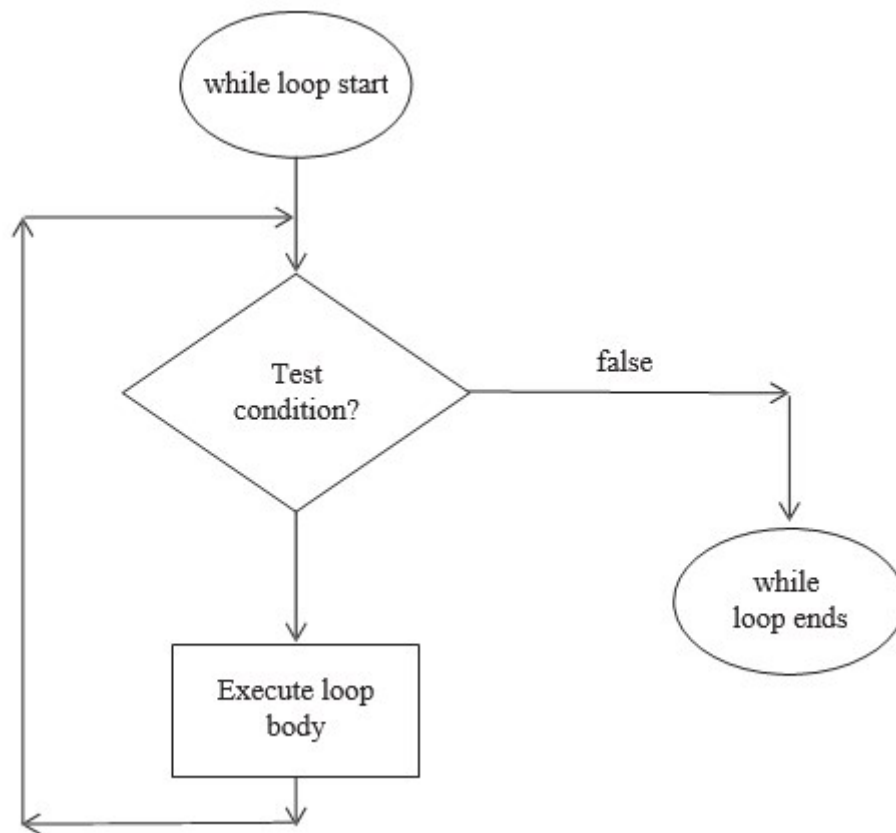
while 循环

“while loop”是一种条件循环。当程序需要评估条件时使用条件循环。如果条件为真，则调用循环；否则，循环终止。

语法：

```
while condition {  
    // 语句块  
}
```

上述语法中的 **while** 循环评估条件。如果条件为真，则执行块语句；否则，循环终止。Rust 中的这个内置结构可以与“loop”、“if”、“else”或“break”声明结合使用。



while loop 流程图

例 1:

```
fn main() {  
    let mut x = 1;  
    while x<=10 {  
        print!("{}", x);  
    }  
}
```

```

        print!(" ");
        x = x+1;
    }
}

```

例 2:

```

fn main() {
    let array = [20,30,35,42,60,90];
    let mut x = 0;
    while x<6 {
        print!("{}", array[x]);
        print!(" ");
        x += 1;
    }
}

```

while 循环的缺点:

- 如果索引长度错误，while 循环可能会导致问题。
- 它也很慢，因为编译器添加了运行时代码来对每个循环迭代执行条件检查。

for 循环

for 循环是一个条件循环，这意味着它会运行特定的时间。rust 语言中 for 循环的行为与其他语言略有不同。for 循环直到满足条件才执行。语法:

```

for var in expression {
    // 语句块
}

```

上述语法中的表达式 **expression** 可以变成一个迭代器，它遍历数据结构的元素。迭代器用于检索每次迭代的值。当没有更多值要获取时，循环结束。

例 1:

```

fn main() {
    for x in 1..12 {
        print!("{}", x);
    }
}

```

例 2:

```

fn main() {
    let mut result;
    for x in 1..12 {
        result = 2*x;
        println!("2*{}={}", x, result);
    }
}

```

```
    }  
}
```

例 3:

```
fn main() {  
    let fruits = ["apple", "orange", "mango", "banana", "watermelon"];  
    for s in fruits.iter() {  
        print!("{}", s);  
    }  
}
```

在上面的示例中，`iter()` 函数用于迭代 `fruits` 变量的每个条目。循环在到达数组的最后一个元素时结束。

while 循环和 for 循环的区别

如果在运行时提高数组的索引长度，则 `while` 循环会出现问题，但在 `for` 循环的情况下不会出现这种情况。因此，我们可以说 `for` 循环增强了代码的安全性并消除了出现问题的可能性。

函数

函数是可理解、可管理和可重用代码的构建块。函数是用于执行指定活动的语句的集合。函数将程序构造成逻辑代码段。函数可以在定义后调用访问代码。因此，代码是可重用的。此外，函数使程序的代码更易于理解和维护。

名称、返回类型和函数参数都在函数声明中指定。函数定义（**function definition**）定义了函数的主体。

1. 定义一个函数 函数的规范解释了给定工作将完成什么以及如何完成。
2. 调用一个函数 要运行一个函数，必须调用它。
3. 返回函数 函数也可以将值和控制权返回给调用者。
4. 参数化的函数 参数是将值发送到函数的一种方式。

函数定义

函数定义描述了给定活动将执行的内容和方式。函数必须先指定才能使用它。函数体包括函数应该运行的代码。命名函数的准则与命名变量的准则相同。`fn` 关键字用于定义函数。以下是定义标准函数的语法：

```
fn function_name(param1, param2..paramN) {  
    // 函数体  
}
```

函数声明可能包含也可能不包含参数。函数使用参数来传递值。

```
// 定义一个函数
fn fn_hello() {
    println!("hello fn_hello ");
}
```

函数调用

要运行一个函数，就必须调用它。这称为函数调用。调用函数时，应该传递参数值。调用另一个函数的函数称为调用者函数（caller function）。

语法：

```
function_name(val1, val2, valN)
```

例子：

```
fn main() {
    // 函数调用
    fn_hello();
}
```

上面的“函数调用”的代码需要与前面的“定义一个函数”的代码结合在一起才能运行。

从函数返回值

函数还可以将值与控制一起返回给调用者。这样的函数称为返回函数（returning function）。

语法：

要定义具有返回类型的函数，请使用以下任何语法。

带 return 语句：

```
// 语法1
fn function_name() -> return_type {
    // 语句
    return value;
}
```

无 return 语句：

```
// 语法2
fn function_name() -> return_type {
    value    // 无分号表示返回该值
}
```

例子：

```
fn main() {
    println!("pi value {}", get_pi());
}
```

```

}
fn get_pi() -> f64 {
    22.0/7.0
}

```

带参数的函数

参数是将值传递到函数的一种方式。函数的签名包括参数。在函数调用期间，参数值提供给函数。除非另有说明，否则提供给函数的值的数量必须等于定义的参数的数量。

以下方法之一可用于向函数传递参数值。

1、值传递

当调用方法时，将为每个值参数生成一个新的存储位置。实际的参数值被转换为它们。因此，在被调用方法中更改参数不会影响参数。

下面的示例声明了一个变量 `no`，它最初设置为 7。该变量作为参数（按值）传递给函数 `mutate_no_to_zero()`，该方法将值转换为零。当函数调用后，控制返回 `main()` 函数时，`no` 变量的值保持不变。

```

fn main() {
    let no:i32 = 7;
    mutate_no_to_zero(no);
    println!("Value of no: {}", no);
}
fn mutate_no_to_zero(mut param_no:i32) {
    param_no = param_no * 0;
    println!("param_no value: {}", param_no);
}

```

2、引用传递

与值参数相反，当我们通过引用传递参数时，不会为这些参数生成新的存储位置。引用参数与方法的实际参数存储在同一内存位置。通过在变量名前面加上 `&`，可以通过引用传递参数值。

在下面的示例中，我们将变量 `no` 初始设置为 7。`mutate_no_to_zero()` 函数把引用传递给变量 `no`。该函数应用于原始变量。当控制在函数调用后返回 `main` 方法时，原始变量的值将为零。

```

fn main() {
    let mut no:i32 = 7;
    mutate_no_to_zero(&mut no);
    println!("Value of no: {}", no);
}
fn mutate_no_to_zero(param_no:&mut i32) {

```

```
    *param_no = 0;
}
```

*运算符用于访问存储在内存地址中的值，作为变量 `param_no`。这有时称为解引用（dereferencing）。

3、字符串传递给函数

`display()` 函数从 `main()` 函数接收 `String` 对象。

```
fn main() {
    let name:String = String::from("RustPoint");
    display(name);
    // display() 后不能再访问 name
    // println!("{}", name);
}
fn display(param_name:String) {
    println!("param_name value: {}", param_name);
}
```

LQ 注：由于变量 `name` 的字符串值传递给函数参数 `param_name`，发生了所有权变化，故如果继续执行 “`println!("{}", name);`” 将会出现编译报错。

注释

每个程序员都努力使他们的代码易于理解，但在某些情况下需要更多的解释。在某些情况下，程序员在其源代码中提供注解（annotations）或注释（comments），编译器会忽略这些注释，但查看代码的个人可能会发现这些注释是有益的。

这是一个基本的注释：

```
// hello, everyone
```

Rust 中的惯用注释样式以两个斜杠开始注释，并一直持续到行尾。对于超出单行范围的注释，请在每行上使用 `//`，如下所示：

```
// doing something complicated here
// multiple lines of comments to do it. Hopefully, the comment will
// explain what is going on.
```

注释也可以添加到代码行的末尾：

```
fn main() {
    let lucky_num = 13; // feeling lucky today
}
```

但是，更常用的编码习惯是，注释放在它所注释的代码上方的单独一行上：

```
fn main() {
    // feeling lucky today
```

```
    let lucky_numb = 13;
}
```

元组

元组是复合数据的一种形式。标量类型只能存储一种类型的数据。例如，一个 `i32` 变量可能只存储一个整数值。在复合类型中，我们可以一次存储多个值，它们可以是各种类型的。

元组有一个设定的长度，一旦声明，它们的大小就不能增加或减少。元组索引从 0 开始。

语法：

```
// 语法1
let tuple_name:(data_type1, data_type2, data_type3) = (value1, value2, value3);
// 语法2
let tuple_name = (value1, value2, value3);
```

例子：

```
fn main() {
    let tuples:(i32,f64,u8) = (-326, 4.8, 23);
    println!("{:?}", tuples);
}
```

下面的示例打印元组中的每个值：

```
fn main() {
    let tuples:(i32,f64,u8) = (-326, 4.8, 23);
    println!("integer: {:?}", tuples.0);
    println!("float: {:?}", tuples.1);
    println!("unsigned integer: {:?}", tuples.2);
}
```

下面的示例使用元组作为参数调用函数。元组按值传递给函数：

```
fn main() {
    let a:(i32,bool,f64) = (120, true, 11.9);
    print(a);
}
fn print(y:(i32,bool,f64)) {
    println!("print method inside");
    println!("{:?}", y);
}
```


解构

解构赋值是 Rust 语言的一个特性，它允许我们解构元组的值。这是通过为每个变量分配元组成员来实现的。

```
fn main() {  
    let a:(i32,bool,f64) = (32, true, 7.8);  
    print(a);  
}  
fn print(y:(i32,bool,f64)) {  
    println!("print method inside");  
    let (age, is_male, cgpa) = y; // 将元组所有成员一一分配给变量  
    println!("Age {}, isMale? {}, cgpa is {}", age, is_male, cgpa);  
}
```

数组

我们将了解数组及其附带的各种功能。在进入数组之前，让我们看一下数组与变量的区别。

变量具有以下约束：

- 变量具有标量性质。换句话说，变量声明一次只能有一个值。这表示在程序中存储 n 个值将需要 n 个变量声明。因此，变量的使用对于存储更大的值集合是不可行的。
- 程序中的变量以随机顺序分配内存，因此无法按其声明顺序检索/读取值。

所有相同的值的集合称为数组。数组是具有相同数据类型的元素的集合。

数组的特性

数组具有以下特征：

- 数组声明按顺序分配内存块。
- 数组不是动态的。这意味着它在初始化数组后无法调整大小。
- 每个内存块对应于单个数组元素。
- 数组元素由称为元素的下标/索引的唯一编号标识。
- 数组初始化是填充数组元素的过程。
- 可以编辑或修改数组元素的值，但不能擦除。

数组声明和初始化

在 Rust 中，使用以下语法来定义和初始化数组。

```
// 语法1  
let array_name = [value1, value2, value3];  
// 语法2  
let array_name:[dataType;size] = [value1, value2, value3];
```

// 语法3

```
let array_name:[dataType;size] = [default_value_for_elements, size];
```

在语法 1 中，数组的类型从初始化期间数组的第一个元素的数据类型推断出来。

示例：简单数组

数组的大小和数据类型在下面的示例中显式指定。println!() 函数的 {:?} 语法用于打印数组的所有元素。数组的大小由 len() 方法确定。

```
fn main() {  
    let arra:[i32;4] = [20, 30, 80, 50];  
    println!("array {:?}", arra);  
    println!("array size: {}", arra.len());  
}
```

示例：不带数据类型的数组

以下示例声明一个四元素的数组。在变量声明期间未显式声明数据类型。在这种情况下，数组的类型将为整数。数组的大小由 len() 方法确定。

```
fn main() {  
    let arra = [20, 30, 80, 50];  
    println!("array {:?}", arra);  
    println!("array size: {}", arra.len());  
}
```

示例：默认值

下面的示例生成一个数组，并为数组的所有元素赋值 -1。

```
fn main() {  
    let arra:[i32;4] = [-1;4];  
    println!("array {:?}", arra);  
    println!("array size: {}", arra.len());  
}
```

示例：带有 for 循环的数组

下面的示例循环访问一个数组，并在传递时打印索引和值。该循环检索从 0 到 4 个索引（最后一个数组元素的索引）的值。

```
fn main() {  
    let arra:[i32;4] = [20,30,80,50];  
    println!("{:?}", arra);  
    println!("array size: {}", arra.len());  
    for index in 0..4 {  
        println!("index is: {}, value: {}", index, arra[index]);  
    }  
}
```

示例：使用 `iter()` 函数

`iter()` 方法返回所有数组项的值。

```
fn main() {
    let arra:[i32;4] = [20,30,80,50];
    println!("{:?}", arra);
    println!("array size: {}", arra.len());
    for vals in arra.iter() {
        println!("value: {}", vals);
    }
}
```

示例：可变数组

可以使用 `mut` 关键字声明可变数组。下面的示例声明一个可变数组，并更改第二个数组元素的值。

```
fn main() {
    let mut arra:[i32;4] = [20,30,80,50];
    arra[1] = 0;
    println!("{:?}", arra);
}
```

将数组作为参数传递给函数

数组可以作为值或引用传递给函数。

示例：按值传递

```
fn main() {
    let arra = [20, 40, 80];
    update(arr);
    print!("Inside-main {:?}", arra);
}
fn update(mut arra:[i32; 3]) {
    for i in 0..3 {
        arra[i] = 0;
    }
    println!("Inside-update {:?}", arra);
}
```

示例：引用传递

```
fn main() {
    let mut arra = [20, 40, 90];
    update(&mut arra);
    print!("Inside-main {:?}", arra);
}
fn update(arr:&mut [i32; 3]) {
    for i in 0..3 {
```

```
        arra[i] = 0;
    }
    println!("Inside-update {:?}", arra);
}
```

数组声明和常量

请考虑以下示例以更好地理解数组声明和常量：

```
fn main() {
    let N:usize = 30;
    let arra = [0;N]; // Error: non-constant used with the constant
    print!("{}", arra[20]);
}
```

编译器将引发异常。由于数组的长度必须在构建时已知，因此就是这种情况。在上面代码的情况下，变量“N”的值将在运行时决定。换句话说，变量不能用于指定数组的大小。

可以稍作修改，使程序有效：

```
fn main() {
    const N:usize = 30;
    let arra = [0;N];
    print!("{}", arra[20]);
}
```

以 `const` 为前缀的标识符的值是在构建（build）时确定的，在运行时无法更改。而 `usize` 是指针的大小；它的实际大小由本地主机环境的体系结构指定。

本章介绍了变量和可变性、数据类型和函数。此外，我们还通过适当的语法和示例学习了注释、元组、数组和控制流。

C3: 理解所有权

- 2022.10.4

在本章中

- 什么是所有权？
- 引用和借用
- 切片类型

上一章介绍了变量、函数和数据类型。此外，我们还讨论了注释和控制语句。本章将覆盖所有权、引用、借用和切片类型。

所有权到底是什么？

Rust 语言的一个主要特性是所有权（ownership）。虽然这个特征描述起来很简单，但它对语言的其他部分有着深刻的影响。

所有程序都必须管理在计算机上运行时如何使用内存。有些语言提供了垃圾收集，在程序运行时搜索不再使用的内存；还有一些语言，程序员必须主动分配和删除内存。**Rust** 则采用了第三种方法：使用所有权系统来控制内存，该系统具有编译器在编译时验证的规则。当我们的软件在运行时，不会有任何所有权的東西使程序变慢。

我们在本章中讨论所有权、引用和借用是什么，还讨论切片类型。好消息是，随着我们掌握 **Rust** 和所有权系统规则的专业知识，我们将能够直观地编写安全高效的代码。

当我们掌握所有权时，我们将有一个坚实的基础来理解区分 **Rust** 的特征。本章将通过几个例子来学习所有权，这些例子侧重于一种非常流行的数据结构：字符串。

栈和堆

许多编程语言不需要我们太多地考虑栈（stack）或堆（heap）。然而，在像 **Rust** 这样的系统编程语言中，项（item）是在栈上还是在堆上，对语言的行为方式以及为什么必须做出某些决定有较大的影响。与栈和堆相关的所有权内容将在本章稍后讨论，这里有一个快速入门。

栈和堆是我们的程序在运行时可能使用的两种可选内存结构。栈按接收顺序保存项，并按相反顺序删除值。这被称为后进先出（LIFO）。考虑一堆盘子：当我们添加额外的盘子时，我们把它们堆在上面，当我们需要一个盘子时，从上面取下一个。从中间或底部添加或删除盘子是无效的。添加数据是指推送到栈上，而删除数据是指从栈中弹出。

放置到栈上的所有数据必须是已知的且是确定的数量。具有不确定数量的数据或在编译时可能变化的数据就必须放在堆中。当我们将数据放在堆上时，需要特定大小的空间。内存分配器在堆中选择足够大的空白空间，将其标记为正在使用，并返回对该位置地址的引用。这称为在堆上分配（**allocating on the heap**），通常简称分配（**allocating**）。分配不包括将值推送到栈中。

因为指针具有已知的常量大小，所以它可以存储到栈上，但我们必须跟随指针来访问实际的数据。

考虑坐在餐厅里。当我们进去时，我们告诉店员我们公司有多少人，店员找到一张可以容纳每个人的空桌子，并引导我们去那里。如果我们有一位成员迟到，他可以咨询店员我们在哪里就座，以便找到我们。

推送到栈（**pushing to the stack**）比堆分配（**heap allocating**）更快，因为在栈中分配器（**allocator**）永远不必寻找新的区域来存储新数据，栈的顶部总是可用的。

另一方面，分配堆空间需要付出更多努力，因为分配器必须首先找到足够大的空间来容纳数据，然后进行记帐，为下一次分配做好准备。

访问堆中的数据比访问栈中的数据慢，因为我们必须跟随指针才能到达那里。现代 CPU 在内存较少的情况下移动速度更快。想象一下，一个餐厅的女服务员从多张桌子上点餐，继续这个类比。最好先完成一张桌子上的所有订单，然后再下一张。

从餐桌 A 中拿到一个订单，然后从餐桌 B 中拿到另一个，再从 A 中拿到另一个，然后又从 B 中拿到另外一个，这将是一个非常缓慢的过程。类似地，如果处理器处理的数据靠近其他数据（如在栈上），而不是远离数据（如可以在堆上），那么处理器的性能会更好。分配大量堆空间也可能需要一些时间。

当代码调用某个函数时，函数中给定的值（可能包括基于堆的指针）和函数的局部变量被放入栈中。函数完成时，值将从栈中移除。

所有权解决了一些困难，例如跟踪代码的哪些部分使用了堆上的哪些数据，最小化堆上的重复数据量，清除堆上不必要的数据，这样就不会耗尽空间。理解所有权之后，我们就不需要考虑栈或堆了，但理解管理堆数据是所有权存在的原因可能有助于解释它为什么会这样工作。

重要的所有权概念

- “所有者（**owner**）”可以根据变量的可变性修改其所有权值。
- 变量的所有权可以转移给另一个变量。
- 在 Rust 中，所有权只是一个语义问题。
- 此外，所有权概念确保了安全。

所有权规则

- 在 Rust 中，每一个值都有一个称为其所有者（**owner**）的变量。
- 在任何时候，只能有一个所有者。

- 当所有者退出范围时，值被销毁。

变量的范围

我们之前看过一个 Rust 程序的示例。既然我们已经学习了基本语法，后面的示例中就不会再包含 `fn main()` 这样的代码。因此，如果我们继续下去，我们必须手动将以下示例的代码放入 `main()` 函数中。因此，示例代码会更简短，使我们能够专注于重要方面，而不是样板代码。

我们将把几个变量的范围作为所有权的第一个示例。范围（scope）是指在程序中一个项的有效范围。假设我们有一个类似这样的变量：

```
let st = "hello";
```

变量 `st` 是指一个文本字符串，其值被硬编码到程序的文本中。变量从声明时起一直有效，直到当前范围过期。清单中包含指示变量 `st` 何时有效的注释。

```
{                                // st 在这里还无效，因为它还没有声明
    let st = "hello";           // 从这里开始，st 有效了
    // 执行 st 相关的一些操作
}                                // 范围到这里终止，st 不再有效
```

换句话说，这里有两个关键时间点：

1. 当 `s` 进入范围时有效。
2. 在超出范围之前仍然有效。

范围和变量有效时间之间的关系与其他编程语言的关系类似。现在，我们将通过引入 `String` 类型来加深对这一点的理解。

String 类型

为了演示所有权规则，我们需要一个比“Rust 数据类型”章节中所教的数据类型更复杂的数据类型。前面提到的所有类型都有一个已知的大小，可以存储在栈中，并在其作用域完成时弹出，如果其他代码需要在不同的作用域中使用相同的值，可以快速简单地克隆以创建新的独立实例。但我们想看看在堆中保存的数据，看看 Rust 是如何理解何时清除这些数据的。

我们将以 `String` 为例，重点介绍 `String` 中与所有权相关的部分。这些考虑也适用于其他复杂的数据类型，无论是标准库提供的还是我们开发的。

我们已经遇到了字符串字面量，它是将字符串值硬编码到软件中的实例。字符串字面量很有用，但并不适用于我们使用文本的所有情况。一个原因是它们是不可改变的。另一个问题是，在创建代码时，并非每个字符串值都是已知的：如果我们想收集用户的输入并存储它，该怎么办？`String` 是 Rust 中的第二种字符串类型，在这些场景中就很有用。这种类型维护堆数据，因此可能包含大量在构建时我们不知道的文本。使用 `from` 函数，我们可以从字符串字面量生成 `String`，如下所示：


```
let st = String::from("hello");
```

双冒号 (::) 是一个运算符，它允许我们使用指定命名空间的函数，而不是使用类似于 `string` 名称的字符串。

此类型的字符串可以更改：

```
let mut st = String::from("hello");
st.push_str(", everyone");    // push_str() 方法将给定的字面量添加到字符串
                              尾部
println!("{}", st);         // 这将输出 hello, everyone
```

那么，这里的区别是什么？为什么 `String` 可以更改而不能字符串字面量不能更改？区别在于这两种类型处理内存的机制不同。

内存和分配

因为我们在编译时知道字符串字面量的内容，所以文本直接硬编码到最终的可执行文件中。正因为如此，字符串字面量快速且高效。然而，这些特性完全源于字符串字面量的不变性。不幸的是，对于在构建时大小未知并且在应用程序执行时可能会变化的文本块，我们不能在二进制文件中包含一个内存块。

要启用 `String` 这种文本可更改、可增长的类型，我们必须在堆上分配编译时未知的内存量来包含内容。这意味着承认：

- 在运行时，内存必须请求内存分配器。
- 当我们处理完 `String` 后，我们需要将这个内存返回给分配器的方法。

我们处理第一部分：当我们调用 `String::from` 时，它的实现请求它的内存。这在编程语言中很常见。

另一方面，第二个组件是唯一的。在带有垃圾收集器（GC）的语言中，GC 会跟踪并清理不再使用的内存，因此我们不必为此烦恼。如果没有 GC，我们就必须识别内存何时不再被使用，并调用代码显式返回它，就像我们请求它一样。从历史上看，做好这项工作一直是一项具有挑战性的编程任务。如果我们忘记了，就会浪费内存。如果执行得太快，我们会得到一个无效变量。如果我们做两次，这也是一个错误。我们必须精确匹配每一个分配与每一个释放。

`Rust` 采用了一种不同的方法：当拥有内存的变量退出范围时，会立即返回内存。下面是范围示例的一个变体，它使用 `String` 而不是字符串字面量：

```
{                                // st 在这里还无效，因为它还没有
声明                             声明
    let st = String::from("hello"); // 从这里开始，st 有效了
    // 执行 st 相关的一些操作
}                                // 范围到这里终止，st 不再有效
```


当 `st` 超出范围时，我们可能会自然地将 `String` 所需的内存返回给分配器。当变量不在范围内时，`Rust` 会为我们调用一个特定的函数。`Drop` 是此方法的名称，也是 `String` 的作者编写代码以返回内存的地方。`Rust` 调用在最后一个花括号处自动丢弃。

这种模式对 `Rust` 代码的编写方式有很大影响。现在看起来似乎很容易，但在更复杂的情况下，如果我们希望多个变量使用堆上的数据，代码行为可能会出乎意料。现在让我们看看其中的一些场景。

变量和数据交互的方式：移动（Move）

在 `Rust` 中，几个变量可以以不同的方式与相同的数据交互。下面我们将查看一个带有整数的示例。

```
let a = 8;
let b = a;
```

“将值 8 绑定到 `a`；然后复制 `a` 中的值并将其绑定到 `b`”，我们可能会这样估计。我们现在有两个变量：`a` 和 `b`，都等于 8。这是正确的，因为整数是具有已知定义大小的简单值，这两个 8 值被放入栈中。

让我们看看 `String` 的版本：

```
let st1 = String::from("hello");
let st2 = st1;
```

这段代码似乎与前面的代码非常相似，因此我们可以断定函数是相同的：第二行将复制 `st1` 中的值并将其绑定到 `st2`。然而，情况并非如此。

`length` 指定了 `String` 的内容当前占用多少内存数量（字节）。`capacity` 是分配器给 `String` 的内存总量（以字节为单位）。了解 `length` 和 `capacity` 之间的区别很重要，但在这里的这种情况下并不重要，因此暂时忽略容量（图 3.1）。

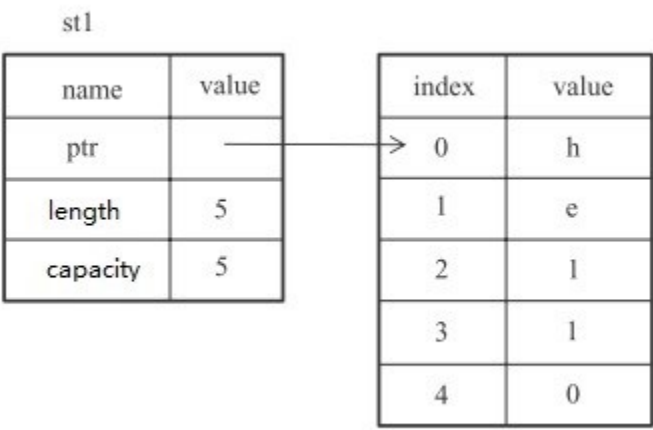


图3.1 值“hello”链接到 `st1` 字符串的内存表示形式

当我们将 `st1` 分配给 `st2` 时，String 数据是重复的，这意味着 Rust 会复制栈的指针（`pointer`）、长度（`length`）和容量（`capacity`）。Rust 不会复制指针指向的堆上的数据。换句话说，就是不会复制内存中的数据表示（图 3.2）。

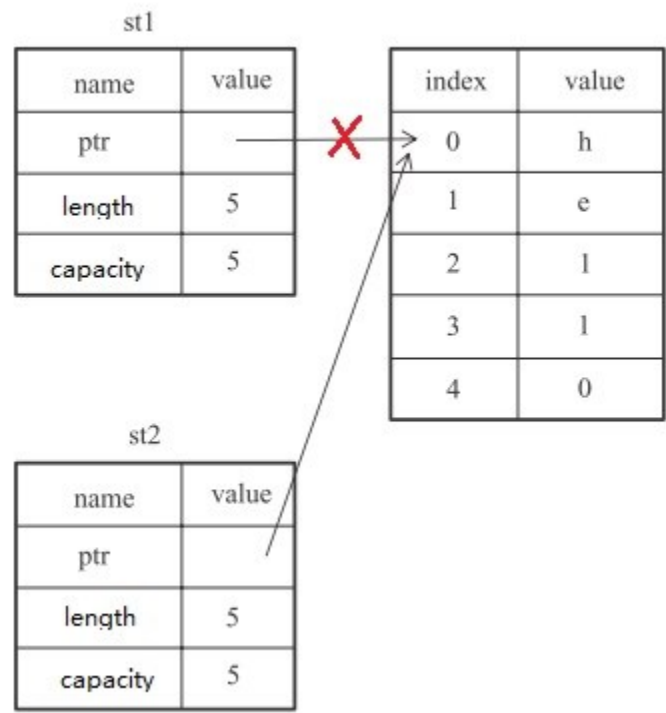


图 3.2 变量 `st2` 的内存表示形式，其中包含 `st1` 的指针、长度和容量的副本

LQ: 打红叉表示 `st1` 在栈内存中已执行 `pop` 弹出操作，指针不存在了。

该表示与下图（图 3.3）不同，它描述了如果 Rust 复制堆数据，内存会是什么样子。如果 Rust 实现了这个，且堆上的数据很大，那么 `st2 = st1` 事务可能会在运行时性能方面付出比较大的开销。

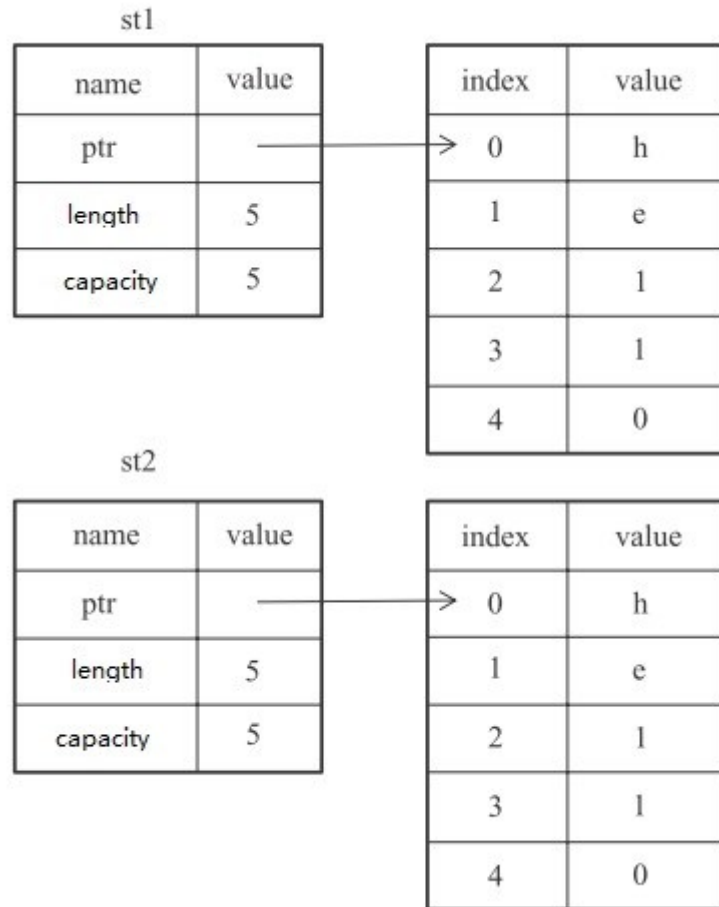


图 3.3 如果 Rust 复制了堆数据，`st2 = st1` 可以做的另一个选择

当变量退出作用域并清除该变量的堆内存时，Rust 会自动执行 `drop` 函数。但是，在图 3.3 中，两个数据指针都指向同一个位置。这是一个问题，因为当 `st2` 和 `st1` 退出范围时，它们将尝试释放相同的内存。这被称为 *双重释放错误* (*double free mistake*)，它是前面描述的内存安全缺陷之一。两次释放内存可能会导致内存损坏，从而导致安全漏洞。

LQ 注：其实不是内存损坏，而是影响堆内存的管理。假设两个指针指向同一块堆内存，一个指针先释放了这个内存块，堆内存管理会回收这块空间，有可能还会进行内存碎片处理，合并相邻的多个小块的空间。这样第二个指针还没释放时，它指向的数据已经改变了，这显然导致错误。

Rust 的这种场景下还有一个额外的机制来确保内存安全。当 Rust 执行 `st2 = st1` 后，就认为 `st1` 无效。因此结果是 `st1` 退出了作用域，Rust 不需要再释放任何东西。如果我们在生成 `st2` 后检查使用 `st1` 会发生什么，它不起作用：

```
fn main() {  
    let st1 = String::from("hello");  
    let st2 = st1;  
    println!("{}", everyone, st2);  
}
```

```

    println!("{}", everyone, st1);
}

```

由于 Rust 限制我们使用无效的引用，我们会得到如下错误：

```

$ rustc test.rs
error[E0382]: borrow of moved value: `st1`
--> test.rs:5:30
   |
2 |     let st1 = String::from("hello");
   |     --- move occurs because `st1` has type `String`, which does
   | not implement the `Copy` trait
3 |     let st2 = st1;
   |               --- value moved here
4 |     println!("{}", everyone, st2);
5 |     println!("{}", everyone, st1);
   |                               ^^^ value borrowed here after move

= note: this error originates in the macro `$crate::format_args_nl` which comes from the expansion of the macro `println` (in Nightly builds, run with -Z macro-backtrace for more info)

```

如果我们在处理其他语言时听说过浅复制（*shallow copy*）和深度复制（*deep copy*）这两个术语，那么复制指针、长度和容量听起来就像一个浅复制而不复制数据。但是，由于 Rust 使第一个变量无效，因此它被称为移动（*move*）而不是浅复制。在本例中，我们将声明 `st1` 已重新定位到 `st2`。图 3.4 描述了发生的情况。

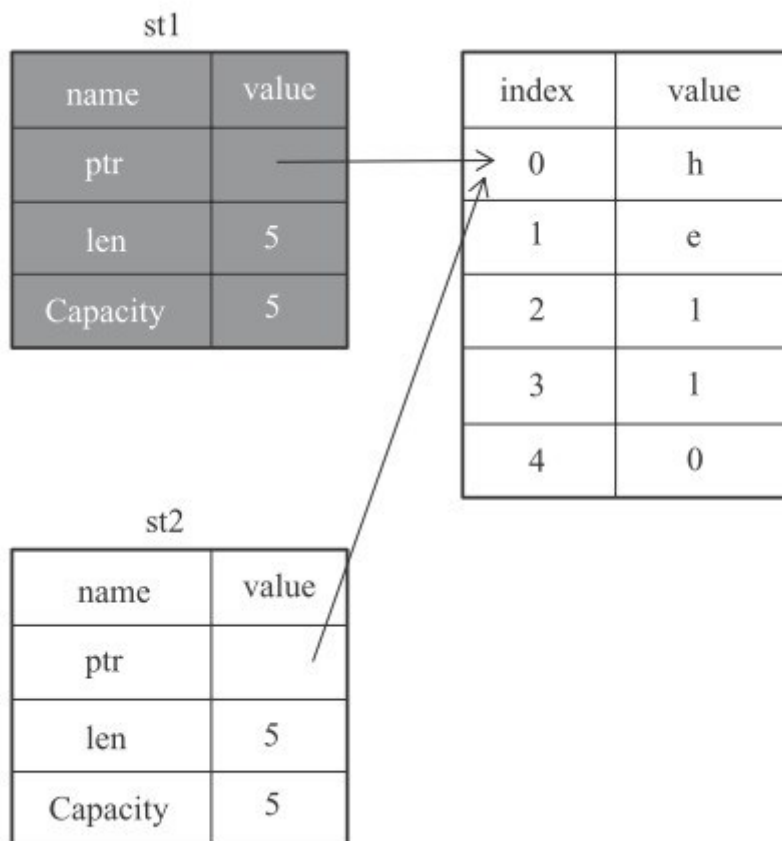


图3.4 `s1` 失效后的内存表示

LQ: 图中的 `st1` 为深色，表示已退出作用域。

这解决了我们的问题！只有 `st2` 有效，当它退出作用域时，它将自行释放内存，我们完成了。

此外，这意味着一个设计选择：**Rust** 永远不会自动构建我们数据的“深度”副本。因此，**Rust** 的任何自动复制都可以被认为在运行时性能方面的开销很小。

变量和数据的交互：克隆

我们可以使用 `clone` 方法来彻底复制 `String` 的堆数据，而不仅仅是栈数据。毫无疑问，我们以前遇到过它们，因为它们在许多编程语言中都很常见。

下面是如何使用 `clone()` 方法的示例：

```
fn main() {  
    let st1 = String::from("hello");  
    let st2 = st1.clone();  
    println!("st1={}, st2={}", st1, st2);  
}
```

上面的代码工作得很好，并生成了如前面图 3.3 中描述的行为，其中堆数据被显式复制。

当我们观察 `clone()` 调用时，我们知道正在执行一些任意代码，并且这些代码可能开销较大。这是一种显式的暗示，表明正在发生一些不寻常的事情。

仅栈数据：复制

还有一个我们没有解决的皱纹。下面的代码使用整数，其中的一部分如图 3.4 所示：

```
let a = 8;
let b = a;
println!("a={}, b={}", a, b);
```

然而，这段代码似乎与我们刚刚学到的相矛盾：没有调用 `clone()`，但 `a` 仍然有效并且没有转移到 `b` 中。

这是因为在构建时具有已知大小的类型（如整数）完全存储在栈中；因此，可以快速生成实际值的副本。在我们创建了变量 `b` 之后，没有理由阻止 `a` 有效。在这种情况下，深度复制和浅复制没有区别。因此，调用 `clone()` 只会执行浅拷贝，这样我们可以忽略它。

Rust 有一个特殊的注释，称为 `Copy` 特征（`trait`），我们可以将其应用于存储在栈中的整数等类型。如果类型具有 `Copy` 特征，那么在赋值后仍可以使用较旧的变量。如果类型或其任何组件实现了 `Drop` 特征，Rust 将不允许我们使用 `Copy` 来注释类型。如果我们将 `Copy` 注释添加到需要在值超出范围时发生任何特定事情的类型，我们将收到编译时错误。

所有权和函数

将值传递给函数的语义类似于给变量赋值。将变量传递给函数将导致它移动或复制，就像赋值一样。以下是一个带有注释的示例，指示变量进入和退出范围的位置：

```
fn main() {
    let st = String::from("hello");    // st 进入作用域
    takes_ownership(st);               // st 的值移动到函数
    // 这里 st 不再有效
    let a = 5;                         // a 进入作用域
    makes_copy(a);                     // a 的值移动到函数
    // 但由于 i32 具备 Copy 特征，因此 a 仍然有效
} // 这里 a 超出作用域，随后是 st，但由于 st 的值之前发生了移动，故这里对 st 而言，什么都不会发生

fn takes_ownership(some_string: String) {    // some_string 进入作用域
    println!("{}", some_string);
} // 这里，some_string 超出作用域，因此会调用 `drop`，其背后的堆内存被释放

fn makes_copy(some_integer: i32) {          // some_integer 进入作用域
```

```
println!("{}", some_integer);
} // 这里, some_integer 超出作用域, 但什么都不会发生
```

返回值和作用域

所有权还可以通过返回值来转移。下面的代码显示了一个示例，其注释与以下示例中的注释相同：

```
fn main() {
    let st1 = gives_ownership(); // give_ownership 将其返回值移动到 st1
    let st2 = String::from("hello"); // st2 进入作用域
    // st2 移动到 takes_and_gives_back(), 且 takes_and_gives_back 也将它
    // 的返回值移动到 st3
    let st3 = takes_and_gives_back(st2);
} // 这里, st3 超出作用域并被丢弃。由于之前 st2 已被移动, 所以这里什么都不会
// 发生
// st1 超出作用域并被删除

fn gives_ownership() -> String { // give_ownership 将它的返回值移动到
调用它的表达式中
    let some_string = String::from("yours"); // some_string 进入作用域
    some_string // some_string 被返回并移出调用函数
}
// 此函数接受 String 并返回一个 a_string
fn takes_and_gives_back(a_string: String) -> String { // a_string 进入
作用域
    a_string // a_string 被返回, 并移出调用函数
}
```

每次，变量的所有权都遵循相同的模式：给另一个变量赋值会改变它。当包含堆数据的变量退出作用域时，该值将被销毁，除非数据已被转移到另一个变量保存。

获取所有权，然后恢复每个函数的所有权的操作是非常耗时的。如果我们想让一个函数使用一个值但不拥有它，该怎么办？之所以不方便，是因为无论我们提交了什么，除了我们可能想要返回的来自函数体的任何数据之外，如果我们想再次使用它，还必须返回它。

元组（tuple）可以用来返回多个值，如下面的代码所示：

```
fn main() {
    let st1 = String::from("hello");
    let (st2, len) = calculate_length(st1);
    println!("length of {} is {}", st2, len);
}
fn calculate_length(st: String) -> (String, usize) {
    let length = st.len(); // len() 方法返回字符串的长度
    (st, length)
}
```

Rust 的引用和借用

引用（reference）是一个作为参数传递给函数的地址。借用（borrowing）类似于我们借了某物，用完后再归还。借用和引用是互斥的，这意味着当一个引用被释放时，借用也就结束了。

为什么需要借用

借用概念的使用有以下原因：

- 借用允许对单个资源的多次引用，同时需要相应的“单个所有者（single owner）”。
- 在 C 语言中，引用类似于指针。
- 引用是对象的一种类型。有两种引用：可变引用（changeable reference）和不可变引用（immutable reference）。不可变引用被复制，而可变引用被重新定位。

例如：

```
fn main() {  
    let st1 = String::from("Rustpoint");  
    let len1 = calculate_length(&st1);  
    println!("length of string {}", len1);  
}  
fn calculate_length(st: &String) -> usize {  
    st.len()  
}
```

在上面的例子中，calculate_length() 函数将字符串 st1 作为参数引用，但不拥有它的所有权。

```
let st1 = String::from("Rustpoint");  
let len1 = calculate_length(&st1);
```

在上面的例子中，&st1 引用变量 st1，但它不拥有它。因此，即使引用超出作用域，它引用的值也不会丢失。

```
fn calculate_length(st: &String) -> usize {  
    st.len()  
}
```

在上面的例子中，变量“st”在控制返回到 main() 方法之前都是有效的。当变量作为引用而不是实际值传递给函数时，我们不需要返回值来重新获得所有权。

让我们看看是否可以改变借来的值：

```
fn main() {  
    let a = 3;  
    value_changed(&a);  
}
```



```
fn value_changed(b:&i32) {
    *b = 8;
}
```

LQ 注：编译会报错。b is a & reference, so the data it refers to cannot be written

可变引用

我们可以通过使用可变引用来纠正上面的错误。可变引用是指那些可以更改的引用。例子：

```
fn main() {
    let mut a = 3;
    value_changed(&mut a);
    println!("After modifying, the value of a is {}", a);
}
fn value_changed(b:&mut i32) {
    *b = 8;
}
```

在上面的例子中，我们构造了一个可变引用&mut a，该引用由 &i32 变量 b 引用。现在我们可以调整由 'b' 变量引用的值。我们分配数字 8(即，*b=8)。结果，a 的值也变成了 8，因为两个变量都指向相同的内存地址。

可变引用的限制

在给定的作用域中，一个数据段只能有一个可更改的引用。

例子：

```
let mut str1 = String::from("hello");
let x = &mut str1;
let y = &mut str1;
```

在上述情况下，编译器会抛出一个错误，因为它包含两个可变引用，这在 Rust 语言中是不允许的。

如果程序中有不可变引用，则不能有可变引用。

例如：

```
let mut str1 = String::from("hello");
let x = &str1;
let y = &str1;
let c = &mut str1;
```

在上述情况下，编译器会抛出一个错误，因为不能同时拥有可变引用和不可变引用。

LQ 注：错误如下error[E0502]: cannot borrow str1 as mutable because it is also borrowed as immutable

悬空引用

在基于指针的语言中，可以通过释放一些内存，但同时保留指向该区域的指针，来构造一个悬空指针（`dangling pointer`），它指向内存中可能已经另作他用的位置。相反，Rust 的编译器可以确保引用不会悬空：如果我们对某些数据有引用，编译器将确保数据不会在对数据的引用之前超出作用域。

让我们尝试创建一个悬空引用，Rust 会拒绝这个引用并产生编译时错误：

```
fn main() {
    let reference_to_nothing = dangle();
}
fn dangle() -> &String {
    let st = String::from("hello");
    &st
}
```

Rust 编译上面的代码，会看到如下错误：

```
error[E0106]: missing lifetime specifier
--> test.rs:4:16
|
4 | fn dangle() -> &String {
|               ^ expected named lifetime parameter
|
= help: this function's return type contains a borrowed value, but there is no value for it to be borrowed from
help: consider using the `'static` lifetime
4 | fn dangle() -> &'static String {
|               +++++++
```

error: aborting due to previous error

这个错误消息暗示了一个我们还没有讨论的主题：生命期（`lifetime`）。然而，如果我们忽略与生命期相关的代码，则该消息包含了为什么这段代码有问题的关键：“这个函数的返回类型包含了借用值，但没有可供借用的值。”

让我们看看悬空代码的每一层都发生了什么：

```
fn dangle() -> &String { // dangle 函数返回对 String 的引用
    let st = String::from("hello"); // 变量 st 是一个新的字符串
    &st // 这里返回字符串对象 st 的引用
} // 这里 st 超出作用域并被删除，它的堆内存空间被丢弃了
```

因为 `st` 是在 `dangle()` 函数的内部生成的，所以在 `dangle()` 的代码执行完成后，`st` 将被释放。但是，我们试图返回对它的引用。因此，该引用将会链接到一个错误的 `String`。这是不可接受的；Rust 不允许我们这么做。

这里的方法是只返回 `String`:

```
fn no_dangle() -> String {  
    let st = String::from("hello");  
    st  
}
```

上面这段代码工作起来就没有任何问题。因为没有任何东西被释放，所有权也已经转移。

引用的指南

让我们从‘引用’这个术语的角度来回顾一下我们讲过的所有内容:

- 我们可以在任何给定时刻拥有一个可变引用或任意数量的不可变引用。
- 引用必须始终正确。
- 接下来，我将介绍一种新的引用类型：切片。

Rust 的切片

切片（slice）是一个指向内存块的指针。切片可以访问存储在小块的连续内存块中的数据。它适用于数组、向量和字符串以及其他数据结构。索引号用于检索数据片。在运行时，计算片的大小。

切片是对底层数据的引用。它们通过引用传递给函数，也称为借用（borrowing）。

例如，可以使用切片来获取字符串值的一部分。指向真正字符串对象的指针是一个切片字符串。因此，必须指定字符串的开始索引和结束索引。索引和数组一样，从 0 开始。

语法:

```
let sliced_value = &data_structure[start-index..end-index]
```

这里有一个小小的编程挑战：创建一个函数，接受一个字符串并返回在其中找到的第一个单词。如果函数在字符串中没有发现空格，则整个字符串必须是一个单词；因此，它应该完整地返回。

考虑以下函数的签名:

```
fn first_word(st: &String) -> ?
```

`first_word` 方法接受 `&String` 参数。这没问题，因为我们不想要所有权。另一方面，我们应该返回什么呢？我们没有很好的方法与字符串片段交互。但是，我们可以返回单词末尾的索引。

```
fn first_word(st: &String) -> usize {  
    let bytes = st.as_bytes();  
    for(x, &item) in bytes.iter().enumerate() {
```

```

        if item==b' ' {
            return x;
        }
    }
    st.len()
}

```

这里我使用 `as_bytes` 方法将 `String` 转换为字节数组，因为我们需要逐个元素遍历它，以确定值是否为空格：

```
let bytes = st.as_bytes();
```

然后使用 `iter` 方法生成一个跨字节数组的迭代器：

```
for(x, &item) in bytes.iter().enumerate() {
```

`iter` 方法返回集合中的每个元素，而 `enumerate` 方法则包装 `iter` 的结果并将每个元素作为元组返回。索引是 `enumerate` 返回的元组的第一个元素，后面是对该元素的引用。这比单独计算索引要方便一些。

我们可以使用模式来分解 `enumerate` 方法返回的元组，因为它返回一个元组。在接下来的章节中，我们将更深入地讨论模式。因此，我们指定了一个模式，在 `for` 循环中使用 `x` 作为元组的索引，并使用 `&item` 作为元组的单字节。我们在模式中使用 `&` 是因为 `.iter().enumerate()` 返回对元素的引用。

使用字面值字节语法，我们在 `for` 循环内搜索表示空格的字节。如果找到空格，则返回该位置。否则，我们使用 `st.len()` 来获取字符串的长度：

```

        if item==b' ' {
            return x;
        }
    }
    st.len()
}

```

我们现在知道了如何查找字符串末尾的第一个单词的索引，但有一个问题。我们返回一个 `usize` 本身，但它只有在与 `&String` 组合时才有用。换句话说，因为它不是 `String` 的一部分，所以不能保证它将来仍然有效。看一下使用 `first_word` 函数的程序。

```

fn main() {
    let mut st = String::from("hello everyone");
    let word = first_word(&st); // 得到值 5
    println!("{}", word);
    println!("{}", &st[0..word]);
    st.clear(); // 这将清空字符串对象 st，使其等于 ""
}

```

担心每个数据中的索引与 `st` 中的数据不同步是令人讨厌和容易出错的。如果我们构建 `second_word` 函数，管理这些索引将变得相当困难。其签字必须如下：

```
fn second_word(st: &String) -> (usize, usize) {
```

我们现在有一个开始索引和结束索引，甚至还有更多从特定状态的数据中派生出来的值，这些值与该状态无关。我们现在有三个不相关的变量必须保持同步。

幸运的是，Rust 为我们提供了一个解决方案：字符串切片。

字符串切片

字符串切片是对字符串部分的引用，它看起来像这样：

```
let st = String::from("hello world");  
let hello = &st[0..5];  
let world = &st[6..11];
```

这类似于使用 [0..5] 位来获取对整个字符串的引用。它只是引用一个子集，而不是引用完整的 String。

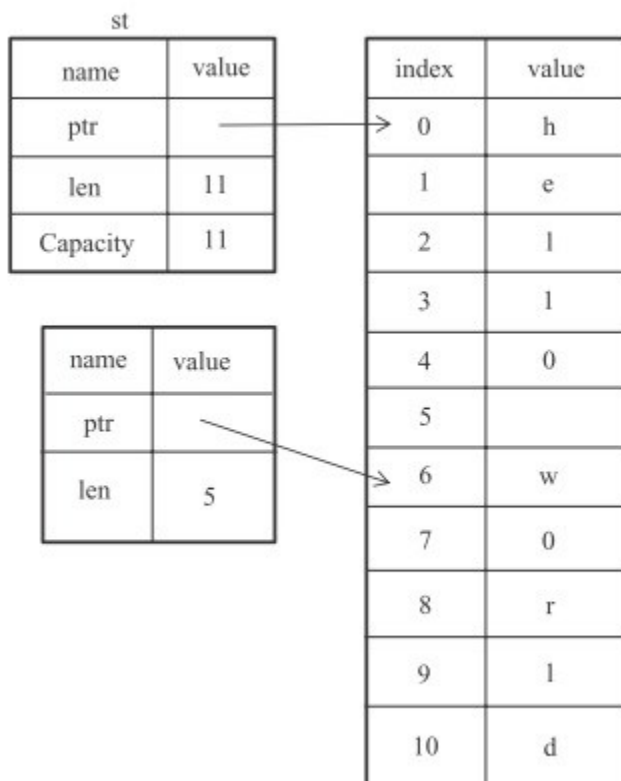


图3.5 术语“字符串切片”指的是字符串部分

声明 [starting_index..ending_index]，其中 starting_index 是切片中的第一个位置，ending_index 是切片中的最后一个位置多一个，我们可以使用括号内的范围生成切片。切片数据结构内部包含片的起始位置和长度，对应于 ending_index 减去

starting_index。例如，“let world = &st[6..11];”，world 将是一个切片，其中包含一个指针，指向 st 的下标为 6 的字节，长度值为 5。

如果我们想从索引 0 开始，那么我们可以使用去掉 starting_index 的语法。换句话说，它们是相等的：

```
let st = String::from("hello");

let slice = &st[0..2];
// 或者这样定义
let slice = &st[..2];
```

相似的是，我们还可以移除尾部的 end_index 索引，表示我们的字符串切片要直到字符串的最后一个字节。换句话说，它们是相等的：

```
let st = String::from("hello");
let len = st.len();

let slice = &st[3..len];
// 或者这样定义
let slice = &st[3..];
```

相似的，我们还可以去掉 start_index 和 end_index，它表示字符串切片包含整个字符串。作为结果，它们是相等的：

```
let st = String::from("hello");
let len = st.len();

let slice = &st[0..len];
// 或者这样定义
let slice = &st[..];
```

字符串切片范围的索引必须出现在有效的 UTF-8 字符的边界上。如果试图在多字节字符中间生成字符串切片，程序将退出并报错。

让我们重做 first_word 以返回包含所有这些信息的切片。&str 是表示“字符串切片”的类型。

```
fn first_word(st:&String) -> &str {
    let bytes = st.as_bytes();
    for(i, &item) in bytes.iter().enumerate() {
        if item==b' ' {
            return &st[0..i];
        }
    }
    &st[..]
}
```

我们获取单词结尾索引的方法与程序中获取单词开头索引的方法相同：我们查找空格的第一次出现。当我们定位一个空格时，我们返回一个字符串切片，其中字符串的开始和空格的索引分别作为开始索引和结束索引。

现在，当调用 `first_word` 时，我们得到一个连接到底层数据的值。对切片的起点的引用和切片中组成该值的组件的数量。

对于 `second_word` 函数，返回一个切片也是可以的：

```
fn second_word(st: &String) -> &str {
```

因为编译器确保了对 `String` 的引用仍然有效，所以我们现在有了一个简单的 API，它很难搞错。记住，在前面的程序中，我们获取了第一个单词末尾的索引，但随后清空了字符串，使索引无效。该代码在概念上是错误的，但它没有立即显示任何错误。如果我们继续使用第一个单词索引和一个空字符串，困难就会稍后出现。切片消除了这个缺陷，并更快地提醒我们代码有问题。如果你使用了 `first_word` 的切片版本，将会发生编译时错误：

```
fn first_word(st:&String) -> &str {
    let bytes = st.as_bytes();
    for(i, &item) in bytes.iter().enumerate() {
        if item==b' ' {
            return &st[0..i];
        }
    }
    &st[..]
}
fn main() {
    let mut st = String::from("hello everyone");
    let word = first_word(&st);
    st.clear();
    println!("First word is: {}", word);
}
```

将出现如下编译错误：

```
error[E0502]: cannot borrow `st` as mutable because it is also borrowed
as immutable
--> r3.rs:13:5
12 |         let word = first_word(&st);
    |                                --- immutable borrow occurs here
13 |         st.clear();
    |         ^^^^^^^^^^ mutable borrow occurs here
14 |         println!("First word is: {}", word);
    |                                ---- immutable borrow later used
    here
```

根据借用规则，如果我们有一个不可变的引用，我们就不能使用可变的引用。`clear` 需要一个可变引用，因为它必须删节 `String`。在执行 `clear` 后调用 `println!`，它使用了 `word` 中的引用；因此，不可变引用在那时仍然是活动的。Rust 编译失败是因为可变引用是 `clear`，和 `word` 中的不可变引用不能同时存在。Rust 不仅使我们的 API 更加对用户友好，而且还消除了这类的编译时错误。

字面量是字符串切片

字符串字面量仅被认为是字符串切片，因为它们是以二进制存储的。让我们来看看：

```
let str = "Hello Rustpoint";
```

`'&str'` 是 `'str'` 的类型，它是一个指向特定二进制点的切片。`'&str'` 是一个不可变引用，字符串字面量也是不可变的。

字符串切片作为参数

如果我们有字符串切片，我们可以直接传递它。为了使 API 更广泛、更有用而又不丧失其功能，我们将字符串切片作为方法的参数而不是引用。

```
fn main() {  
    let str = String::from("technology");  
    let first_word = first_word(&str[..]); // first_word 函数找到字符串  
    的第一个单词  
  
    let st = "technology"; //string literal  
    let first_word = first_word(&st[..]); // first_word 函数找到字符串  
    的第一个单词  
  
    let first_word = first_word(st); // 字符串切片同样作为字符串字面量，因  
    此，它也可以写成这种形式  
}
```

其它的切片

切片还可以用于处理数组。它们具有与字符串切片相同的行为。切片的类型是 `[&i32]`。它们的工作方式与字符串切片同，引用作为第一个元素，而长度作为第二个元素。

考虑以下数组：

```
let arra = [110, 220, 320, 440, 510]; // 数组初始化  
let x = &arra[1..=4]; // 取回数组的第 2、3、4、5 个元素
```

例子：

```
fn main() {  
    let arra = [120, 210, 330, 420, 540, 630];  
    let mut c = 0;
```



```
let x = &arra[1..=4];
let len = x.len();
println!("Elements of 'x' array: ");
while c<len {
    println!("{}", x[c]);
    c += 1;
}
```

我们在本章讨论了什么是所有权、引用和借用，还讨论了切片类型。

C4: 为相关数据使用结构

- 2022.10.9

在这一章

- 定义和实例化结构
- 使用结构的示例程序
- 方法的语法

我们在前一章讨论了所有权、借用和切片类型。本章将讨论 `struct` 的定义以及相关的示例和语法。

结构的定义是什么？

在第二章的“元组类型”一节中讨论的元组类似于结构 `struct`。`struct` 的各个部分，如同元组，可以是多种数据类型。与元组不同的是，我们将为每一项数据进行命名，这样值的含义就很清楚。结构比元组更通用，因为它们的名字：我们不需要依赖数据的顺序来指定或检索实例的值。

要创建结构，我们使用关键字 `struct` 并给它一个名称。结构的名称应该能体现放在一起的项的重要性。数据位的名称和类型，我们称之为字段，在花括号内定义。

```
struct User {  
    actives: bool,  
    usernames: String,  
    e_mail: String,  
    sign_in_counts: u64,  
}
```

我们在创建结构之后创建它的实例，并为使用它的每个字段指定特定的值。我们通过指定结构的名称来创建一个实例，后面是花括号，花括号内是一个个键值对，其中键是字段名，值是我们希望放入这些字段中的数据。字段不必按照结构中声明的顺序指定。换句话说，结构定义充当类型的通用模板，实例用特定的数据填充空白，以提供类型值。例如，我们可以声明一个特定的用户，如下所示。

```
let user1 = User {  
    e_mail: String::from("example@example.com"),  
    usernames: String::from("example1234"),  
    actives: true,  
    sign_in_counts: 1,  
};
```

我们可以使用点表示法从结构体中获取特定的值。我们可以使用 `user1`。如果我们只想要该用户的电子邮件地址，则可以在任何地方使用此值。如果实例是可变的，

则可以通过使用点表示法将值赋给特定字段来更改值。下面的示例解释如何更新可变 `User` 实例的电子邮件字段的值。

```
let mut user1 = User {
    e_mail: String::from("example@example.com"),
    usernames: String::from("example1234"),
    actives: true,
    sign_in_counts: 1,
};
user1.email = String::from("anotherexample@example.com");
```

重要的是要注意，整个实例必须修改；Rust 不能使我们简单地使某些字段可变。作为函数体中的最后一个表达式，我们可以生成结构的一个新实例来自动返回这个新实例，就像任何其他表达式一样。

在以下代码中的 `build_user` 函数返回一个 `user` 对象，其中包含指定的电子邮件和用户名。`actives` 字段设置为 `true`，`sign_in_counts` 字段设置为 `1`。

```
fn build_user(e_mail: String, usernames: String) -> User {
    User {
        e_mail: e_mail,
        usernames: usernames,
        actives: true,
        sign_in_counts: 1,
    }
}
```

尽管将函数参数命名为与结构字段相同是明智的，但重复 `e_mail` 和 `usernames` 字段名和变量还是有些不方便。如果结构有更多的字段，重复每个名称将变得更加乏味。幸运的是，有一种简便的速记方法。

当变量和字段名称相同时，使用字段初始化简写。

因为在上面的代码中参数和结构字段名是相同的，所以我们可以使用字段初始化简写语法来重写 `build_user`。如示例所示，它的操作相同，但没有重复的 `e_mail` 和 `usernames`。

```
fn build_user(e_mail: String, usernames: String) -> User {
    User {
        e_mail,
        usernames,
        actives: true,
        sign_in_counts: 1,
    }
}
```

如上所示，这种简写很简洁。

使用结构更新语法从其他实例创建新实例

通常，基于已有的一个结构来构建新实例很方便，该实例保留了旧实例的大部分值，且按需修改其他值。这可以通过结构的更新语法来实现。

首先，在不使用更新语法的情况下，下面的程序演示了如何在 `user2` 中创建一个新 `User` 实例。我们更改了电子邮件的值，但保留了在前面代码中创建的 `user1` 的值。

```
let user2 = User {
    actives: user1.actives,
    usernames: user1.usernames,
    e_mail: String::from("example@example.com"),
    sign_in_counts: user1.sign_in_counts,
};
```

而通过使用结构的更新语法，我们可以用更少的代码达到同样的效果。语法 `..` 暗示所有未显式设置的字段与当前实例中的字段应具有相同的值。

```
let user2 = User {
    e_mail: String::from("example@example.com"),
    ..user1
};
```

上面代码生成的 `user2` 实例具有不同的 `e_mail` 值，但 `usernames`、`actives` 和 `sign_in_counts` 列的值与 `user1` 相同。`..user1` 必须排在最后，以指示任何剩余字段都应该从 `user1` 中的相应字段中获取它们的值。尽管如此，我们仍然可以按任何顺序为任意数量的字段提供值，而不管结构规范中元素的定义顺序。

因为它移动（move）数据，结构更新语法类似于使用运算符 `=` 的赋值，正如我们在第 3 章的“变量和数据交互的方式：移动”一节中看到的。我们在建立 `user2` 之后就不能再使用 `user1` 了，因为 `user1` 的 `username` 字段中的 `String` 被移到了 `user2`。如果我们为 `user2` 提供了 `e_mail` 和 `username` 新 `String` 值，因此只使用 `user1` 中的 `active` 和 `sign_in_counts` 变量，那么在创建 `user2` 之后，`user1` 仍然有效。因为 `actives` 和 `sign_in_counts` 类型实现了 `Copy` 特征，所以第三章“仅栈数据：复制”一节中描述的行为适用于此情况。

没有命名字段的元组结构来创建不同的类型

我们还可以定义元组结构（tuple struct），它看起来像元组。元组结构体增加了结构体名称的含义，但它们的字段没有名称；相反，它们有字段的类型。当我们想给整个元组命名并使其与其他元组的类型不同时，将每个字段命名为典型结构中的字段显得太长或太多余，可以使用元组结构。

要创建一个元组结构，首先是结构关键字和结构名称，然后是元组类型。下面是两个分别名为 `Color` 和 `Point` 的元组结构的定义和用法，例如：

```
struct Colors(i32,i32,i32);
struct Points(i32,i32,i32);
```

```
let black = Colors(0,0,0);
let origin = Points(0,0,0);
```

因为它们是不同的元组结构的实例，所以 `black` 和 `origin` 是不同的类型。即使结构字段具有相同的类型，我们构造的每个结构也有其惟一的类型。尽管这两种类型都由三个 `i32` 值组成，但接受 `Colors` 参数的方法不能接受 `Points` 作为输入。

结构看起来像单元，但没有任何字段

我们还可以创建没有任何字段的结构！这些被称为类单元结构（`unit-like struct`），因为它们的操作类似于我们在第二章“元组类型”一节中讨论的单元类型。当我们需要在类型上实现一个 `trait` 但无需存储任何数据时，类单元结构非常方便。下面是一个声明和实例化 `AlwaysEqual` 类单元结构的例子：

```
struct AlwaysEqual;

let subjects = AlwaysEqual;
```

我们使用 `struct` 关键字、所需名称和分号来定义类单元结构 `AlwaysEqual`。不需要花括号或圆括号！然后，在声明变量 `subject` 中，我们可以使用我们定义的名称检索 `AlwaysEqual` 的实例，而不使用任何花括号或圆括号。考虑到我们将为这种类型实现行为。每个示例总是等于每个其他类型的每个实例，这可能是为了为测试提供可预测的结果。要实现该行为，我们不需要任何数据。

结构数据的所有权

我们没有使用 `&str` 字符串切片类型，而是在 `User` 结构定义中使用了拥有的 `string` 类型。这是故意的，因为我们希望这个结构的实例拥有它的所有数据，并且该数据在与结构的其他部分相同的时间内保持有效。

`struct` 可以存储对属于其他人的数据的引用，但这样做需要使用生命周期（`lifetime`），这是 `Rust` 的一个特性，我将在后面介绍。生命周期保证结构引用的数据与结构本身具有相同的有效时间。考虑下面的例子，尝试没有定义生命期的情况下在结构中保存一个引用：

```
struct User {
    actives: bool,
    usernames: &str,
    e_mail: &str,
    sign_in_counts: u64,
}

fn main() {
    let user1 = User {
        e_mail: "example@example.com",
        usernames: "example1234",
        actives: true,
        sign_in_counts: 1,
```

```
    };
}
```

编译器会抱怨需要生命周期说明符：

```
error[E0106]: missing lifetime specifier
--> test.rs:3:16
   |
3  |     usernames: &str,
   |                  ^ expected named lifetime parameter
help: consider introducing a named lifetime parameter
   |
1  ~ struct User<'a> {
2  |     actives: bool,
3  ~     usernames: &'a str,
   |

error[E0106]: missing lifetime specifier
--> test.rs:4:13
   |
4  |     e_mail: &str,
   |              ^ expected named lifetime parameter
help: consider introducing a named lifetime parameter
   |
1  ~ struct User<'a> {
2  |     actives: bool,
3  |     usernames: &str,
4  ~     e_mail: &'a str,
   |

error: aborting due to 2 previous errors
```

更新语法

使用结构的更新语法，可以基于现有实例创建一个新实例。

当新实例使用了旧实例的大部分值时，我们使用结构更新语法。考虑以下两个工人：employee1 和 employee2。

- 首先创建 Employee 结构的 employee1 实例。


```
let employee1 = Employee {
    employee_name: String::from("Ankush Jain"),
    employee_id: 12,
    employee_profile: String::from("Computer Science"),
    active: true
};
```

其次，创建 employee2 实例。employee2 实例中的一些值与 employee1 实例中的值相同。employee2 实例可以用以下两种方法声明。

第一种方法是声明 `employee2` 对象而不更改语法。

```
let employee2 = Employee {
    employees_name : String::from("Aarushi Sharma"),
    employees_id: 11,
    employees_profile : employees1.employees_profile,
    active : employees1.active,
};
```

第二种方法是使用更新语法来声明 `employee2` 实例。

```
let employee2 = Employee {
    employees_name : String::from("Aarushi Sharma"),
    employees_id: 11,
    ..employee1
};
```

“..”语法指示未显式设置的其余字段，拥有与给定实例中的相应字段相同的值。

例如：

```
struct Triangles {
    base: f64,
    height: f64,
}
fn main() {
    let triangle = Triangles{ base:22.0, height:32.0};
    print!("Area of right angled triangle {}", area(&triangle));
}
fn area(t: &Triangles) -> f64 {
    0.5 * t.base * t.height
}
```

上面这个例子中建立了三角形的结构，它包含两个变量：直角三角形的底和高。在 `main()` 方法中，生成了一个 `Triangles` 的实例。

结构的一个例子

让我们构造一个程序来计算矩形的面积，以确定何时可能需要使用结构。我们将从单个变量开始，逐步重构代码，直到使用结构。

我用 `Cargo` 创建一个名为 `rectangles` 的新二进制项目，它将接受以像素为单位指定的矩形的宽度和高度，并计算矩形的面积。

```
# 创建项目
$ cargo new rectangles
$ cd rectangles
```

`src/main.rs`:

```
fn main() {
    let width = 40;
    let height = 60;
    println!("Area of the rectangle {} square pixels.", area(width, height));
}
fn area(width:u32, height:u32) -> u32 {
    width * height
}
```

我们可以做得更好，即通过调用面积函数的每个维度来计算矩形的面积并使程序工作。因为宽和高组合成一个矩形，它们彼此相连。

这段代码的问题可以在 `area` 函数的签名中看到：

```
fn area(width:u32, height:u32) -> u32 {
```

虽然 `area` 函数的目的是计算单个矩形的面积，但我们创建的函数包含了两个参数。虽然参数是链接的，但在程序的其他地方没有指定。如果把宽度和高度结合起来，就会更容易阅读和管理。

用元组重构

下面的程序显示了基于元组重构后的版本。

```
fn main() {
    let rec1 = (40, 60);
    println!("Area of the rectangle {} square pixels.", area(rec1));
}
fn area(dimensions: (u32, u32)) -> u32 {
    dimensions.0 * dimensions.1
}
```

这个程序有一个优点。元组允许我们添加一些结构，现在我们只需传递一个参数。然而，这个版本在另一方面不那么明显。因为元组的每个片段没有名称，所以我们的计算变得更加复杂，因为我们必须索引到元组的各个部分。

在计算面积时，如果我们弄乱了宽度和高度也没关系，但如果我们想在屏幕上绘制矩形，就需要记忆宽度是元组索引 `0`，高度是元组索引 `1`。如果代码共享给别人使用，其他从事代码工作的人必须弄清楚这一点并记住它。

因为我们没有在代码中传达数据的含义，所以很容易忘记或混淆这些值，从而导致问题。

使用结构进行重构：增加额外的含义

我们使用结构通过标记数据来为其提供意义。如图所示，我们可以将元组转换为一个数据类型，其中包含一个整体名称和一个片段名称。


```

struct Rectangle {
    width: u32,
    height: u32,
}
fn main() {
    let rec1 = Rectangle{width:40, height:60};
    println!("Area of the rectangle {} square pixels.", area(&rec1));
}
fn area(rectangle: &Rectangle) -> u32 {
    rectangle.width * rectangle.height
}

```

在这里，我创建了一个名为 `Rectangle` 的结构体。我将字段定义为 `width` 和 `height`，它们的类型都是 `u32`，位于花括号内。然后，在主程序中，我生成了一个宽 40、高为 60 像素的矩形实例。

`area` 方法现在有一个名为 `rectangle` 的参数，它的类型是一个不可变的借用矩形实例。正如前一章所指出的，我们不想承担结构的所有权。这确保 `main` 保留 `rec1` 的所有权并可以继续使用它，因此我在 `area` 函数签名中使用 `&` 并调用它。

`area` 函数访问 `Rectangle` 实例的宽度和高度字段，使用矩形的宽度和高度字段计算矩形的面积，如 `area` 函数签名中所述。这表明宽度和高度是相连的，而不是使用 0 和 1 的元组索引值，而是为这些值提供了描述性的名称。这是透明度的胜利。

使用派生 `trait` 添加有用的功能

在调试程序时，最好能打印 `Rectangle` 的实例并查看其所有字段的值。下面演示如何使用 `println!` 宏。然而，这是行不通的。

```

struct Rectangle {
    width: u32,
    height: u32,
}
fn main() {
    let rec1 = Rectangle{width:40, height:60};
    println!("rect1 is {}", rec1);
}

```

当我们编译这段代码时，这个核心信息出现了问题：

```
error[E0277]: `Rectangle` doesn't implement `std::fmt::Display`
```

`println!` 宏可以执行各种格式，花括号告诉 `println!` 使用默认情况的 `Display` 输出，它旨在直接供终端用户的消费。因为向用户显示 1 或任何其他基元类型只有一种方法，所以到目前为止我们看到的基元类型都默认实现了 `Display`。然而，由于结构体有额外的显示选项，`println!` 应该格式化的结果就不那么清楚了：我们到底想不想在句子中使用逗号？我们想打印花括号吗？是否有必要显示所有字段？由于这些不确定性，`Rust` 不会试图推断我们想要什么，而且结构体也没有 `Display` 实现。

如果我们继续阅读这些错误，我们会发现下面这个有用的注释：

```
= help: the trait `std::fmt::Display` is not implemented for `Rectangle`  
= note: in format strings you may be able to use `{:?}` (or `{:#?}` for pretty-print) instead
```

`println!` 宏现在将通过 `println!("rect1 is {:?}", rec1);` 语句输出。在花括号内，说明符 `?:` 通知 `println!` 宏表示希望使用 Debug 输出格式。Debug trait 允许以开发人员友好的方式打印结构，以便在调试代码时查看其值。

在完成此更改后编译代码。见鬼！我们仍然会得到这个错误：

```
error[E0277]: `Rectangle` doesn't implement `std::fmt::Display`
```

但是，编译器再一次给我们提供了一个有用的提示：

```
= help: the trait `Debug` is not implemented for `Rectangle`  
= note: add `#[derive(Debug)]` to `Rectangle` or manually `impl Debug` for `Rectangle`
```

尽管 Rust 包含打印调试信息，但我们还必须显式地选择使该特性对我们的类型可用。在结构定义之前，我们添加外部属性 `#[derive(Debug)]`。修改后的代码如下：

```
#[derive(Debug)]  
struct Rectangle {  
    width: u32,  
    height: u32,  
}  
fn main() {  
    let rec1 = Rectangle{width:40, height:60};  
    println!("rect1 is {:?}", rec1);  
}
```

现在当我运行程序时，就不再产生任何错误了，可以得到如下输出：

```
$ cargo run  
  Compiling rectangles v0.1.0 (D:\workspace\rust-works\rectangles)  
  Finished dev [unoptimized + debuginfo] target(s) in 0.05s  
  Running `target\debug\rectangles.exe`  
rect1 is Rectangle { width: 40, height: 60 }
```

这不是最吸引人的输出，但它显示了该实例的所有字段的值，这对故障排除很有用。当我们有较长的结构时，如果有更容易阅读的输出就更好了；在这些情况下，我们可以在 `println!` 字符串内使用 `{:#?}` 而不是 `{:?}`。如下使用 `{:#?}` 样式：

```
$ cargo run  
  Compiling rectangles v0.1.0 (D:\workspace\rust-works\rectangles)  
  Finished dev [unoptimized + debuginfo] target(s) in 1.04s  
  Running `target\debug\rectangles.exe`  
rect1 is Rectangle {
```

```

    width: 40,
    height: 60,
}

```

`dbg!` 宏也可以打印出一个 Debug 格式的值。`dbg!` 宏获得表达式的所有权，发布 `dbg!` 宏在代码中调用时的文件和行号，以及表达式的结果值，然后返回该值的所有权。与 `println!` 宏将输出打印到标准输出控制台流(`stdout`)不同，`dbg!` 宏将输出打印到标准错误控制台流(`stderr`)。这里有一个例子，其中对分配给 `width` 字段的值和 `rect1` 中整个结构的值都是感兴趣的。

```

#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}
fn main() {
    let scale = 3;
    let rect1 = Rectangle {
        width: dbg!(40*scale),
        height: 60,
    };
    dbg!(&rect1);
}

```

由于 `dbg!` 宏返回表达式值的所有权，故我们可以将 `dbg!` 宏放在表达式 `40*scale` 上，这里无论有没有 `dbg!` 宏，`width` 字段的值不变。由于我们不需要 `dbg!` 宏拥有 `rect1` 的所有权，因此在下一个调用中，我们在 `dbg!` 宏内使用了引用。下面是这个例子的输出：

```

$ cargo run
   Compiling rectangles v0.1.0 (D:\workspace\rust-works\rectangles)
   Finished dev [unoptimized + debuginfo] target(s) in 1.08s
   Running `target\debug\rectangles.exe`
[src\main.rs:9] 40 * scale = 120
[src\main.rs:12] &rect1 = Rectangle {
    width: 120,
    height: 60,
}

```

调试信息输出的第一行来自于表达式 `40 * scale`，它的值为 120。

方法语法

方法的声明方式与函数相同：使用 `fn` 关键字和它们的名称。它们可能具有参数和返回值。它们包括从另一个位置调用时执行的代码。另一方面，方法是在结构的上下文中定义的，它的第一个参数总是 `self`，它表示被调用的结构的实例。

定义方法

让我们在 `Rectangle` 结构中定义一个 `area` 方法来替换 `area` 函数，该结构以 `Rectangle` 实例作为参数。

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}
impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }
}
fn main() {
    let rect1 = Rectangle { width: 40, height: 60, };
    println!("Area of the rectangle {} square pixels.", rect1.area());
}
```

我们为 `Rectangle` 编写了一个 `impl` 实现块，在 `Rectangle` 的上下文中指定函数。`Rectangle` 类型将应用于 `impl` 块中的所有内容。然后，在 `impl` 花括号中，我们传递 `area` 函数，并将签名中的第一个参数更改为 `self`，并贯穿整个主体。我们可以使用方法语法来调用 `Rectangle` 实例的 `area` 方法，而不是在 `main` 中使用 `rect1` 作为参数调用 `area` 函数。在实例之后，添加方法语法：一个点、后跟方法名、圆括号和任何参数。

我们在 `area` 签名中使用 `&self` 而不是 `&Rectangles`。`&self` 是“self”的缩写形式。`Self` 类型是 `impl` 块中用于自身类型的别名。由于方法必须包含一个 `Self` 类型的形参，故 `self` 形参作为第一个实参，`Rust` 允许我们将其缩写为 `&self`。为了表明此方法借用了 `Self` 实例，我们必须在 `self` 简写前使用 `&`，就像我们在 `Rectangle` 中所做的那样：`&rectangle`。方法可以假定 `self` 的所有权，像我们上面所做的那样，不可变地借用它，或者像任何其他参数那样可变地借用它。

LQ 注：parameter 是指函数定义中的参数，而 argument 指的是函数调用时的实际参数。通常将 parameter 译为“形参(formal parameter)”，而将 argument 译为“实参(actual parameter)”。

我们选择 `&self` 的原因与我们在函数版本中使用 `&rectangle` 的原因相同：我们不想拥有结构的所有权，并且我们只想读取它的数据，而不是写入它。如果我们想要改变作为方法功能的一部分调用方法的实例，我们将使用 `&mut self` 作为第一个参数。很少有方法通过只传递 `self` 作为初始参数来获得示例的所有权；当一个人把自己变成别的东西时，通常会使用这种技巧。我们不希望调用者在转换之后使用原始实例。

除了使用方法语法和不在每个方法的签名中重复 `self` 类型之外，使用方法而不是函数的最显著优点是组织。我们没有让代码的未来使用者在我们提供的库中到处搜索 `Rectangle` 的功能，而是把可以用一个类型实例做的所有事情都放入一个 `impl` 块中。

```

impl Rectangle {
    fn width(&self) -> bool {
        self.width > 0
    }
}
fn main() {
    let rect1 = Rectangle { width: 40, height: 60, };
    if rect1.width() {
        println!("Rectangle has a nonzero width; it is {}", rect1.width
    ));
    }
}

```

我们决定，如果 `Rectangle` 实例的 `width` 字段的值大于 0，那么 `width` 方法返回 `true`，如果该值为 0，则返回 `false`：我们可以将同名方法中的字段用于各种目的。当我们在 `main` 函数中的 `rect1.width` 后面使用圆括号时，`Rust` 理解我们指的是 `width` 方法。`Rust` 知道当我们不使用括号时，我们指的是 `width` 字段。

与字段名称相同的方法通常(但并非总是)被定义为只返回字段中的值而不做其他任何事情。`getter` 就是这样的方法，不像其他一些语言，`Rust` 不会为结构字段自动实现 `getter`。`getter` 很重要，因为它们提供了对字段的只读访问，方法是将字段设为私有，而将方法设为公共，作为类型的公共 API 的一部分。

具有更多参数的方法

让我们通过向 `Rectangle` 结构中添加第二个方法来做练习。这一次，我们希望一个 `Rectangle` 实例能够容纳另一个 `Rectangle` 实例，如果第二个 `Rectangle` 可以完全装入第一个 `Rectangle`，则返回 `true`；否则，该方法应该返回 `false`。也就是说，一旦我们编写了 `can_hold` 方法，我们希望能够编写如下所示的程序：

```

#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}
fn main() {
    let rect1 = Rectangle {width:30, height:50,};
    let rect2 = Rectangle {width:20, height:30,};
    let rect3 = Rectangle {width:70, height:55,};
    println!("Can rect1 hold rect2? {}", rect1.can_hold(&rect2));
    println!("Can rect1 hold rect3? {}", rect1.can_hold(&rect3));
}

```

因为 `rect2` 的宽和高两个尺寸都比 `rect1` 的小，但 `rect3` 比 `rect1` 宽，所以预测结果如下：

```

Can rect1 hold rect2? true
Can rect1 hold rect3? false

```

由于我们知道我们将定义一个方法，我们将把该方法放在 `Rectangle` 的 `impl` 块中。该方法将命名为 `can_hold`，它将接受一个不可变的、借来的、另一个 `Rectangle` 作为参数。通过查看调用该方法的代码，我们可以找出它的参数类型：`&rect2`，对 `rect2` 的不可更改的借用，一个 `Rectangle` 的实例，由 `rect1.can_hold(&rect2)` 传入。这是有道理的，因为我们只需要读取 `rect2`（而不是写入，这需要一个可变的借用），而且我们希望 `main` 在执行 `can_hold` 方法后保持 `rect2` 的所有权。该实现将验证自己的宽度和高度是否大于另一个矩形的宽度和高度，`can_hold` 的返回值将是一个布尔值。

```
impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }
    fn can_hold(&self, other:&Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}
```

如果我们使用 `main` 函数运行这段代码，就会得到想要的结果。多个参数可以添加到一个方法的签名中，在自参数之后，这些参数的行为与函数中的参数类似。

关联函数

因为它们与 `impl` 后命名的类型有关，所以在 `impl` 块中创建的任何函数都被称为关联函数（associated function）。因为我们不需要与该类型的实例进行交互，我们可以编写不以 `self` 作为第一参数的关联函数（因此不是方法）。`String::from` 函数，定义在 `String` 类型上，就是这样的一个函数的例子。

对于返回一个新结构体实例的构造函数，我们经常使用不是方法的关联函数。例如，我们可以提供一个关联函数，其中的一个维度参数可以同时作为宽度和高度使用，这样就可以更容易地生成一个正方形的矩形，而不是需要两次指定相同的值。

```
impl Rectangle {
    fn square(size:u32) -> Rectangle {
        Rectangle {width:size, height:size,}
    }
}
```

我们使用带有结构名称的 `::` 语法来调用这个关联函数；例如，让 `sq = Rectangles::squares(3)`；结构 `::` 语法既用于关联函数，也用于模块生成的命名空间来命名这个函数。

多个 `impl` 块

每个结构中都可以有多个 `impl` 块。

```
impl Rectangle {
    fn area(&self) -> u32 {
```

```
        self.width * self.height
    }
}
impl Rectangle {
    fn can_hold(&self, other:&Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}
```

在上面的情况下，无需将这些方法拆分为多个 `impl` 块，但这是 Rust 可以接受的语法。

在本章中，我们介绍了结构和结构的实例化。我们还讨论了使用结构和方法语法的程序。

C5: 枚举和模式匹配

- 2022.10.27

在本章中

- 定义一个枚举
- 匹配控制流操作符
- 使用 `if let` 的简明控制流

在上一章中，我们通过相关示例介绍了结构的定义和实例化。在本章中，我们将讨论枚举（`enum`）和匹配控制流操作符。我们还将介绍使用 `if let` 的简明控制流。

定义一个枚举

LQ 注：本节的内容照抄了 Rust 官方提供的文档，见<https://doc.rust-lang.org/book/ch06-01-defining-an-enum.html>

让我们看一下想在代码中表示的情况，看看为什么枚举在某些情况下比结构更有用和更合适。让我们假设我们必须对 IP 地址进行操作。现在有两个主要的 IP 地址标准：第四版（IPv4）和第六版（IPv6）。这些是我们的程序将遇到的唯一的 IP 地址的选项：我们可以列举出所有可以想象的“枚举”一词的版本。

任何 IP 地址都可以是 IPv4 或 IPv6，但不能同时是两个版本。使用枚举数据结构是合适的，因为枚举值只能是 IP 地址的几种类型中的一种。因为 IPv4 和 IPv6 的地址从根本上说都是 IP 地址，当代码处理涉及任何类型的 IP 地址的情况时，它们应该视为同一类型来处理。

这个概念可以在代码中通过定义 `IpAddrKind` 枚举来表达，它包含了可能存在的许多类型的 IP 地址，如 IPv4 和 IPv6。这些是该枚举的变体。

```
enum IpAddrKind {  
    V4,  
    V6,  
}
```

我们现在可以在代码的其他部分将 `IpAddrKind` 作为一个自定义的数据类型使用它。

Enum 值

我们可以对两个 `IpAddrKind` 版本的每个实例进行如下处理：

```
let five = IpAddrKind::V4;  
let seven = IpAddrKind::V6;
```


注意，枚举的变体在其标识符下被命名，并且用双冒号将两者分开。这很有利，因为 `IpAddrKind::V4` 和 `IpAddrKind::V6` 现在都是同一类型：`IpAddrKind`。然后，举例来说，我们可以写一个接受任何 `IpAddrKind` 的函数。

```
fn route(ip_kind: IpAddrKind) {}
```

我们可以用以下两种方式调用这个函数：

```
route(IpAddrKind::V4);
route(IpAddrKind::V6);
```

使用枚举还有很多好处。但就我们的 IP 地址类型而言，这里还没有一种机制可以保存实际的 IP 地址数据；我们只知道地址是什么类型。由于我们最近在第 4 章学习了 `struct` 结构，所以下面可以在下面解决这个问题。

```
enum IpAddrKind {
    V4,
    V6,
}
struct IpAddr {
    kind: IpAddrKind,
    address: String,
}
fn main() {
    let home = IpAddr {
        kind: IpAddrKind::V4,
        address: String::from("127.0.0.1"),
    };
    let loopback = IpAddr {
        kind: IpAddrKind::V6,
        address: String::from("::1"),
    };
}
```

我们已经构建了一个具有两个字段的结构 `IpAddr`：一个类型为 `String` 的 `address` 地址字段和一个类型为 `IpAddrKind` 的 IP 类型字段（我们前面定义的 `enum`）。该结构体被复制了两次。第一个是 `home`，它的类型是 `IpAddrKind::V4`，关联的地址数据是 `127.0.0.1`。第二个实例是 `loopback`，它的类型是 `IpAddrKind::V6`，关联的地址数据是 `::1`。因为我们使用了一个结构来组合 `kind` 和 `address` 值，所以现在变体被链接到值。

通过将数据直接放入每个 `enum` 版本中，我们可以更紧凑地使用 `enum` 而不是使用结构中的 `enum` 来表示相同的概念。根据 `IpAddr` `enum` 的更新规范，`V4` 和 `V6` 变体都将有关联的 `String` 值。

```
enum IpAddr {
    V4(String),
    V6(String),
}
```

```
fn main() {
    let home = IpAddr::V4(String::from("127.0.0.1"));
    let loopback = IpAddr::V6(String::from("::1"));
}
```

因为我们立即将数据附加到每个 `enum` 版本，所以不再需要之前的 `IpAddr` 结构体。枚举操作方式的另一个方面是，我们声明的每个枚举变量的名称也成为创建枚举实例的函数。换句话说，`IpAddr::V4()` 是一个接受 `String` 作为参数并返回 `IpAddr` 实例的函数。声明 `enum` 的结果是自动定义这个构造函数。

使用枚举而不是结构的另一个好处是，每个版本都可以有与之关联的不同类型和数量的数据。IPv4 地址总是由四个数值组成，数值范围从 0 到 255。而使用结构体，我们不会将 V4 地址存储为四个 `u8` 值，同时仍然将 V6 地址表示为一个 `String` 值。而这对于枚举来说，是一个很容易处理的情况：

```
enum IpAddr {
    V4(u8, u8, u8, u8),
    V6(String),
}
fn main() {
    let home = IpAddr::V4(127, 0, 0, 1);
    let loopback = IpAddr::V6(String::from("::1"));
}
```

在 IP 地址的 V4 和 V6 中，我们已经介绍了几种创建用于存储 IP 地址数据结构的不同方法。然而，事实证明，存储 IP 地址和编码它们的类型是如此普遍，以至于我们可以使用标准库中的定义。让我们看看在标准库如何定义 `IpAddr`。它使用与我们使用的相同枚举和变量，但它使用为每个变量单独定义的两个不同结构将地址数据嵌入到变量中：

```
struct Ipv4Addr {
}
struct Ipv6Addr {
}
enum IpAddr {
    Ipv4Addr,
    Ipv6Addr,
}
```

这段代码演示了枚举版本可以包含任何类型的数据，例如文本、数值类型或结构。我们甚至可以包含另一个枚举！此外，标准库类型通常并不比我们设计的复杂多少。

即使标准库提供了 `IpAddr` 定义，我们仍然可以开发和使用我们自己的 `IpAddr`，而不会与标准库的定义发生冲突，因为我们还没有将标准库的定义引入我们的范围。

让我们看看另一个枚举，它的变体中嵌入了广泛的类型。

```
enum Messages {
    Quit,
```

```

    Move {x:i32, y:i32},
    Write(String),
    ChangeColor(i32, i32, i32),
}

```

这个枚举有四种不同的类型：

1. 没有与 Quit 相关的数据。
2. Move 和结构体一样，也有命名字段。
3. Write 中只包含一个字符串。
4. ChangeColor 中包含三个 i32 值。

定义带有变量的枚举（比如前面程序中的那些变量）与定义几种类型的结构定义相似，唯一的例外是枚举不使用 `struct` 关键字，并且所有变量都被分组在 `Message` 类型下。以下结构可以存储与它们之前的枚举变量相同的数据：

```

struct QuitMessage; // 单元结构
struct MoveMessage {
    x: i32,
    y: i32,
} // LQ: 注意这里没有分号
struct WriteMessage(String); // 元组结构
struct ChangeColorMessage(i32, i32, i32); // 元组结构

```

但是，如果使用不同的结构（每个结构都有自己的类型），就无法快速编写函数来接受任何这些类型的消息，就像前面示例中定义的 `Message enum` 一样，它只有一个类型。

枚举和结构之间的另一个相似之处是，我们可以使用 `impl` 在枚举上定义方法，就像我们可以使用 `impl` 在结构上定义方法一样。下面是我们可以添加到 `Message` 枚举中的调用方法：

```

impl Messages {
    fn call(&self) {
        // do something
    }
}
fn main() {
    let m1 = Messages::Write(String::from("hello"));
    m1.call();
}

```

方法的主体将通过使用 `self` 获取调用方法的值。我们已经用这个例子中的值创建了一个变量 `m1`。 `Messages::Write(String::from("hello"))`，当 `m1.call()` 被调用时，`self` 将在调用方法的主体中。

让我们看看标准库中另一个经常使用的枚举 `:Option`。

Option 枚举相对于 Null 空值的优点

在上一节中，我们看到了 `IpAddr` 枚举如何允许我们使用 Rust 的类型系统在程序中嵌入更多的信息，而不仅仅是数据。`option` 是标准库定义的另一个枚举，本节将对其进行案例研究。因为它封装了相对典型的情况，即值可以是任意值或无值，所以 `Option` 类型被广泛使用。

当我们用类型系统来表达这个概念时，编译器可能会检查我们是否处理了所有应该管理的情况；这个功能可以帮助我们避免其他编程语言中的常见问题。

通常认为编程语言设计应该包含哪些特性，但考虑去掉哪些特性也很重要。Rust 缺少许多其他语言中的 `null` 特性。`Null` 是一个值，表示它没有值。支持 `null` 的语言中的变量可以始终为 `null` 或不为 `null`。

`Null` 的问题是，试图使用一个非空值将导致某种类型的错误。因为 `null` 或 `not-null` 属性的使用非常广泛，所以很容易犯这样的错误。

然而，`null` 试图表示的概念仍然有用：`null` 是当前由于某种原因无效或不存在的值。

问题不在于概念，而在于如何实施。因此，Rust 缺少 `null` 值，尽管它包含了一个枚举来表示值存在或不存在的。`Option` 就是这种枚举，在标准库中定义如下：

```
enum Option<T> {  
    None,  
    Some(T),  
}
```

`Option` 枚举非常有用，它被包含在序言中；不需要明确地规定其范围。它的变体也得到支持：我们可以使用 `Some` 和 `None` 而无需带 `Option::` 前缀。`Some(T)` 和 `None` 仍然是 `Option` 类型的变体，而 `Option` 枚举仍然只是一个标准的枚举。

我们还没有讨论 语法，它是 Rust 的一个特性。因为它是一个泛型类型参数，所以我们需要知道表示 `Option` 枚举的某些变体可以携带任何类型的一块数据，并且任何替换为 `T` 的具体类型都会改变整个 `Option` 类型。下面是如何使用 `Option` 值存储数字类型和字符串类型的示例：

```
let some_number = Some(6);  
let some_string = Some("string");  
let absent_number: Option<i32> = None;
```

`Option` 是一个 `Some` 数值的类型。`Option<&str>` 是一个字符串的类型，它是另一种类型。因为我们在 `Some` 变体中指定了一个值，所以 Rust 可以推断出这些类型。Rust 需要我们为 `absent_number` 标注整个 `Option` 类型：因为编译器不能仅根据一个 `None` 值推断相关的某个变体将持有的类型。故我们通知 Rust，这里 `absent_number` 应该是 `Option` 类型。

当我们有一个 `Some` 值时，我们知道存在一个值，因为该值被保存在 `Some` 中。在某些方面，`None` 值与空值相同：我们没有有效值。那么，为什么 `Option` 优于 `null` 呢？

换句话说，编译器不允许我们使用 `Option` 值，似乎它是一个有效值一样，因为 `Option` 和 `T`（其中 `T` 可以是任何类型）是不同的类型。例如，这段代码将无法编译，因为它试图将 `i8` 添加到 `Option`：

```
let a:i8 = 6;
let b:Option<i8> = Some(6);
let sum = a + b;
// 编译将会报如下错误:
error[E0277]: cannot add `Option<i8>` to `i8`
...
4 |         let sum = a + b;
  |                   ^ no implementation for `i8 + Option<i8>`
```

此错误消息表明 `Rust` 无法对 `i8` 和 `Option` 进行求和运算，因为它们是不同的类型。在 `Rust` 中，当我们有一个像 `i8` 这样类型的值时，编译器会确保我们在任何时候都有一个有效的值。我们不需要在使用该值之前检查空值，以便有信心地向前推进。我们只需要在处理 `Option`（或我们正在处理的任何类型的值）时担心没有值，编译器将确保我们在使用该值之前处理该场景。

那么，如果我们有一个 `Option` 类型的值，如何从一些变体中获得 `T` 的值，以便我们可以利用它？`Option` 枚举包含了在各种情况下都有用的许多方法；相关的文档可以在[这里](#)找到。在我们的 `Rust` 冒险中，熟悉 `Option` 方法将非常有益。

一般来说，如果我们要使用 `Option` 值，我们需要代码来处理每个变体。我们需要一些只有当你有一个 `Some(T)` 值时才会运行的代码，这些代码允许使用内部的 `T`。我们还需要有当存在 `None` 值时对应的代码，这些代码中没有 `T` 值。当与枚举一起使用时，`match` 匹配表达式正是这样做的：它根据所包含的枚举的任何变体运行不同的代码，并且该代码可以使用匹配值中的数据。

匹配控制流操作符

`match` 是 `Rust` 语言中一个功能强大的控制流操作符，它允许我们将值与模式序列进行比较，然后根据匹配的模式执行代码。模式可以由文字值、变量名、通配符和各种其他元素组成。模式的表达性以及编译器验证所有可能的场景都得到了处理这一事实，都与它的能力相匹配。

考虑一个类似于硬币分拣机的 `match` 表达式：硬币沿着带有不同大小孔的轨道滑动，每枚硬币都掉进它遇到的第一个孔中。类似地，值在匹配中的每个模式中传递，如果值“符合”第一个模式，则将它放在执行期间使用的相关代码块中。

让我们用硬币来利用 `match` 因为我们刚刚说过。如图所示，我们可以构造一个函数，它接受一枚未知的美元硬币，类似于计数机，确定它是哪枚硬币，并返回其价值(以美分为单位)。

```
enum Coins {
    Penny,
    Nickel,
    Dime,
    Quarter,
}
fn value_in_cents(coin: Coins) -> u8 {
    match coin {
        Coins::Penny => 1,
        Coins::Nickel => 5,
        Coins::Dime => 10,
        Coins::Quarter => 25,
    }
}
```

让我们看一下 `value_in_cents` 函数中的 `match`。首先列出 `match` 关键字，然后紧跟一个表达式，在本例中是 `coin` 的值。这看起来很像 `if` 表达式，但有一个关键的区别：如果是 `if`，表达式必须返回布尔值，而这里它可以是任何类型。在本例中，我们在第 1 行定义的 `Coins` 枚举是硬币的类型。

接下来是 `match` 的匹配臂。匹配有两部分：一个模式（`pattern`）和一段代码。第一个匹配臂包含一个值为 `Coins::Penny` 的模式，后面跟着 `=>` 操作符，它将模式与要执行的代码段分离开来。在上面这种情况下，代码只是值 `1`。用逗号将每条匹配臂与前一条匹配臂分隔开。

当 `match` 匹配表达式运行时，它会将结果与每个匹配臂的模式逐一进行比较。如果值与模式匹配，则运行与该模式关联的代码。如果模式与值不匹配，就会移动到下一个匹配臂，类似于硬币分拣机。

与每个匹配臂相关联的代码是一个表达式，完整匹配表达式的返回值来自匹配臂中的表达式。

当匹配臂代码很短时，在每个臂返回一个值的示例中通常不使用花括号。花括号可以在一个匹配臂中运行多行代码。例如，下面的代码将输出 “Lucky penny!” 每次用 `Coins::Penny` 调用该方法时，它仍然返回块的最后一个值 `1`：

```
fn value_in_cents(coin: Coins) -> u8 {
    match coin {
        Coins::Penny => {
            println!("Lucky penny!");
            1
        }
        Coins::Nickel => 5,
        Coins::Dime => 10,
    }
}
```

```

        Coins::Quarter => 25,
    }
}

```

绑定到值的模式

匹配臂还可以绑定到与模式匹配的值的部分。我们可以通过这种方式从枚举变量中检索值。

让我们以更新一个枚举变量来保存数据为例。从 1999 年到 2008 年，美国为 Quarter 硬币搞了定制化设计，50 个州的 Quarter 硬币设计了不同的图案。但只有 25 美分的 Quarter 硬币有这种额外的价值，其他硬币都没有这种定制化的图案。我们可以通过修改 Quarter 变量来在枚举中包含存储在其中的 UsState 值，就像上面所做的那样。

```

enum UsState {
    Alabama,
    Alaska,
}
enum Coins {
    Penny,
    Nickel,
    Dime,
    Quarter(UsState),
}

```

假设我们的某个朋友想要收集所有 50 州的 Quarter 硬币。当我们按照硬币的种类来分类零钱时，我们会喊出与每个 Quarter 硬币相关的州名，所以如果我们的朋友没有该 Quarter 硬币，就可以把它添加到他们的收藏中。

我们将一个名为 state 的变量添加到这段代码的匹配表达式中的模式中，以匹配变体 Coins::Quarter 的值。当匹配到 Coins::Quarter 时，将 state 变量设置为该 Quarter 的 state 值。然后，在该匹配臂的代码中，我们可以使用如下 state:

```

fn value_in_cents(coin: Coins) -> u8 {
    match coin {
        Coins::Penny => {
            println!("Lucky-penny");
            1
        }
        Coins::Nickel => 5,
        Coins::Dime => 10,
        Coins::Quarter(state) => {
            println!("State quarter {:?}!", state);
            25
        }
    }
}

```


如果调用 `value_in_cents(Coins::Quarter(UsState::Alaska))`，那么 `Coins` 将是 `Coins::Quarter(UsState::Alaska)`。当我们将该值与每个匹配臂进行比较时，我们发现它们都不匹配，直到我们到达 `Coins::Quarter(state)`。值 `UsState::Alaska` 将是该 `state` 在该点的绑定。然后，可以通过在 `println!` 表达式的绑定使用该值。

LQ 注：完整的代码如下

```
#[derive(Debug)]
enum UsState {
    Alabama,
    Alaska,
}
enum Coins {
    Penny,
    Nickel,
    Dime,
    Quarter(UsState),
}
fn value_in_cents(coin: Coins) -> u8 {
    match coin {
        Coins::Penny => {
            println!("Lucky-penny");
            1
        }
        Coins::Nickel => 5,
        Coins::Dime => 10,
        Coins::Quarter(state) => {
            println!("State quarter {:?}!", state);
            25
        }
    }
}
fn main() {
    let a = Coins::Quarter(UsState::Alaska);
    println!("{}", value_in_cents(a));
}
```

带 Option 的匹配

在上一节中使用 `Option` 时，我们需要获取某些情况下的内部 `T` 值；我们还可以用 `match` 来处理 `Option`，就像我们处理 `Coins` 枚举那样！我们将比较 `Option` 的版本，但匹配表达式仍然以相同的方式运行。

让我们想象一下，我们想要构造一个函数，它接受 `Option<32>`，如果有值存在，则对其中包含的任何值加 1；如果没有值，函数应该返回 `None`，并且不尝试执行任何操作。

```
fn plus_one(a: Option<i32>) -> Option<i32> {
    match a {
```



```

        None => None,
        Some(x) => Some(x+1),
    }
}
fn main() {
    let five = Some(5);
    let six = plus_one(five);
    let none = plus_one(None);
    println!("{:?}", six);
    println!("{:?}", none);
}

```

让我们看一下 `plus_one` 的执行。当调用 `plus_one(five)` 时，`plus_one` 函数体中的变量 `x` 的值为 `Some(5)`。之后，我们将其与每个匹配臂进行比较。

```
None => None,
```

由于值 `Some(5)` 不匹配模式 `None`，我们继续下一个匹配臂。

```
Some(x) => Some(x+1),
```

`Some(5)` 和 `Some(i)` 一样吗？它的确是！版本对我们来说是一样的。因为 `i` 被绑定到 `Some` 中的值，它是 5。然后运行匹配臂的代码，因此我们将 `i` 的值加 1，并构造一个新的 `Some` 值，其中的数为 6。

对于 `none` 变量，让我们看看上面代码中的第二个 `+1` 调用。当匹配开始时，我们先比较第一只匹配臂和第二只匹配臂。

```
None => None,
```

已经匹配上了！由于没有需要 `add` 的内容，所以程序停止并在 `=>` 的右侧返回 `None` 值。因为第一个臂匹配，所以不再比较其他臂。

在许多情况下，将 `match` 匹配和 `enum` 枚举相结合是有益的。在 `Rust` 代码中，我们会经常看到这种模式：匹配枚举，将变量绑定到其中的数据，并根据它运行代码。刚开始会有点挑战，但一旦我们掌握了窍门，我们就会希望每种语言都能使用它。这是一种很受用户欢迎的选择。

匹配是详尽的

匹配还有一个问题需要解决。考虑一下这个 `plus_one` 函数的 `bug` 版本，它将无法编译：

```

fn plus_one(a: Option<i32>) -> Option<i32> {
    match a {
        Some(x) => Some(x+1),
    }
}

```

因为我们没有处理各种情况，所以这段代码将失败。幸运的是，Rust 很擅长捕捉这种 Bug。如果我们试图编译这段代码，我们将得到以下错误：

```
error[E0004]: non-exhaustive patterns: `None` not covered
  --> r3.rs:2:11
   |
2  |         match a {
   |         ^ pattern `None` not covered
= note: the matched value is of type `Option<i32>`
help: ensure that all possible cases are being handled by adding a match arm with a wildcard pattern or an explicit pattern as shown
```

Rust 很清楚我们没有覆盖所有的情况，甚至知道我们忽略了哪个模式！在 Rust 中，匹配是穷尽的：我们必须穷尽所有可能的选项以使代码合法。Rust 防止我们忽略显式处理 None 情况，从而避免我们犯错，特别是在 Option 的情况下。

全抓（catch-all）模式和 _ 占位符

考虑以下场景：我们希望对几个特定值采取独特的操作，同时对其他所有值采取单一的默认操作。假设我们正在设计一款游戏，如果掷骰子的数值为 2，玩家将获得一顶新帽子而不是继续移动。如果我们掷出 6，玩家就会失去一顶漂亮的帽子。对于掷出的其它值，我们的玩家将在游戏棋盘上移动相同数量的方格。下面是一个实现该逻辑的匹配，骰子滚动的结果使用了硬编码而不是随机值，所有其他逻辑都没有主体的函数表示，因为实际实现超出了本示例的范围：

```
fn main() {
    let dice_roll = 8;
    match dice_roll {
        2 => add_fancy_hat(),
        6 => remove_fancy_hat(),
        other => move_player(other),
    }
}

fn add_fancy_hat() {
    println!("add a fancy hat!");
}

fn remove_fancy_hat() {
    println!("subtract a fancy hat!");
}

fn move_player(n: u8) {
    println!("move {}", n);
}
```

LQ 注：我把示例代码补全了，以便能调试代码。

前两个匹配臂的模式是数字值 2 和 6。最后一个匹配臂的模式，它覆盖了所有其他可能的值，是我们命名为 `other` 的变量。在为 `other` 匹配臂运行的代码中，该变量被传递给 `move_player` 函数。

尽管我们没有声明 `u8` 的所有潜在值，但由于最后一个模式匹配所有没有特别列出的值，所以这段代码会通过编译。这种全抓模式满足了穷尽匹配的标准。因为这些模式是按顺序检查的，所以我们必须把包罗一切的那条放在最后。`Rust` 会通知我们，如果我们在全抓匹配后继续添加匹配臂，那么后面的手臂将永远不会被匹配。

当我们不想使用来自全抓模式的值时，我们可以使用下面这种模式，这是一种特殊的模式，它匹配任何值，但不绑定它。这将通知 `Rust`，我们不打算使用该值来警告我们。

让我们调整游戏规则，如果我们摇出的点数不是 2 或 6，你必须再摇一次。在这种情况下，我们不需要使用该值，因此我们可以将代码修改为使用 `_` 而不是其他变量：

```
fn main() {
    let dice_roll = 8;
    match dice_roll {
        2 => add_fancy_hat(),
        6 => remove_fancy_hat(),
        _ => reroll(),
    }
}

fn add_fancy_hat() {
    println!("add a fancy hat!");
}

fn remove_fancy_hat() {
    println!("subtract a fancy hat!");
}

fn reroll() {
    println!("reroll!");
}
```

这个例子也通过了穷尽标准（`exhaustiveness criteria`），因为最后一个分支中的所有其他值都被明确地忽略了；我们没有忽视任何东西。

如果我们再次改变游戏规则，即如果我们滚动的点数不是 2 或 6，那么在我们的回合中什么也不会发生，我们便可以使用 `unit` 单元值(即我们在第二章“`tuple type`”章节中所阐述的空元组类型)作为 `_` 匹配臂的代码：

```
fn main() {
    let dice_roll = 8;
    match dice_roll {
        2 => add_fancy_hat(),
        6 => remove_fancy_hat(),
        _ => (),
    }
}
```

```

}
fn add_fancy_hat() {
    println!("add a fancy hat!");
}
fn remove_fancy_hat() {
    println!("subtract a fancy hat!");
}

```

我们显式地告诉 Rust，我们不会使用任何与前面匹配臂中模式不匹配的其他值，并且在这种情况下我们不会运行任何代码。

现在，我们将转到 `if let` 语法，当匹配表达式有点长时，它会很方便。

带 `if let` 的简洁控制流

如果我们将 `let` 组合到 `if let` 语法中，就可以处理匹配一个模式的值，而不必那么繁琐地忽略其余的值。考虑这样一个程序，它检查 `config_max` 变量的 `Option` 值，但只希望在该值为 `Some` 变体时运行代码。

```

let config_max = Some(3u8);
match config_max {
    Some(max) => println!("Maximum is configured to be {}", max),
    _ => (),
}

```

如果值是 `Some`，我们希望打印 `Some` 变体中的值，这是通过将值赋给模式中的变量 `max` 来实现的。对于 `None` 值，我们不想做任何事情。在分析了一个版本之后，我们必须添加 `_=>()` 来满足匹配表达式，这属于添加的繁琐样板代码。

```

let config_max = Some(3u8);
if let Some(max) = config_max {
    println!("Maximum is configured to be {}", max);
}

```

在 `if let` 语法中，模式和表达式用等号分隔。它的功能类似于 `match` 匹配，即短语被传递给匹配，而模式是它的第一个匹配臂。在本例中，模式是 `Some(max)`，而 `max` 绑定到 `Some` 中的值。然后，就像我们对类似的匹配臂所做的一样，可以在 `if let` 块的主体中使用 `max`。如果值与模式不匹配，则不执行 `if let` 块中的函数。

使用 `if let` 可以减少输入、缩进和样板代码，从而节省时间。然而，我们确实失去了匹配的详尽检查。`match` 匹配和 `if` 允许之间的决定是基于具体情况下所做的事情，以及获得简洁是否是失去彻底检查的良好权衡。

换句话说，考虑 `if let` 是作为 `match` 匹配的语法糖，当值匹配一个模式并忽略所有其他值时执行代码。

对于 `if let`，我们可以包含 `else`。与 `else` 一起使用的代码与 `match` 表达式中与 `_case` 一起使用的代码相同，后者与 `if let` 和 `else` 相比较。还记得 `Coin` 枚举的 `Quarter` 变

量是如何包含 `UsState` 值的吗？我们可以使用像这样的匹配表达式来计数我们看到的所有非 `Quarter` 硬币，同时通知 `Quarter` 的 `state`：

```
let mut counts = 0;
match coin {
    Coin::Quarter(state) => println!("State quarter {:?}", state),
    _ => counts += 1,
}
```

或者，我们也可以像这样使用 `if let` 和 `else` 表达式：

```
let mut counts = 0;
if let Coin::Quarter(state) = coin {
    println!("State quarter {:?}", state);
} else {
    counts += 1;
}
```

请记住，如果我们发现我们的程序推理太长而无法用 `match` 匹配描述，那么 `if let` 就应该在我们的 `Rust` 工具包中。

本章涵盖了枚举定义、匹配控制流操作符以及使用 `if let` 的简洁控制流。

C6: 包、crate 和模块

- 2022.11.3

在这一章

- 包和 crate
- 定义模块来控制范围和隐私
- 在模块树中用于引用项的路径
- 使用 `use` 关键字将路径引入作用域
- 将模块分离到不同的文件中

在前一章中，我们介绍了枚举和模式匹配及其相应的语法和示例。本章将讨论包和 `crate`，定义模块，引用路径，以及如何将模块分离到不同的文件中。

组织我们的代码将变得越来越重要，因为当我们构建更大的程序时，在大脑中跟踪我们的整个程序将变得不切实际。我们将发现实现特定特性的代码，以及通过比较分组功能和将具有单独特性的代码分离，如何改变特性的工作方式。

到目前为止，我们构建的所有程序都包含在单个模块和单个文件中。随着项目的发展，我们可以通过将代码分解为不同的模块和多个文件来组织代码。一个包中可以包含多个二进制 `crate` 和一个库 `crate`。

我们可以将包的组件隔离到不同的 `crate` 中，当它扩展时，这些 `crate` 成为外部依赖性。所有这些策略都将在本章中介绍。

除了分组功能之外，封装实现细节还允许我们在更高的级别重用代码：在我们实现了一个操作之后，其他程序可以通过代码的公共接口调用它，而无需知道它是如何工作的。

代码的哪些部分是公开的，供其他代码使用，哪些部分是我们有权更改的秘密实现细节，这取决于我们如何设计它。这是另一种将大脑中的信息量降到最低的方法。

作用域是一个类似的概念：在编写代码的分层上下文中，名称集合被定义为“在作用域中”。在读取、编写和编译代码时，程序员和编译器必须确定给定名称是指变量、函数、结构、枚举、模块、常量还是其他项，以及该项的含义。我们可以设置作用域并调整包含或排除的名称。

同一个作用域中不能有两个名称相同的东西；可以使用解决名称冲突的机制。

`Rust` 包含了一些用于管理代码组织的功能，例如哪些细节是公开的，哪些细节是私有的，以及应用程序中的每个作用域中有哪些名称。这些特性，有时作为一个整体被称为模块系统（`module system`），包括：

- **Package（包）**：包是允许我们创建、测试和共享 `crate` 的 `cargo` 功能。

- **Crate**（板条箱）：板条箱是构成库或可执行文件的模块的集合。
- **Module and use**（模块和使用）：模块和它们的应用程序允许我们管理路径的组织、范围和隐私。
- **Path**（路径）：路径是结构、函数和模块等名称。

我们将在本章中讨论所有这些特性，以及它们如何相互作用，以及如何使用它们来管理作用域。我们应该对模块系统有一个牢固的掌握，并在最后像专业人士一样使用作用域！

*LQ 注：我不愿意将 **crate** 翻译为板条箱，这样的直译除了引入干扰，对理解 **Rust** 并无帮助，故我的选择不译。*

包和 crate

包和 **crate** 是我们将要研究的模块系统的第一个方面。一个二进制文件或库可以称为 **crate**。**crate** 根文件是一个源代码文件，**Rust** 编译器用它来构建 **crate** 根模块（我们将在下一节“定义模块来控制范围和隐私”中更详细地介绍模块）。包是提供一组特性的 **crate** 的集合。**Cargo.toml** 文件定义了如何构造这些 **crate**。

一个包可以包含什么是由一组规则决定的。一个包中只能有一个 **crate** 库，它可以包含我们想要的任意多个二进制 **crate**，但必须至少有一个(库或二进制)。

让我们看看创建包所涉及的步骤。首先，我们将使用 **cargo new** 命令：

```
$ cargo new myproject
  Created binary (application) `myproject` package
$ ls myproject
Cargo.toml
src
$ ls myproject/src
main.rs
```

cargo 创建了一个 **Cargo.toml** 文件，并在我们输入命令时给了我们一个包。**Cargo.toml** 文件中没有提到 **src/main.rs**，因为 **Cargo** 使用了约定，**src/main.rs** 是与包同名的二进制 **crate** 的根文件。类似地，如果包目录中包含 **src/lib.rs**，**Cargo** 会认为包中包含了一个与包同名的 **crate** 库，以及 **src/lib.rs** 是这个 **crate** 库的根文件。要构建库或二进制文件，**Cargo** 将 **crate** 根文件发送到 **rustc**。

我们有一个带 **src/main.rs** 的包，它只有一个名为 **myproject** 的二进制 **crate**。如果 **src/main.rs** 和 **src/lib.rs** 包含在一个包中，那么这将包含两个 **crate**：一个库和一个二进制文件，两者具有与包相同的名称。通过将文件存储在 **src/bin** 目录，一个包可以有多个二进制 **crate**：其中的每一个文件都是一个单独的二进制 **crate**。

可以对 **crate** 在作用域并按功能进行分组，以便在多个项目之间轻松共享它。

在我们的项目范围内使用 **rand crate**，就可以在我们的项目中使用该功能。**rand crate** 的名字 **rand** 提供了对 **crate** 所有功能的访问。

保持一个 `crate` 的功能在它的作用域内，可以澄清一个功能是在我们的 `crate` 中定义的还是在 `rand crate` 中定义的，从而避免潜在的冲突。例如，`rand crate` 有一个名为 `Rng` 的特性。

在我们的 `crate` 中，我们还可以定义一个名为 `Rng` 的结构。当我们添加 `rand` 作为依赖项时，编译器不会对名称 `Rng` 感到困惑，因为 `crate` 的功能是在其作用域的命名空间。它引用了我们在 `crate` 中定义的结构 `Rng`。我们将使用 `rand::Rng` 从 `rand crate` 中获得 `Rng` 特性。

为控制作用域和隐私定义模块

在本节中，我们将讨论模块和模块系统的其他方面，包括路径（允许命名对象）、`use` 关键字（将方法放入作用域）和 `pub` 关键字（使项公开）。还将讨论 `as` 关键字、外部包和 `glob` 操作符。下面让我们先暂时聚焦于模块。

模块（`module`）允许我们将代码分组到一个 `crate` 中，以便于阅读和重用。模块还可以管理私有（`private`）项，这决定了项是否可以被外部代码（`public`）公开利用，或者是外部（`private`）无法访问的内部实现细节。

假设我们想要创建一个具有餐厅功能的 `crate` 库。为了将重点放在代码的组织上，而不是在代码中实现餐厅，我们将指定函数的签名，但将函数体保留为空。

在餐饮业中，一些餐厅区域被称为前厅，另一些被称为后厅。顾客在前厅受到招待，在那里主人让他们坐下，服务员接受订单和付款，调酒师混合饮料。厨师和厨师在厨房工作，洗碗工打扫卫生，而经理在后厅做行政工作。

我们可以将函数组织到嵌套的模块中，像真正的餐厅那样构造我们的 `crate`。通过执行 `cargo new -lib restaurant`，您可以创建一个名为 `restaurant` 的新库，然后将程序源代码放入 `src/lib` 中来定义模块和函数签名。

```
// restaurant/src/lib.rs
mod front_of_houses {
    mod hostings {
        fn add_to_waitlist() {}
        fn seat_at_table() {}
    }
    mod servings {
        fn take_order() {}
        fn serve_order() {}
        fn take_payment() {}
    }
}
```

我们首先使用 `mod` 关键字，然后指定模块的名称（本例中模块名为 `front_of_houses`），并在模块体周围放入花括号。其他模块可以包含在模块中，如本例中的 `hostings` 模块和 `servings` 模块，如上所述。模块还可以保存其他元素的定义，如结构、枚举、常量、特征和程序函数。

我们可以将相关的定义分组在一起，并通过使用模块命名它们之间的连接原因。程序员可以基于分组来研究代码，而不是通读所有的定义，这使得找到所需的定义更容易。想要在代码中添加额外功能的程序员应该知道把它放在哪里，以便程序保持有序。

`crate` 的根文件是 `src/main.rs` 和 `src/lib.rs`，就像我们之前所述。这两个文件的所有内容构成了一个 `crate` 模块，该 `crate` 以模块结构的根作为模块名，模块结构称为模块树（`module tree`），这就是它们命名的原因。

```
crate
├── front_of_houses
│   ├── hostings
│   │   ├── add_to_waitlist
│   │   └── seat_at_table
│   └── servings
│       ├── take_order
│       ├── serve_order
│       └── take_payment
```

这个树描述了一些模块是如何嵌套的（例如，`hostings` 如何嵌套在 `front_of_houses`）。该树还揭示了一些模块是兄弟模块，这意味着它们是在同一个模块中创建的（例如，`hostings` 和 `servings` 模块都定义在 `front_of_houses` 中）。为了继续这个模块树的比喻，如果模块 A 包含在模块 B 中，我们可以说模块 A 是模块 B 的子模块，模块 B 是模块 A 的父模块。值得注意的是，整个模块树都根植于 `crate` 隐式模块。

模块树可能会使我们想起计算机磁盘上的目录树；这是一个有趣的观察。模块被用来排列我们的代码，就像文件系统中使用目录一样。我们需要一种机制来查找模块，就像我们需要一种方法在目录中查找文件那样。

用于引用模块树中项的路径

我们以与浏览文件系统相同的方式使用路径，向 `Rust` 显示在模块树的何处查找项。我们需要知道调用函数的路径。

路径可以是下面两种类型之一：

1. 一个 `crate` 名可以使用从 `crate` 根文件开始的绝对路由（`absolute route`）
2. 从当前模块开始，并使用 `self`、`super` 或标识符的相对路径（`relative path`）

在绝对路由或相对路径中的标识符之间使用双冒号（`::`）进行分隔。

让我们回到前面的例子。调用 `add_to_waitlist` 函数的语法是什么？`add_to_waitlist` 函数的路径是什么？我们通过删除下面几个模块和函数来减少代码。我们将演示从 `crate` 根通过公开的 `eat_at_restaurant` 函数调用 `add_to_waitlist` 函数的两种方式。`pub` 关键字用于指示 `eat_at_restaurant` 方法是库 `crate` 的公开 API 的一部分。我们

将在“使用 pub 关键字公开路径”一节中进一步深入研究 pub。此示例目前不会编译；我们稍后会解释原因。

```
mod front_of_houses {
    mod hostings {
        fn add_to_waitlist() {}
    }
}
pub fn eat_at_restaurant() {
    // 绝对路径
    crate::front_of_houses::hostings::add_to_waitlist();
    // 相对路径
    front_of_houses::hostings::add_to_waitlist();
}
```

我们在 eat_at_restaurant 中第一次运行 add_to_waitlist 方法时使用绝对路由。因为 add_to_waitlist 方法与 eat_at_restaurant 定义在同一个 crate 中，所以可以使用 crate 关键字启动绝对路由。

在 crate 之后，我们包含每个后续模块，直到到达 add_to_waitlist。想象一个具有相同结构的文件系统，我们将从 /front_of_houses/hostings/add_to_waitlist 路径执行 add_to_waitlist 程序；使用 crate 名称从 crate 根开始与在 shell 中使用从文件系统根开始类似。

第二次在 eat_at_restaurant 中运行 add_to_waitlist 时，我们使用了一个相对路径。路径以 front_of_houses 开头，模块名在模块树中与 eat_at_restaurant 声明在同一层。在本例中，文件系统的等效内容是 front_of_houses/hostings/add_to_waitlist。如果路径以名称开头，则路径是相对的。

The decision should be based on whether we're more likely to relocate item definition code independently from code that utilizes the item or together.

我们必须根据我们的项目来决定是使用相对路径还是绝对路由。这个决定应该基于我们是更可能独立地重新定位项定义代码，还是更可能与利用项的代码一起重新定位。如果我们将 front_of_houses 模块和 eat_at_restaurant 函数移动到名为 customer_experience 的模块中，则需要调整 add_to_waitlist 的绝对路由，但相对路径仍然可以接受。

如果我们将 eat_at_restaurant 函数重新定位到名为 eating 的模块中，则到 add_to_waitlist 调用的绝对路由将保持不变，但需要调整相对路径。我们使用绝对路由是因为代码定义和项调用更可能彼此独立地移动。

让我们看看能否构建代码并找出它不能工作的原因。

```
D:\workspace\rust-works\restaurant> cargo build
Compiling restaurant v0.1.0 (D:\workspace\rust-works\restaurant)
error[E0603]: module `hostings` is private
```

```

--> src\lib.rs:8:29
8 |     crate::front_of_houses::hostings::add_to_waitlist();
  |                                ^^^^^^^^ private module
note: the module `hostings` is defined here
--> src\lib.rs:2:5
2 |     mod hostings {
  |     ^^^^^^^^^^^^^

error[E0603]: module `hostings` is private
--> src\lib.rs:10:22
10 |     front_of_houses::hostings::add_to_waitlist();
   |                      ^^^^^^^^ private module
note: the module `hostings` is defined here
--> src\lib.rs:2:5
2 |     mod hostings {
   |     ^^^^^^^^^^^^^

```

根据上面的错误消息提示，模块 `hostings` 是私有的。换句话说，`hostings` 模块和 `add_to_waitlist` 函数有正确的位置，但 Rust 不让我们使用它们，因为它不能访问私有部分。

模块的用途不仅仅是构建代码。它们还定义了 Rust 的隐私边界（`privacy border`），它是一个界限，包含了在 Rust 中不允许外部代码知道、调用或依赖的实现细节。因此，如果我们想要一个函数或结构是私有的，我们必须把它放在一个模块中。

在 Rust 中，默认情况下对所有事物（函数、方法、结构、枚举、模块和常量）强制执行 `privacy` 私有性。父模块中的项不能访问子模块中的私有内容，而子模块可以访问祖先模块中的项。尽管子模块封装并隐藏了它们的实现细节，但它们可以观察定义它们的上下文，情况就是这样。把隐私规则（`privacy rule`）想象成一家餐厅的后台办公室：那里发生的事情是餐厅顾客的隐私，但办公室经理可以看到并做他们经营的餐厅里的一切。

Rust 以这种方式设计模块系统，使其默认隐藏核心实现细节。我们将能够知道我们可以在不破坏外部代码的情况下修改内部代码的哪些部分。然而，使用 `pub` 关键字使对象公开，我们可以将子模块代码的内部部分公开给外部祖先模块。

用 `pub` 关键字公开路径

回到上面程序中的错误，`hostings` 模块是私有的；我们再看一遍。我们希望父模块的 `eat_at_restaurant` 函数访问子模块的 `add_to_waitlist` 方法，因此我们使用 `pub` 关键字指定 `hostings` 模块，如下所示：

```

mod front_of_houses {
    pub mod hostings {
        fn add_to_waitlist() {}
    }
}
pub fn eat_at_restaurant() {
    // 绝对路径
    crate::front_of_houses::hostings::add_to_waitlist();
    // 相对路径
    front_of_houses::hostings::add_to_waitlist();
}

```

遗憾的是，代码构建时仍然报错。

LQ 注：原文中作者把 route 和 path 两个单词进行了混用，一会儿使用“absolute route”，一会儿使用“relative path”，其本意想表达的是模块内的绝对路径和相对路径。故我统一翻译为“路径”。

当你在 mod hosting 前面使用 pub 关键字时，模块就会变成公共的。有了这个更新后，如果我们可以访问 front_of_houses 模块，那么我们就可以访问 hostings 模块。另一方面，hostings 模块的内容是私有的；公开 hostings 模块并不会公开 hostings 的内容。模块的 pub 关键字只允许其祖先模块中的代码引用它。

根据前面的代码，add_to_waitlist 方法是私有的，如生成的错误所示。隐私法（privacy law）涵盖结构、枚举、函数、方法和模块。

在 add_to_waitlist 函数的定义之前添加 pub 关键字，使它成为公共的。

```

mod front_of_houses {
    pub mod hostings {
        pub fn add_to_waitlist() {}
    }
}
pub fn eat_at_restaurant() {
    // 绝对路径
    crate::front_of_houses::hostings::add_to_waitlist();
    // 相对路径
    front_of_houses::hostings::add_to_waitlist();
}

```

代码现在可以通过编译了。让我们看看绝对路径和相对路径，看看为什么在 add_to_waitlist 中使用 pub 关键字允许我们在遵守隐私限制的情况下使用这些路径。

我们从绝对路径中的 crate（crate 模块树的根）开始。然后，在 crate 根中定义 front_of_houses 模块。尽管 front_of_houses 模块不是公共的，但是我们可以从 eat_at_restaurant 中引用它，因为 eat_at_restaurant 函数和 front_of_houses 定义在同一个模块中（也就是说，eat_at_restaurant 和 front_of_houses 是兄弟关系）。接下来是 hostings 模块，使用关键字 pub 修饰。我们可以转到 hostings 的父模块，

它允许我们访问 `hostings`。最后，`add_to_waitlist` 函数被标记为 `pub`，它的父模块可以访问；因此，这个函数调用是有效的！

相对路径遵循与绝对路径相同的推理，除了第一步，它从 `front_of_houses` 而不是 `crate` 根开始。因为 `front_of_houses` 模块与 `eat_at_restaurant` 在相同的模块中定义，所以从 `eat at restaurant` 模块开始的相对路径是有效的。剩下的路径可以正常工作，这个函数调用是有效的，因为 `hosting` 和 `add_to_waitlist` 都被标记为 `pub`。

用 `super` 开始相对路径

我们还可以在路径的开始处使用 `super` 来创建从父模块开始的相对路径。这与在文件系统路径中使用 `..` 用于开始的语法相同。我们为什么要这么做呢？

考虑下面的代码，它模拟了厨师纠正错误的订单并亲自将其交付给客户的情况。通过指定以 `super::` 开头的服务顺序的路径，该方法修复了调用函数 `serve_order` 的错误顺序。

```
fn serve_order() {}
mod back_of_houses {
    fn fix_incorrect_order() {
        cook_order();
        super::serve_order();
    }
    fn cook_order() {}
}
```

我们可以使用 `super` 转到 `back_of_houses` 的父模块，即 `crate`，也即根模块，因为 `fix_incorrect_order` 方法在该模块中。然后搜索 `server_order` 并定位它，成功！如果我们决定重新组织 `crate` 的模块树，我们相信 `back_of_houses` 模块和 `serve_order` 函数将继续保持相同的关系并一起移动。因此，如果将来代码被重新定位到不同的模块中，我们使用 `super` 来减少更新代码的位置。

让结构和枚举公开

我们还可以使用 `pub` 将结构和枚举设置为公开的，但还有一些额外的考虑。当我们在结构定义之前使用 `pub` 时，结构将变成公开的，但结构中的字段仍然是私有的。以具体分析为基础，我们可以使每个字段公开或私有。在下面的例子中，我们定义了一个公共 `back_of_houses::Breakfast` 结构体，它有一个公共 `toast` 字段，但有一个私有 `seasonal_fruit` 字段。这类似于在餐馆里，顾客可以选择面包的种类，但厨师根据当季和可用的水果来选择与之配套的水果。

顾客无法选择或者甚至看不到他们将收到的水果，因为可用的水果经常变化。

```
mod back_of_houses {
    pub struct Breakfast {
        pub toast: String,
        seasonal_fruit: String,
```

```

    }
    impl Breakfast {
        pub fn summer(toast: &str) -> Breakfast {
            Breakfast {
                toast: String::from("bread"),
                seasonal_fruit: String::from("grapes"),
            }
        }
    }
}

pub fn eat_at_restaurant() {
    // 在夏天点一份有黑麦吐司的早餐
    let mut meal = back_of_houses::Breakfast::summer("Rye");
    // 改变我们想吃什么面包的主意
    meal.toast = String::from("brown");
    println!("I would like {} toast", meal.toast);
}

```

因为 `back_of_houses::Breakfast` 结构中的 `toast` 字段是公共的，所以我们可以可以在 `eat_at_restaurant` 中使用点表示法对其进行写入和读取。我们不能在 `eat_at_restaurant` 中使用 `seasonal_fruit` 字段，因为它属于其他人。要检查我们得到了什么错误，取消注释修改 `seasonal_fruit` 字段值的行。

当我们将一个枚举设为 `public` 时，它的所有变体都变成 `public`。如下所示，`pub` 关键字只需要在 `enum` 关键字之前。

```

mod back_of_houses {
    pub enum Appetizer {
        Kebab,
        Noodles,
    }
}

pub fn eat_at_restaurant() {
    let order1 = back_of_houses::Appetizer::Kebab;
    let order2 = back_of_houses::Appetizer::Noodles;
}

```

我们可以在 `eat_at_restaurant` 中使用 `Kebab` 和 `Noodles` 变量，因为我们将 `Appetizer` 枚举设置为公共的。枚举变体除非是公开的，否则是没有好处的；在每种情况下用 `pub` 注释每个枚举变体是不方便的；因此，公共是默认的。因为结构字段通常在不为公共的情况下是有用的，所以它们遵循一般规则，默认情况下所有内容都是私有的，除非用 `pub` 注释。

我们还没有讨论使用 `pub` 的另一个场景：`use` 关键字，这是我们的最后一个模块系统特性。我们将从 `use` 本身开始，然后说明如何混合使用 `pub` 和 `use`。

使用 `use` 关键字将路径引入作用域

到目前为止，我们编写的调用函数的路径可能看起来很长且重复。例如，在前面的示例中，我们使用 `add_to_waitlist` 函数的绝对或相对路径；每次调用 `add_to_waitlist` 时，我们必须指定 `front_of_houses` 和 `hostings`。有一种方法可以使事情更简单，使用 `use` 关键字，我们可以将一个路径引入作用域一次，然后将该路径中的函数调用都视为本地项。

在下面的例子中，我们将 `crate::front_of_houses::hostings` 模块引入到 `eat_at_restaurant` 函数的范围内，因此我们只需提供 `hostings::add_to_waitlist` 在 `eat_at_restaurant` 函数中使用 `add_to_waitlist` 方法。

```
mod front_of_houses {
    pub mod hostings {
        pub fn add_to_waitlist() {}
    }
}

use crate::front_of_houses::hostings;

pub fn eat_at_restaurant() {
    hostings::add_to_waitlist();
    hostings::add_to_waitlist();
    hostings::add_to_waitlist();
}
```

添加 `use` 和一个路径与在作用域中的文件系统中创建符号链接类似。因为 `use crate::front_of_houses::hostings` 被添加到 `crate` 根目录中，所以 `hostings` 现在是该作用域中的有效名称，就像 `hostings` 模块在那里声明了一样。像所有其他路径一样，路径引入了使用检查隐私的范围。

`use` 和相对路径也可以将项带入作用域。下面的程序演示了如何定义一个相对路由来实现与上面的程序相同的结果。

```
mod front_of_houses {
    pub mod hostings {
        pub fn add_to_waitlist() {}
    }
}

use self::front_of_houses::hostings;

pub fn eat_at_restaurant() {
    hostings::add_to_waitlist();
    hostings::add_to_waitlist();
    hostings::add_to_waitlist();
}
```

创建合乎习惯的 use 路径

我们可能想知道为什么在前面的代码中，我们使用了 `use crate::front_of_houses::hostings`，然后在 `eat_at_restaurant` 中使用了 `hostings::add_to_waitlist`，而不是使用 `add_to_waitlist` 函数的使用路径。

```
mod front_of_houses {
    pub mod hostings {
        pub fn add_to_waitlist() {}
    }
}

use self::front_of_houses::hostings::add_to_waitlist;

pub fn eat_at_restaurant() {
    add_to_waitlist();
    add_to_waitlist();
    add_to_waitlist();
}
```

尽管前面的示例实现了相同的目标，但这是将函数引入使用范围的惯用方法。当使用将函数的父模块放入作用域时，必须在调用函数时指定父模块。*在调用函数时，指定父模块可以明显地表明函数不是本地的，同时还可以减少整个路径的重复次数。*

但是，当引入结构、枚举和其他 `use` 的东西时，指定完整路径是惯用手段。下面的代码演示了如何以惯用的方式将 `HashMap` 结构从标准库引入到二进制 `crate` 的作用域中。

```
use std::collections::HashMap;

fn main() {
    let mut map1 = HashMap::new();
    map1.insert(2, 1);
}
```

这种习惯用法没有令人信服的理由：它只是一种已经形成的编码习惯，`Rust` 开发者已经习惯了以这种方式阅读和创建 `Rust` 代码。

只有当我们使用语句将两个具有相同名称的项引入作用域时，才会被破坏这种习惯用法，而 `Rust` 不允许这样做。下面的示例演示如何作用域名称相同但父模块不同的两个 `Result` 类型，以及如何引用它们。

```
use std::fmt;
use std::io;
fn function1() -> fmt::Result {}
fn function2() -> io::Result<()> {}
```


正如我们所见，这两种 `Result` 类型是通过使用父模块来区分的。我们可以使用 `std::io::Result` 而不是 `std::fmt::Result`。如果我们使用 `Result`，我们将在相同的作用域中有两个 `Result` 类型，Rust 将不知道我们指的是哪一个。

通过 `as` 关键字提供新命名

解决将两个具有相同名称的类型带入相同作用域的另一种方法是为路径之后的类型指定 `as` 和一个新的本地名称或别名。下面的示例展示了一种编写上述代码的不同方法，它使用 `as` 重命名其中一个 `Result` 类型。

```
use std::fmt::Result;
use std::io::Result as IoResult;
fn function1() -> Result {}
fn function2() -> IoResult<()> {}
```

我们在第二个 `use` 语句中为 `std::io::Result` 类型选择了新名称 `IoResult`，这样它就不会与我们也引入到作用域中的 `std::fmt` 的 `Result` 发生冲突。

使用 `pub use` 再导出命名

当我们使用 `use` 关键字将名称引入作用域中时，新作用域中出现的名称默认是私有的。我们可以混合使用 `pub use`，以允许代码调用我们的代码时能引用该名称，就好像它是在该代码的作用域中定义的一样。这种方法称为再导出（re-exporting），因为我们将一个项带入作用域，并使其他人可以将其带入他们的作用域。

```
mod front_of_houses {
    pub mod hostings {
        pub fn add_to_waitlist() {}
    }
}

pub use crate::front_of_houses::hostings;

pub fn eat_at_restaurant() {
    hostings::add_to_waitlist();
    hostings::add_to_waitlist();
    hostings::add_to_waitlist();
}
```

利用 `pub` 的用法，外部代码现在可以使用 `hostings::add_to_waitlist` 调用 `add_to_waitlist` 方法。如果我们没有指定 `pub` 用法，尽管 `eat_at_restaurant` 函数可以在它的作用域中调用 `hostings::add_to_waitlist`，但是外部代码却不能利用这个新路径。

当我们的代码的内部结构与调用您的代码的程序员考虑作用域的方式不同时，再导出是有用的。例如，在上面餐厅的比喻中，负责餐厅的人会考虑“前厅”和“后厅”。另一方面，光顾一家餐厅的顾客不太可能从这些方面来考虑餐厅的组成部分。我们

可以用一种结构写代码，但用 `pub` 的用法揭示另一种结构。因此，我们的库组织得很好，对开发它的程序员和调用它的程序员来说都是好事。

将模块分离到不同的文件中

到目前为止，本章中上面的例子都是在一个文件中声明了几个模块。为了使代码更容易浏览，我们可能需要将大模块的定义转移到另一个文件中。

从前面的代码开始，我们可以将 `front_of_houses` 模块重新定位到它的文件 `src/front_of_house.rs`，通过修改 `crate` 根文件来包含下面的代码。在本例中，`crate` 根文件是 `src/lib`。但是这种方法也适用于使用 `src/main.rs` 作为 `crate` 根文件的二进制 `crate`。

```
// src/lib.rs
mod front_of_houses;

pub use crate::front_of_houses::hostings;

pub fn eat_at_restaurant() {
    hostings::add_to_waitlist();
    hostings::add_to_waitlist();
    hostings::add_to_waitlist();
}
```

此外，还有 `src/front_of_houses.rs`。如前所述，Rust 从 `front_of_houses` 模块的主体中获取定义。

```
// src/front_of_houses.rs
pub mod hostings {
    pub fn add_to_waitlist() {}
}
```

不是在 `mod front_of_houses` 后面使用块，而是用分号通知 Rust 从同名的另一个文件加载模块的内容。为了继续我们的示例并将 `hostings` 模块移到它的文件中，我们更新 `src/front_of_houses.rs` 只包含 `hostings` 模块的声明：

```
// src/front_of_houses.rs
pub mod hostings;
```

然后我们创建一个 `src/front_of_houses` 目录和一个 `src/front_of_houses/hostings.rs` 文件，用来保存 `hostings` 模块的定义：

```
// src/front_of_houses/hostings.rs
pub fn add_to_waitlist() {}
```

即使定义在不同的文件中，模块树也保持不变，并且在 `eat_at_restaurant` 中的函数调用将不用进行修改即可正常工作。随着模块大小的增长，您可以使用此技术将它们重新定位到新文件中。

值得注意的是，在 `src/lib.rs` 中 `pub use crate::front_of_houses::hostings` 语句并没有改变，使用也没有改变，这与作为 `crate` 的一部分编译哪些文件没有关系。`mod` 关键字声明模块，`Rust` 在与模块同名的文件中查找进入该模块的代码。

本章涵盖了包和 `crate`，定义控制作用域和隐私的模块，引用模块树中项的路径，以及使用 `use` 关键字将路径引入作用域。我们还讨论了将模块分离到不同的文件中。

C7: 错误处理

- 2022.11.16

在本章中，

- 不可恢复的错误用 `panic!`
- 可恢复的错误用 `Result`

在前一章中，我们讨论了使用包、`crate` 和模块管理不断增长的项目。在本章中，我们将讨论错误处理，讨论不可恢复的错误用 `panic!` 和可恢复错误用 `Result`。

错误处理

`Rust` 对可靠性的贡献也体现在错误处理上。错误是软件开发中不可避免的一部分；因此，`Rust` 提供了几种处理它们的方法。在编译代码之前，`Rust` 通常需要我们认识到潜在的错误并采取行动。这个标准保证在我们的代码投入生产之前发现错误并有效地处理错误，从而提高了我们程序的健壮性。

`Rust` 将错误分为两种：可恢复的错误和不可恢复的错误。恰当的做法是将可恢复的错误（比如文件未找到错误）通知用户，然后重试操作。对于不可恢复的错误，例如试图访问数组末端以外的位置，总会报错。

大多数编程语言并不区分这两种类型的问题，而是使用异常等技术以相同的方式对待它们。`Rust` 也不例外。但是，`Rust` 提供了 `panic!` 宏，当程序遇到不可恢复的错误时停止执行，类型为 `Result` 则表示可恢复错误。本章从 `panic!` 开始，然后继续返回 `Result` 值。我们还将考虑在选择是尝试从错误中恢复还是终止执行时要考虑的因素。

LQ 注: `panic` 意为“恐慌”，我思考再三，还是决定不译最佳。

不可恢复的错误用 `panic!`

不好的事情在我们的代码中不时发生，我们对此无能为力。`Rust` 提供了 `panic!` 宏来应对某些情况。我们的程序将打印一个失败消息，展开（`unwind`）和清理栈，然后在 `panic!` 宏运行时终止。这种情况通常发生在发现 `bug` 而程序员不确定如何处理错误的情况下。

撤消栈或中止栈以响应 `panic`

默认情况下，当出现 `panic` 时，程序会展开，这意味着 `Rust` 会沿着栈向上走，清理它遇到的每个函数的数据。然而，返回和清理会涉及很多工作。另一种选择是立即停止程序，这将在没有清理的情况下结束它。然后由操作系统来清理程序的内存。如果我们需要在项目中让最终的二进制文件尽可能小，我们可以通过在项目的 `Cargo.toml` 配置文件的 `[profile]` 节添加 `panic = 'abort'` 来关闭当出现 `panic` 时从展

开到中止的过程。例如，如果我们想要在 `release` 模式中出现 `panic` 时中止，我们可以添加以下内容：

```
[profile.release]
panic = 'abort'
```

让我们在一个简单程序中建立一个 `panic!` 调用：

```
fn main() {
    panic!("crash & burn");
}
```

当运行这个程序时，可以看到如下内容：

```
thread 'main' panicked at 'crash & burn', src/main.rs:2:5
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

`panic!` 导致出现两行错误消息。我们的 `panic` 消息和源代码中发生 `panic` 的位置显示在第一行，“`src/main.rs:2:5`”表示 `src/main.rs` 中第二行的第五个字符。

在这种情况下，我们可以看到 `panic!` 宏调用，如果运行到那一行。在其他情况下，`panic!` 宏可能在代码调用的代码中调用，错误消息显示的文件名和行号将是其他的代码，而不是最终导致 `panic` 的代码行。我们可以利用 `panic` 的回溯追踪调用的方法来找出导致问题的代码部分。接下来，我们将更深入地讨论什么是回溯。

用 `panic!` 回溯

让我们看看另一个场景，看看我们在 `panic` 时是什么感觉，因为我们的代码有漏洞。在本例中，调用来自库，而不是直接接触宏的代码。有些代码试图通过索引检索程序中 `vector` 中的元素。

```
fn main() {
    let v1 = vec![1,2,3];
    v1[99];
}
```

我们试图找到向量的第 100 个元素（索引号为 99，因为索引从 0 开始），但是向量实际上只有 3 个元素。Rust 在这种情况下会发生 `panic`。虽然 `[]` 应该返回一个元素，但如果我们给出一个无效的索引，Rust 就不能返回正确的元素。

在 C 语言中，读取数据结构末端以外的内容是未定义的行为。即使内存不属于该结构，我们也可以检索与数据结构中该元素对应的内存位置上的任何内容。这被称为缓冲区超读（`buffer overread`），如果攻击者可以更改索引，以便读取数据结构保存后不允许读取的数据，就会导致安全缺陷。

如果我们试图读取索引上不存在的元素，Rust 将停止执行并拒绝继续，从而保护我们的应用程序免受这种类型的漏洞。

```
thread 'main' panicked at 'index out of bounds: the len is 3 but the index is 99', main.rs:3:5
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

当我们尝试访问 main.rs 的第 3 行上的索引号 99 时，会出现这个问题。可以设置环境变量 RUST_BACKTRACE=1，根据下一个注释行获取触发问题的具体事件的反向跟踪。回溯记录列出了到达这一点所用的所有函数。Rust 的回溯功能与其他语言中的回溯功能相似：读取回溯的关键是从头开始读，直到看到创建的文件。那就是问题开始的地方。

上面和下面提到文件的行是由我们的代码调用的代码，下面的行是由我们的代码调用的代码。这些行可以包含 Rust 核心代码、标准库代码或我们正在使用的 crate。将 RUST_BACKTRACE 环境变量设置为 0 以外的任意值，以查看是否可以获得反向跟踪。

```
D:\workspace\rust-works> set RUST_BACKTRACE=2
D:\workspace\rust-works> rustc main.rs
D:\workspace\rust-works> main
thread 'main' panicked at 'index out of bounds: the len is 3 but the index is 99', main.rs:3:5
stack backtrace:
 0: std::panicking::begin_panic_handler
    at /rustc/a55dd71d5fb0ec5a6a3a9e8c27b2127ba491ce52/library\std\src\panicking.rs:584
 1: core::panicking::panic_fmt
    at /rustc/a55dd71d5fb0ec5a6a3a9e8c27b2127ba491ce52/library\core\src\panicking.rs:142
 2: core::panicking::panic_bounds_check
    at /rustc/a55dd71d5fb0ec5a6a3a9e8c27b2127ba491ce52/library\core\src\panicking.rs:84
 3: core::slice::index::<impl core::ops::index::Index<I> for [T]>::index
 4: core::slice::index::<impl core::ops::index::Index<I> for [T]>::index
 5: <alloc::vec::Vec<T,A> as core::ops::index::Index<I>>::index
 6: <() as std::process::Termination>::report
 7: core::ops::function::FnOnce::call_once
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose backtrace.
```

输出的信息很多。根据我们的操作系统和 Rust 版本的不同，输出的准确信息可能会有所不同。必须启用调试符号才能使用此信息获取回溯。当运行 cargo build 或 bargo run 且不带 ——release 标志时，如我们上面所述，调试符号默认是启用的。

输出信息的一开始就给出了导致出现问题的代码行：main.rs 的第 3 行。如果不想让程序 panic，我们应该开始查看第一行所指示的位置，其中提到了我们创建的文件。在前面的代码中，我们有目的地构建了解释如何使用回溯的代码，纠正这种恐

慌的解决方案是不向只包含三个元素的 `vector` 请求索引 99 处的元素。如果我们的代码再次出现恐慌，我们需要弄清楚它执行什么操作，用什么值来产生 `panic`，以及它应该做什么。

我们返回到 `panic!`，以及什么时候应该和不应该使用它来处理错误场景，请参见本章的第 3 节。同时，我们将了解如何使用 `Result` 从错误中恢复。

可恢复的错误用 `Result`

大多数错误都不会严重到导致软件完全关闭。当一个函数失败时，其原因可能很容易理解和响应：如果我们试图访问一个文件，但由于该文件不存在而失败，那么与其终止进程，不如创建该文件。

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

`T` 和 `E` 是泛型类型参数，我们将在后面深入地讨论它们。现在只需知道，`T` 代表成功场景中返回的 `Ok` 变体的值类型，`E` 代表失败事件中接收到的 `Err` 变体错误类型。由于 `Result` 包含了这些通用类型参数，所以在希望返回的成功值和错误值不同的各种情况下，我们可以使用它和标准库中提供的函数。

因为函数可能会失败，所以我们将其命名为返回 `Result` 值的函数。我们试着打开一个文件。

```
use std::fs::File;  
fn main() {  
    let f1 = File::open("helloo.txt");  
}
```

我们有什么证据证明 `File::open` 返回一个结果？我们可以参考标准库的 API 文档，也可以直接询问编译器。如果我们为 `f1` 提供一个不是函数返回类型的类型注释，然后尝试编译代码，编译器将报类型不匹配的错误。然后，错误消息将通知我们 `f` 类型。让我们试一试！因为我们知道 `File::open` 的返回类型不是 `u32`，我们将把 `let f1` 语句更新为：

```
let f1: u32 = File::open("helloo.txt");
```

当编译源码时，编译器将报如下错误：

```
error[E0308]: mismatched types  
--> r3.rs:3:19  
3 |         let f1: u32 = File::open("helloo.txt");  
  |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ expected `u32`, found enum `Result`  
um `Result`
```



```
|         expected due to this
|
```

这表明 `File::open` 函数的返回类型是 `Result`。成功值的类型 `std::fs::File` 是一个文件句柄，已经为泛型参数 `T` 填充了。`std::io::Error` 则是错误值的类型。

这个返回类型表明 `File::open` 方法可能成功并返回一个文件句柄，我们可以通过它进行读或写；函数调用也可能会失败，因为文件不存在或者我们没有访问它的权限。`File::open` 函数必须能够指示它是成功还是失败，并提供文件句柄或错误信息。`Result` 枚举给出的正是这一信息。

如果 `File::open` 成功，变量 `f1` 的值将是一个包含文件句柄的 `Ok` 实例。如果失败，`f1` 的值将是 `Err` 的一个实例，包含关于所发生错误类型的附加详细信息。

我们必须添加到上述代码中，以根据值执行不同的操作。命令 `File::open` 返回。如下面的示例所示，匹配表达式是一种可以用于处理 `Result` 的简单技术。

```
use std::fs::File;
fn main() {
    let f = File::open("helloo.txt");
    let f1 = match f {
        Ok(file) => file,
        Err(error) => panic!("Problem opening file: {:?}", error),
    };
}
```

序言（**preamble**）将 **Result** 枚举及其变体带入了作用域，就像 **Option** 枚举一样，所以我们不需要在匹配臂中的 **Ok** 和 **Err** 变体之前提到 **Result::**。

当结果为 `Ok` 时，我们指示 `Rust` 从 `Ok` 变量返回内部文件值，然后将该文件句柄值添加到变量 `file`。匹配后，我们可以通过文件句柄进行读写。

匹配的另一部分处理 `File::open` 返回 `Err` 值的情况。我们选择 `panic!` 宏。当我们运行这段代码时，如果当前目录中没有名为 `helloo.txt` 文件时，则 `panic!` 宏将提供以下输出：

```
thread 'main' panicked at 'Problem opening file: 0s { code: 2, kind: No
tFound, message: "系统找不到指定的文件。" }', r3.rs:6:23
note: run with `RUST_BACKTRACE=1` environment variable to display a bac
ktrace
```

这个输出像往常一样，向我们准确地展示了哪里出了问题。

上面的代码会导致 **panic!**，而不管 `File::open` 失败的原因。相反，我们希望根据失败的原因执行不同的操作：如果 `File::open` 因为文件不存在而失败，我们希望创建它并将句柄返回给新文件。如果 `File::open` 因为任何其他原因而失败，因为我们没

有打开文件的权限，那么我们仍然希望代码能够导致 `panic`。请看下面的代码，其中包含一个内部匹配表达式。

```
use std::fs::File;
use std::io::ErrorKind;
fn main() {
    let f = File::open("helloo.txt");
    let f1 = match f {
        Ok(file) => file,
        Err(error) => match error.kind() {
            ErrorKind::NotFound => match File::create("helloo.txt") {
                Ok(fc) => fc,
                Err(e) => panic!("Problem opening file: {:?}", e),
            },
            other_error => {
                panic!("Problem opening file: {:?}", other_error)
            }
        },
    };
}
```

`File::open` 返回的 `Err` 变体值的类型为 `io::Error`，是标准库给出的结构体。我们可以通过调用该结构的 `kind` 方法来检索 `io::ErrorKind` 值。标准库提供了枚举 `io::ErrorKind`，它有描述 `io` 操作期间可能发生的多种错误类型的变体。我们希望使用的 `ErrorKind::NotFound` 变体表明我们试图打开的文件还不存在。所以我们在 `f1` 上有一个外部匹配，但在 `error.kind()` 上有一个内部匹配。

我们想看看内部的错误返回的值是否匹配。`ErrorKind` 枚举的 `kind` 方法是 `NotFound` 变体。如果是这种情况，则使用 `File::create` 创建文件。我们需要在内部匹配表达式中添加第二个分支，因为 `File::create` 也可能失败。如果文件无法创建，则会显示一个单独的错误消息。因为外部匹配的第二臂保持不变，所以当程序遇到除丢失文件错误以外的任何错误时，它就会导致 `panic`。

这是一个比较大的匹配！匹配表达式非常有用，但它也是一个非常原始的表达式。

错误中的 `panic` 的快捷方式：展开（`unwrap`）并期待（`expect`）

匹配运行良好，尽管它有点啰嗦，但并不总是能清楚地解释意图。在 `Result` 类型上定义了许多辅助器方法来执行各种操作。其中一个方法是 `unwrap`，它是一种简写方法，其工作原理类似于我们前面编写的匹配表达式。如果 `Result` 值是 `Ok` 变体，`unwrap` 将返回 `Ok` 中的值。如果 `Result` 值是 `Err` 变体，`unwrap` 会调用 `panic!` 宏。下面是一个 `unwrap` 如何工作的例子：

```
use std::fs::File;
fn main() {
    let f1 = File::open("helloo.txt").unwrap();
}
```

如果我们在没有 `helloo.txt` 文件的情况下运行此代码，我们会收到 `panic!` 宏的错误信息。`unwrap()` 方法生成以下调用信息：

```
thread 'main' panicked at 'called `Result::unwrap()` on an `Err` value:
  Os { code: 2, kind: NotFound, message: "系统找不到指定的文件。" }',
r3.rs:3:39
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

另一个方法 `expect` 与 `unwrap` 类似，它允许我们选择 `panic!` 宏作为错误通知。使用 `expect` 而不是 `unwrap` 并提供明确的错误消息可能有助于我们传达我们的目标并找到 `panic` 的根源。`expect` 具有以下语法：

```
use std::fs::File;
fn main() {
    let f1 = File::open("helloo.txt").expect("打开 helloo.txt 文件失败");
}
```

`expect` 与 `unwrap` 类似，它返回文件句柄或调用 `panic!` 宏。传递给 `expect` 的参数将是 `expect` 在调用 `panic!` 宏输出的错误消息，而不是 `panic!` 宏默认用于展开的错误消息。它是这样出现的：

```
thread 'main' panicked at '打开 helloo.txt 文件失败: Os { code: 2, kind:
NotFound, message: "系统找不到指定的文件。" }', r3.rs:3:39
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

由于此错误消息以“打开 `helloo.txt` 文件失败”开头，因此更容易确定此错误消息在代码中产生的位置。因为所有的展开都调用 `panic!` 打印相同的消息，所以如果在多个地方使用展开，则需要更长的时间来确定哪个展开产生了 `panic`。

传播错误

与其处理函数内的错误，不如将其传递给调用代码，以便在开发实现调用可能失败的函数时决定如何处理。这被称为传播错误（`propagating errors`），它给了调用代码更大的能力，因为调用代码可能比我们的代码有更多的信息或逻辑来指示应该如何处理问题。

下面的示例描述了一个从文件中读取用户名的函数。如果文件不存在或无法读取，此函数将向调用它的代码返回错误。

```
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let f = File::open("helloo.txt");
    let mut f1 = match f {
        Ok(file) => file,
```

```

        Err(e) => return Err(e),
    };
    let mut s1 = String::new();
    match f1.read_to_string(&mut s1) {
        Ok(_) => Ok(s1),
        Err(e) => Err(e),
    }
}

```

尽管这个函数可以用更简短的方式来编写，但我们将从大量手动操作开始，以便学习错误处理，然后在最后展示更简洁的写法。首先，让我们看看函数的返回类型 `Result<String, io::Error>`，一个包含 `io::Error` 的字符串。这表示该函数返回一个 `Result<T,E>` 类型的值，泛型参数 `T` 用具体类型 `String` 填充，泛型类型 `E` 则用具体类型 `io::Error` 填充。如果此函数成功完成，那么调用它的代码将收到一个 `Ok` 值，该值包含一个 `String`，其中包含此函数从文件中检索到的用户名。

如果该方法遇到任何问题，调用它的代码将收到一个包含 `io::Error` 实例的 `Err` 值。关于这些问题的更多细节可以在错误消息中找到。我们选择 `io::Error` 作为此函数的返回类型，因为这是我们在这个函数体中调用的两个可能失败的操作：`read_to_string` 方法和 `File::open` 函数。

函数体以调用 `File::open` 函数开始。然后，取代之前的调用 `panic!` 宏，我们用一个类似于前面代码中的匹配来处理 `Result` 返回值。在 `Err` 场景中，我们提前退出该方法，并从 `File::open` 向调用代码返回错误值作为此过程的错误值。我们将文件句柄保留在变量 `f1` 中，如果 `file::open` 成功则继续。

然后，在 `f1` 中的文件句柄上使用 `read_to_string` 函数，在变量 `s1` 中创建一个新的 `String` 并将文件的内容读入 `s1`。因为 `read_to_string` 方法可能会失败，即使 `File::open` 成功，它也会返回一个 `Result`。因此我们需要另一个匹配来处理 `Result`：如果 `read_to_string` 成功，则我们的函数成功，我们从文件 `s1` 中返回用户名，它被包装在 `Ok` 中；如果读取字符串失败，则返回错误值的方式与处理 `File::open` 返回错误值的匹配相同。但是，因为这是函数中的最后一个表达式，所以不需要显式指定 `return`。

然后，运行此过程的代码将处理获取包含用户名的 `Ok` 值或获取包含 `io::Error` 的 `Err` 值。我们不知道调用代码将使用这些值做什么。如果调用代码接收到 `Err` 值，它可能会发出 `panic!` 信号。例如，为了避免软件崩溃，使用默认用户名或从文件以外的其他地方搜索用户名。我们没有足够的知识来弄清楚调用代码试图做什么，所以我们将所有的成功和错误信息发送给它来处理。

由于这种传播错误模式（`propagating error pattern`）在 `Rust` 中非常普遍，问号运算符 `?` 为其提供了更简洁的操作方式。

传播错误的快捷方式：? 问号运算符

下面的 `read_username_from_file` 函数实现了与上一个示例相同的功能，不同之处在于它使用了 ? 问号运算符。

```
use std::fs::File;
use std::io;
use std::io::Read;

fn read_username_from_file() -> Result<String, io::Error> {
    let mut f1 = File::open("helloo.txt"?);
    let mut s1 = String::new();
    f1.read_to_string(&mut s1)?;
    Ok(s1)
}
```

? 跟随在 `Result` 值之后，定义的行为类似于我们之前创建的处理 `Result` 值的匹配表达式。如果 `Result` 值是 `Ok`，该表达式将返回 `Ok` 中的值，程序将继续执行。如果值是 `Err`，则 `Err` 将从整个函数返回，就像使用了 `return` 关键字一样，允许将错误值传播到调用代码。

匹配表达式和 ? 运算符的区别：有 ? 号运算符的调用，错误值通过 `from` 函数进行传递，该函数在标准库的 `From trait` 中定义，用于将错误从一种类型转换为另一种类型。当 `from` 函数通过 ? 运算符调用时，接收到的错误类型将转换为当前函数的返回类型中定义的错误类型。

当函数返回单一的错误类型来表示所有可能的失败时，这很有用，即使部分可能由于各种原因而失败。? 运算符关心转换的自动化，只要每个错误类型实现了 `from` 函数，可以讲自身转换为返回的错误类型即可。

`Ok` 中的值将通过以 ? 结尾的 `File::open` 调用返回给变量 `f1`。如果发生错误，? 运算符提前退出函数并将任何 `Err` 值返回给调用代码。在以 ? 结尾的 `read_to_string` 调用有相同的效果。

? 运算符去除了大量的样板代码，简化了函数的实现。我们可以通过在 ? 后面链接方法调用来使代码更短。

```
use std::fs::File;
use std::io;
use std::io::Read;
fn read_username_from_file() -> Result<String, io::Error> {
    let mut s1 = String::new();
    File::open("helloo.txt")?.read_to_string(&mut s1)?;
    Ok(s1)
}
```

在 `s1` 中生成新 `String` 的过程被移到了函数的开头；其他一切都没有改变。我们已经把 `read_to_string` 的调用链接到 `File::open("helloo.txt")?` 的结果上而不是创建一

个变量 `f1`。在这行代码的结尾我们还有一个 `?` 运算符，当 `File::open` 和 `read_to_string` 都成功时，我们仍然会返回一个包含 `s1` 中用户名的 `Ok` 值，而不是返回错误。功能与前面的程序相同；只是这次用了一种新的，更符合人体工程学的方法。

下面的例子演示了在使用其他方法创建这个函数时，如何使它更短：

```
use std::fs;
use std::io;
fn read_username_from_file() -> Result<String,io::Error> {
    fs::read_to_string("helloo.txt")
}
```

因为将文件读入字符串是一个典型的任务，Rust 包含 `fs::read_to_string` 函数，该函数打开文件，生成一个新的 `string`，读取文件内容，将它们放入该 `string` 中，然后返回它。但是，使用 `fs::read_to_string` 不允许我们演示所有的错误处理，所以我们之前先用代码冗长的方式来演示它。

? 运算符可用于返回结果的函数

`?` 运算符被定义为与前面代码中定义的匹配表达式相同的方式工作，并且可以在返回类型为 `Result` 的函数中使用。返回 `Err(e)` 是匹配中需要 `Result` 返回类型的部分；因此，函数的返回类型必须是 `Result`，以与此返回一致。

让我们看看如果用 `?` 运算符在 `main` 函数中，它的返回类型为 `()`：

```
use std::fs::File;
fn main() {
    let f1 = File::open("helloo.txt"?);
}
```

在编译此代码时，我们会得到一个错误通知。

此错误表明 `?` 运算符只能用于返回 `Result`、`Option` 或实现 `std::ops::Try` 的其他类型的函数。当在尚未返回这些类型之一的函数中编写代码时，我们希望使用 `?` 运算符。在调用返回 `Result` 的其他函数时，我们有两个选项来解决这个问题。如果没有任何约束，则将函数的返回类型更改为 `Result` 是一个选项。另一种选择是使用匹配或 `Result` 方法之一，以任何有意义的方式处理 `Result`。

`main` 主函数是唯一的，它的返回类型必须遵循一定的准则。`()` 是 `main` 的一个有效返回类型，而 `Result` 是另一个方便的返回类型，如下所示：

```
use std::error::Error;
use std::fs::File;
fn main() -> Result<(), Box<dyn Error>> {
    let f1 = File::open("helloo.txt"?);
    Ok(())
}
```

一个名为 `Box` 类型的 `trait` 对象被调用。目前，您可以将 `Box` 解释为“任何类型的错误”。它是在 `main` 函数中允许使用 `?` 运算符时使用这个返回类型。

让我们回到如何决定在哪些情况下使用哪种方法的问题上，现在我们已经讨论了调用 `panic!` 或返回 `Result` 的细节。

用 `panic!` 或不用 `panic!`

我们怎么知道什么时候该调用 `panic!`，什么时候应该返回 `Result`，什么时候不应该返回 `Result` 呢？当代码出现 `panic` 时，没有恢复机制。如果我们选择调用 `panic!`，我们代表调用代码决定一个场景是不可恢复的，是否有可能的方法恢复。

我们提供调用代码选项，而不是决定何时选择返回 `Result` 值。调用代码可以选择以一种适合该情况的方式进行恢复，或者它可以决定 `Err` 值（在本例中）是不可恢复的，在这种情况下，它可以使用 `panic!` 将可恢复的错误变为不可恢复的错误。因此，在定义可能失败的函数时，返回 `Result` 是一个明智的默认选择。

在某些情况下，最好是编写产生 `panic` 的代码，而不是产生 `Result`。让我们看看为什么在例子、原型代码和测试等情况下 `panic` 是合适的。然后我们将讨论编译器无法识别某些事情是否不可能的场景，但我们可以。本章最后将介绍一些通用规则，以确定在编写库代码时是否 `panic`。

例子，原型代码和测试

在构建演示概念的示例时，包含健壮的错误处理代码会使示例不太容易理解。在示例中，假设对 `unwrap` 等方法的调用可以引起 `panic`，它被用作我们希望应用程序如何处理错误的占位符，这些错误可以根据代码的其余部分而变化。

类似地，在决定如何处理错误之前，`unwrap` 和 `expect` 方法可以用于实验。当我们准备让程序更健壮时，它们会在代码中留下显式的指示符。

在测试中如果一个方法调用失败，那么整个测试也应该失败，即使该方法不是被测试的功能。如果想弄清 `panic!` 是如何将测试报告为失败的，则应该改为调用 `unwrap` 或 `expect`。

我们比编译器拥有更多信息的情况

当我们有一些逻辑保证 `Result` 有一个 `Ok` 值，但编译器不理解这个逻辑时，我们应该调用 `unwrap`。我们仍然必须处理 `Result` 值：我们调用的任何操作通常都有可能失败，即使在您的特定情况下逻辑上是不可能的。如果我们可以手动分析代码来验证我们永远不会出现 `Err` 变化，那么调用 `unwrap` 是完全可以的。这里有一个例子：

```
use std::net::IpAddr;
```

```
let home:IpAddr = "127.0.0.1".parse().unwrap();
```


我们正在处理硬编码文本以创建一个 `IpAddr` 实例。因为可以看到 `127.0.0.1` 是一个合法的 IP 地址，所以在本例中可以使用 `unwrap` 展开。然而，硬编码的有效字符串并不影响解析方法的返回类型：我们仍然获得 `Result` 值，编译器将它视为可能存在 `Err` 变体，因为编译器不够聪明，无法识别该字符串始终是有效的 IP 地址。如果 IP 地址字符串来自用户而不是硬编码到程序中，就有可能失败，那么我们肯定希望更健壮地处理 `Result`。

错误处理的指南

当我们的代码可能处于一个糟糕的状态时，最好让您的代码处于 `panic` 状态。当一个假设、保证、约定或不变量被违反时，就会发生坏状态，例如当向我们的代码提供无效、冲突或缺失的值时，并伴随着以下一个或多个值：

- 坏状态不是经常发生的事情。
- 在这一点之后，我们的代码必须依赖于我们没有处于这种糟糕状态的事实。
- 没有一种很好的方式来编码我们所使用的信息。

如果有人使用我们的代码并传入不合理的值，最好的选择是调用 `panic!` 并将代码中的错误通知使用我们库的人，以便他们可以在开发时进行更正。同样，`panic!` 通常适用于访问我们不控制的外部代码，并且它返回我们不能更改的无效状态。

然而，当预测失败时，返回 `Result` 比调用 `panic!` 更适合。有两个例子，接收畸形数据的解析器或返回指示速率限制状态的 HTTP 请求。在这些情况下返回 `Result` 意味着调用函数必须处理失败的可能性。

当我们的代码对值执行操作时，它应该首先检查值是否合法，如果不合法则 `panic`。这主要是出于安全原因：操作错误的数据会使我们的代码暴露于漏洞。在标准库中这是导致 `panic` 的主要原因。当我们尝试越界访问内存：尝试访问不属于当前数据结构的内存时，经常会出现安全漏洞。约定在函数中很常见：只有当输入满足特定的标准时，它们的行为才会得到保证。当约定被打破时 `panic` 是有道理的，因为违反约定总是表明调用方有问题，我们不希望调用代码必须显式地处理这个问题。实际上，调用代码并没有实际的恢复方法；调用的程序员必须修改代码。函数的约定应该在函数的 API 描述中解释，特别是当违反约定可能导致 `panic` 时。

另一方面，在我们的所有函数中进行大量的错误检查将是冗长和不愉快的。幸运的是，我们可以通过使用 `Rust` 的类型系统(以及编译器的类型检查)自动化许多检查。如果在函数中使用特定的类型作为参数，我们可以继续执行代码的逻辑，因为编译器已经检查了可接受的值。例如，如果我们使用类型而不是 `Option`，我们的程序期望看到一些东西而不是什么也看不到。

我们的代码将不再需要处理 `Some` 和 `None` 变体的两种情况；相反，它必须处理一个值的一种情况。因为试图向函数发送任何内容的代码将会构建失败，所以我们的方法不需要在运行时检查这种情况。例如，使用像 `u32` 这样的无符号整数类型可以确保参数永远不会为负。

为验证创建自定义类型

让我们进一步利用 Rust 的类型系统来确保有效值，为验证定义一个新类型。我们从不检查用户的猜测和我们的密码，看看它是否在这些数字之间；我们只是检查了一下它是正数。在这个例子中，影响很小：我们的输出“太高”或“太低”仍然是准确的。

然而，这将是一个有用的升级，以帮助用户进行准确的猜测，并在用户猜测超出范围的数字与用户输入字母时产生不同的反应。

要接受潜在的负值，将猜测解析为 `i32` 而不仅仅是 `u32`，然后添加一个在范围内的数字检查，如下所示：

```
loop {
    let guess:i32 = match guess.trim().parse() {
        Ok(num) => num,
        Err(_) => continue,
    };
    if guess < 1 || guess > 100 {
        println!("Secret number will be between 1 & 100.");
        continue;
    }
    match guess.cmp(&secret_number) {}
}
```

如果我们的值超出范围，`if` 表达式将通知用户问题，并继续调用开始下一个循环迭代并要求再次猜测。我们可以继续进行猜测和 `if` 表达式后面的密码之间的比较，知道猜测在 1 到 100 之间。

然而，这不是最好的解决方案：如果程序必须只对 1 到 100 之间的值进行操作，那么在每个函数中进行这样的检查将不方便(并且可能影响性能)。

我们可以构建一个新类型，并将验证放在一个函数中，以生成该类型的实例，而不是到处重复验证。因此，函数可以安全地在其签名中使用新类型并信任它们获得的值。下面的示例展示了建立 `Guess` 类型的一种方法，该方法仅在新函数被分配 1 到 100 之间的值时才会产生 `Guess` 实例。

```
pub struct Guess {
    value: i32,
}
impl Guess {
    pub fn new(value:i32) -> Guess {
        if value < 1 || value > 100 {
            panic!("Guess value between 1 and 100, got{}.", value);
            Guess {value}
        }
    }
    pub fn value(&self) -> i32 {
```

```
        self.value
    }
}
```

首先，我们创建一个名为 `Guess` 的结构，其中包含了一个类型为 `i32` 的 `value` 字段，它是数值将被保存的位置。

然后，我们在 `Guess` 上使用名为 `new` 的相关函数创建 `Guess` 值的实例。新函数被定义为接受一个名为 `value` 的 `i32` 类型形参并返回一个 `Guess`，且 `new` 函数体中的代码会验证 `value` 是否在 1 到 100 之间。

如果 `value` 未能通过此测试，我们将发出调用 `panic!`，通知编写调用代码的程序员需要修复缺陷，因为创建一个具有超出此范围值的 `Guess` 将违反 `Guess::new` 所依赖的约定。`Guess::new` 可能会 `panic` 的情况应该在其面向公众的 API 文档中提及；我们将讨论表明 `panic!` 可能性的文档约定。在 API 文档中。如果值通过了测试，则返回 `Guess` 并构建一个新的 `Guess`，将 `value` 字段设置为 `value` 参数。

接下来，我们创建一个借用 `self` 的 `value` 方法，它没有其他参数，并返回一个 `i32`。因为它的功能是从字段中检索数据并返回数据，所以这种类型的方法有时被称为 `getter`。因为 `Guess` 结构的 `value` 字段是私有的，所以需要这个公开函数。因为 `value` 字段必须是秘密的，使用 `Guess` 结构的代码不允许直接设置 `value`：模块外部的代码必须使用 `Guess::new` 函数构造一个 `Guess` 的实例，确保 `Guess` 不能有 `Guess::new` 函数中的条件没有检查过的值。

只接受或返回 1 到 100 之间值的函数可以在其签名中指定它接受或返回 `Guess` 而不是 `i32`，从而消除了进行额外检查的需要。

在本章中，我们讨论了不可恢复的错误。和可恢复错误的结果。

C8: 泛型类型、Trait 和生命期

- 2022.11.22

在本章中

- 泛型数据类型
- Trait: 定义共同行为
- 用生命期 (lifetime) 验证引用

LQ 注: Trait 可译为特征, 但我觉得不译更好。另外, lifetime 译为生命期, 而不是生命周期, 以避免与 lifecycle (生命周期) 相混淆。

在前一章中, 我们讨论了不可恢复的错误。和可恢复错误的结果。本章将讨论通用数据类型、Trait: 定义共享行为, 以及使用生命期验证引用。

泛型数据类型

泛型可以定义诸如函数签名或结构之类的项, 然后这些项可以用于各种具体的数据类型。让我们从使用泛型来定义函数、结构、枚举和方法开始。然后我们将讨论泛型如何影响代码性能。

在函数定义中

我们将泛型放在函数的签名中, 通常在此定义形参和返回值的数据类型。这使我们的代码更加灵活, 并为函数的调用者提供更多的功能, 同时避免代码重复。

与我们的最大函数保持一致, 下面的代码演示了两个函数, 它们发现切片中的最大值。

```
fn large_i32(list: &[i32]) -> i32 {
    let mut large = list[0];
    for &item in list {
        if item > large {
            large = item;
        }
    }
    large
}

fn large_char(list: &[char]) -> char {
    let mut large = list[0];
    for &item in list {
        if item > large {
            large = item;
        }
    }
    large
}
```

```

}
fn main() {
    let number_list = vec![34, 50, 25, 100, 65];
    let result = large_i32(&number_list);
    println!("Large number is {}", result);
    let char_list = vec!['y', 'm', 'a', 'q'];
    let result = large_char(&char_list);
    println!("Large char is {}", result);
}

```

`large_i32` 函数定位切片中最大的整数。`large_char` 方法返回切片中最大的 `char`。因为函数体是相同的，所以我们可以通过在单个函数中提供泛型类型参数来避免重复。

必须为类型形参命名，就像为函数中的值形参命名一样，以参数化将要构建的新函数中的类型。作为类型参数名，我们可以使用任何标识符。但是我们将选择 `T`，因为 Rust 中的参数名很短，通常只有一个字母，而且 Rust 使用驼峰类型命名标准。`T` 代表“类型”，大多数 Rust 程序员将其作为默认值。

在函数体中使用形参时，必须在签名中声明形参名称，以便编译器理解它的含义。同样，在函数签名中使用类型参数名时，必须首先指定类型参数名。将类型名称声明放在尖括号 `<>` 内，在函数名和形参列表之间，以定义泛型最大函数，如下所示：

```

fn large<T>(list: &[T]) -> T {
    let mut large = list[0];
    for &item in list {
        if item > large {
            large = item;
        }
    }
    large
}
fn main() {
    let number_list = vec![34, 50, 25, 100, 65];
    let result = large(&number_list);
    println!("Large number is {}", result);
    let char_list = vec!['y', 'm', 'a', 'q'];
    let result = large(&char_list);
    println!("Large char is {}", result);
}

```

编译源码，会报如下错误：

```

error[E0369]: binary operation `>` cannot be applied to type `T`
--> r3.rs:4:17
4 |         if item > large {
  |                   ^      T
  |                   |
  |                   T

```

```
|
help: consider restricting type parameter `T`
1 | fn large<T: std::cmp::PartialOrd>(list: &[T]) -> T {
    |      ++++++
```

error: aborting due to previous error

注释提到了 `trait std::cmp::PartialOrd`。在下一部分中，我们将讨论特性（characteristics）。目前，这个错误表明，`large` 函数体不会对所有可以想象到的 `T` 形式起作用。我们只能使用那些值可能被排序的类型，因为我们想比较体中类型 `T` 的值。标准库中的 `std::cmp::PartialOrd` trait 可用于类型以启用比较（有关该 trait 的更多信息，请参阅附录 C）。我们将学习如何指示泛型类型具有特定的 trait，但首先，让我们看看使用泛型类型参数的其他一些方法。

在结构定义中

使用 `<>` 语法，我们还可以构建结构，以便在一个或多个字段中使用泛型类型参数。下面的代码演示了如何创建一个 `Point` 结构体，该结构体可以保存任意类型的 `a` 和 `b` 坐标值。

```
struct Point<T> {
    a: T,
    b: T,
}
fn main() {
    let integer = Point{a:4, b:14};
    let float = Point{a:2.0, b:3.0};
}
```

在结构定义中使用泛型的语法类似于函数定义。首先，直接在结构名称之后，在尖括号内声明类型参数的名称。然后，我们可以在结构定义中使用泛型类型，而不是指定实际的数据类型。

因为我们只使用了一个泛型类型来描述 `Point`，所以这个定义说 `Point` 结构体对于某种类型 `T` 是泛型的，并且字段 `a` 和 `b` 都是该类型的，不管它是什么。如果用不同类型的值构造 `Point` 的实例，则代码将无法编译。

```
struct Point<T> {
    a: T,
    b: T,
}
fn main() {
    let wont_work = Point{a:4, b:3.0};
}
```

在本例中，当我们将整型值 4 赋给 a 时，我们告诉编译器，对于 Point 的实例，泛型类型 T 将是一个整数。然后，如果我们为 b 指定 3.0，我们会得到一个类型不匹配的错误，我们已经指定了 b 的类型与 a 相同：（编译时报错如下）

```
error[E0308]: mismatched types
--> r3.rs:6:34
6 |         let wont_work = Point{a:4, b:3.0};
  |                                ^^^ expected integer, found floati
ng-point number
```

我们可以利用多个泛型类型参数来构建 Point 结构体，其中 a 和 b 都是泛型且有不同的类型。例如，如果 a 的类型是 T，b 的类型是 U，我们可以将 Point 的定义更改为对类型 T 和 U 的泛型。

```
struct Point<T, U> {
    a: T,
    b: U,
}
fn main() {
    let both_integer = Point{a:5, b:10};
    let both_float = Point{a:1.0, b:4.0};
    let integer_and_float = Point{a:5, b:4.0};
}
```

现在允许显示 Point 的所有实例。在定义中，我们可以使用任意多的泛型类型参数，但是添加过多的泛型类型参数会使代码难以阅读。当我们的代码中需要大量泛型类型时，这可能是我们需要将其分解为更小块的迹象。

枚举与结构体一样，可用于在其变体中存储泛型数据类型。让我们回顾一下标准库提供的 Option 枚举，我们在第 5 章中使用过它：

```
enum Option<T> {
    Some(T),
    None,
}
```

现在我们应该能够理解这个定义了。我们可以看到，Option 是类型 T 上的泛型枚举，有两个变量：Some，它存储一个类型 T 值；而 None，它不携带任何值。我们可以使用 Option 枚举表示具有可选值的抽象概念。因为 Option 是泛型的，所以不管可选值的类型是什么，我们都可以利用这种抽象。

枚举还可以使用各种泛型类型。一个例子是我们在第 7 章中使用的 Result 枚举的定义：

```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Result 枚举是跨 **T** 和 **E** 两种类型的泛型，并且有两个变体：**Ok** 和 **Err**。**Ok** 保存类型 **T** 值，而 **Err** 保存类型 **E** 值。这个定义允许我们在操作成功（返回一个类型为 **T** 的值）或失败（返回一个类型为 **E** 的错误）的任何时候使用 **Result** 枚举。实际上，在即将到来的第 9 章的例子中，我们使用它来打开一个文件，当文件被成功打开时，其中 **T** 由类型 **std::** 填充；当打开文件出现问题时，**E** 被填充为类型 **std::io::Error**。

当我们在代码中发现许多结构或枚举定义只在它们所携带的值的类型上有所不同时，我们可以通过更改为泛型类型来避免冗余。

在方法定义中

方法可以实现结构和枚举，泛型类型可以在它们的定义中使用。

```
struct Point<T> {
    a: T,
    b: T,
}
impl<T> Point<T> {
    fn a(&self) -> &T {
        &self.a
    }
}
fn main() {
    let p = Point{a:5, b:10};
    println!("p.a = {}", p.a());
}
```

我们在 **Point** 上创建了一个名为 **a** 的方法，它返回对字段 **a** 中数据的引用。

值得注意的是，我们需要在 **impl** 之后声明 **T**，以使用它来表明我们正在实现 **Point** 类型的方法。**Rust** 可以通过在 **impl** 之后将 **T** 声明为泛型类型来判断 **Point** 中尖括号中的类型是泛型类型而不是具体类型。

例如，我们可以只在 **Point** 实例上实现它们，而不是在 **Point** 实例上实现任何泛型类型的方法。我们在下面的代码中使用具体类型 **f32**，这意味着我们在 **impl** 之后不定义任何类型。

```
impl Point<f32> {
    fn distance_from_origin(&self) -> f32 {
        (self.a.powi(2) + self.b.powi(2)).sqrt()
    }
}
```

这段代码表明 **Point** 类型将具有 **distance_from_origin** 方法，而 **Point** 的其他实例（其中 **T** 不是类型 **f32**）则没有。该方法使用仅对浮点类型可访问的数学操作来确定我们的点到坐标 **(0.0, 0.0)** 处点的距离。

结构定义中的泛型类型参数并不总是与该结构的方法签名中使用的参数相同。例如，从前面的代码中定义 `Point<T,U>` 结构上的方法 `mixup`。该方法接受另一个 `Point` 作为参数，该参数可能与我们调用 `mixup` 的 `self Point` 的类型相同，也可能不同。该方法返回一个新的 `Point` 实例，其中 `a` 的值来自传入的 `Point`（类型 `W`），`b` 的值来自 `self Point`（类型 `T`）。

```
struct Point<T,U> {
    a: T,
    b: U,
}
impl<T,U> Point<T,U> {
    fn mixup<V,W>(self, other: Point<V,W>) -> Point<T,W> {
        Point {
            a: self.a,
            b: other.b,
        }
    }
}
fn main() {
    let p1 = Point{a:4, b:10.3};
    let p2 = Point{a:"Helloo", b:'c'};
    let p3 = p1.mixup(p2);
    println!("p3.a={}, p3.b={}", p3.a, p3.b);
}
```

我们在 `main` 中定义了一个 `Point`，即 `p1`，它的 `a` 为 `i32`（值 4），`b` 为 `f64`（值 10.3）。`p2` 也是一个 `Point` 结构体，它有一个字符串切片用于 `a`（值为“Helloo”）和一个 `char` 值用于 `b`（值为 `c`）。因为 `fn` 起始于 `p1`，在 `p1` 上调用参数为 `P2` 的 `mixup` 返回 `p3`，它将有一个 `i32` 用于 `a`。因为 `b` 来自 `p2`，`p3` 的 `b` 将是 `char` 值。故使用 `println!` 宏函数将输出 `p3.a=4, p3.b='c'`。

本示例旨在展示如何使用 `impl` 声明一些泛型参数，以及使用方法描述声明另一些泛型参数。因为它们与结构定义一起使用，`T` 和 `U` 的泛型参数在 `impl` 之后声明。因为泛型参数 `V` 和 `W` 只与方法相关，所以它们在 `fn mixup` 之后被声明。

使用泛型的代码性能

如果我们使用泛型类型参数，我们可能想知道是否有运行时成本。好消息是 `Rust` 以这样一种方式实现泛型，即使用泛型类型不会比使用具体类型更降低代码的速度。

`Rust` 通过对使用泛型的代码进行编译时的单一化（`monomorphization`）来做到这一点。单一化是通过在编译期间填充所需的具体类型，将泛型代码转换为特定代码的过程。

编译器执行的步骤与创建泛型函数的步骤相反：它检查调用泛型代码的所有位置，并为调用泛型代码的具体类型生成代码。

让我们通过一个使用标准库中的 `Option` 枚举的例子来看看这是如何工作的：

```
let integer = Some(4);
let float = Some(4.0);
```

Rust 在编译此代码时执行单一化。在此过程中，编译器检查 `Option` 实例中使用的值，并区分出 `Option` 有 `i32` 和 `f64` 两种类型。因此，它将 `Option` 泛型定义扩展为 `Option_i32` 和 `Option_f64`，从而用特定的描述替换泛型定义。

以下是该代码的单一化版本。编译器的个别定义被用来代替泛型 `Option`：

```
enum Option_i32 {
    Some(i32),
    None,
}
enum Option_f64 {
    Some(f64),
    None
}
fn main() {
    let integer = Option_i32::Some(4);
    let float = Option_f64::Some(4.0);
}
```

我们不需要为使用泛型付出运行时成本，因为 Rust 将泛型代码编译为在每个实例中指定类型的代码。当代码运行时，它的行为与手动复制每个声明完全相同。得益于单一化方法，Rust 的泛型在运行时非常高效。

Trait：定义共享的行为

`trait` 通知 Rust 编译器给定类型所具有的功能以及可以与其他类型共享的功能。从抽象意义上讲，`trait` 可用来定义共享行为。`trait` 边界可以将泛型类型定义为具有特定行为的任何类型。

定义一个 `trait`

我们可以调用的方法定义了类型的行为。如果我们对所有类型调用相同的方法，它们就具有相同的行为。方法签名在 `trait` 定义中分组，以定义实现目标所需的一组行为。

假设我们有多包含不同类型和数量的文本结构，例如 `NewsArticle` 结构包含在特定位置归档的新闻故事，`Tweet` 结构包含最多 280 个字符和元数据，元数据指示它是新 `Tweet`、转发 `Tweet` 还是对另一个 `Tweet` 的回复。

我们希望创建一个媒体聚合器库，以显示来自 `NewsArticle` 和 `Tweet` 实例的数据摘要。为此，我们需要每个类型的摘要，我们将通过对一个示例调用 `summarize` 方法来获得该摘要。表达此行为的 `Summary trait` 的定义在代码中如下所示：

```
pub trait Summary {
    fn summarize(&self) -> String;
}
```

我们使用 `trait` 关键字后面跟着 `trait` 的名称来定义一个 `trait`，在本例中是 `Summary`。定义实现该 `trait` 类型的行为的方法签名（在本例中为 `fn summarize(&self) -> String`）在花括号内声明。

我们使用分号，而不是在方法签名后的花括号中提供实现。每个实现此 `trait` 的类型都必须为方法主体提供其特定的行为。编译器将确保每个具有 `Summary trait` 的类型都有使用此签名创建的 `summarize` 方法。

方法签名按行列出，每行以分号结尾。一个 `trait` 在它的主体中可以有多个方法。

在类型上实现 Trait

现在，我们可以使用 `Summary trait` 来表达适当的行为，并将其应用于媒体聚合器中的类型。下面的代码显示了 `Summary trait` 的 `NewsArticles` 结构实现，该特征使用 `headline`（标题）、`author`（作者）和 `location`（位置）创建摘要的返回值。我们将 `Tweets` 结构的 `summary` 定义为用户名后面跟着 `Tweet` 的全文，假设 `Tweet` 的内容已经限制为 280 个字符。

```
pub trait Summary {
    fn summarize(&self) -> String;
}

pub struct NewsArticles {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

impl Summary for NewsArticles {
    fn summarize(&self) -> String {
        format!("{}", by {} ({}))", self.headline, self.author, self.location)
    }
}

pub struct Tweets {
    pub usernames: String,
    pub contents: String,
    pub reply: bool,
    pub retweets: bool,
}

impl Summary for Tweets {
    fn summarize(&self) -> String {
        format!("{}", self.usernames, self.contents)
    }
}
```

在类型上实现 `trait` 等同于在类型上放置普通方法。不同之处在于，我们将想要实现的 `trait` 名写在 `impl` 之后，然后使用 `for` 关键字来定义我们想要实现的 `trait` 类型。`trait` 定义的方法签名放在 `impl` 块中。我们不是在每个签名后面使用分号，而是使用花括号，用我们希望针对给定 `trait` 类型的方法具有的精确行为填充方法体。

我们可以在 `NewsArticles` 和 `Tweets` 实例上调用方法，就像我们在实现 `trait` 后调用普通方法一样，例如：

```
fn main() {
    let tweets = Tweets {
        usernames: String::from("horse_ebooks"),
        contents: String::from("of course, as we probably already know,
people"),
        reply: false,
        retweets: false,
    };
    println!("1 new tweet: {}", tweets.summarize());
}
```

这个代码打印出以下内容：“1 new tweet: horse_ebooks: of course, as we probably already know, people”。

因为 `Summary trait` 以及 `NewsArticles` 和 `Tweets` 类型都是在同一个库（`lib.rs`）中指定的。它们都在同一个作用域。让我们假装这个 `lib.rs` 是针对名为 `aggregator crate` 的，而其他入则希望利用此 `crate` 的功能，在他们库范围内声明的结构上实现 `Summary trait`。他们必须首先将属性包含在他们的作用域中。

可以通过指定 `use aggregator::Summary;` 来实现这一点，允许它们为自己的类型实现 `Summary`。对于实现 `Summary` 特性的另一个 `crate`，它必须是一个公开的 `trait`，这就是为什么我们在 `trait` 之前插入了 `pub` 关键字。

记住 `trait` 的实现，如果 `trait` 或类型是 `crate` 本地的，那么我们只能在类型上实现一个 `trait`。To keep in mind trait implementations, we can only implement a trait on a type if the trait or the type is local to our crate.

因为类型 `Tweets` 对于我们的聚合器 `crate` 来说是本地的，所以我们可以利用标准库特性，比如在自定义类型 `Tweets` 上使用 `Display`，作为聚合器 `crate` 功能的一部分。因为 `trait Summary` 是我们聚合器 `crate` 的局部特征，我们也可以在我们聚合器 `crate` 中的 `Vec` 上实现 `Summary`。

但是，我们不能将外部属性应用于外部类型。因为 `Display` 和 `Vec` 是在标准库中定义的，而不是我们的聚合器 `crate` 本地的，所以我们不能在聚合器 `crate` 的 `Vec` 上实现 `Display` 特性。这个约束是程序一致性属性（`coherence property`）的一部分，特别是孤儿规则（`orphan rule`），以缺少父类型而命名。这个规则保证了其他人的代码不会破坏我们的代码，反之亦然。如果规则不存在，两个 `crate` 可能为同一类型实现相同的特征，`Rust` 将不知道选择哪一个。

用生命期验证引用

Rust 中的每个引用都有一个有效期（lifespan），也就是该引用有效的范围，这是我们在第 3 章的“Rust 中的引用和借用”一节中没有涉及到的。大多数情况下，生命期是假设的和推断的，就像大多数情况下类型是假设的和推断的一样。当可能存在大量类型时，我们必须对它们进行注释。类似地，当引用的生命期可以以各种方式关联时，我们必须对它们进行注释。为了验证运行时使用的实际引用是真实的，Rust 需要我们使用通用寿命参数注释关系。

生命期的概念与其他编程语言的工具有所不同，这可能是 Rust 最显著的特征。尽管我们不会在本章中详细讨论生命期，但我们将介绍一些可能遇到生命期语法的最典型方法，以便更好地理解这些思想。

防止使用生命期的悬空引用

生命期的主要目标是避免悬空引用，悬空引用会导致程序引用不打算被引用的数据。看一下这个程序，它有一个外部作用域和一个内部作用域。

```
{
    let s;
    {
        let y = 6;
        s = &y;
    }
    println!("s: {}", s);
}
```

外部作用域声明了一个没有起始值的名为 `s` 的变量，而内部作用域声明了一个名为 `y` 的变量，其第一个值为 6。我们尝试将 `s` 的值设置为内部作用域中对 `y` 的引用。然后，内部作用域结束，我们尝试打印 `s` 中的值。因为 `s` 所指的值在我们尝试使用它之前已经超出作用域，所以这段代码将无法编译。错误信息如下：

```
error[E0597]: `y` does not live long enough
--> r2.rs:6:17
   |
6 |         s = &y;
   |             ^^ borrowed value does not live long enough
7 |     }
   |     - `y` dropped here while still borrowed
8 |     println!("s: {}", s);
   |                       - borrow later used here
```

变量 `y` “活得不够久”。这是因为当内部作用域在第 7 行结束时，`y` 将超出作用域。但是，`s` 对于外部作用域仍然有效；我们说它“寿命更长”是因为它的作用域范围更大。如果 Rust 让这段代码运行，`s` 将寻址在 `y` 退出作用域时已被释放的内存，我们试图对 `s` 做的任何事情都将失败。那么，Rust 是如何判断这段代码是否合法的呢？使用借用检查器（borrow checker）。

借用检查器

Rust 编译器中的借用检查器会检查作用域，以查看是否所有的借用都是有效的：与前面的代码相同，但带有指示变量生存期的注释。

```
{
    let s; // -----+-- 'a
    ///
    {///
        let y = 6; // -+-- 'b|
        s = &y; ///|
    } // -+|
    println!("s: {}", s); ///
} // -----+
```

s 的生命期用 'a 标记，y 的生命周期用 'b 标记。正如我们所看到的，内部的 'b 块明显小于外部的 'a 生命期块。Rust 在构建时分析这两个生命期的大小，发现 s 的生命期为 'a，但引用的内存的生命期为 'b。由于 'b 比 'a 短，程序会被拒绝：引用的主题没有引用存在的时间长。

下面修复了代码中的悬空引用，并确保它的编译没有问题。

```
{
    let y = 6; // -----+-- 'b
    /// |
    s = &y; // --+-- 'a|
    /// | |
    println!("s: {}", s); // | |
    /// --+ |
} // -----+
```

y 的生命期是 'b，在这种情况下 'b 比 'a 大。这意味着 s 可以指向 y，因为 Rust 知道，虽然 y 是有效的，但 s 中的引用总是有效的。

LQ 注：原书中上面的描述与源代码不一致，如果要 'b 生命期大于 'a，那就需要把 y 的定义放在 s 的定义的更外层。

现在我们知道了引用的生存期在哪里，以及 Rust 如何计算生存期以确保引用始终有效，让我们看看函数上下文中参数和返回值的通用生存期。

函数中泛型的生命期

让我们创建一个方法，它返回两个字符串切片之间最长的字符串切片。这个函数从两个字符串切片返回一个字符串切片。当我们实现了 longest 函数时，代码应该输出“The efgh is longest string”。

```
fn main() {
    let string1 = String::from("efgh");
    let string2 = "abc";
```

```

    let result = longest(string1.as_str(), string2);
    println!("The {} is longest string", result);
}

```

因为我们不希望 `longest` 函数获得其形参的所有权，所以我们希望函数接受字符串切片，即引用。更多关于为什么我们使用的参数是我们想要的参数的信息可以在第 3 章的“字符串切片作为参数”一节中找到。

如果我们尝试实现如下所示的 `longest` 函数，它将编译失败：

```

fn longest(y: &str, z: &str) -> &str {
    if y.len() > z.len() {
        y
    } else {
        z
    }
}

```

错误消息与生命期有关。

error: 1 positional argument in format string, but no arguments were given

```

--> r2.rs:5:19
|
5 |     println!("The {} is longest string");
|                   ^^

```

error[E0106]: missing lifetime specifier

```

--> r2.rs:7:33
|
7 | fn longest(y: &str, z: &str) -> &str {
|           ----      ----      ^ expected named lifetime parameter
|
= help: this function's return type contains a borrowed value, but the signature does not say whether it is borrowed from `y` or `z`
help: consider introducing a named lifetime parameter
|
7 | fn longest<'a>(y: &'a str, z: &'a str) -> &'a str {
|           +++++   ++           ++           ++

```

根据错误提示将 `longest` 函数定义修改为如下即可。

```

fn longest<'a>(y: &'a str, z: &'a str) -> &'a str {

```

本章讨论了共享行为和使用生命周期验证引用。

c9: 输入/输出项目：构建一个命令行程序

- 2022.11.24

在本章中

- 接受命令行参数
- 读取文件
- 重构以改进模块化和错误处理
- 用测试驱动开发（TDD）开发库的功能
- 使用环境变量
- 改进我们的 I/O 项目

在上一章中，我们讨论了泛型数据类型、`trait`，其中我们讨论了定义共享行为，以及用生命周期验证引用。在本章中，我们将讨论接受命令行参数、读取文件和重构以改进模块化和错误处理。此外，我们还将讨论使用测试驱动开发（TDD）开发库的功能，使用环境变量，以及改进我们的 I/O 项目。

本章回顾了到目前为止我们所学到的各种技能，并研究了一些更常见的库特性。为了将您学到的一些 Rust 原理用于测试，我们将创建一个命令行工具，它与文件和命令行输入/输出交互。

因为 Rust 的速度、安全性、单二进制输出和跨平台支持使它非常适合构建命令行工具，所以我们将为我们的项目开发传统命令行搜索工具 `grep`（全局搜索正则表达式和打印）的自定义版本。`Grep` 在最基本的用例中搜索指定文件中的指定字符串。

文件名和字符串作为参数发送给 `grep`。然后，程序扫描文件，查找包含字符串参数的行，并打印它们。

我们将教您如何使我们的命令行工具使用终端特性。为了允许用户调整工具的功能，我们将读取一个环境变量的值。我们还将把错误消息打印到标准错误控制台流（`stderr`）而不是标准输出（`stdout`），这样用户就可以，例如，将成功的输出重定向到文件，同时仍然在屏幕上查看错误消息。

Rust 社区的成员 Andrew Gallant 已经创建了一个 `ripgrep`，这是一个功能齐全且速度极快的 `grep` 版本。相比之下，我们的 `grep` 版本是很简单的，但本章将为我们提供一些背景信息，我们需要了解像 `ripgrep` 这样的真实项目。

接受命令行参数

像往常一样，让我们用 `cargo new` 开始一个新的项目。为了将它与机器上可能已有的 `grep` 实用程序区分开来，我们把项目命名为 `minigrep`。

```
$ cargo new minigrep
Created binary (application) `minigrep` project
$ cd minigrep
```

第一步是让 `minigrep` 接受文件名和要搜索的字符串作为命令行参数。也就是说，我们希望能够使用 `cargo run`、待搜索的字符串、和待搜索的文件路径来运行程序，如下所示：

```
$ cargo run searchstring examplefilename.txt
```

目前，由 `cargo new` 创建的程序无法处理我们提供的参数。虽然在 `crate.io` 上已经有现成的库了，可以帮助我们开发一个接受命令行参数的程序，但因为我们刚刚学习这个概念，所以还是让我们自己来手动实现。

我们需要 Rust 标准库的 `std::env::args` 函数，以使 `minigrep` 能够读取我们传递给它的命令行参数的值。该方法生成一个迭代器，其中包含在命令行上传递给 `minigrep` 的参数。关于迭代器，我们只需要知道两件事：迭代器生成一系列值，我们可以使用 `collect` 函数将它们放入一个集合中，例如一个 `vector`，其中包含迭代器生成的所有元素。

允许我们的 `minigrep` 应用程序读取任何命令行参数，然后使用代码将值收集到一个向量中。

```
use std::env;
fn main() {
    let args: Vec<String> = env::args().collect();
    println!("{:?}", args);
}
```

首先，我们使用 `use` 语句将 `std::env` 模块引入作用域，以便使用它的 `args` 函数。正如我们所看到的，`std::env::args` 函数嵌套在两层模块中。当期望的函数嵌套在许多模块中时，就像我们在第 6 章中描述的那样，通常会引入父模块而不是函数引入范围。然后，我们可以轻松地使用 `std::env` 的函数。这样也比添加 `use std::env::args` 然后只使用 `args` 调用函数更清晰，因为 `args` 很容易与当前模块的函数相混淆。

我们在 `main` 的第一行调用 `env::args`，然后使用 `collect` 将迭代器转换为一个包含迭代器产生的所有值的向量。由于 `collect` 方法可以用来生成各种各样的集合，所以我们显式地给出 `args` 的类型来表示我们想要一个字符串的向量。虽然在 Rust 中我们不需要经常注释类型，但是我们确实需要注释集合，因为 Rust 不能推断我们想要的集合类型。

最后，我们使用调试格式化器打印向量，`:?` 让我们先不带任何参数地执行代码，然后带两个参数：

```
D:\workspace\rust-works\minigrep>cargo run
Compiling minigrep v0.1.0 (D:\workspace\rust-works\minigrep)
```

```
Finished dev [unoptimized + debuginfo] target(s) in 2.09s
Running `target\debug\minigrep.exe`
["target\\debug\\minigrep.exe"]
```

```
D:\workspace\rust-works\minigrep>cargo run needle haystack
Finished dev [unoptimized + debuginfo] target(s) in 0.02s
Running `target\debug\minigrep.exe needle haystack`
["target\\debug\\minigrep.exe", "needle", "haystack"]
```

我们的二进制文件的名称是“target/debug/minigrep.exe”，这是向量中的第一个值。这与 C 中的参数列表的行为类似，后者允许程序在执行过程中使用调用它们的名称。如果我们想要在消息中打印程序名，或者根据调用它的命令行别名调整程序的行为，那么知道程序名是很有用的。然而，为了本章的目的，我们将忽略它，只保留我们需要的两个参数。

在变量中保存参数值

程序可以访问作为命令行参数指定的值，这一点通过打印参数向量的值来证明。现在我们必须将两个参数的值保存在变量中，以便稍后在程序中使用它们。

```
use std::env;
fn main() {
    let args: Vec<String> = env::args().collect();
    let query = &args[1];
    let filenames = &args[2];
    println!("Search for {}", query);
    println!("File {}", filenames);
}
```

再次运行程序：

```
D:\workspace\rust-works\minigrep>cargo run needle haystack
Finished dev [unoptimized + debuginfo] target(s) in 0.01s
Running `target\debug\minigrep.exe needle haystack`
Search for needle
File haystack
```

读取一个文件

现在，我们将实现前述的命令行选项中提供文件名的功能。首先，我们需要一个示例文件（Emily Dickinson, I’m Nobody! Who are you? (260), in Poems, Series 2, 1891）来测试它：用于确保 minigrep 工作的最佳文件类型是在大量的行和一些重复的术语中分布少量的文本。

用于测试的示例文本文件 poem.txt：

```
Are you nobody, too?
Then there’s a pair of us – don’t tell!
They’d banish us, you know.
```

```
How dreary to be somebody!  
How public, like a frog  
To tell your name the livelong day  
To an admiring bog!
```

此文本文件创建完成后，编辑 `src/main.rs` 并添加以下代码以读取文件：

```
use std::env;  
use std::fs;  
fn main() {  
    let args: Vec<String> = env::args().collect();  
    let query = &args[1];  
    let filename = &args[2];  
    println!("Search for {}", query);  
    println!("File {}", filename);  
    let contents = fs::read_to_string(filename).expect("Something went  
wrong reading file");  
    println!("With the text:\n{}", contents);  
}
```

首先，我们添加另一个 `use` 行以包含标准库的必要部分：我们需要 `std::fs` 来处理文件。

我们在 `main` 中添加了一个新语句：`fs::read_to_string` 获取文件名，打开文件，并生成一个包含文件内容的 `Result`。

在该声明之后，我们添加了一个临时 `println!` 在读取文件后打印 `contents` 值的语句，允许我们验证程序到目前为止是否正常工作。

让我们使用任意字符串作为第一个命令行选项的输入（因为搜索部分尚未实现）和 `poem.txt` 作为第二个命令行参数来执行此代码：

```
D:\workspace\rust-works\minigrep>cargo run needle poem.txt  
    Finished dev [unoptimized + debuginfo] target(s) in 0.02s  
    Running `target\debug\minigrep.exe needle poem.txt`  
Search for needle  
File poem.txt  
With the text:  
Are you nobody, too?  
Then there's a pair of us - don't tell!  
They'd banish us, you know.  
How dreary to be somebody!  
How public, like a frog  
To tell your name the livelong day  
To an admiring bog!
```

非常好！代码读取了文件的内容，然后打印它们。但是，代码中仍然存在一些问题。主函数中有很多子功能：而只负责单个功能的函数通常更清晰，更容易维护。另一个问题是，我们没有尽可能地处理错误。因为程序仍然很小，所以这些错误不算重

要问题，但当程序扩展时，要干净地修复它们将更加困难。所以最好在程序开发的早期就开始重构，因为在早期重构代码要容易得多。这就是我们接下来要做的事情。

重构以改进模块化和错误处理

为了使我们的程序更好，我们将修复程序结构以及它如何处理潜在错误的四个问题。

我们的主函数现在有两个函数：它解析输入和读取文件。对于这么短的函数来说，这不是一个大问题。但是，如果我们继续在 `main` 中扩展我们的程序，`main` 函数处理不同任务的数量将会增加。随着函数复杂性的增加，在不破坏其组件之一的情况下推理、测试和更新变得更加困难。所以最好将功能分开，以便每个函数只负责一项任务。

第二个问题是，虽然查询和文件名是我们程序中的配置变量，但内容等变量用于执行程序的逻辑。主要内容越长，我们需要引入范围的变量就越多；我们范围内的变量越多，就越难记住每个变量要执行什么。要使配置变量的用途显而易见，应该将它们分组到一个结构中。

第三个问题是，如果读取文件失败，我们希望它打印错误消息，但错误消息只是说“读取文件时出错”。读取文件可能由于各种原因而失败，例如文件丢失或没有打开文件的权限。而无论现在的情况如何，我们都会输出“读取文件时出错”的错误消息，这样用户会得不到有价值的信息。

第四，我们多次使用 `expect` 来处理各种问题，如果用户在没有提供足够输入的情况下执行我们的程序，`Rust` 将抛出一个索引越界错误，而这无法解释问题。所有错误处理代码都应该放在一个地方。这样，如果必须修改错误处理的逻辑，未来的维护者只需在一个地方查看。将所有错误处理代码放在一个地方还可以确保我们输出对用户有帮助的消息。

让我们重新设计我们的项目来处理这四个问题。

二进制项目的关注点分离

许多二进制项目都面临着将各种任务的责任分配给核心职能的组织挑战。因此，`Rust` 社区建立了一个流程，作为在 `main` 变得太大时打破二进制程序的各种问题的指南。该过程的步骤如下：

- 将我们的程序拆分为两个文件，`main.rs` 和 `lib.rs`，并将逻辑重新定位到 `lib.rs`。
- 我们可以将命令行解析逻辑保留在 `main.rs` 中，只要它是最小的。
- 从 `main.rs` 中提取命令行分析逻辑，并在它变得过于复杂时将其重新定位到 `lib.rs`。

在此过程之后，主要功能的职责应限制为以下内容：

- 使用参数值调用命令行解析逻辑

- 任何其他配置的设置
- 在 `lib.rs` 中，调用 `run` 函数
- 如果 `run` 返回错误，我们必须处理它。

此模式强调关注点分离：`main.rs` 负责运行程序。而 `lib.rs` 负责任务的逻辑。因为我们不能直接测试 `main` 函数，所以这种结构允许我们将程序的所有功能移动到 `lib.rs` 的函数中。`main.rs` 中剩下的唯一代码将很适中，可以阅读和检查其准确性。让我们使用此方法重写程序。

提取参数解析器

为了准备将命令行解析逻辑重新定位到 `src/lib.rs`，我们将解析参数的机制隔离为 `main` 将调用的函数。下面的示例显示了 `main` 的修订启动，它调用一个新的函数解析配置，我们将暂时在 `src/main.rs` 中创建该配置。

```
use std::env;
fn main() {
    let args: Vec<String> = env::args().collect();
    let (query, filename) = parse_config(&args);
}
fn parse_config(args: &[String]) -> (&str, &str) {
    let query = &args[1];
    let filename = &args[2];
    (query, filename)
}
```

取代之前的将索引 1 处的 `args` 值分配给 `main` 函数中的 `query` 变量，将索引 2 处的 `args` 值分配给 `main` 函数中的 `filename` 变量，而是将整个向量分配给 `parse_config` 函数。选择哪个参数属于存储在 `parse_config` 函数中的变量逻辑，然后将数据返回到 `main`。`query` 和 `filename` 变量仍在 `main` 中创建，但它不再负责决定命令行输入和变量之间的关系。

我们正在一点点的渐进步骤进行重组，因此对于我们的小程序来说，这种返工可能看起来有点过分了。进行此更改后再次运行程序，以确保参数分析仍然有效。跟踪我们的进度是个好主意，这样我们就可以在出现问题时找出导致问题的原因。

对配置值进行分组

我们可以通过采取另一小步来进一步改进解析配置方法。我们现在返回一个元组，但我们很快就会再次将其分解为单独的组件。这表明我们还没有找到正确的抽象。

`parse_config` 的 `config` 组件是另一个存在改进机会的指标，因为它意味着我们返回的两个值是连接的，并且都是一个配置值的一部分。我们目前没有在数据结构中传达这种含义，只是通过将两个值分组到一个元组中；我们可以将这两个值组合成一个结构，并将每个结构字段命名为重要内容。这将使未来的代码维护者更容易理解各种值如何相互作用以及它们的目的是什么。

该程序演示了如何改进 `parse_config` 函数。

```
use std::env;
use std::fs;
fn main() {
    let args: Vec<String> = env::args().collect();
    let config = parse_config(&args);
    println!("Search for {}", config.query);
    println!("File {}", config.filename);
    let contents = fs::read_to_string(config.filename)
        .expect("Something went wrong reading file");
    println!("With the text:\n{}", contents);
}

struct Config {
    query: String,
    filename: String,
}

fn parse_config(args: &[String]) -> Config {
    let query = args[1].clone();
    let filename = args[2].clone();
    Config {query, filename}
}
```

为 `Config` 创建一个构造函数

到目前为止，解析命令行参数的代码是从 `main` 中提取出来的，放在 `parse_config` 函数中。结果，我们能够观察到 `query` 和 `filename` 值是连接的，并且这种关系需要在我们的代码中反映出来。然后，为了能够从 `parse_config` 方法中返回值的名称作为结构字段名，我们构建了一个 `Config` 结构来标识 `query` 和 `filename` 的相关目的。

我们可以将 `parse_config` 从一个普通函数更改为一个名为 `new` 的函数，它与 `Config` 结构相关联，因为该函数的目的是生成一个 `Config` 实例。由于这个改进，代码将变得更加习惯化。通过调用 `String::new`，我们可以从标准库中创建类型的实例，比如 `String`。我们还可以调用 `Config::new`，通过将 `parse_config` 转换为与 `Config` 关联的新函数来生成 `Config` 实例。我们需要做的更改显示在代码中。

```
use std::env;
use std::fs;
fn main() {
    let args: Vec<String> = env::args().collect();
    let config = Config::new(&args);
    println!("Search for {}", config.query);
    println!("File {}", config.filename);
    let contents = fs::read_to_string(config.filename)
        .expect("Something went wrong reading file");
    println!("With the text:\n{}", contents);
}

struct Config {
```



```

        query: String,
        filename: String,
    }
    impl Config {
        fn new(args: &[String]) -> Config {
            let query = args[1].clone();
            let filename = args[2].clone();
            Config {query, filename}
        }
    }
}

```

我们已经将 `main` 中的代码更改为调用 `Config::new` 而不是 `parse_config`。
`parse_config` 的名称已更改为 `new`，并已移动到一个 `impl` 块中，该块将新函数与 `config` 关联起来。通过再次编译来确保代码正常工作。

修复错误处理

现在我们将重点关注改进错误处理。请记住，在索引 1 或 2 处访问 `args` 向量中的值将导致程序出现错误，如果该向量的项少于 3 个。如果你不带任何参数运行程序，它将如下所示：

```

D:\workspace\rust-works\minigrep>cargo run
    Finished dev [unoptimized + debuginfo] target(s) in 0.01s
    Running `target\debug\minigrep.exe`
thread 'main' panicked at 'index out of bounds: the len is 1 but the index is 1', src\main.rs:18:21
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

```

错误消息“index out of bounds: The len is 1 but The index is 1”是为程序员准备的。它对我们的最终用户理解发生了什么以及他们应该做什么没有帮助。现在让我们来解决这个问题。

改进错误消息

在访问索引 1 和 2 之前，我们在新函数中添加一个检查，以确保切片足够长。如果切片不够长，程序会 `panic`，并显示比索引越界更好的错误消息。

```

// 代码片段
fn new(args: &[String]) -> Config {
    if args.len() < 3 {
        panic!("not enough argument");
    }
}

```

我们在 `Guess::new` 方法中进行的 `panic!` 调用代码类似于之前当 `value` 值参数超出有效范围时的 `panic!` 调用。这里我们不检查 `value` 值的范围，而是检查 `args` 数组至少长度为三，并且假设满足这个条件，函数的其余部分可以继续。如果 `args` 数组包含少于三个元素，则满足此条件，并且立即调用 `panic!` 宏以终止程序。

让我们在不带任何参数的情况下运行程序，看看现在的错误是什么样子的：

```
D:\workspace\rust-works\minigrep>cargo run
Compiling minigrep v0.1.0 (D:\workspace\rust-works\minigrep)
Finished dev [unoptimized + debuginfo] target(s) in 0.68s
Running `target\debug\minigrep.exe`
thread 'main' panicked at 'not enough argument', src\main.rs:19:13
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

用测试驱动开发（TDD）开发库的功能

既然我们已经将逻辑移到 `src/lib.rs` 中，那么为代码的基本功能构建测试就容易得多了。将参数收集和错误处理放在 `src/main.rs` 中。不必从命令行执行二进制文件，我们可以直接使用各种参数调用函数并检查返回值。请随意为 `Config::new` 特性开发测试，并自行运行函数。

在本部分中，我们将使用 TDD 过程向 `minigrep` 程序添加搜索逻辑。该软件开发技术的步骤如下：

- 创建一个失败的测试并执行它，以确保它因预期的原因而失败。
- 应该编写或修改足够的代码以通过新的测试。
- 重构我们刚刚添加或更新的代码，并再次检查测试是否仍能通过。
- 步骤 1 必须重复！

TDD 只是构建软件的众多技术之一，但它对代码设计有帮助。首先编写测试，然后编写通过测试的代码，确保在整个过程中有良好的测试覆盖率。我们将运行该特性的实现，该特性在文件内容中搜索查询字符串并返回与查询匹配的行列表。该功能将被添加到一个名为 `search` 的函数中。

删除 `src/lib.rs` 和 `src/main.rs` 中我们用来检查程序行为的 `println!` 语句，因为我们不再需要它们。然后，我们将在 `src/lib.rs` 中添加一个带有测试函数的测试模块。`test` 函数定义了 `search` 函数的行为：它接受一个 `query` 其中包含了要 `search` 查询的文本，并且只返回包含查询文本的行。该测试显示在上面的代码中，但目前还不能编译。

```
// src/lib.rs
#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn one_result() {
        let query = "duct";
        let content = "\
Rust:
safe, fast, productive.
Pick three.";
    }
```

```

        assert_eq!(vec!["safe, fast, productive."], search(query, content));
    }
}

```

这个测试寻找的是“duct”这个词。我们在三行文本中检索，但其中只有一行包含单词“duct”(注意，在开始的双引号之后的反斜杠告诉 Rust 不要在这个字符串字面量的内容开始放置换行符)。我们断言搜索函数提供的结果只包含我们期望的行。

我们不能运行这个测试并看着它失败，因为它甚至没有编译：search 函数甚至还没有实现！现在我们要添加足够的代码来构建和执行测试，如前所述，通过定义 search 函数总是返回一个空向量。然后，测试应该运行并失败，因为空向量不匹配包含“safe, fast, productive.”行的向量。

```

// src/lib.rs
pub fn search<'c>(query: &str, content: &'a str) -> Vec<&'c str> {
    vec![]
}

```

值得注意的是，我们需要指定显式生命周期 'c，它在 search 签名中定义，并与 contents 参数和返回值一起使用。记住，在第 8 章中，生命周期参数决定了哪个参数的生命周期与返回值的生命周期相链接。在本例中，我们指定返回的向量应该包含与 content 参数切片相关的字符串切片(而不是 query 参数)。

换句话说，我们告诉 Rust，search 函数返回的数据将与 contents 参数中提供的数据一样持久。这是至关重要的！切片引用的数据必须是合法的，这样引用才有效；如果编译器认为我们生成的是 query 字符串切片而不是 content 切片，它将不能正确地执行安全检查。

如果试图在没有生命周期注释的情况下编译此代码，将会得到一个错误。

与环境变量一起工作

Minigrep 将通过添加不区分大小写的 search 选项进行增强，用户可以通过环境变量启用该选项。我们可以让它成为一个命令行选项，用户每次需要它工作时都必须输入它，但我们将使用一个环境变量。这允许我们的用户只需设置一次环境变量，并让他们在该终端会话中的所有搜索不区分大小写。

不区分大小写的 search 函数编写失败测试

当环境变量被设置为 true 时，我们将调用新的不区分大小写的 search 方法。这次我们将继续 TDD 过程；因此，第一步是编写一个失败的测试。为了澄清两个测试之间的区别，我们将为新的不区分大小写的 search 函数编写一个新的测试，并将旧的测试更改为区分大小写。

```

// src/lib.rs
#[cfg(test)]

```

```

mod tests {
    use super::*;
    #[test]
    fn case_sensitive() {
        let query = "duct";
        let content = "\
Rust:
safe, fast, productive.
Pick three.
Duct tape.";
        assert_eq!(vec!["safe, fast, productive."], search(query, content));
    }
    #[test]
    fn case_insensitive() {
        let query = "rUsT";
        let content = "\
Rust:
safe, fast, productive.
Pick three.
Trust me.";
        assert_eq!(
            vec!["Rust:", "Trust-me."],
            search_case_insensitive(query, content)
        );
    }
}

```

我们还改变了旧测试的内容。当以区分大小写的方式进行搜索时，我们插入了一个带有短语“Duct tape.”的新行。它使用了大写的 D，故不应该与查询“duct”匹配。通过对现有测试进行这种更改，我们可以确保不会破坏已经开发的区分大小写的搜索功能。这个测试现在应该通过了，并且在我们处理不区分大小写的搜索时，它应该继续通过。

“rUsT”则是用于新的不区分大小写搜索测试的查询。即使这两行与查询的大小写不同，查询“rUsT”也应该匹配包含“Rust:”的行（大写 R）和“Trust me.”行。在我们即将实现的搜索不区分大小写的函数中。这是我们的失败测试，它将无法编译，因为还没有定义搜索不区分大小写的函数。要查看测试构建和失败，添加一个总是返回空向量的框架实现，与我们处理 `search` 函数的方式类似。

实现 `search_case_sensitive` 函数

下面的示例显示了 `search_case_sensitive` 函数，它与 `search` 函数几乎相同。唯一的区别是，我们将查询和每一行都小写，所以当我们检查该行是否包含 `query` 时，无论输入参数是什么情况都是相同的情况。

```

// src/lib.rs
pub fn search_case_insensitive<'a>(<

```

```

        query: &str,
        content: &'a str
    ) -> Vec<&'a str> {
        let query = query.to_lowercase();
        let mut result = Vec::new();
        for line in content.lines() {
            if line.to_lowercase().contains(&query) {
                result.push(line);
            }
        }
        result
    }
}

```

query 字符串首先小写，并存储在具有相同名称的阴影变量（shadowed variable）中。我们需要在 query 上调用 to_lowercase，以便无论用户查询的是“rust”、“RUST”、“Rust”还是“rUsT”，我们都将其忽略大小写并视为“rust”。虽然 to_lowercase 可以处理基本的 Unicode 字符，但它并不准确。如果我们要开发一个真正的程序，那么我们需要在这里做更多的工作，但本节讲述关于环境变量，而不是 Unicode，所以点到为止。

因为调用 to_lowercase 会创建新数据而不是引用当前数据，所以 query 现在是一个 String 而不是字符串切片。假设查询是“rUsT”：该字符串片缺少可以使用的小写 u 或 t，因此我们必须分配一个包含“rUsT”的新字符串。因为 includes 方法的签名被定义为接受字符串切片，所以现在在传递 query 作为参数时必须包含一个 & 号。

然后，在每一行上，在检查它是否包含对所有小写字母的 query 之前，添加一个对 to_lowercase 的调用。现在我们已经将 line 行和 query 查询都转换为小写，我们将能够定位匹配，而不管查询的大小写。

// src/lib.rs 的完整代码

// 原书本章的源代码内容比较零碎，这里 LQ 把代码汇总

```

#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn case_sensitive() {
        let query = "duct";
        let content = "\
Rust:
safe, fast, productive.
Pick three.
Duct tape.";
        assert_eq!(vec!["safe, fast, productive."], search(query, content));
    }
    #[test]
    fn case_insensitive() {
        let query = "rUsT";

```

```

        let content = "\
Rust:
safe, fast, productive.
Pick three.
Trust me.";
        assert_eq!(
            vec!["Rust:", "Trust me."],
            search_case_insensitive(query, content)
        );
    }
}

pub fn search<'a>(query: &str, content: &'a str) -> Vec<&'a str> {
    let mut result = Vec::new();
    for line in content.lines() {
        if line.contains(&query) {
            result.push(line);
        }
    }
    result
}

pub fn search_case_insensitive<'a>(
    query: &str,
    content: &'a str
) -> Vec<&'a str> {
    let query = query.to_lowercase();
    let mut result = Vec::new();
    for line in content.lines() {
        if line.to_lowercase().contains(&query) {
            result.push(line);
        }
    }
    result
}

```

让我们看看这些测试，看看这个实现是否通过：

```

D:\workspace\rust-works\minigrep>cargo test
Compiling minigrep v0.1.0 (D:\workspace\rust-works\minigrep)
Finished test [unoptimized + debuginfo] target(s) in 0.78s
Running unittests src\lib.rs (target\debug\deps\minigrep-41749b9e6
2da306d.exe)

```

```

running 2 tests
test tests::case_sensitive ... ok
test tests::case_insensitive ... ok

```

```

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered
out; finished in 0.00s

```

```

Running unittests src\main.rs (target\debug\deps\minigrep-bed007b5

```

```
57971686.exe)
```

```
running 0 tests
```

```
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered  
out; finished in 0.00s
```

```
Doc-tests minigrep
```

```
running 0 tests
```

```
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered  
out; finished in 0.00s
```

太棒了！编写的测试都通过了。让我们使用 `run` 函数来调用新的 `search_case_sensitive` 函数。首先，我们将在 `Config` 结构中添加一个开关，它允许我们在区分大小写和不区分大小写的搜索之间进行选择。因为我们还没有初始化这个字段，添加它会导致编译器错误：

```
// src/main.rs
pub struct Config {
    pub query: String,
    pub filename: String,
    pub case_sensitive: bool,
}
```

添加的 `case_sensitive` 字段，包含了布尔值。接下来，如下所示，我们需要 `run` 函数来验证 `case_sensitive` 字段的值，并使用它来确定是调用 `search` 函数还是调用 `search_case_sensitive` 函数。值得注意的是，这仍然无法编译。

```
// src/lib.rs
pub fn run(config: Config) -> Result<(), Box<dyn Error>> {
    let content = fs::read_to_string(config.filename)?;
    let result = if config.case_sensitive {
        search(&config.query, &content)
    } else {
        search_case_insensitive(&config.query, &content)
    };
    for line in result {
        println!("{}", line);
    }
    Ok(())
}
```

最后但并非最不重要的是，我们必须寻找环境变量。与环境变量交互的函数在标准库的 `env` 模块中，因此我们将在 `src/lib.rs` 顶部行使用 `use std::env;`。把那个模块引入范围。然后，如示例所示，我们将利用 `env` 模块中的 `var` 函数查找名为 `case_sensitive` 的环境变量。


```
// src/lib.rs
use std::env;
impl Config {
    pub fn new(args: &[String]) -> Result<Config, &str> {
        if args.len() < 3 {
            return Err("not enough argument");
        }
        let query = args[1].clone();
        let filenames = args[2].clone();
        let case_sensitive = env::var("CASE_INSENSITIVE").is_err();
        Ok(Config {
            query,
            filenames,
            case_sensitive,
        })
    }
}
```

我们将创建一个名为 `case_sensitive` 的新变量。我们调用 `env::var` 函数，并将 `CASE_INSENSITIVE` 环境变量的名称提供给它以设置其值。如果设置了环境变量，`env::var` 函数将生成一个 `Result`，该 `Result` 是成功的 `Ok` 变体，其中包含环境变量的值。如果没有设置环境变量，它将返回 `Err` 变量。

我们正在检查它是否是一个错误，因此在 `Result` 上使用 `is_err` 方法取消设置，这意味着它应该进行区分大小写的搜索。如果 `CASE_INSENSITIVE` 环境变量被设置为任何值，`is_err` 将返回 `false`，应用程序将执行不区分大小写的搜索。我们不关心环境变量的值；我们所关心的是它是 `set` 还是 `unset`，所以我们使用 `is_err` 而不是 `unwrap`、`expect` 或任何其他 `Result` 方法。

我们将 `case_sensitive` 变量的值提供给 `Config` 实例，以便运行函数可以读取它，并决定是调用 `search` 还是调用 `search_case_insensitive`，就像前面所做的那样。

让我们来看看它是如何进行的！首先，我们不使用环境变量，只使用 `to` 查询来运行程序，它应该匹配所有小写单词“to”的行：

```
$ cargo run to thepoem.txt
Compiling minigrep v0.1.0 (file:///projects/minigrep)
  Finished dev [unoptimized + debuginfo] target(s) in 0.1s
  Running `target/debug/minigrep to thepoem.txt`
Are we nobody, too?
How dreary to be somebody.
```

这似乎仍然有效！让我们再次运行程序，但这一次将 `CASE_INSENSITIVE` 设置为 1，并使用相同的 `query`：

```
PS> $Env:CASE_INSENSITIVE=1; cargo run to thepoem.txt
```

现在，`CASE_INSENSITIVE` 将在我们的 Shell 的其余部分持续存在。

```
PS> Remove-Item Env:CASE_INSENSITIVE
```

我们应该得到带有“to”字的词，可能还有大写字母：

```
$ CASE_INSENSITIVE=1 cargo run to poem.txt
  Finished dev [unoptimized + debuginfo] target(s) in 0.0s
  Running `target/debug/minigrep to poem.txt`
Are we nobody, too??
How dreary to be somebody.
To tell our name the livelong day
To an admiring bog.
```

我们还得到了含有“To”的行，这真是太棒了！我们的 `minigrep` 程序现在可以根据环境中的变量进行不区分大小写的搜索。我们现在知道了如何使用命令行参数或环境变量来处理选项。

对于相同的配置，有几个程序提供参数和环境变量。在这些情况下，程序会选择哪个优先。另一个实践是尝试使用命令行选项或环境变量来管理大小写不敏感的问题，进行另一种练习。

如果程序运行时，一个设置为区分大小写，另一个设置为不区分大小写，那么决定是命令行输入优先还是环境变量优先。

在 `std::env` 模块中可以找到更多处理环境变量的有用功能：要查看哪些是可访问的，请查看文档。

将错误消息写入标准错误，而不是标准输出

我们目前正在使用 `println!` 宏将所有的输出写到终端。大多数终端都提供用于一般信息的标准输出(`stdout`)和用于错误消息的标准错误(`stderr`)。这种区别允许用户将程序的成功输出指向文件，同时仍然在屏幕上打印错误警告。

因为 `println!` 宏只能打印到标准输出，所以我们必须用另一种方法打印标准错误。

检查错误写入的位置

让我们首先看看 `minigrep` 的输出现在是如何写入标准输出的，包括我们想要发送到标准错误的任何错误消息。我们将通过将标准输出流转发到一个文件，同时有意地产生一个错误来实现这一点。因为标准错误流不会被重定向，所以提交给标准错误的任何内容将继续出现在屏幕上。

即使我们将标准输出流重定向到文件，我们仍然可以在屏幕上看到错误消息，因为命令程序的目的是将错误信息发送到标准错误流。我们的程序目前行为不端：我们即将发现，它反而将错误信息的输出保存到了一个文件中！这就是我们的程序。

用 `>` 和文件名 `output.txt`（我们希望将标准输出流重定向到该文件）运行该程序，将展示这种行为。我们将不传递任何参数，这将导致以下错误：

```
$ cargo run > output.txt
```

(>) 语法指示 shell 将标准输出写入 `output.txt`，而不是屏幕。我们在屏幕上没有看到预期的错误消息，所以它一定已经保存到文件中了。

Problem parsing argument: not enough argument

正如预期的那样，错误消息被写入标准输出。向标准错误输出这样的错误消息要有用得着，这样只有成功运行的数据才会出现在文件中。我们要改变这一点。

打印错误到标准错误

为了改变错误消息的打印方式，我们将使用示例中的代码。由于我们在本章之前完成了重构，所有产生错误警告的代码现在都在 `main` 函数中。因为标准库包含了 `eprintln!` 宏，它可以将输出打印到标准错误流，我们将在之前两处调用 `println!` 来打印错误的语句中改用 `eprintln!` 来打印错误。

```
fn main() {
    let args: Vec<String> = env::args().collect();
    let config = Config::new(&args).unwrap_or_else({err} {
        eprintln!("The Problem parsing argument: {}", err);
        process::exit(1);
    });
    if let Err(e) = minigrep::run(config) {
        eprintln!("The Application error: {}", e);
        process::exit(1);
    }
}
```

让我们在把 `println!` 改为 `eprintln!` 后再次运行该程序，这次没有任何参数，并使用 `>` 来转移标准输出。

```
$ cargo run > theoutput.txt
Problem parsing argument: not enough argument
```

现在我们在屏幕上看到了错误，而且 `output.txt` 是空的，正如我们从一个命令行程序所期望的那样。

让我们再试一次，用不出错的参数，但仍将标准输出重定向到一个文件。

```
$ cargo run to thepoem.txt > theoutput.txt
```

我们不会在终端得到任何输出，但我们的结果会在 `output.txt` 中。

```
Are we nobody, too??
How dreary to be somebody.
```

这表明，在适当的情况下，我们现在使用标准输出进行成功输出，使用标准错误进行错误输出。

在本章中，我们讨论了如何构建命令行程序，其中讨论了接受命令行参数、读取文件以及重构以改进模块化和错误处理。我们还介绍了开发库的功能和使用环境变量。

C10: Cargo 和 crates.io

- 2022.11.27

在本章中

- 使用 release profile 配置自定义构建过程
- 发布 crate 到 crates.io 仓库
- Cargo 工作区
- 使用 cargo install 命令从 crates.io 安装二进制文件
- 使用自定义命令扩展 Cargo

在前一章中，我们介绍了如何构建命令程序。本章将讨论使用 release profile 配置项自定义构建过程，发布一个 crate 到 crates.io 和 cargo 工作区。此外，通过 cargo install 从 crates.io 安装二进制文件，以及使用自定义命令扩展 Cargo。

使用 release profile 配置自定义构建过程

Rust 的 release profile 配置项是预定义的和可自定义的配置项，具有各种配置，让程序员可以针对特定的编译选项有更多的控制。每个 profile 配置项都是独立设置的。

Cargo 有两个主要的 profile 配置项：dev 和 release。当我们运行 cargo build 进行构建时，Cargo 会利用 dev 配置项；当我们运行 cargo build --release 进行构建时，Cargo 会利用 release 配置项。release 配置项为发布构建提供了良好的默认值，而 dev 配置项为开发提供了良好的默认值。

这些 profile 配置项的名称可以从构建的输出中得知：

```
$ cargo build
Finished dev [unoptimized + debuginfo] target in 0.2s
$ cargo build --release
Finished release [optimized] target(s) in 0.1s
```

编译器使用不同的 profile 配置项，正如构建输出中的 dev 值和 release 值所示。

如果在 Cargo.toml 配置文件中没有 [profile.] 节的时候，cargo 为每个 profile 配置项使用默认设置。开发者可以通过添加 [profile.*] 节来来覆盖默认的 profile 设置。例如，对于 dev 和 release 配置项，opt-level 的设置默认如下：

```
[profile.dev]
opt-level = 0

[profile.release]
opt-level = 3
```

`opt-level` 的取值范围为 0~3，此参数决定 Rust 将对我们的代码进行多少优化。当使用额外的优化时，会增加编译时间。因此，如果我们正在开发并频繁编译代码，即使最终代码运行较慢，我们也会喜欢更快的编译。因此，开发的默认选项级别是 0。当你准备发布你的代码时，编译时花费额外的时间是明智的。在 `release` 发布模式下我们只编译一次，但是您将多次运行构建的程序；因此，发布模式以换取更长的构建时间为我们提供了运行更快的代码。因此，`release profile` 配置项的默认选项级别是 3。

我们可以覆盖 `Cargo.toml` 配置文件中的任何默认设置项。例如，如果我们想在 `dev profile` 开发配置项中使用优化级别 1，我们可以将下面这两行添加到 `Cargo.toml` 配置文件中：

```
[profile.dev]
opt-level = 1
```

这段代码将覆盖默认值 0。当我们运行 `cargo build` 构建时，Cargo 现在将使用 `dev profile` 开发配置项的默认值和我们的自定义的优化级别。Cargo 将应用比默认值更多的优化，因为 `opt-level` 设置为 1，但没有 `release` 构建的优化那么多。

发布 crate 到 crates.io

我们使用 `crates.io` 包作为项目依赖，而且我们还可以通过将包发布以便与他人共享代码。因为 `crate` 在 `crates.io` 注册和发布我们产品的源代码，`crates.io` 站点主要用于托管开源代码。

Rust 和 Cargo 都包含让用户更容易使用和找到我们发布的包的功能。接下来，在展示如何发布包之前，我们将介绍其中一些特性。

提供有用的文档注释

花时间为我们的包生成文档是非常重要的，因为它将帮助其他用户理解如何以及何时使用它们。我们在第 2 章中介绍了如何用两个斜杠 `//` 注释 Rust 代码。文档注释，通常称为标准注释，是 Rust 中生成 HTML 文档的注释类型。HTML 显示了公开 API 项的文档注释内容，这些公开 API 项是为那些想要学习如何使用我们的 `crate` 而不是如何实现我们的 `crate` 的程序员准备的。

文档注释使用 Markdown 语法进行格式化，并使用三个斜杠 `///` 而不是两个。文档注释应该直接放在文档记录之前。在名为 `my_crate` 的 `crate` 中使用 `add_one` 方法的文档说明如下例所示：

```
/// Add one to the number given.
///
/// # Example
///
/// ```
/// let arg = 6;
/// let answer = my_crate::add_one(arg);
```

```

///
/// assert_eq!(7, answer);
/// ```
pub fn add_one(x: i32) -> i32 {
    x+1
}

```

我们首先解释 `add_one` 函数，然后转到名为 `Example` 的部分，在那里我们将展示描述如何使用 `add_one` 方法的代码。使用 `cargo doc`，我们可以从这个文档注释生成 HTML 文档。该命令启动 `rustdoc` 工具，创建 HTML 文档并将其保存在 `target/doc` 目录中。

在 web 浏览器中运行 `cargo doc -open` 将会使用浏览器打开我们当前创建的 `crate` 文档（以及所有 `crate` 依赖项的文档）生成的 HTML。点击 `add_one` 链接转到 `add_one` 函数，我们可以看到文档注释中的内容生成后的样子。

Function doc1::add_one 

source · [-]

```
pub fn add_one(x: i32) -> i32
```

[-] Add one to the number given.

Example

```

let arg = 6;
let answer = my_crate::add_one(arg);

assert_eq!(7, answer);

```

add_one 函数文档

常用的章节

在前面的例子中，我们使用“# Examples”的 Markdown 语法标题在 HTML 中创建一个标题为“Examples”的章节部分。`crate` 作者经常在他们的文档中使用的其他章节部分包括：

- **Panic 章节：**用文档记录可能会出现 `panic` 场景的函数。如果不希望程序出现 `panic`，函数的调用者应该避免在这些条件下进行调用。
- **Error 章节：**如果函数返回 `Result`，用文档记录可能发生的错误类型以及可能导致这些问题返回的情况，可能有助于调用者在构建代码时以多种方式处理各种类型的错误。
- **Safety 章节：**如果调用的函数是不安全的，应该有相关的文档章节说明为什么它不安全，以及函数希望调用者维护的不变量。

尽管大多数文档注释不需要所有这些章节部分，但这是一个有用的清单，可以提醒我们调用代码的人会对代码的哪些方面感兴趣。

文档注释作为测试

在文档注释中包含示例代码块有助于展示如何使用我们的库。它还有一个额外的好处：`cargo test` 将执行我们文档中的代码示例作为测试！没有什么比文档包含示例更好的做法了。反之，没有什么比文档的示例执行失败更令人沮丧了。因为自文档创建后，代码可能已经发生了更改。如果我们使用前面例子的 `add_one` 函数的文档运行 `cargo test`，我们将在测试结果中看到如下内容：

```
running 1 test
test src/lib.rs - add_one (line 5)..... ok
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered
out; finished in 0.98s
```

现在，如果我们改变函数或 `Example` 中的任何一个，在示例中再次运行 `cargo test`，那么 `assert_eq!` 将会 `panic`，我们会发现文档测试捕获了示例和代码不同步的事实！

注释包含的项

`//!` 是另一种类型的文档注释，它将文档添加到包含注释的项，而不是注释之后的项。为了将 `crate` 或模块作为一个整体来文档化记录，我们通常在 `crate` 根文件（`src/lib.rs`）或模块内部中使用这些文档注释。

例如，如果我们想在 `src/lib.rs` 文件的开头添加文档注释，此文档注释指定了包含 `add_one` 函数的用途，那么我们可以使用以 `//!` 开始的文档注释，如下所示：

```
/// # My Crate
///
/// !`my_crate` is collection of utilities to make performing certain
calculations more convenient.

/// Add one to number given.
// --snip--
```

在以 `//!` 开头的最后一行的代码之后没有进一步的代码了。我们记录的是包含此注释的项，而不是此注释之后的项，因为我们以 `//!` 开始注释而不是以 `/` 开始。`src/lib.rs` 文件，也就是 `crate` 根，在这种情况下包含了这个注释。这些说明适用于整个 `crate`。

当我们运行 `cargo doc --open` 时，这些注释将出现在 `my_crate` 文档的首页，位于 `crate` 中公开 API 列表的上方。

使用 `pub use` 导出一个方便的公共 API

在第 6 章中，我们学习了如何使用 `mod` 关键字将代码安排到模块中，如何使用 `pub` 关键字将项公开，以及如何使用 `use` 关键字将项引入范围。然而，在设计

crate 时，对 crate 开发者而言有意义的结构可能对用户来说不那么实用。我们可能希望使用多层次的分层结构来组织结构，但对于使用的用户就可能出现查找所需时遇到困难，而且用户可能还会因为必须输入 use

my_crate::some_module::another_module::UsefulType; 而不是
my_crate::UsefulType;

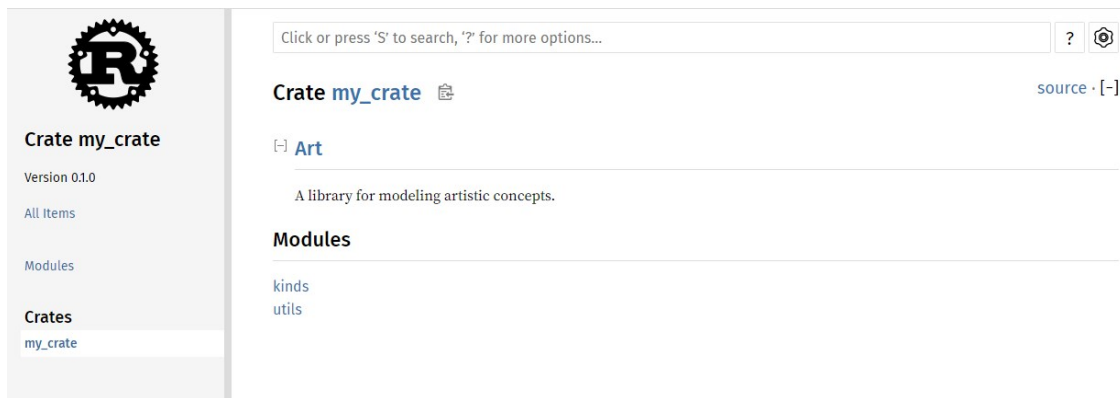
在发布 crate 时，我们公开 API 的结构是至关重要的。我们的 crate 用户并不像我们那样熟悉结构，如果我们的 crate 有一个很复杂的模块层次结构，那么用户可能很难找到他们所需 API 的位置。

好消息是，即使用户不容易从某个 crate 库使用这个结构，对于开发者而言也不必重新组织该 crate 的内部结构：相反，我们可以使用 pub use 来重新导出项，以创建一个不同于内部结构的公开结构。重新导出（re-exporting）是将一个处于某个位置（第一个位置）的公共对象放入另一个位置（第二个位置）的过程，就好像它是在那里（第二个位置）定义的一样。

假设我们为艺术概念建模创建了一个艺术库。这个库中包含了两个模块：其中一个 kinds 模块，它有两个名为 PrimaryColor 和 SecondaryColor 的枚举；另一个是 utils 模块，它有一个名为 mix 的方法，如下所示：

```
///! # Art
///!
///! A library for modeling artistic concepts.
pub mod kinds {
    /// The primary colors according to the RYB color model.
    pub enum PrimaryColor {
        Red,
        Blue,
        Yellow,
    }
    // the secondary colors according to the RYB color model.
    pub enum SecondaryColor {
        Green,
        Orange,
        Purple,
    }
}
pub mod utils {
    use crate::kinds::*;
    /// Combines two primary colors in the equal amounts to create
    /// a secondary color.
    pub fn mix(c1: PrimaryColor, c2: PrimaryColor) -> SecondaryColor {
        // --snip--
    }
}
```

运行 cargo doc -open 可以看到生成的文档如下图所示：



生成的文档

`mix` 函数和 `PrimaryColor` 和 `SecondaryColor` 枚举类型没有出现在首页。我们必须首先单击 `kinds` 或 `utils` 超链接来查看它们。

另一个依赖于这个库的 `crate` 需要利用语句将对象从 `art` 引入作用域，并指定当前的模块结构。下图是一个使用 `PrimaryColor` 并结合 `art crate` 组件的示例：

```
use art::kinds::PrimaryColor;
use art::utils::mix;
fn main() {
    let red = PrimaryColor::Red;
    let blue = PrimaryColor::Blue;
    mix(red, blue);
}
```

根据 `art crate` 代码作者的文档所述，`PrimaryColor` 在 `kinds` 模块中，而 `mix` 在 `utils` 模块中。`art crate` 的开发人员比使用 `art crate` 的开发人员对模块结构更感兴趣。将 `crate` 的部分安排到 `kinds` 和 `utils` 模块中的内部结构不包含对理解如何使用 `art crate` 有用的信息。相反，`art crate` 的模块结构让开发人员感到困惑，因为它要求开发人员知道在哪里查找，而且这很不方便，因为开发人员必须在使用语句中指定模块名称。

我们可以修改 `art crate` 的代码，包括 `pub use` 语句，以重新导出顶层的项，从公开 API 中删除内部结构，如下所示：

```
/// # Art
///
/// A library for modeling artistic concepts.

pub use self::kinds::PrimaryColor;
pub use self::kinds::SecondaryColor;
pub use self::utils::mix;
pub mod kinds {
    // --snip--
}
pub mod utils {
```

```
    // --snip--  
}
```

设置 crate.io 帐户

我们必须先在 crate.io 站点上注册一个帐户，并在发布任何 `crate` 之前获取一个 `API token` 令牌。要做到这一点，访问 `crate.io` 主页，然后用我们的 `GitHub` 帐户登录。

（目前，需要一个 `GitHub` 帐户，但该网站可能在未来会启用其他帐户的创建方法）登录后，转到 <https://crates.io/me/> 并从帐户设置中检索 `API` 密钥。然后，使用我们的 `API` 密钥，运行 `cargo login` 登录命令：

```
$ cargo login abcdefghijklmnopqrstuvwxyz012345
```

这个命令会告知 `Cargo` 我们的 `API` 令牌，它将保存到本地的 `~/.Cargo/credentials` 中。这个令牌是一个密钥，因此不要告诉任何人。如果我们因为各种原因与其他人分享了令牌，那么我们应该撤销此令牌并重新创建一个新令牌。

Cargo 工作区

我们在第 9 章中使用二进制 `crate` 和库 `crate` 创建了一个包。随着项目的进展，我们可能会发现库 `crate` 的大小在不断增长，这促使我们将包分割为额外的库 `crate`。`Cargo` 有一个称为工作区的特性，它可以帮助管理在本例中同时开发的几个相关包。

创建工作区

工作区（`workspace`）是具有相同 `Cargo.lock` 和输出目录的包集合。让我们在工作区的帮助下创建一个项目。我们将使用简单的代码来专注于工作区的结构。要结构化一个工作站（`workstation`）有许多不同的方法，我们将展示一种最常见的方法。我们将创建一个包含二进制文件和两个库的工作区。二进制文件的主要功能将依赖于这两个库。其中一个库提供 `add_one` 函数，另一个库提供 `add_two` 函数。这三个盒子的工作区是相同的。我们将首先创建一个新的工作区目录：

```
D:\workspace\rust-works>md add  
D:\workspace\rust-works>cd add
```

然后在 `add` 目录下创建 `Cargo.toml` 文件，该文件将设置整个工作区。该文件缺少 `[package]` 节和在其他 `Cargo.toml` 中看到的元数据。相反，此文件将从一个 `[workspace]` 节开始，这部分允许我们通过提供与我们的二进制 `crate` 相关联的包路径来向工作区添加成员。在本例中，该路径是 `adder`：

```
[workspace]  
members = [  
    "adder",  
]
```

然后，在 `add` 根目录下，运行 `cargo new` 创建 `adder` 的二进制 `crate`：

```
D:\workspace\rust-works\add>cargo new adder
Created binary (application) `adder` package
```

此时，我们可以运行 `cargo build` 来创建工作区。以下是 `add` 目录的文件结构：

```
D:\workspace\rust-works\add>cargo build
Compiling adder v0.1.0 (D:\workspace\rust-works\add\adder)
Finished dev [unoptimized + debuginfo] target(s) in 1.90s
```

```
D:\workspace\rust-works\add>tree /f
```

卷 dev 的文件夹 PATH 列表

卷序列号为 48FB-5163

```
D:.\
├──Cargo.lock
├──Cargo.toml
├──adder
│   ├──Cargo.toml
│   └──src
│       └──main.rs
└──target
```

LQ 注：target 目录下的内容省略，因为它们是构建时生成的文件。

编译后的工件（artifacts）被放置在工作区顶层的 `target` 目录中（LQ：即 `add` 目录）；`adder` 包没有自己的 `target` 目标目录。如果我们从 `adder` 目录下运行 `cargo build`，结果会是编译后的工件仍然放在 `add/target` 目录中，而不是 `add/adder/target` 目录。因为工作区中的 `crate` 是相互依赖的，所以 Cargo 以这种方式在工作区中构造 `target` 目标目录。如果每个 `crate` 都有自己的 `target` 目标目录，那么每个 `crate` 都必须重新编译工作区中的每个其他 `crate`，以便在其 `target` 目标目录中拥有工件。而通过共享单个 `target` 目标目录，`crate` 可以最小化不必要的重建。

在工作区中创建第二个包

接下来，在工作区中创建一个新的成员包，并将其命名为 `add-one`。在顶层的 `Cargo.toml` 中指定成员列表中 `add-one` 的路径：

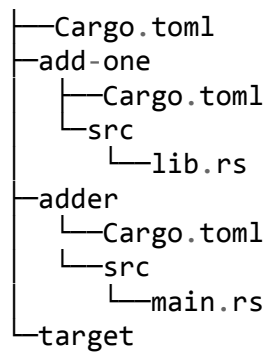
```
[workspace]
members = [
    "adder",
    "add-one",
]
```

接下来，创建一个名为 `add-one` 的库：

```
D:\workspace\rust-works\add>cargo new add-one --lib
Created library `add-one` package
```

这下目录结构应该是这样：

```
D:.\
├──Cargo.lock
```



让我们在 `add-one/src/lib.rs` 文件中添加一个 `add_one` 函数:

```
pub fn add_one(c: i32) -> i32 {
    c + 1
}
```

我们可以将 `adder` 包与二进制文件一起使用，具体取决于它们依赖的库 `add-one` 包，这是我们在工作区拥有的另一个包。首先，我们必须在 `adder/Cargo.toml` 中包含对 `add-one` 库的路由依赖。

```
[dependencies]
add-one={path="../add-one"}
```

由于 `Cargo` 不会假设工作区中的 `crate` 存在依赖，因此我们必须明确指定 `crate` 的依赖关系。

然后，我们在 `adder crate` 中从 `add-one` 库 `crate` 中使用 `add_one` 函数。要将新的 `add-one` 库 `crate` 纳入作用域，请打开 `adder/src/main.rs` 文件并在开头添加一个 `use` 行。然后，如下面的代码所示，更改 `main` 函数以调用 `add_one` 方法。

```
use add_one;
fn main() {
    let numb = 13;
    println!("Hello, everyone. {} plus one is {}!", numb, add_one::add_one(numb));
}
```

LQ 补充: 执行程序, 如下

```
D:\workspace\rust-works\add>cargo run
Compiling add-one v0.1.0 (D:\workspace\rust-works\add\add-one)
Compiling adder v0.1.0 (D:\workspace\rust-works\add\adder)
Finished dev [unoptimized + debuginfo] target(s) in 2.48s
Running `target\debug\adder.exe`
Hello, everyone. 13 plus one is 14!
```

通过 cargo install 从 crates.io 安装二进制文件

我们可以使用 `cargo install` 命令在本地安装和使用二进制 crate。这些二进制 crate 并不是系统包的替代品，相反，它旨在成为让 Rust 开发者在 crates.io 上分享包的简单方法。请务必注意，您只能安装具有二进制目标（**binary target**）的软件包。与库目标（**library target**）相反，二进制目标是一个可运行的程序，它在当 crate 具有 `src/main.rs` 文件或以二进制（**binary**）定义的文件时构建。crate 通常在 README 文件中说明它是库、或者是二进制目标，又或者是两者兼而有之。

所有使用 `cargo install` 安装的二进制文件都保存在安装根目录的 `bin` 文件夹中。如果我们使用 `rustup.rs` 安装了 Rust 并且没有任何指定的配置，那么此目录将位于 `$HOME/.cargo/bin`。要方便地运行使用 `cargo install` 安装的应用程序，请确保该目录包含在 `$PATH` 环境变量中。

例如，我们在第 9 章中解释过 `ripgrep` 是用于搜索文件的 `grep` 工具的 Rust 实现。我们可能会使用以下命令来安装 `ripgrep`：

```
D:\workspace\rust-works\add>cargo install ripgrep
  Updating `rsproxy` index
  Downloaded ripgrep v13.0.0 (registry `rsproxy`)
  Downloaded 1 crate (272.1 KB) in 2.36s
  Installing ripgrep v13.0.0
  Downloaded quote v1.0.21 (registry `rsproxy`)
  Downloaded itoa v1.0.4 (registry `rsproxy`)
  Downloaded serde_json v1.0.89 (registry `rsproxy`)
  Downloaded unicode-width v0.1.10 (registry `rsproxy`)
  Downloaded serde_derive v1.0.148 (registry `rsproxy`)
  Downloaded grep v0.2.10 (registry `rsproxy`)
  Downloaded num_cpus v1.14.0 (registry `rsproxy`)
  Downloaded regex v1.7.0 (registry `rsproxy`)
  Downloaded thread_local v1.1.4 (registry `rsproxy`)
  Downloaded unicode-ident v1.0.5 (registry `rsproxy`)
  Downloaded grep-searcher v0.1.10 (registry `rsproxy`)
  Downloaded serde v1.0.148 (registry `rsproxy`)
  Downloaded grep-cli v0.1.6 (registry `rsproxy`)
  Downloaded base64 v0.13.1 (registry `rsproxy`)
  Downloaded grep-matcher v0.1.5 (registry `rsproxy`)
  Downloaded encoding_rs v0.8.31 (registry `rsproxy`)
  Downloaded crossbeam-utils v0.8.14 (registry `rsproxy`)
  Downloaded encoding_rs_io v0.1.7 (registry `rsproxy`)
  Downloaded bytecount v0.6.3 (registry `rsproxy`)
  Downloaded grep-regex v0.1.10 (registry `rsproxy`)
  Downloaded grep-printer v0.1.6 (registry `rsproxy`)
  Downloaded regex-syntax v0.6.28 (registry `rsproxy`)
  Downloaded ryu v1.0.11 (registry `rsproxy`)
  Downloaded memmap2 v0.5.8 (registry `rsproxy`)
  Downloaded ignore v0.4.18 (registry `rsproxy`)
  Downloaded fnv v1.0.7 (registry `rsproxy`)
```



```
Downloaded aho-corasick v0.7.20 (registry `rsproxy`)
Downloaded log v0.4.17 (registry `rsproxy`)
Downloaded syn v1.0.105 (registry `rsproxy`)
Downloaded proc-macro2 v1.0.47 (registry `rsproxy`)
Downloaded globset v0.4.9 (registry `rsproxy`)
Downloaded 31 crates (3.2 MB) in 3.13s (largest was `encoding_rs` at
1.4 MB)
  Compiling memchr v2.5.0
  Compiling cfg-if v1.0.0
  Compiling winapi v0.3.9
  Compiling log v0.4.17
  Compiling proc-macro2 v1.0.47
  Compiling unicode-ident v1.0.5
  Compiling lazy_static v1.4.0
  Compiling quote v1.0.21
  Compiling regex-automata v0.1.10
  Compiling syn v1.0.105
  Compiling serde_derive v1.0.148
  Compiling regex-syntax v0.6.28
  Compiling encoding_rs v0.8.31
  Compiling serde v1.0.148
  Compiling fnv v1.0.7
  Compiling serde_json v1.0.89
  Compiling unicode-width v0.1.10
  Compiling bstr v0.2.17
  Compiling aho-corasick v0.7.20
  Compiling grep-matcher v0.1.5
  Compiling once_cell v1.16.0
  Compiling regex v1.7.0
  Compiling thread_local v1.1.4
  Compiling encoding_rs_io v0.1.7
  Compiling textwrap v0.11.0
  Compiling globset v0.4.9
  Compiling winapi-util v0.1.5
  Compiling ryu v1.0.11
  Compiling same-file v1.0.6
  Compiling termcolor v1.1.3
  Compiling memmap2 v0.5.8
  Compiling strsim v0.8.0
  Compiling itoa v1.0.4
  Compiling bitflags v1.3.2
  Compiling bytecount v0.6.3
  Compiling crossbeam-utils v0.8.14
  Compiling grep-searcher v0.1.10
  Compiling clap v2.34.0
  Compiling atty v0.2.14
  Compiling base64 v0.13.1
  Compiling grep-cli v0.1.6
  Compiling walkdir v2.3.2
  Compiling grep-regex v0.1.10
```

```
Compiling ignore v0.4.18
Compiling num_cpus v1.14.0
Compiling ripgrep v13.0.0
Compiling grep-printer v0.1.6
Compiling grep v0.2.10
  Finished release [optimized + debuginfo] target(s) in 1m 00s
Installing C:\Users\chszs\.cargo\bin\rg.exe
Installed package `ripgrep v13.0.0` (executable `rg.exe`)
```

已安装二进制文件的位置和名称（在 ripgrep 例子中为 rg.exe）显示在输出的倒数第二行。然后我们可以运行 `rg -help` 来开始使用更快、更 Rust 化的扫描文件工具，只要它所在的目录包含在 `$PATH` 环境变量中，就可以随时随地运行。

LQ 注：运行 `rg --help` 输出的信息太多，这里用 `rg --version` 来代替，如下

```
D:\workspace\rust-works\add>rg --version
ripgrep 13.0.0
-SIMD -AVX (compiled)
+SIMD +AVX (runtime)
```

使用自定义命令扩展 cargo

Cargo 旨在添加新的子命令，而无需更改它们。如果我们的 `$PATH` 环境变量中有一个名为 `cargo-something` 的二进制文件，我们可以通过键入 `cargo something` 将其作为 cargo 子命令启动。当我们运行 `cargo -list` 时，还会列出这样的自定义命令。Cargo 的设计使得使用 `cargo install` 来安装扩展并且像内置 cargo 功能一样，执行它们变得非常简单。

本章介绍了使用 `release profile` 配置项自定义构建、将 crate 发布到 `crates.io` 以及 cargo 工作区。我们还讨论了使用 `cargo install` 从 `crates.io` 安装二进制文件，以及使用自定义命令扩展 cargo。

LQ 注：原书中 Cargo 扩展这一节的内容过于简陋，仅稍微做了简绍。

C11: 并发和状态

- 2022.12.2

在本章中

- 使用线程同时运行代码
- 使用消息传递在线程之间传输数据
- 共享状态的并发
- 具有 `Sync`（同步）和 `Send`（发送）`trait` 的可扩展的并发性

在前一章中，我们详细讨论了 `cargo` 和 `crates.io`，其中我们介绍了自定义构建、发布 `crate` 和 `cargo` 工作区。此外，我们还介绍了通过 `cargo install` 从 `crates.io` 安装二进制文件，以及使用自定义命令扩展 `cargo`。本章将讨论使用线程同时运行代码，使用消息传递在线程之间传输数据，共享状态的并发，以及具有同步和发送 `trait` 的可扩展的并发。

无畏并发

`Rust` 的另一个主要目标是安全高效地处理并发编程。随着越来越多的计算机利用其众多的处理器，并发编程（`concurrent programming`），其中程序的各个部分独立执行；和并行编程（`parallel programming`），程序的不同部分同时运行，变得越来越重要。在这些环境中编程历来都很复杂，容易出错；`Rust` 旨在补救这一问题。

最初，`Rust` 团队认为确保内存安全和防止并发问题是两个不同的挑战，需要使用不同的方法来解决。随着时间的推移，团队意识到所有权和类型系统是处理内存安全和并发问题的强大工具！由于所有权和类型检查，`Rust` 中的许多并发问题都是编译时错误，而不是运行时错误。因此，不正确的代码将拒绝编译并显示详细说明问题的错误消息，而不是要求您花费大量的精力尝试复制出现运行时并发错误的确切情况。

因此，我们将能够在编写代码时修复代码，而不是在代码发布到生产环境后修复代码。`Rust` 的这一特性被称为无畏并发（`Fearless Concurrency`）。无畏并发使我们能够编写无 `bug` 的代码，这些代码很容易重构，而不会产生新 `bug`。

许多语言对于同时处理多种困难所提供的答案持坚定态度。例如，`Erlang` 提供了漂亮的消息传递并发特性，但只有几种令人困惑的线程间状态通信方式。对于高级语言来说，只支持可行解决方案的子集是一种明智的技术，因为放弃一些控制以换取抽象会带来好处。另一方面，低级语言必须在任何给定的上下文中提供最高的性能，并且对硬件的抽象更少。因此，`Rust` 提供了几种工具，用于以任何与我们的需求最相关的方法来建模问题。

使用线程同时运行代码

可执行程序的代码在大多数现代操作系统中运行，操作系统同时维护多个进程。我们可以在程序中同时运行独立的组件。线程是连接这些独立组件的特性。

将程序的计算拆分为多个线程可以通过允许它同时执行多个任务来提高效率，但这也增加了复杂性。因为线程可以同时运行，所以不能保证我们在不同线程上的代码片段以相同的顺序运行。这可能会导致以下问题：

- 当线程以不确定的顺序访问数据或资源时，会出现争用情况。
- 当两个线程等待对方完成使用另一个线程拥有的资源时，会发生死锁，从而阻止两个线程继续执行。
- 错误只在特定情况下发生，难以一致地重现和修复。

Rust 试图减少使用线程的缺点，但在多线程上下文中编程仍然需要思考不同于单线程系统的代码结构。

线程在编程语言中以各种方法实现。为了创建新线程，各个操作系统提供了一套 API。在此模型中，一种编程语言利用操作系统的 API 生成线程，称为 1:1，即每个语言线程对应一个操作系统线程。

线程在不同的计算机语言中以不同的方式实现。绿色线程（green thread）是编程语言提供的线程，使用它们的编程语言将在不同数量的操作系统线程的上下文中执行它们。绿色线程范式被称为 M:N 模型，因为每 N 个操作系统线程有 M 个绿色线程，其中 M 和 N 的数值并不总是相同。

每个模型都有自己的优点和权衡，以及 **Rust** 最关键的 **Runtime**（运行时）支持。术语“运行时”可能有点误导，因为它在不同的情况下可能意味着不同的东西。

从这个意义上说，运行时指在每个二进制文件中引用语言的代码。根据语言的不同，这些代码可以是广泛的，也可以是次要的，但每种非汇编（non-assembly）语言都有一些运行时代码。因此，当有人说一种语言“没有运行时（no runtime）”时，他们通常意味着“最小的运行时（minimal runtime）”。较小的运行时包含较少的功能，但它们生成的二进制文件较小，使得在更多设置下，将该语言与其他语言混合起来更容易。尽管许多语言都可以用更大的运行时来换取额外的功能，但 **Rust** 需要一个较小的运行时，并且不能牺牲调用 C 以保持性能的能力。

绿色线程 M:N 模型需要更大的语言运行时来管理线程。因此，**Rust** 标准库只提供了 1:1 的线程实现。因为 **Rust** 是一种低级语言，所以如果我们希望用性能开销来换取更好的线程运行控制权和更方便的上下文切换成本，可以使用社区实现的一些 crate，它们实现了 M:N 模型。

我们已经在 **Rust** 中定义了线程，现在让我们看看如何使用标准库提供的线程相关的 API。

用 spawn 创建新线程

我们可以调用 `thread::spawn` 函数并为其提供一个闭包（closure），其中包含我们要在新线程中运行以启动新线程的代码。下面的示例显示了来自主线程的一些文本和来自新线程的一些文本：

```
use std::thread;
use std::time::Duration;
fn main() {
    thread::spawn(|| {
        for x in 1..10 {
            println!("hello number {} from spawned thread!", x);
            thread::sleep(Duration::from_millis(1));
        }
    });
    for x in 1..5 {
        println!("hello number {} from main thread!", x);
        thread::sleep(Duration::from_millis(1));
    }
}
```

请注意，当主线程终止时，无论新线程是否完成执行，它都将停止。该程序的输出可能会不时变化，但它应该看起来像这样：

```
hello number 1 from main thread!
hello number 1 from spawned thread!
hello number 2 from main thread!
hello number 2 from spawned thread!
hello number 3 from main thread!
hello number 3 from spawned thread!
hello number 4 from spawned thread!
hello number 4 from main thread!
hello number 5 from spawned thread!
```

对 `thread::sleep` 的调用会导致线程暂停一小段时间，从而使另一个线程能够运行。线程很可能会轮流执行，但这并不能保证：操作系统如何调度线程是一个因素。在此迭代中，`main` 主线程首先打印输出，即使派生线程的 `print` 语句先出现在代码中。而且，尽管我们指示派生线程打印输出到 `x` 为 9，但主线程在关闭之前只打印输出到 5。

如果我们执行此代码并且只从主线程获得输出，或者如果没有重叠，请尝试提高范围值，以便让操作系统有更多的机会在线程之间切换。

使用 JoinHandle 等待所有线程执行完成

前面示例中的代码不仅由于主线程的终止而使得派生线程提前终止，而且还不能确保派生线程的正确运行。而且，实际上线程的运行顺序是不可预测的。

通过将 `thread::spawn` 的返回值保存到变量中，我们可以解决派生线程无法运行或无法完全运行的问题。`JoinHandle` 是 `thread::spawn` 的返回类型。

`JoinHandle` 是一个有所有权的项，它将在调用 `join` 函数之前等待它的线程完成。以下示例演示了如何使用线程的 `JoinHandle` 并执行 `join`，以确保派生线程在主线程退出之前执行完成：

```
use std::thread;
use std::time::Duration;
fn main() {
    let handle = thread::spawn(|| {
        for x in 1..10 {
            println!("hello number {} from spawned thread!", x);
            thread::sleep(Duration::from_millis(1));
        }
    });
    for x in 1..8 {
        println!("hello number {} from main thread!", x);
        thread::sleep(Duration::from_millis(1));
    }
    handle.join().unwrap();
}
```

`handle` 上的 `join` 操作会阻止当前线程（即主线程），直到 `handle` 代表的线程终止。当线程被阻塞时，将阻止它执行工作或退出。运行此代码应该产生与之前类似的输出，因为我们在主线程的 `for` 循环之后放置了对 `join` 的调用。

```
hello number 1 from main thread!
hello number 1 from spawned thread!
hello number 2 from spawned thread!
hello number 2 from main thread!
hello number 3 from main thread!
hello number 3 from spawned thread!
hello number 4 from spawned thread!
hello number 4 from main thread!
hello number 5 from spawned thread!
hello number 5 from main thread!
hello number 6 from main thread!
hello number 6 from spawned thread!
hello number 7 from spawned thread!
hello number 7 from main thread!
hello number 8 from spawned thread!
hello number 9 from spawned thread!
```

两个线程持续交错输出，但主线程由于调用了 `handle.join()` 而暂停，并且在派生线程执行完成之前不会终止。

让我们看看如果我们将 `handle.join()` 移到 `main` 中的 `for` 循环之前会发生什么，如下所示：

```

use std::thread;
use std::time::Duration;
fn main() {
    let handle = thread::spawn(|| {
        for x in 1..10 {
            println!("hello number {} from spawned thread!", x);
            thread::sleep(Duration::from_millis(1));
        }
    });
    handle.join().unwrap();
    for x in 1..8 {
        println!("hello number {} from main thread!", x);
        thread::sleep(Duration::from_millis(1));
    }
}

```

主线程在启动其 for 循环之前等待生成的线程完成，因此输出不再交错，如下所示：

```

hello number 1 from spawned thread!
hello number 2 from spawned thread!
hello number 3 from spawned thread!
hello number 4 from spawned thread!
hello number 5 from spawned thread!
hello number 6 from spawned thread!
hello number 7 from spawned thread!
hello number 8 from spawned thread!
hello number 9 from spawned thread!
hello number 1 from main thread!
hello number 2 from main thread!
hello number 3 from main thread!
hello number 4 from main thread!
hello number 5 from main thread!
hello number 6 from main thread!
hello number 7 from main thread!

```

一些小细节（例如调用 join 的位置），可能会影响我们的线程是否并发运行。

在线程之间使用消息传递传输数据

消息传递（线程或参与者通过相互发送包含数据的消息进行通信）是确保安全并发的一种越来越流行的方法。用 Go 语言官方文档的话来说，“不要通过共享内存相互通信；与之相反，通过相互通信来共享内存。”

通道（channel）是 Rust 标准库实现的编程概念，是实现消息发送并发性的关键工具。程序中的一个通道可以比作一条水道（例如一条小溪或一条河流），如果我们把一只橡皮鸭扔到小溪里，它会顺流而下，直到到达水道的尽头。

在编程中，一个通道被分成两半：发射器（transmitter）和接收器（receiver）。发射器那一半是我们把橡皮鸭放进的河上游，而接收器那一半是橡皮鸭到达的河下

游。代码的一部分通过调用发射器上的方法发送数据，而另一部分检查接收端的消息。当通道上的发射器或接收器被丢弃时，称为通道关闭。

在本节中，我们将构建一个程序，其中一个线程产生值并将其发送到通道，另一个线程接收值并将它们打印输出。

我们将通过通道在线程之间发送简单的值来演示该特性。一旦你掌握了这项技术，你就可以使用通道来构建一个聊天系统或一个这样的系统，在这个系统中，许多线程执行计算的不同部分，并将结果发送给一个聚合结果的线程。

首先，我们将创建一个通道，但不对其进行任何操作。值得注意的是，这还不能编译，因为 Rust 不知道我们要通过通道发送什么样的值。

```
use std::sync::mpsc;
fn main() {
    let (tx,rx) = mpsc::channel();
}
```

使用 `mpsc::channel` 函数，我们创建了一个新通道；`mpsc` 代表多个生产者、单个消费者（multiple producers, single consumer 的缩写）。Rust 标准库实现通道的方式允许多个发送端产生值，但只有一个接收端使用这些值。设想多条河流合并成一条大河：任何一条河流中的任何东西最终都会流入同一条河流。我们现在从一个生产者开始，但一旦这个例子成功，我们将添加更多的生产者。

`mpsc::channel` 函数返回一个元组，第一个元素表示发送端，第二个元素代表接收端。我们相应地命名变量，因为缩写 `tx` 和 `rx` 在许多领域中通常用于表示发射器和接收器。我们将 `let` 语句与模式结合使用以解构元组。这种使用 `let` 语句的方法是提取 `mpsc::channel` 返回元组片段的方便方法。

如示例所示，让我们将发射端移动到派生线程中，并发送一个字符串，以便派生线程可以与主线程通信。这相当于向上游扔一只橡皮鸭，或者说从一个线程向另一个线程发送聊天消息。

```
use std::sync::mpsc;
use std::thread;
fn main() {
    let (tx,rx) = mpsc::channel();
    thread::spawn(move || {
        let vals = String::from("hello");
        tx.send(vals).unwrap();
    });
}
```

同样，我们使用 `thread::spawn` 来启动一个新线程，然后将 `tx` 移动到闭包中，以便派生线程拥有 `tx`。要通过通道发送消息，派生线程必须拥有通道的发射端。

发射端有一个 `send` 方法，它接受我们要发送的值。由于 `send` 方法返回 `Result` 类型，所以如果接收端已被丢弃并且无处发送值，那么它将返回错误。在本例中，我们指示解包在发生错误时进行 `panic`。然而，我们将在实际应用程序中正确处理它；返回第 7 章，回顾错误处理策略。

在主线程中，我们将从通道的接收端获取值。这类似于从河口的水中取回一只橡皮鸭或接收聊天信息。

```
use std::sync::mpsc;
use std::thread;
fn main() {
    let (tx,rx) = mpsc::channel();
    thread::spawn(move || {
        let vals = String::from("hello");
        tx.send(vals).unwrap();
    });
    let receive = rx.recv().unwrap();
    println!("Got {}", receive);
}
```

`recv` 和 `try_recv` 是通道接收端的两种有用方法。我们使用 `recv`（receive 接收的缩写）来阻止主线程的执行，并等待一个值沿通道发送。发送值后，`recv` 将返回 `Result<T, E>`。当通道的发送端关闭时，`recv` 将返回一个错误，表示不再发送任何值。

`try_recv` 方法不会阻塞，而是立即返回 `Result<T, E>`：如果有消息可用，则为包含消息的 `Ok` 值；如果没有可用消息，则为 `Err` 值。假设该线程在等待消息时有其他任务要完成。在这种情况下，我们可以编写一个循环，每隔一段时间调用 `try_recv`，处理一条消息（如果有的话），否则在再次检查之前短时间内执行其他工作。

为了简单起见，我们在本例中使用了 `recv`；除了等待消息之外，主线程没有其他工作要做，因此阻塞主线程是合适的。

当我们执行代码时，将从主线程打印输出以下值：

Got hello

通道和所有权转让

```
use std::sync::mpsc;
use std::thread;
fn main() {
    let (tx,rx) = mpsc::channel();
    thread::spawn(move || {
        let vals = String::from("hello");
        tx.send(vals).unwrap();
        println!("vals is {}", vals);
    });
    let receive = rx.recv().unwrap();
}
```

```
    println!("Got {}", receive);
}
```

在这里，我们尝试在使用 `tx.send` 将 `vals` 发送到通道后打印输出 `vals`。但这是一个坏主意，因为一旦值被发送到另一个线程（假设为 B 线程），此 B 线程可能会在再次使用它之前修改它或删除它。由于数据不一致或不存在，其他线程的修改可能会导致错误或意外结果。而且，当我们试图编译代码时，Rust 会抛出一个错误：

```
error[E0382]: borrow of moved value: `vals`
--> r3.rs:8:32
6 |         let vals = String::from("hello");
  |         ---- move occurs because `vals` has type `String`, which
  |         does not implement the `Copy` trait
7 |         tx.send(vals).unwrap();
  |         ---- value moved here
8 |         println!("vals is {}", vals);
  |                                   ^^^^ value borrowed here after move

= note: this error originates in the macro `crate::format_args_nl` which comes from the expansion of the macro `println` (in Nightly builds, run with -Z macro-backtrace for more info)
```

我们的并发错误导致了编译时错误。当发送参数时，`send` 函数将获得该参数的所有权，而当值被移动时，接收方将获得该参数的所有权。这可以防止我们在发送后意外重用该值；所有权机制确保一切井然有序。

发送多个值并看到接收方在等待

代码已编译并运行，但它没有澄清两个独立的线程正在通过通道进行通信。我们做了一些更改，允许我们在并发运行代码之前测试代码：生成的线程现在将发送多条消息，每一条消息之间暂停一秒钟。

```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;
fn main() {
    let (tx, rx) = mpsc::channel();
    thread::spawn(move || {
        let vals = vec![
            String::from("hello"),
            String::from("everyone"),
            String::from("from"),
            String::from("thread"),
        ];
        for val in vals {
            tx.send(val).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });
}
```

```

    });
    for receive in rx {
        println!("Got {}", receive);
    }
}

```

这一次，派生线程包含一个我们要发送到主线程的字符串向量。我们遍历此向量它们，每次发送一个，并通过调用 `thread::sleep` 函数实现每次发送后暂停 1 秒。

我们不再在主线程中显式调用 `recv` 函数；相反，我们将 `rx` 视为迭代器。我们正在打印输出每个接收到的值。迭代将在通道关闭时结束。

当您运行代码时，我们应该看到下面的输出，每行之间有 1 秒的暂停：

```

Got hello
Got everyone
Got from
Got thread

```

我们告诉主线程正在等待来自派生线程的值，因为主线程中的 `for` 循环中没有暂停或延迟的代码。

通过克隆发射器创建多个生产者

我们之前提到过，`mpsc` 是多个生产者，单个消费者的首字母缩写。让我们使用 `mpsc` 并扩展前面的代码来创建多个将值发送到同一接收器的线程。我们可以通过克隆通道的传输部分来实现这一点：

```

use std::sync::mpsc;
use std::thread;
use std::time::Duration;
fn main() {
    let (tx,rx) = mpsc::channel();
    let tx1 = tx.clone();
    thread::spawn(move || {
        let vals = vec![
            String::from("hello"),
            String::from("everyone"),
            String::from("from"),
            String::from("thread"),
        ];
        for val in vals {
            tx1.send(val).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });
    thread::spawn(move || {
        let vals = vec![
            String::from("many"),
            String::from("messages"),

```

```

        String::from("from"),
        String::from("us"),
    ];
    for val in vals {
        tx.send(val).unwrap();
        thread::sleep(Duration::from_secs(1));
    }
});
for receive in rx {
    println!("Got {}", receive);
}
}

```

在这次创建第一个派生线程之前，我们在发射器上调用 `clone` 方法。这将为我们提供一个新的发送句柄，我们可以将其传递给生成的第一个线程。我们将通道的原始发送端传递给派生的第二个线程。这会导致两个线程，每个线程向通道的接收端发送不同的消息。

共享状态并发

消息传递是处理并发性的一种优秀方法，但并不是唯一的方法。思考一下 Go 语言官方文档中的以下语句：“不要通过共享内存进行通信。”

通过共享内存进行通信会是什么样子？此外，为什么消息传递爱好者不使用它，而是做相反的事情？

通道类似于单一所有权，因为一旦值通过通道转移，就不应再使用它。通过共享内存并发，多个线程可以同时访问相同的内存位置，这类似于多个所有权。Rust 的类型系统和所有权规则非常有助于实现正确的管理。考虑互斥锁，它是共享内存中最常用的并发原语之一。

使用互斥体（mutex）一次只允许一个线程访问数据

互斥（mutex）是 **mutual exclusion**（相互排斥）的缩写，因为互斥只允许一次只能一个线程访问某些数据。线程必须首先通过请求互斥锁（mutex's lock）来发出其访问互斥锁中数据的意图。锁是一种互斥数据结构，用于跟踪谁在任何给定时间对数据具有独占访问权限。因此，互斥锁被描述为使用锁定系统保护其保存的数据。

互斥体因难以使用而名声不佳，因为需要遵守以下两条规则：

1. 在使用数据之前，我们必须首先尝试获取锁。
2. 当我们完成互斥锁所保护的数据时，我们必须将其解锁，以便其他线程可以获取锁。

考虑在一个只有一个麦克风的会议上进行小组讨论，作为互斥锁的现实隐喻。在小组成员发言之前，他必须要求或表明希望使用麦克风。当他拿到麦克风时，可以在将麦克风传给下一个要求发言的小组成员之前，说他想说的话。如果发言人在发言

结束时忘记关闭麦克风，其他人则无法发言。如果共享麦克风管理不当，讨论小组将无法正常工作。

互斥管理很难掌握，这就是为什么这么多人热衷于通道。然而，由于 Rust 的类型系统和所有权规则，我们在锁定和解锁方面不会出错。

Mutex API

让我们从单线程上下文中的互斥体开始，作为如何使用互斥体的示例：

```
use std::sync::Mutex;
fn main() {
    let mt = Mutex::new(5);
    {
        let mut numb = mt.lock().unwrap();
        *numb = 6;
    }
    println!("mt = {:?}", mt);
}
```

我们使用其相关的函数 `new` 创建了 `Mutex`，就像我们对许多其他类型所做的那样。我们使用 `lock` 方法获取锁，以访问互斥锁内的数据。此调用将阻止当前线程，阻止它执行任何操作，直到释放互斥锁为止。

如果持有锁的另一个线程发生 `panick`，那么对锁的调用将失败。在这种情况下，没有任何线程能够获得锁，因为我们决定 `unwrap`，如果我们发现自己处于这种情况，就让这个线程 `panick`。

获得锁后，我们可以将返回值（在本例中为 `numb`）视为对其中包含数据的可变引用。在使用 `mt` 中的值之前，类型系统确保我们获得一个锁：`Mutex<i32>` 而不是 `i32`，因此我们必须在使用 `i32` 值之前获取锁。这一点我们决不能忘记；否则，类型系统将不允许我们访问内部 `i32`。

正如您所料，`Mutex` 是一个聪明的指针。更具体地说，对 `lock` 的调用将返回一个封装在 `LockResult` 中的 `MutexGuard` 智能指针，我们使用 `unwrap` 解包调用来处理该指针。

`MutexGuard` 智能指针使用 `Deref` 来指向我们的内部数据，智能指针还有一个 `Drop` 实现，当 `MutexGuard` 退出作用域时，它会自动释放锁，这发生在内部作用域的末尾。因此，我们不必担心忘记释放锁并阻止其他线程使用互斥锁，因为锁释放是自动的。

我们可以在释放锁后打印输出互斥值，并看到可以已经将内部 `i32` 类型的值更改为 6。

在多个线程之间共享 Mutex

现在，让我们看看是否可以在多个线程之间使用 **Mutex** 以共享值。我们将启动十个线程，每个线程将使计数器值加 1，使计数器总数达到 10。下面的示例将包含一个编译器错误，我们将使用它来了解 **Mutex** 相关的更多信息以及 **Rust** 如何帮助我们正确使用它。

```
use std::sync::Mutex;
use std::thread;
fn main() {
    let counter = Mutex::new(0);
    let mut handles = vec![];
    for _ in 0..11 {
        let handle = thread::spawn(move || {
            let mut numb = counter.lock().unwrap();
            *numb += 1;
        });
        handles.push(handle);
    }
    for handle in handles {
        handle.join().unwrap();
    }
    println!("Results: {}", *counter.lock().unwrap());
}
```

和前面一样，我们创建一个计数器变量以便将 `i32` 类型的值保存到互斥锁中。然后，通过在一组数字上迭代生成 10 个线程。我们使用 `thread::spawn` 并为所有线程分配相同的闭包，这个闭包将计数器移动到线程中，通过调用 `lock` 方法获得互斥锁，然后给互斥锁中的值加 1。当一个线程完成闭包时，`numb` 退出作用域并释放锁，允许另一个线程获取它。

我们收集主线程中的所有连接句柄。然后，像之前一样，在每个句柄上调用 `join`，以确保所有线程都能执行完成。主线程将获取互斥锁并在此时打印输出计数器的值。

我暗示了这个示例无法编译的事实。现在让我们来研究一下原因!

```
error[E0382]: use of moved value: `counter`
--> r3.rs:7:36
|
|
4 |         let counter = Mutex::new(0);
|         ----- move occurs because `counter` has type `Mutex<i32>`,
|         which does not implement the `Copy` trait
|
...
7 |         let handle = thread::spawn(move || {
|                                     ^^^^^^^ value moved into closure
|         here, in previous iteration of loop
8 |         let mut numb = counter.lock().unwrap();
|         ----- use occurs due to use in closure
```


错误消息表明计数器值在前一个循环迭代期间被移动了。Rust 告诉我们，我们不能跨多个线程移动锁计数器的所有权。

LQ 注：原书中此处的错误没做进一步的解释和改进，经检索，发现内容来自于[官方文档](<https://doc.rust-lang.org/book/ch16-03-shared-state.html?highlight=mutex#multiple-ownership-with-multiple-threads>)，正确的代码如下：

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];
    for _ in 0..11 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut numb = counter.lock().unwrap();
            *numb += 1;
        });
        handles.push(handle);
    }
    for handle in handles {
        handle.join().unwrap();
    }
    println!("Results: {}", *counter.lock().unwrap());
}
```

运行代码，输出：Results: 11。

具有 Sync 和 Send 特征（trait）的可扩展并发性

令人惊讶的是，Rust 编程语言几乎没有并发特性。我们在本章中讨论的几乎所有并发特性都是标准库的一部分，而不是语言的一部分。我们没有限制用语言或标准库来处理并发；您可以编写自己的并发特性，还可以使用其他人编写的并发特性。

然而，Rust 语言包含了两个并发概念：std::marker 中的 Sync trait 和 Send trait。

允许在线程之间使用 Send 转移所有权

Send 特征表明实现 Send 的类型的值可以在线程之间传输。几乎每个 Rust 数据类型都具备 Send，但也有一些例外，包括 Rc：这不能发送，因为如果我们克隆了 Rc 的值并试图将所有权转移到另一个线程，两个线程可能会同时更新引用计数。因此，Rc 设计用于我们不想支付线程安全性能损失的单线程情况。

因此，Rust 的类型系统和 trait 边界确保我们不会以不安全的方式跨线程发送 Rc 值。当我们尝试在我们的例子中这样做时，我们收到了错误 Rc<Mutex> 没有实现 Copy trait。我们切换到 Arc，即 Send 编译的代码。

任何完全由 **Send** 类型组成的类型都会被自动标记为 **Send**。除了原始指针之外，几乎所有的基元类型都是 **Send**。

可以通过 **Sync** 多线程访问

Sync 特征表明实现 **Sync** 的类型可以从多个线程安全地引用。换句话说，如果 **&T** 是 **Send**，那么类型 **T** 都是 **Sync**，这表明引用可以安全地传递给另一个线程。像 **Send** 样的基元类型都是 **Sync**，完全由 **Sync** 类型组成的类型也是 **Sync**。

由于同样的原因，智能指针 **Rc** 不是 **Send**，也不是 **Sync**。**RefCell** 类型及其相关的 **Cell** 类型不是 **Sync**。借用检查的 **RefCell** 实现不是线程安全的。**Mutex** 是 **Sync**，可以用于与多个线程共享访问，如“在多个线程之间共享互斥锁”小节中所示。

手动实现 **Send** 和 **Sync** 是不安全的

我们不必手动实现 **Send** 和 **Sync** 特征，因为由这些特征组成的类型是自动 **Send** 和 **Sync** 的。他们甚至没有实现 **marker** 特征的方法。它们只对强制并发的不变量有用。

手动实现这些 **trait** 需要使用不安全的 **Rust** 代码。重要的是要记住，创建不由 **Send** 和 **Sync** 组件组成的新并发类型需要仔细考虑以维护安全保证。

我们在本章中讨论了使用线程同时运行代码，使用消息传递在线程之间传输数据，共享状态并发，以及具有 **Sync** 和 **Send** 特征的可扩展并发。

*LQ 总结：本章很多内容照搬了 **Rust** 官方文档，而不是自己原创，挺失望的。*

C12: Rust 的面向对象编程

- 2022.12.5

在本章中

- 实现面向对象的设计模式
- 宏 (Macros)

在前一章中，我们介绍了如何使用线程运行代码和消息传递来传输数据。我们还学习了共享状态并发和具有 `Sync` 和 `Send` 特征的可扩展并发。本章将讨论实现面向对象的设计模式和宏。

实现面向对象的设计模式

状态模式 (State Pattern) 是面向对象编程中使用的一种设计模式。该模式的关键在于值具有由一组状态对象表示的内部状态，并且值的行为取决于内部状态的变化。状态对象的功能是共享的：在 Rust 中，我们使用 `struct` (结构) 和 `trait` (特征)，而不是对象和继承。每个状态对象负责其行为并决定何时更改状态。保存状态对象的值不知道状态的各种行为，也不知道何时在状态之间转换。

使用状态模式意味着，如果程序的业务需求发生变化，我们不必更改保存状态的值的代码或使用该值的代码。要更改其中一个状态对象的规则或添加更多的状态对象，我们只需更新其中一个状态对象中的代码。让我们看一个状态设计模式的示例，以及如何在 Rust 中使用它。

我们将逐步实现一个博客文章的工作流程。博客的最终功能看起来是这样的：

- 一个空白的草稿是一篇博文的起点。
- 当草稿完成后，帖子被审核。
- 一旦帖子被批准，就会发布。
- 未经批准的帖子不能意外地发表，因为只有发表的博文才会返回内容打印输出。

对一个帖子所做的任何额外修改都不应该对工作流程产生任何影响。例如，如果我们试图在要求审查之前批准一篇博文的草稿，那么该博文应该保持未发表。

下面的代码说明了这个工作流程：

这是一个如何在名为 `blog` 的库 `crate` 中使用 `API` 的示例。这将无法编译，因为我们还没有实现 `blog crate`。

```
use blog::Post;
fn main() {
    let mut post = Post::new();
```

```

    post.add_text("We ate a salad for dinner today.");
    assert_eq!("", post.content());
    post.request_review();
    assert_eq!("", post.content());
    post.approve();
    assert_eq!("We ate a salad for dinner today.", post.content());
}

```

使用 `Post::new`，我们想让用户创建一个新的博文草稿。然后，当博文仍处于草稿模式时，我们想允许向草稿中添加文本。如果我们试图在博文批准之前立即获得帖子的内容，那么应该什么也看不到，因为博文仍然是草稿状态。为了演示，我们向代码中添加了 `assert_eq!`。一个优秀的单元测试应该断言一篇博文草稿从 `content()` 方法返回一个空字符串，但我们不会为这个例子编写测试。

接下来，我们想要对博文发起审查的请求，我们希望 `content()` 在等待审查时仍然返回一个空字符串。当博文被批准后，它应该被发布，这意味着当调用 `content()` 时，博文的文本将被返回。

值得注意的是，`Post` 类型是我们在 `crate` 中与之交互的唯一类型。这种类型将使用状态模式，并包含一个值，该值将是三个状态对象之一，代表一篇博文处于草稿、等待审查或发表的各种状态。`Post` 类型将在内部处理从一种状态到另一种状态的转换。这些状态往往会随着我们的库用户在 `Post` 实例上使用的方法而相应变化，但他们并不需要直接管理状态的变化。此外，用户不能在状态上犯错误，比如在帖子被审查之前就进行发布。

在 Draft（草稿）状态下定义 `Post` 并创建新实例

让我们开始库的实现。我们需要一个带有一些内容的 `public Post` 结构体，因此我们将从结构定义和相关的 `public new` 函数开始创建 `Post` 实例。我们还将创建一个 `State trait`。然后，在一个名为 `state` 的私有字段中，`Post` 将在 `Option` 中存储 `Box` 的 `trait` 对象。稍后，我们将理解为什么需要 `Option`。

```

pub struct Post {
    state: Option<Box<dyn State>>,
    content: String,
}

impl Post {
    pub fn new() -> Post {
        Post {
            state: Some(Box::new(Draft {})),
            content: String::new(),
        }
    }
}

trait State {}

```

```
struct Draft {}
impl State for Draft {}
```

State 特征定义了不同帖子状态的共享行为，它将由 Draft（草稿）、PendingReview（等待审查）和 Published（发布）三种状态实现。目前，该特征还没有包含任何方法，我们将只从定义 Draft 状态开始，因为这是我们希望博文帖子开始的状态。

当我们创建一个新的帖子（即 Post）时，我们将其状态（即 state）字段设置为 Some，其中包括一个 Box。此盒子指向一个新的草稿（即 Draft）结构实例。这就保证了每当我们创建一个新的帖子实例时，它将从草稿（即 Draft）开始。没有办法在其他状态下创建新帖子（即 Post），因为 Post 的状态（即 state）字段是私有的！我们在 Post::new 函数中将 content 内容字段设置为 new()，空的字符串。

存储帖子内容的文本

前面的示例演示了我们想要调用一个名为 add_text 的方法并传递给它一个 &str，然后将其添加到博文帖子的文本内容中。我们没有将内容字段公开为 pub，而是将其实现为一个方法。这意味着我们以后可以实现一个方法来控制如何读取内容字段中的数据。add_text 的方法非常简单直接，所以让我们将这个实现添加到 impl Post 块中：

```
impl Post {
    // -- 代码片段 --
    pub fn add_text(&mut self, text: &str) {
        self.content.push_str(text);
    }
}
```

当我们在调用 add_text 时更改 Post 实例，add_text 方法需要一个对 self 的可变引用。然后，我们调用 push_str 并传递 text 参数以便向 content 中添加 String 上保存的内容。因为这个行为独立于 post 的状态，所以它不是状态模式的一部分。尽管 add_text 方法与 state 字段没有交互，但它是我们希望支持的行为的一部分。

确保草稿的内容为空

因为博文帖子仍然处于草稿状态，所以即使在使用了 add_text 并向其添加了一些内容之后，我们也希望 content 方法返回一个空的字符串切片。现在，让我们用最基本的东西来实现 content 方法来满足这个需求：总是返回一个空字符串切片。稍后，当我们可以改变一个帖子的状态，使其可以被发布时，我们将改变这个。帖子目前只能处于草稿状态，所以帖子的内容应该始终为空。这是一个占位符的实现，如下面的代码所示。

```
impl Post {
    // -- 代码片段 --
    pub fn content(&self) -> &str {
        ""
    }
}
```

```
    }  
}
```

请求审查会改变帖子的状态

接下来，我们必须添加请求审查帖子的功能，这应该将帖子的状态从草稿（即 Draft）更改为待审（即 PendingReview）。

```
impl Post {  
    // -- 代码片段 --  
    pub fn request_review(&mut self) {  
        if let Some(sm) = self.state.take() {  
            self.state = Some(sm.request_review())  
        }  
    }  
}  
  
trait State {  
    fn request_review(self: Box<Self>) -> Box<dyn State>;  
}  
  
struct Draft {}  
impl State for Draft {  
    fn request_review(self: Box<Self>) -> Box<dyn State> {  
        Box::new(PendingReview {})  
    }  
}  
  
struct PendingReview {}  
impl State for PendingReview {  
    fn request_review(self: Box<Self>) -> Box<dyn State> {  
        self  
    }  
}
```

我们为 Post 提供了一个公开的 request_review 方法，该方法接受一个对 self 的可变引用。然后，在 Post 的当前状态上，我们调用一个内部的 request_review 方法，这个第二个 request_review 方法使用当前状态并返回一个新状态。

request_review 方法已添加到 State trait 中；现在所有实现这个特征的类型都必须实现 request_review 方法。方法的第一个参数不是 self、&self 或 &mut self，而是 self: Box。此语法表明，该方法仅在对包含该类型的盒子上调用时才有效。此语法获得了 Box 的所有权，使旧的状态无效，并允许 Post 的状态值转换为新的状态。

request_review 方法必须取得状态值的所有权才能使用旧的状态。这就是 Post 的 state 字段中的 Option 的作用：我们使用 take 方法从 state 字段中删除 Some 值并将其替换为 None，因为 Rust 不允许我们在结构中使用未填充的字段。这允许我们从 Post 中删除状态值，而不是借用它。Post 的 state 值将被设置为这个操作的结果。

为了获得 `state` 值的所有权，我们必须暂时将 `state` 设置为 `None`，而不是直接使用 `self.state = self.state.request_review()` 这样的代码来设置它。这可以防止 `Post` 在转换为新状态后重新使用旧的 `state` 值。

`Draft` 上的 `request_review` 方法必须返回一个新的 `PendingReview` 结构体的盒子实例，它代表帖子处于等待审核时的状态。`PendingReview` 结构体也实现了 `request_review` 方法，但是它不执行任何转换。相反，它会返回自身，因为当我们请求对一个已经处于 `PendingReview` 状态的帖子进行审查时，它应该保持这个状态。

现在我们可以看到状态模式的好处：`Post` 上的 `request_review` 方法无论其状态值如何都是一样的。每个状态都负责自己的一套规则。

我们将保留 `Post` 的 `content()` 方法，返回一个空的字符串切片。现在我们可以让一个 `Post` 同时处于 `Draft` 和 `PendingReview` 状态，但是我們希望在 `PendingReview` 状态下有相同的行为。

`approve` 方法的功能类似于 `request_review` 方法，它将 `state` 设置为当前状态表明它在该状态被批准时应具有的值，如下所示：

```
impl Post {
    // -- 代码片段 --
    pub fn approve(&mut self) {
        if let Some(sm) = sm.state.take() {
            self.state = Some(sm.approve())
        }
    }
}

trait State {
    fn request_review(self: Box<Self>) -> Box<dyn State>;
    fn approve(self: Box<Self>) -> Box<dyn State>;
}

struct Draft {}
impl State for Draft {
    // -- 代码片段 --
    fn approve(self: Box<Self>) -> Box<dyn State> {
        self
    }
}

struct PendingReview {}
impl State for PendingReview {
    // -- 代码片段 --
    fn approve(self: Box<Self>) -> Box<dyn State> {
        Box::new(Published {})
    }
}

struct Published {}
impl State for Published {
```



```

    fn request_review(self: Box<Self>) -> Box<dyn State> {
        self
    }
    fn approve(self: Box<Self>) -> Box<dyn State> {
        self
    }
}

```

然后，我们将 `approve` 方法添加到 `State` 特征和一个实现 `State` 的新结构体，`Published` 状态。

类似于 `request_review`，在 `Draft` 上调用 `approve` 方法没有效果，因为它返回 `self`。当我们在 `PendingReview` 上使用 `approve` 方法时，我们得到了 `Published` 结构的一个新的盒子实例。语句中的 `Published` 结构体实现了 `State` 特征，并且对于 `request_review` 和 `approve` 方法它都返回自己，因为在这两种情况下帖子应该保持 `Published` 状态。

下一步是更新 `Post` 上的 `content` 方法，以便如果状态为 `Published`，则返回 `Post` 的 `content` 字段中的值；否则，我们返回一个空的字符串切片，如下所示：

```

impl Post {
    // -- 代码片段 --
    pub fn content(&self) -> &str {
        self.state.as_ref().unwrap().content(self)
    }
    // -- 代码片段 --
}

```

因为我们的想法是在实现 `State` 的结构中维护所有这些规则，所以我们对 `State` 中的值调用一个 `content` 方法，并将 `post` 实例(即 `self`) 作为参数发送。然后返回对状态值调用 `content` 方法的结果。

我们在 `Option` 上调用 `as_ref` 方法，是因为我们想要一个对包含在 `Option` 中的值的引用，而不是拥有该值的所有权。这是因为状态是一个 `Option<Box>`，当我们调用 `as_ref` 时，我们得到一个 `Option<&Box>`。如果我们不调用 `as_ref`，就会得到一个错误，因为我们不能将状态从借来的函数形参 `&self` 移出。

此外，我们调用 `unwrap` 方法，我们知道它永远不会 `panic`，因为 `Post` 上的方法保证了当这些方法完成后，状态总是会包含一个 `Some` 值。这是第七章“我们拥有比编译器更多信息的情况”一节中讨论的情况之一，我们知道 `None` 值永远不可能，尽管编译器无法理解它。

当我们在 `&Box` 上调用 `content` 时，强制解除引用将影响 `&` 和 `Box`，导致在实现 `State` 特征的类型上调用 `content` 方法。这意味着我们需要将 `content` 添加到 `State trait` 定义中，这就是我们将根据我们所处的状态返回什么内容的逻辑，如下所示：

```

trait State{
    // -- 代码片段 --
    fn content<'a>(&self, post: &'a Post) -> &'a str {
        ""
    }
}
struct Published {}
impl State for Published {
    // -- 代码片段 --
    fn content<'a>(&self, post: &'a Post) -> &'a str {
        &post.content
    }
}

```

`content` 方法现在有一个默认实现，返回一个空的字符串切片。这意味着我们不需要向 `Draft` 和 `PendingReview` 结构体添加 `content` 方法。`content` 方法将被 `Published` 结构体重写，以便返回 `post.content` 中的值。

这个方法需要生命期注释，如第 8 章所述。我们将一个 `post` 引用作为参数，并返回对该 `post` 的一部分的引用，因此返回的引用的生命期与 `post` 参数的生命期相关。

状态模式的权衡

我们已经演示了 Rust 可以实现面向对象的状态模式来封装 `post` 在每个状态中应该具有的各种行为。`Post` 的方法对各种行为并不了解。由于我们组织代码的方式，我们只需要在一个地方就可以了解已发布帖子的各种行为方式：在 `published` 结构体上实现 `State` 特征。

如果我们不使用状态模式，我们可以在 `Post` 的方法中使用匹配表达式，甚至在检查帖子状态的主代码中使用匹配表达式，在这些地方改变行为。这意味着我们必须在多个地方查看，以了解一个帖子被发布的所有影响！这意味着我们必须在多个地方查看。随着我们增加更多的状态，这种情况只会更糟：每个匹配表达式都需要另一个分支。

状态模式消除了对 `Post` 方法和使用 `Post` 的地方使用匹配表达式的需要，要添加一个新状态，我们只需要添加一个新的结构体，并在该结构体上实现 `trait` 方法。

状态模式的实现很容易扩展，可以添加更多的功能。尝试考虑这些建议，看看维护使用状态模式的代码是多么容易：

- 添加一个 `reject` 方法，将文章从 `PendingReview` 状态返回到 `Draft` 状态。
- 在状态更改为 `Published` 之前，必须批准两个调用。
- 只有当帖子处于 `Draft` 状态时，才允许用户向其添加文本内容。提示：让状态对象负责对内容的任何更改，但不负责修改 `Post`。

状态模式的一个缺点是有些状态是相互耦合的，因为这些状态实现了状态之间的转换。如果我们在 `PendingReview` 和 `Published` 之间添加一个新状态，例如

Scheduled，为了从 PendingReview 转换到 Scheduled，我们将需要修改 PendingReview 中的代码。如果 PendingReview 不需要随着新状态的添加而改变，那么工作量就会降到最低，但这就需要切换到一个不同的设计模式。

状态模式的另一个缺点是我们重复了一些逻辑。为了减少重复，我们可以为 State trait 的 request_review 和 approve 方法创建默认实现，返回 self；然而，这将违反对象安全，因为 trait 不知道具体的 self 将是什么。因为我们想把 State 作为一个 trait 对象，它的方法必须是对象安全的。

其他重复的例子包括 Post 对 request_review 和 approve 方法的相似实现。这两个方法都将 Option 的 state 字段中值的实现委托给同一个方法，并将 state 字段的新值设置为结果。如果我们在 Post 上有多个方法遵循这个模式，我们可以定义一个宏来消除这种重复的现象。

这有一个缺点，因为我们在这里没有充分利用 Rust 的优势，完全按照面向对象语言的定义来实现状态模式。让我们深入了解一下我们可以对 blog crate 进行的一些更改，这些更改会导致无效的状态和转换，从而导致编译时错误。

宏 (macros)

在本书中，我们使用了像 println! 等宏，但我们还没有完全探索什么是宏，以及它是如何工作的。术语宏指的是 Rust 的一个特性家族：带有 macro_rules! 声明的宏以及三种类型的过程宏：

- 用于结构和枚举的自定义 #[derived] 宏，指定使用 derive 派生属性添加的代码。
- 类属性宏，定义可应用于任何项的自定义属性。
- 类似函数的宏，看起来像函数调用，但在作为参数传递给它们的标记上执行操作。

我们将依次讨论这些内容，但首先考虑一下，既然我们已经有了函数，为什么还需要宏。

宏和函数的区别

宏从根本上说是一种编写代码的方法，它可以编写代码的代码，这种技术称为元编程 (meta-programming)。我们在附录 C 中讨论了派生属性，它为我们生成了各种 trait 的实现。在本书中，我们还使用了 println! 和 vec! 宏。所有这些宏都能扩展出比我们手工写的更多的代码。

LQ 注：原书中对元编程的定义没解释清楚，换用这个解释就比较清晰了。元编程是值通过操作程序实体(program entity)，在编译时(compile time)计算出运行时(runtime)需要的常数、类型、代码的方法。

元编程是一种有用的编程工具，可以减少必须编写和维护的代码量，这也是函数的作用之一。另一方面，宏有一些函数所不具备的额外功能。

参数的数量和类型必须在函数签名中声明。另一方面，宏可以接受无限数量的参数：我们可以用一个参数调用 `println!("hello")`，或者用两个参数调用 `println!("hello {}", name)`。此外，宏在编译器解释代码的含义之前就被展开了，因此宏可以在给定的类型上实现一个特征。而函数就做不到，因为它在运行时被调用，而特征必须在编译时实现。

使用宏而不是函数的缺点是，宏的定义比函数的定义更复杂，因为我们编写的是编写 Rust 代码的 Rust 代码。由于这种间接关系，宏定义通常比函数定义更难阅读、理解和维护。

宏和函数之间的另一个重要区别是，在文件中调用宏之前必须定义宏或将宏引入作用域，而函数可以在任何地方定义和调用。

LQ 注：本书的内容完全照搬了 Rust 官方文档，见<https://doc.rust-lang.org/book/ch19-06-macros.html>，下面一节同样如此，缺少自己的见解和总结。

面向通用元编程的带 `macro_rules!` 的声明性宏

声明性宏是 Rust 中最常见的宏类型。这些宏也被称为“示例宏”、“`macro_rules!` 宏”，或者简单地称为“宏”。声明性宏，其核心是允许我们编写类似 Rust 匹配表达式的东西。如第 5 章所述，匹配表达式是一种控制结构，它接受一个表达式，将其结果值与模式进行比较，然后运行与匹配模式相关的代码。

宏还可以将一个值与特定代码相关的模式进行比较：在这种情况下，值是传递给宏的 Rust 源代码的字面量值；模式与该源代码的结构进行比较；与每个模式相关联的代码，如果匹配，将替换传递给宏的代码。这一切都发生在编译过程中。

`macro_rules!` 构造用于定义一个宏。让我们看看 `vec!` 宏是如何定义的，看看我们如何使用 `macro_rules!`。例如，下面这个宏生成一个包含三个整数的新向量：

```
let vs: Vec<u32> = vec![1,2,3];
```

`vec!` 宏也可以创建一个有两个整数的向量或一个有五个字符串切片的向量。我们不能使用一个函数来完成同样的事情，因为我们无法提前知道值的数量或类型。

下面是对 `vec!` 宏的一个稍微简化的定义：

```
#[macro_export]
macro_rules! vec {
    ($($c:expr),*) => {
        {
            let mut temp_vec = Vec::new();
            $(
                temp_vec.push($c);
            )*
            temp_vec
        }
    }
}
```

```
    };  
}
```

“#[macro_export]”注释指出，当定义宏的 crate 进入作用域时，这个宏应该可用。除非存在此注释，否则宏不能引入作用域。

然后，宏定义从 `macro_rules!` 以及我们定义的宏的名字且不带感叹号开始。名字（在本例中是 `vec`）后面跟着花括号，花括号代表宏定义的主体。

`vec!` 主体的结构类似于匹配表达式。一个匹配臂有模式 `((x:expr),*)`，后面跟着 `=>` 和与此模式相关的代码块。如果找到该模式，则执行相关的代码块。因为这是这个宏中唯一的模式，所以只有一种匹配方式；任何其他模式都会导致错误。更复杂的宏将有多个匹配臂。

因为宏模式是针对 Rust 代码结构而不是针对值匹配的，所以宏定义中的有效模式语法不同于模式语法。让我们看一看模式的片断；完整的宏模式语法请阅读参考文献。

首先，整个模式用一组圆括号括起来。后面是一个美元符号 `()`，后面是一组括号，用于捕获与括号内模式匹配的值，以便在替换代码中使用。`c:expr` 是 `$()` 中的一个函数，它匹配任何 Rust 表达式，并将其命名为 `$c`。

`$()` 后面的逗号表示在与 `$()` 中的代码相匹配的代码之后可以出现逗号分隔符字面量。`(*)` 表示该模式（即之前出现的内容）可以匹配 0 次或多次。

当我们使用 `vec[1,2,3]`；要调用这个宏，`$c` 模式需要与 1、2 和 3 表达式匹配三次。

让我们来看看与此匹配臂相关的代码中的模式：`$()*` 中的 `temp_vec.push()` 会为模式中与 `$()` 匹配的每个部分生成 0 次或多次，这取决于模式匹配的次数。每个匹配的表达式替换 `$c`。当我们用 `vec[1,2,3]`；调用这个宏时，生成的替换这个宏调用的代码如下：

```
{  
    let mut temp_vec = Vec::new();  
    temp_vec.push(1);  
    temp_vec.push(2);  
    temp_vec.push(3);  
    temp_vec  
}
```

我们已经创建了一个宏，它可以接受任意数量的任何类型的参数，并生成代码来创建具有指定元素的向量。

对于 `macro_rules`，有一些奇怪的边缘情况！在未来的几年里，Rust 将会有第二种类型的声明性宏，其功能类似，但将会解决其中一些边缘情况。`macro_rules!` 将会该更新生效后弃用。有鉴于此，大多数 Rust 程序员只需会使用宏而无需掌握编写宏的技能，故我们不会再详细讨论 `macro_rules!` 更多的内容。

用于从属性中生成代码的程序性宏

程序性宏，其行为更像函数，是第二种类型的宏(也是一种过程类型)。程序性宏接受一些代码作为输入，对这些代码进行操作，并生成一些代码作为输出，这与声明性宏相反，声明性宏与模式相匹配，并以其他代码替换代码。

三种类型的程序性宏（自定义派生、类属性和类函数）都以相同的方式发挥作用。

在创建程序性宏时，定义必须保存在它们自己的 `crate` 中，并有一个独特的 `crate` 类型。这是由于各种技术问题造成的，我们希望在将来解决这些问题。使用程序性宏的代码看起来像下面这样，其中 `some_attribute` 是用于使用特定宏的占位符。

LQ 注：原书中缺少了下面这段代码，但是可以从 [Rust 官方文档](#) 中找回。

```
use proc_macro;

#[some_attribute]
pub fn some_name(input: TokenStream) -> TokenStream {
}
```

定义程序性宏的函数接受一个 `TokenStream` 参数作为输入并输出一个 `TokenStream`。Rust 中包含的 `proc_macro crate` 定义了 `TokenStream` 类型，表示一个标记的系列。这是宏的核心：宏所操作的源代码是输入的 `TokenStream`，而宏生成的代码是输出的 `TokenStream`。该函数还有一个属性，用于指定我们正在创建的程序性宏的类型。在同一个 `crate` 中，我们可以有不同类型的程序性宏。

让我们深入了解程序性宏的各种类型。我们将从一个自定义派生宏开始，然后解释其他形式之间的微小差异。

如何编写一个自定义派生宏

让我们创建一个名为 `hello_macro` 的 `crate`，它定义了一个名为 `HelloMacro` 的 `trait` 和一个名为 `hello_macro` 的函数。我们将提供一个程序性宏，允许用户用 `#[derived(HelloMacro)]` 注释他们的类型，以获得 `hello_macro` 函数的默认实现，而不是要求我们的 `crate` 用户为他们的每个类型实现 `HelloMacro` 特征。默认情况下，将会打印：“Hello, Macro! My name is TypeName,” `TypeName` 是定义这个特征的类型名称。换句话说，我们将创建一个 `crate`，允许另一个程序员使用我们的 `crate` 来编写代码。

```
use hello_macro::HelloMacro;
use hello_macro_derive::HelloMacro;

#[derive(HelloMacro)]
struct Pancake;

fn main() {
```



```
    Pancake::hello_macro();  
}
```

这段代码将产生一个打印输出：“Hello, Macro! My name is Pancake!”当我们完成后，第一步是创建一个新的库 crate，如下所示：

```
$ cargo new hello_macro --lib
```

LQ 注：这节的内容没讲完，但原书中没有了，我从 Rust 官方文档中补全内容。

接下来，我们将定义 HelloMacro 特征及其相关函数：

```
// src/lib.rs  
pub trait HelloMacro {  
    fn hello_macro();  
}
```

我们有了一个特征及其函数。此时，我们的 crate 用户可以实现这个特征来实现想要的功能，如下所示：

```
use hello_macro::HelloMacro;  
  
struct Pancake;  
  
impl HelloMacro for Pancake {  
    fn hello_macro() {  
        println!("Hello, Macro! My name is Pancakes!");  
    }  
}  
  
fn main() {  
    Pancake::hello_macro();  
}
```

然而，此 crate 的用户需要为他们想要使用 hello_macro 的每个类型编写实现块；我们想让他们省去这项工作。

此外，我们还不能为 hello_macro 函数提供默认实现，该实现将打印特征实现类型的名称：Rust 没有反射功能，因此它不能在运行时查找类型的名称。我们需要一个宏来在编译时生成代码。

下一步是定义程序性宏。在撰写本文时，程序性宏需要放在它们自己的 crate 中。最终，这一限制可能会被取消。构造 crate 和宏 crate 的约定如下：对于名为 foo 的 crate，自定义派生程序性宏 crate 被称为 foo_derived。让我们在 hello_macro 项目中启动一个名为 hello_macro_derived 的新 crate：

```
$ cargo new hello_macro_derive --lib
```


这两个 crate 是紧密相关的，因此我们在 `hello_macro` crate 的目录中创建程序性宏 crate。如果我们改变了 `hello_macro` 中的 `trait` 定义，我们也必须改变 `hello_macro_derived` 中程序性宏的实现。这两个 crate 需要分别单独发布，使用这些 crate 的程序员需要将两者作为依赖项添加，并将它们都引入作用域。我们可以让 `hello_macro` crate 使用 `hello_macro_derived` 作为依赖项，并重新导出程序性宏代码。然而，我们的项目结构使程序员有可能使用 `hello_macro`，即使他们不需要 `derive` 功能。

我们需要将 `hello_macro_derived` crate 声明为程序性宏 crate。稍后您将看到，我们还需要来自 `syn` 和 `quote` crate 的功能，因此我们需要将它们作为依赖项添加。在 `Cargo.toml` 中添加如下内容：

```
// hello_macro_derive/Cargo.toml
[lib]
proc-macro = true

[dependencies]
syn = "1.0"
quote = "1.0"
```

要开始定义程序性宏，请将下面的代码放入 `src/lib.rs` 文件中，用于 `hello_macro_derived` crate。注意，在为 `impl_hello_macro` 函数添加定义之前，这段代码不会被编译。

```
// hello_macro_derive/src/lib.rs
use proc_macro::TokenStream;
use quote::quote;
use syn;

#[proc_macro_derive(HelloMacro)]
pub fn hello_macro_derive(input: TokenStream) -> TokenStream {
    // 将 Rust 代码构建为我们可以操作的语法树的表现形式
    let ast = syn::parse(input).unwrap();
    // 建立特征的实现
    impl_hello_macro(&ast)
}
```

注意，我们将代码分成了 `hello_macro_derived` 函数（负责解析 `TokenStream`）和 `impl_hello_macro` 函数（负责转换语法树）：这使得编写程序性宏更加方便。外层函数（在本例中为 `hello_macro_derived`）中的代码对于您看到或创建的几乎每个程序性宏 crate 都是相同的。在内层函数体中指定的代码（在本例中为 `impl_hello_macro`）将根据程序性宏的目的而有所不同。

我们引入了三个新的 crate： `proc_macro`、`syn` 和 `quote`。 `proc_macro` crate 是 Rust 自带的，所以我们不需要在 `Cargo.toml` 的依赖项中添加它。 `proc_macro` crate 是编译器的 API，它允许我们从我们的代码中读取和操作 Rust 代码。

`syn crate` 将 Rust 代码从字符串解析为我们可以执行操作的数据结构。`quote crate` 将 `syn` 数据结构转换回 Rust 代码。这些 `crate` 使得解析我们可能想要处理的任何类型的 Rust 代码变得更加简单：为 Rust 代码编写一个完整的解析器不是一件简单的任务。

当我们库的用户在一个类型上指定 `#[derived(HelloMacro)]` 时，将调用 `hello_macro_derived` 函数。这是可能的，因为我们在这里用 `proc_macro_derived` 注释了 `hello_macro_derived` 函数，并指定了 `HelloMacro` 名称，这与我们的 `trait` 名称相匹配；这是大多数程序性宏遵循的约定。

`hello_macro_derived` 函数首先将输入从 `TokenStream` 转换为一个数据结构，然后我们可以对其进行解释和执行操作。这就是 `syn` 发挥作用的地方。`syn` 中的解析函数接受一个 `TokenStream` 并返回一个代表解析后的 Rust 代码的 `DeriveInput` 结构。下面的代码显示了我们通过解析 `struct Pancakes;` 字符串得到 `DeriveInput` 结构的相关部分：

```
DeriveInput {  
    // --snip--  
  
    ident: Ident {  
        ident: "Pancakes",  
        span: #0 bytes(95..103)  
    },  
    data: Struct(  
        DataStruct {  
            struct_token: Struct,  
            fields: Unit,  
            semi_token: Some(  
                Semi  
            )  
        }  
    )  
}
```

这个结构的字段表明，我们所解析的 Rust 代码是一个单元结构，其 `ident`（标识符，指名称）为 `Pancakes`。这个结构上还有更多的字段用于描述各种 Rust 代码；可以阅读 `DeriveInput` 的 `syn` 文档以了解更多信息。

很快我们就会定义 `impl_hello_macro` 函数，这个函数中我们将构建我们想要包含的新 Rust 代码。但在此之前，请注意派生宏的输出也是一个 `TokenStream`。返回的 `TokenStream` 被添加到我们的 `crate` 用户编写的代码中，因此当他们编译他们的 `crate` 时，他们将获得我们在修改后的 `TokenStream` 中提供的额外功能。

您可能已经注意到，如果在这里对 `syn::parse` 函数的调用失败，则调用 `unwrap` 会导致 `hello_macro_derived` 函数 `panic`。我们的程序性宏有必要在出错时 `panic`，因为 `proc_macro_derived` 函数必须返回 `TokenStream` 而不是 `Result` 以符合程序性宏

的 API。我们通过使用 `unwrap` 简化了这个例子；在生产代码中，您应该使用 `panic!` 或 `expect` 来提供更具体的错误消息，说明哪里出了问题。

现在我们有将注释的 Rust 代码从 `TokenStream` 转换为 `DeriveInput` 实例的代码，让我们生成代码，在注释的类型上实现 `HelloMacro` 特征，如下所示：

```
// hello_macro_derive/src/lib.rs
fn impl_hello_macro(ast: &syn::DeriveInput) -> TokenStream {
    let name = &ast.ident;
    let gen = quote! {
        impl HelloMacro for #name {
            fn hello_macro() {
                println!("Hello, Macro! My name is {}", stringify!(#name));
            }
        }
    };
    gen.into()
}
```

我们使用 `ast.ident` 获得一个 `Ident` 结构实例，其中包含被注释类型的名称(标识符)。上面代码中的结构表明，当我们将前面代码运行 `impl_hello_macro` 函数时，我们得到的 `ident` 将具有 `Ident` 字段，其值为“pancake”。因此，代码中的 `name` 变量将包含一个 `Ident` 结构实例，当打印时，该实例将是字符串“pancake”，即代码中的结构名称。

`quote!` 宏允许我们定义我们想要返回的 Rust 代码。编译器对 `quote!` 宏的直接执行结果有不同的期望，所以我们需要把它转换成一个 `TokenStream`。我们通过调用 `into` 方法来实现这一点，该方法使用这个中间表示，并返回所需的 `TokenStream` 类型的值。

`quote!` 宏还提供了一些非常酷的模板机制：我们可以输入 `#name`，而 `quote!` 宏将用变量名中的值来替换它。您甚至可以做一些类似于常规宏工作方式的重复操作。可以查看 `quote crate` 的文档以了解更多信息。

我们希望我们的程序性宏为用户注释的类型生成 `HelloMacro trait` 的实现，我们可以通过使用 `#name` 来获得。`trait` 的实现有一个 `hello_macro` 函数，它的主体包含了我们想要提供的功能：打印输出“printing Hello, Macro! My name is”，然后是被注释的类型的名字。

这里使用的 `stringify!` 宏是内置于 Rust 中的。它接受一个 Rust 表达式，例如 `1+2`，并在编译时将表达式转换为字符串文字，例如“1+2”。这与 `format!` 或 `println!` 不同，后者是对表达式进行评估，然后将结果变成一个字符串。`#name` 输入可能是一个要按字面意思打印的表达式，因此我们使用 `stringify!`。使用 `stringify!` 还可以在编译时将 `#name` 转换为字符串字面量，从而节省一个分配。

此时，`cargo build` 应该在 `hello_macro` 和 `hello_macro_derived` 中成功完成。让我们将这些 crate 与前面的代码连接起来，看看程序性宏的运行情况。使用 `cargo new pancake` 在 `projects` 目录中创建一个新的二进制项目。我们需要在 `pancake` 的 `Cargo.toml` 中添加 `hello_macro` 和 `hello_macro_derived` 作为依赖项。如果您已经将 `hello_macro` 和 `hello_macro_derived` 的版本发布到 `crates.io`，那它们将是常规的依赖项；如果没有发布，你可以将它们指定为路径依赖项，如下所示：

```
[dependencies]
hello_macro = {path="./hello_macro"}
hello_macro_derive = {path="./hello_macro_derive"}
```

将前面的代码放入 `src/main.rs` 中，并运行 `cargo run`，它应该能打印输出“Hello, Macro! My name is Pancakes!”。程序性宏中 `HelloMacro` 特征的实现被包含在内，而不需要 `pancakes` 来实现它，`#[derived (HelloMacro)]` 添加了 `trait` 实现。

LQ 注：我执行的结果如下

```
D:\workspace\rust-works\ch12> cargo run
    Updating `rsproxy` index
    Compiling proc-macro2 v1.0.49
    Compiling unicode-ident v1.0.6
    Compiling quote v1.0.23
    Compiling syn v1.0.107
    Compiling hello_macro v0.1.0 (D:\workspace\rust-works\ch12\hello_macro)
    Compiling hello_macro_derive v0.1.0 (D:\workspace\rust-works\ch12\hello_macro_derive)
    Compiling ch12 v0.1.0 (D:\workspace\rust-works\ch12)
    Finished dev [unoptimized + debuginfo] target(s) in 6.08s
    Running `target\debug\ch12.exe`
Hello, Macro! My name is Pancakes!
```

接下来，让我们探讨其他类型的程序性宏与自定义派生宏有什么不同。

类属性宏

类属性宏与自定义派生宏相似，因为它们为派生属性生成代码，而不是让我们创建新的属性。它们的用途也更广泛：派生只适用于结构体和枚举，而属性可以适用于其他类型的对象，如函数。下面是一个类属性宏的例子：假设我们有一个路由属性，在一个 Web 应用框架中注释函数：

```
#[route(GET, "/")]
fn index() {
```

这个 `#[route]` 属性将被框架定义为一个程序性宏。宏定义函数的签名看起来是这样的：

```
#[proc_macro_attribute]
pub fn route(attr: TokenStream, item: TokenStream) -> TokenStream {
```

在这里，我们有两个 `TokenStream` 类型的参数。第一个是属性的内容：`GET, "/"` 部分。第二个是属性所连接的项的主体：在本例中是 `fn index(){}` 和函数主体的其余部分。

除此之外，类属性宏的工作方式与自定义派生宏相同：使用 `proc-macro crate` 类型创建一个 `crate`，并实现一个能生成你想要的代码的函数。

类函数宏

类函数宏定义了看起来像函数调用的宏。与 `macro_rules!` 宏类似，它们比函数更灵活。例如，它们可以接受未知数量的参数；而 `macro_rules!` 宏只能使用我们在前面的“面向通用元编程的带 `macro_rules!` 的声明性宏”一节中讨论过的类似匹配的语法来定义。类函数宏接受一个 `TokenStream` 参数，它们的定义使用 Rust 代码操作该 `TokenStream`，就像其他两种程序性宏一样，类函数宏的一个例子是 `sql!` 宏，它可以这样调用：

```
let sql = sql!(SELECT * FROM posts WHERE id=1);
```

这个宏会解析其中的 SQL 语句，并检查它的语法是否正确，这比 `macro_rules!` 宏能做的处理要复杂得多。`sql!` 宏的定义如下：

```
#[proc_macro]
pub fn sql(input: TokenStream) -> TokenStream {
```

这个定义类似于自定义派生宏的签名：我们接收圆括号内的标记，并返回我们想要生成的代码。

Appendix: 附录

- 2022.1.1

附录 A: 关键字

下面列出的关键字保留给 Rust 语言，当前或将来使用。因此，函数、变量、参数、结构字段、模块、`crate`、常量、宏、静态值、属性、类型、特征或生命周期的名称不能用作标识符（原始标识符除外，见“原始标识符(raw identifiers)”节所述）。

在使用的关键词

下面描述的功能目前附加到以下关键字：

- `as`: 执行原语强制转换，消除包含项的特定特征的歧义，或重命名正在使用的项和 `extern crate` 语句
- `async`: 返回 `Future`，而不是阻塞当前线程
- `await`: 暂停执行，直到 `Future` 的结果准备就绪
- `break`: 立即退出循环
- `const`: 定义常量项或常量原始指针
- `continue`: 继续进行下一次循环迭代
- `crate`: 链接外部 `Crate` 或宏变量，表示定义宏的 `Crate`
- `dyn`: 对 `trait` 对象的动态调度
- `else`: `if` 和 `if let` 控制流结构的回退
- `enum`: 定义枚举
- `extern`: 链接外部 `crate`、函数或变量
- `false`: 布尔型假的字面量
- `fn`: 定义函数或函数指针类型
- `for`: 遍历迭代器中的项、实现 `trait` 或指定更高的生命周期
- `if`: 根据条件表达式的结果进行分支
- `impl`: 实现固有的或特征的功能
- `in`: `for` 循环语法的一部分
- `let`: 绑定变量
- `loop`: 无条件循环
- `match`: 将值匹配到模式
- `mod`: 模块定义
- `move`: 使闭包拥有它捕获的所有对象
- `mut`: 表示引用、原始指针或模式绑定中的可变性
- `pub`: 表示 `struct` 字段、`impl` 块或模块中的公共可见性

- `ref`: 引用绑定
- `return`: 从函数返回
- `Self`: 定义或实现的类型的类型别名
- `self`: 当前模块或方法主题
- `static`: 持续整个程序执行的全局变量或生命周期
- `struct`: 定义结构
- `super`: 当前模块的父模块
- `trait`: 定义特征
- `true`: 布尔值真的字面量
- `type`: 定义类型别名或关联类型
- `union`: 定义 `Union`, 仅在联合声明中使用关键字
- `unsafe`: 表示不安全的代码、功能、特征或实现
- `use`: 将符号带入作用域内
- `where`: 表示约束类型的子句
- `while`: 根据表达式的结果有条件地循环

保留给未来使用的关键字

以下关键字没有任何功能，但已经被 `Rust` 保留，以便在将来可能使用。

- `abstract`
- `become`
- `box`
- `do`
- `final`
- `macro`
- `override`
- `priv`
- `try`
- `typeof`
- `unsized`
- `virtual`
- `yield`

原始标识符（`Raw Identifiers`）

原始标识符是允许我们在通常禁止使用关键字的地方使用关键字的语法。原始标识符是通过在关键字前加上 `r#` 来创建的。

例如，`match` 是一个关键字。如果我们尝试编译以下函数，它具有名称匹配：


```
fn match(needles: &str, haystack: &str) -> bool {
    haystack.contains(needles)
}
```

我们将得到如下错误：

```
error: expected identifier, found keyword `match`
--> src/main.rs:1:4
1 | fn match(needles: &str, haystack: &str) -> bool {
  |      ^^^^^ expected identifier, found keyword
help: escape `match` to use it as an identifier
1 | fn r#match(needles: &str, haystack: &str) -> bool {
  |      ++
```

该错误表明关键字 `match` 不能被用作函数标识符。要使用 `match` 作为函数名，请使用原始标识符的语法，它看起来像这样。

```
fn r#match(needles: &str, haystack: &str) -> bool {
    haystack.contains(needles)
}
fn main() {
    assert!(r#match("foo", "foobar"));
}
```

这段代码编译没有任何问题。注意函数名定义中的 `r#` 前缀，以及函数在 `main` 中的位置。

原始标识符允许我们使用任何我们想要的单词作为标识符，即使它是保留的关键字。此外，原始标识符使我们能够使用不同 Rust 版本编写的库，而不是我们的 `crate` 所使用的版本。

附录 B：运算符和符号

本附录包括 Rust 语法术语表，其中提供了运算符和其它符号，这些符号单独出现或组合出现在路径、泛型、`trait` 边界、宏、属性、注释、元组或括号中。

操作符

表 B.1 包括了 Rust 操作符，操作符在上下文中出现的例子，简短的解释，以及操作符是否可以重载。如果一个操作符可以重载，则列出重载该操作符的相关特征。

运算符	例子	解释	是否可重载
!	ident!(...), ident!{...}, ident![...]	宏展开	
!	!expr	按位或逻辑补码	Not
!=	var != expr	不等的比较	PartialEq
%	expr % expr	算术余数	Rem
%=	var %= expr	算术余数和赋值	RemAssign
&	&expr, &mut expr	借用	
&	&type, &mut type, &'a type, &'a mut type	借用指针类型	
&	expr & expr	位和	BitAnd
&=	var &= expr	位和与赋值	BitAndAssign
&&	expr && expr	逻辑与运算	
*	expr * expr	乘法运算	Mul
*=	var * = expr	算术乘法和赋值	MulAssign
*	*expr	解除引用	Deref
*	*const type, *mut type	原始指针	
+	trait + trait, 'a + trait	复合类型约束	
+	expr + expr	算术加法	Add
+=	var += expr	算术加法和赋值	AddAssign
,	expr, expr	参数和元素分隔符	
-	- expr	算术求反	Neg
-	expr - expr	算术减法	Sub
-=	var -= expr	算术减法和赋值	SubAssign
->	fn(...) -> type, [...] -> type	函数和闭包返回类型	
.	expr.ident	成员访问	
..	..., expr..., ..expr, expr..expr	右包含的范围	PartialOrd
..=	..=expr, expr..=expr	右包涵的范围和赋值	PartialOrd
..	..expr	结构字面值更新语法	
..	variant(x...), struct_type {x...}	“其余”模式绑定	
...	expr...expr	(已废弃,使用 ..= 代替)在模式中:包含范围的模式	
/	expr / expr	算术除法	Div
/=	var /= expr	算术除法与赋值	DivAssign
:	pat: type, ident: type	限制条件	
:	ident: expr	结构字段的初始化器	
:	'a: loop {...}	循环标签	
:	expr;	语句和项结束符	
:	[...; len]	数组固定大小语法的一部分	
<<	expr << expr	左移	Shl
<<=	var <<= expr	左移和赋值	ShlAssign
<	expr < expr	小于比较	PartialOrd
<=	expr <= expr	小于或等于比较	PartialOrd
=	var = expr, ident = type	赋值/等价	
==	expr == expr	相等的比较	PartialEq
=>	pat => expr	match 匹配臂语法的一部分	
>	expr > expr	大于比较	PartialOrd
>=	expr >= expr	大于或等于比较	PartialOrd
>>	expr >> expr	右移	Shr
>>=	var >>= expr	右移和赋值	ShrAssign
@	ident @ pat	模式绑定	
^	expr ^ expr	位异或	BitXor
^=	var ^= expr	位异或和赋值	BitXorAssign
	pat pat	模式的选择	
	expr expr	按位或	BitOr
=	var = expr	位或和赋值	BitOrAssign
	expr expr	逻辑或运算	
?	expr?	错误传播	

表B.1

非运算符的符号

下面的列表包括了所有不能作为运算符的符号，它们的行为不像函数或方法调用。

表 B.2 中的符号是单独出现的，在各种上下文中都有效。

符号	解释
<code>'ident</code>	命名的生命周期或循环标签
<code>...u8, ...i32, ...f64, ...usize, etc.</code>	特定类型的数字字面量
<code>" ..."</code>	字符串字面量
<code>r" ...", r#"..."#, r##"..."##, etc.</code>	原始字符串字面量，未处理转义字符
<code>b" ..."</code>	字节字符串字面量；构造一个 <u>[u8]</u> 而不是一个字符串
<code>br" ...", br#"..."#, br##"..."##, etc.</code>	原始字节字符串字面量，原始字节字符串字面值的组合
<code>'...'</code>	字符串字面量
<code>b'...'</code>	ASCII字节字面量
<code>[...] expr</code>	封闭（闭包）
<code>!</code>	分歧函数的底部类型始终为空
<code>_</code>	"忽略"模式绑定；也用于使整数字面量可读

表 B.2

表 B.3 中的符号出现在从模块层次结构到一个项的路径上下文中。

符号	解释
<code>ident::ident</code>	命名空间路径
<code>::path</code>	相对于crate根的路径(即显式的绝对路径)
<code>self::path</code>	相对于当前模块的路径(即显式的相对路径)。
<code>super::path</code>	相对于当前模块的父模块的路径
<code>type::ident, <type as trait>::ident</code>	相关的常量、函数和类型
<code><type>::...</code>	不能直接命名的类型的关联项(例如, <code><&T>::...</code> , <code><[T]>::...</code> 等)
<code>trait::method(...)</code>	通过命名定义方法调用的trait来消除方法调用的歧义
<code>type::method(...)</code>	通过命名定义方法调用的类型来消除方法调用的歧义
<code><type as trait>::method(...)</code>	通过命名特征和类型来消除方法调用的歧义

表 B.3

表 B.4 显示了使用泛型类型参数时出现的符号。

符号	解释
<code>path<...></code>	在一个类型中指定泛型类型的参数(例如 <code>Vec<u8></code>)
<code>path::<...>, method::<...></code>	在一个表达式中为泛型类型、函数或方法指定参数；通常被称为涡轮鱼(turbofish)，例如 <code>"42".parse::<i32>()</code>
<code>fn ident<...> ...</code>	定义泛型函数
<code>struct ident<...> ...</code>	定义泛型结构
<code>enum ident<...> ...</code>	定义泛型枚举
<code>impl<...> ...</code>	定义泛型实现
<code>for<...> type</code>	更高等级的生命周期界限
<code>type<ident=type></code>	泛型类型，其中一个或多个关联类型具有特定赋值(例如 <code>Iterator<Item=T></code>)

表 B.4

表 B.5 描述了用 trait 边界约束泛型类型参数时出现的符号。

符号	解释
T: U	泛型参数T被约束为实现U的类型
T: 'a	泛型类型T必须超过生命周期 'a
T: 'static	泛型类型T除“静态引用”外不包含任何借用引用
'b: 'a	泛型生命周期 'b 的寿命必须超过比生命周期 'a
T: ?Sized	允许泛型类型参数为动态大小的类型
'a + trait, trait + trait	复合型的约束

表 B.5

表 B.6 显示了在调用或定义宏和指定项上的属性时出现的符号。

符号	解释
#[meta]	外部属性
#![meta]	内部属性
\$ident	宏替换
\$ident:kind	宏捕获
\$(...)	宏重复
ident!(...), ident!{...}, ident![...]	宏调用

表 B.6

表 B.7 显示了产生注释的符号。

符号	解释
//	行注释
//!	内行文档注释
///	外行文档注释
/* ... */	块注释
/*! ... */	内部块文档注释
/** ... */	外部块文档注释

表 B.7

表 B.8 显示了使用元组时出现的符号。

符号	解释
()	空元组，包括字面量和类型
(expr)	圆括号表达式
(expr.)	单元素元组表达式
(type.)	单元素元组类型
(expr,...)	元组表达式
(type,...)	元组类型
expr(expr,...)	函数调用表达式；也用于初始化元组结构和元组枚举变量
expr.0, expr.1, etc.	元组索引

表 B.8

表 B.9 显示了大括号使用的上下文情况。

上下文	解释
<code>{...}</code>	块表达式
<code>Type {...}</code>	结构字面量

表 B.9

表 B.10 显示了使用方括号的上下文情况。

上下文	解释
<code>[...]</code>	数组字面量
<code>[expr; len]</code>	数组字面量，包含表达式的len长度复制
<code>[type; len]</code>	数组类型，包含类型的len长度实例
<code>expr[expr]</code>	集合索引，可重载(<code>Index</code> , <code>IndexMut</code>)
<code>expr[..], expr[a..], expr[..b], expr[a..b]</code>	集合索引假装是一个集合切片，使用 <code>Range</code> 、 <code>RangeFrom</code> 、 <code>RangeTo</code> 或 <code>RangeFull</code> 作为“索引”

表 B.10

附录 C：可衍生的特征（Derivable Traits）

在本书中，我们讨论了 `derive` 派生属性，它可以应用于结构体或枚举的定义。派生属性生成的代码，该代码将在使用派生语法注释的类型上实现其默认实现的 `trait`。

这里包含了所有可用于派生的标准库特征的列表。每个部分都讨论了：

- 这个 `trait` 将启用哪些操作符和方法。
- 派生提供的 `trait` 实现所做的工作。
- `trait` 的实现对于类型意味着什么。
- 在什么情况下你可以或不可以使用这个 `trait`。
- 需要该 `trait` 操作的例子。

如果我们想要重写派生属性提供的行为，请参考每个 `trait` 的标准库文档，了解如何做到这一点。

其余的标准库 `trait` 不能在使用派生的类型上实现。因为这些特征没有合理的默认行为，所以我们要以一种对你想要实现的目标有意义的方式来实现它们。

为终端用户处理格式化的 `Display` 就是不能派生 `trait` 的一个例子。我们应该始终考虑向用户显示一个类型的最佳方式。用户应该能够看到类型的哪些部分？哪些部分他们会觉得有趣？什么样的数据格式对他们最有利？因为 `Rust` 编译器缺乏这方面的知识，所以它不能为我们提供适当的默认行为。

本附录中的可派生 `trait` 列表并不是详尽的：库可以为自己的 `trait` 实现派生，从而使我们可以使用派生的 `trait` 列表真正开放。

用于程序员输出的 Debug trait

Debug 特征允许在格式字符串中进行调试格式化，这可以通过在 `{}` 中插入 `:?` 占位符。

Debug 特征使您能够为调试目的打印一个类型的实例，使我们和其他使用该类型的程序员能够在程序执行的特定点上检查一个实例。

例如在使用 `assert_eq!` 宏时，需要使用 Debug 特征。如果相等断言失败，此宏将打印作为参数传递的实例的值，以便程序员可以看到两个实例不相等的原因。

与 PartialEq 和 Eq 的相等性比较

PartialEq 特征对一个类型的实例进行比较以检查是否相等，并支持 `==` 和 `!=` 运算符。

`eq` 方法在 PartialEq 派生特征中实现。当 PartialEq 应用于结构时，只有当两个实例的所有字段都相等时，它们才相等，如果它们中的任何一个字段不相等，它们就不相等。当从枚举派生时，每个变体都等于它自己，但不等于其他变体。

例如，在使用 `assert_eq!` 宏时需要 PartialEq 特征，它要求比较一个类型的两个实例是否相等。

Eq 特征没有任何方法。它的目的是表明对于带注释类型的每一个值来说，该值都等于它自己。Eq 特征只适用于实现了 PartialEq 特征的类型，但并不是所有实现了 PartialEq 特征的类型都可以实现 Eq 特征。浮点数类型就是一个例子：浮点数的实现说明两个非数字（not-a-number，NaN）值的实例彼此不相等。

对于 `HashMap<K,V>` 中的键，需要 Eq，以便 `HashMap<K,V>` 可以确定两个键是否相同。

用于排序比较的 PartialOrd 和 Ord

PartialOrd 特征比较同一类型的实例以进行排序。实现 PartialOrd 特征的类型可以与运算符 `<`、`>`、`<=` 和 `>=` 一起使用。PartialOrd 特征只能应用于同时实现了 PartialEq 特征的类型。

派生 PartialOrd 特征使用 `partial_cmp` 方法，该方法返回一个 `Option`，如果给定的值没有产生排序，则该方法返回 `None`。非数字 (NaN) 浮点值是不产生排序值的一个例子，尽管该类型的大多数值都可以进行比较。当使用任何浮点数和 NaN 浮点值进行调用时，`partial_cmp` 将返回 `None`。

PartialOrd 通过比较每个字段的值来比较两个实例，当在结构上派生时，按照字段在结构定义中出现的顺序来进行比较。当在枚举上派生时，在枚举定义中较早声明的枚举变体被认为比后面列出的变体少。

例如，`rand crate` 的 `gen_range` 方法需要 `PartialOrd` 特征，该方法在 `range` 表达式指定的范围内生成一个随机值。

`Ord` trait 表明，对于注释类型的任何两个值来说，存在一个有效的排序。因为有效的排序总是可能的，`Ord` 特征实现了 `cmp` 方法，它返回一个 `Ordering` 而不是 `Option`。`Ord` 特征只能应用于同时实现了 `PartialOrd` 和 `eq` 的类型。`cmp` 在结构体和枚举上派生时的行为与 `partial cmp` 的派生实现对 `PartialOrd` 的行为相同。

当在 `BTreeSet` 中存储数值时，即根据数值的排序顺序来存储数据的数据结构，需要 `Ord`。

复制值的 `Clone`（克隆）和 `Copy`（拷贝）

`Clone` 特征允许我们创建一个值的显式深度副本，复制过程可能包括运行任意代码和复制堆数据。有关克隆的更多信息，请参阅第 3 章“变量和数据交互：Clone”节。

派生 `Clone` 实现了 `clone` 方法，当对整个类型实现时，它对类型的每个部分都调用 `clone`。这意味着为了派生 `Clone`，类型中的所有字段或值也必须实现 `Clone`。

例如，在一个切片上调用 `to_vec` 方法时，`Clone` 是必须的。切片不拥有它所包含的类型实例，但 `to_vec` 返回的向量拥有，因此 `to_vec` 对每个项调用 `clone`。因此，切片中存储的类型必须支持 `Clone` 特征。

`Copy` trait 允许我们通过简单地从栈中复制位来复制一个值；不需要任何代码。有关复制的更多信息，请参见第 3 章的“仅栈数据：复制”节。

`Copy` 特征没有定义方法，以防止程序员重载这些方法，从而违反没有任意代码被运行的假设。

`Copy` 可以从任何部分都实现 `Copy` 的类型中推导出来。因为实现 `Copy` 的类型有一个简单的 `Clone` 实现，它执行与 `Copy` 相同的任务，所以实现 `Copy` 的类型也必须实现 `Clone`。

很少需要 `Copy` 特征；实现 `Copy` 的类型具有可用的优化，这意味着我们不必调用 `clone`，从而使代码更简短。

`Copy` 能做的所有事情，`Clone` 也能做，但代码可能更慢或需要在某些地方使用 `clone`。

将值映射到固定尺寸值的哈希函数

`Hash` trait 允许您使用哈希函数将任意大小的类型实例映射到一个固定尺寸的值。派生 `Hash` 是一个实现了 `hash` 方法的类。`hash` 方法的派生实现将对类型的每个部分调用哈希的结果组合在一起，这意味着为了派生哈希，所有字段或值也必须实现 `Hash` 特征。

一个需要 `Hash` 特征的例子是在 `HashMap<K,V>` 中存储键以有效地存储数据。

面向默认值的 Default

`Default trait` 允许我们定义一个类型的默认值。派生 `Default` 是一个实现了 `default` 函数的函数。为了派生 `Default`, `Default` 函数的派生实现在类型的每个部分上调用 `default` 函数，这意味着类型中的所有字段或值也必须实现 `Default`。

`Default::default` 函数经常与第 4 章“使用结构更新语法从其他实例创建实例”节中讨论的结构更新语法一起使用。

通过使用 `Default::default()`，我们可以自定义结构的一些字段，然后为其余字段设置和使用默认值。

例如，在对 `Option` 实例使用 `unwrap_or_default` 方法时，需要使用 `Default` 特征。如果 `Option` 为 `None`，方法 `unwrap_or_default` 返回存储在 `Option` 中的类型 `T` 的 `Default::default` 结果。

评价（Appraisal）

Rust 是一种多范式、高阶、静态类型的脚本语言。这种语言主要强调安全和性能，帮助开发人员开发强大且安全的应用程序。与 `C/C++` 相比，**Rust** 已经克服了内存错误和并发编程的问题。

Rust 语言最初是为了解决 `C/C++` 的困难而设计的。然而，事实证明它非常成功，以至于现在许多顶级公司都在使用它，包括 **Dropbox**、**Firefox** 和 **Cloudflare**，包括初创公司和大型企业。

由于 **Rust** 的巨大优势，全世界有成百上千家企业都在使用它。它速度快，内存占用低。**Rust** 语言可以提供许多性能函数，与其他语言进行接口，并在不需要运行时或垃圾收集器的情况下在嵌入式设备上运行。

Rust 健壮的类型系统和所有权概念使其能够在构建时消除许多错误。它集成了包管理、支持多编辑器的类型分析和自动补全功能。

Rust 的背景

Mozilla Research 的 Graydon Hoare 在一个合作者团队的协助下创建了 **Rust**。Mozilla 基金会是其背后的支持者。

Rust 是一种社区驱动的开源语言。许多知名企业，如亚马逊、谷歌和微软，都在使用它。**Rust** 是一种适合于编程代码的并发且安全的语言。

为什么借用检查在 Rust 中很重要？

借用检查器（`borrow checker`）是 **Rust** 编程语言的一个基本元素，也是 **Rust** 的一个特点。它有助于所有权的管理。**Rust** 的另一个显著特征是所有权。它允许语言在不需要使用垃圾收集器的情况下提供内存安全。

首先，借用检查器为我们完成了什么，以及它如何连接到其他内存管理策略（如垃圾收集器和所有权）？让我们把事情简单化，因为有很多事情要做。

那么，借用检查器到底是什么？大多数脚本语言不需要我们考虑你的变量保存在哪里，因为垃圾收集器会为我们做这件事。然而，在 Rust 代码中，借用检查器处理一切。虽然 Rust 中没有指定的内存模型，但应用程序可以将数据存储两种类型的内存中：栈和堆。保存在栈上的数据必须指定其大小，而保存在堆上的数据则可以有任何大小。

数据访问是直接和快速的，但数据必须符合某些标准。由于没有严格的数据需求，堆方法较慢，但更具可配置性。当栈不可用时，它很有用。

在垃圾收集语言中，我们不必考虑数据的去向：栈还是堆。在 C 语言中，内存必须显式地分配给堆。记住，一旦我们避免了内存泄漏，就必须删除内存，但只能删除一次。

Rust 的目标是什么？

现在是时候看看 Rust 软件程序能做什么了。至少，它可以用于编程。该语言非常适合开发操作系统和微控制器应用程序。Rust 已经被用于构建几个强大的操作系统，包括 QuiltOS、Rux、Redox 和 intermezzOS。Mozilla 也在其网络引擎中使用了该语言。通常，Rust 编程语言可以用来创建以下软件：

LQ 注：显然这里没写完，但原书这里缺少内容。

Rust 语言的特殊特性

Rust 相对于其他脚本语言的最大优势是它的私密性。这部分是通过错误管理来实现的。如果在编译过程中出现无法更正的错误，则 `panic!` 宏被使用。这将终止应用程序并显示错误消息，以确保不会发生损害。

Rust 的内存管理也很安全。这样做的好处是 Rust 在不使用垃圾收集器的情况下确保了内存安全。内存一直是各种计算机语言的黑客经常攻击的目标。当内存耗尽时，可能会导致系统出现故障，从而导致可以利用的漏洞。“垃圾收集器”保证从内存中删除不需要的东西。然而，这会降低代码的执行速度。“垃圾收集器”已被 Rust 编译器淘汰。取而代之的是，它在编译期间检查任何内存错误。

但是，增强的安全保护不会以牺牲性能为代价。Rust 是一个运行速度与 C/C++ 相同的系统软件应用程序。一方面，这与拒绝“垃圾收集器”有关。快速运行时还可以通过“零成本抽象”得到保证，这意味着您可以使用具有高度抽象的语言进行编写，而不会遇到性能下降的问题。

因此，Rust 是高级和低级计算机语言的混合体。Rust 和 C/C++ 一样，接近于硬件，确保了快速的性能，同时又像高级语言一样易于开发。

Rust 对于新手和专业程序员来说都很容易学习。该语言在用法上类似于已知的替代方案。另一方面，设计错误警报的工作量是一个显著的优势。当其他脚本语言显示错误时，**Rust** 给出了如何解决这些错误的实用且有价值的信息。

Rust 编程语言的好处

Rust 最初是 **Mozilla** 的一个研究项目，其目标是重新实现 **Firefox** 浏览器的基本组件。这个选择的动机有几个重要因素促成的：**Firefox** 应该更多地利用当前的多核 CPU；网络浏览器的绝对流行需要它们的安全性。

然而，这些优点是所有软件都需要的，而不仅仅是浏览器，这就是为什么 **Rust** 从一个浏览器组件发展成为一个成熟的语言项目。**Rust** 通过以下特性实现了安全性、速度和可用性：

- **Rust** 提供内存安全性：**Rust** 不会构建试图使用不安全内存的程序。大多数内存问题是在程序执行时发现的。**Rust** 的语法和语言隐喻确保了其他语言中常见的内存相关问题——空指针或悬空指针、数据竞争等等——永远不会实现。编译器会检测到这些问题，并在程序执行之前强制解决这些问题。
- **Rust** 有很强的适应性：在必要的时候，**Rust** 让我们可以不计后果地存活。当我们需要直接修改内存时，**Rust** 的安全性可以部分挂起，例如在 **C/C++** 中取消引用一个原始引用，**Rust** 的安全机制可以部分暂停。因为 **Rust** 的内存安全操作永远不能完全停用，所以基本术语是“部分(停用)”。即便如此，在大多数常见的使用情况下，我们几乎从来没有拆除安全带，所以最终的结果是软件在默认情况下更安全。
- **Rust** 是跨平台的：它可以在三个主要的操作系统上运行：**Linux**、**Windows** 和 **MacOS**。除了这三个，还支持其他的。为不同的架构或平台交叉编译或生成二进制文件，而不是我们目前正在运行的架构或平台，只需要极少的工作量，但 **Rust** 的总体目标之一是减少此类任务所需的繁重工作量。此外，虽然 **Rust** 可以在大多数可用的系统上运行，但它的作者并不打算让 **Rust** 在任何地方都完全编译，而是只在不需要对语言做出过多牺牲的流行平台上编译。
- **Rust** 提供了一个方便的标准库：**Rust** 的更大目标包括鼓励 **C** 和 **C++** 开发人员尽可能选择 **Rust** 而不是其他语言。然而，**C** 和 **C++** 用户需要一个好的标准库——他们希望能够利用容器、集合和迭代器；操作字符串；管理进程和线程；进行网络和文件的 I/O 操作；等等。**Rust** 标准库执行所有这些操作，甚至更多。**Rust** 标准库只能包含可以安全地跨平台传输的特性，因为它本来就是跨平台的。特定于平台的函数，如 **Linux** 的 **epoll**，必须由第三方库提供。
- **Rust** 是一种强大的语言：很少有开发人员愿意开始使用一种新的语言，如果它提供的功能比他们习惯的语言更少或更弱。**Rust** 的原生语言特征可以与 **C++** 等语言相媲美：**Rust** 将宏、泛型、模式匹配和组合（通过“**trait**”（特

征)”)视为一等公民。一些在其他语言中流行的功能，比如内联汇编器（`inline assembler`），在 Rust 中也可以访问，尽管带有“不安全”标记。

- Rust 的设置很简单：如果不使用 Rust 的安全性和完整性功能，它们就没有任何意义。这就是为什么 Rust 程序员和社区一直在努力使这门语言对新手尽可能有帮助和友好。

构建 Rust 二进制文件所需的一切都包含在同一个包中。只有在 Rust 生态系统之外构建组件时才需要外部编译器，比如 GCC（例如，从源代码编译的 C 库）。微软 Windows 用户不会被视作二等公民；Rust 工具链在 Windows 上和 Linux 和 MacOS 上一样强大。

为什么公司要用 Rust？

如果我们关注内存的安全性，Rust 是一个极好的解决方案。尽管如此，许多人在对 C 和 C++ 编程语言的局限性感到不满后开始使用它。在设计 Rust 计算机语言时，工程师们专注于构建一个具有高效工作流程的环境。

与其他脚本语言相比，企业更喜欢 Rust 的主要原因如下：

1. 一个繁荣和参与活跃的社区
2. 稳定的软件性能保证
3. 具有卓越效率的内存安全性
4. 允许同时进行编程
5. 越来越多的 Rust 文档

LQ 注：最后的“参考文献”和“索引”节就不译了。